



Facultad de
**Ciencias Sociales y
Tecnologías de la Información**
Talavera de la Reina. UCLM

UNIVERSIDAD DE CASTILLA-LA MANCHA
Tecnologías y Sistemas de Información

GRADO EN INGENIERÍA INFORMÁTICA

SISTEMAS DE INFORMACIÓN

Trabajo Fin de Grado

Entangle: Mapeo, Análisis y Visualización del Ecosistema de Computación Cuántica en GitHub

**Entangle: Mapping, Analysis and Visualization
of the Open-Source Quantum Computing Ecosystem on GitHub**

Ángel Luis Lara Martín

Julio, 2026

ENTANGLE: MAPEO, ANÁLISIS Y VISUALIZACIÓN
DEL ECOSISTEMA DE COMPUTACIÓN CUÁNTICA EN GITHUB



UNIVERSIDAD DE CASTILLA-LA MANCHA

GRADO EN INGENIERÍA INFORMÁTICA

Trabajo Fin de Grado

Entangle: Mapeo, Análisis y Visualización del Ecosistema de Computación Cuántica en GitHub

Autor: Ángel Luis Lara Martín

Tutor: Ricardo Pérez del Castillo

Departamento: Tecnologías y Sistemas de Información

Intensificación: Sistemas de Información

Julio, 2026

Ángel Luis Lara Martín

Talavera de la Reina – España

© 2026 Ángel Luis Lara Martín

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled "GNU Free Documentation License".

Se permite la copia, distribución y/o modificación de este documento bajo los términos de la Licencia de Documentación Libre GNU, versión 1.3 o cualquier versión posterior publicada por la *Free Software Foundation*; sin secciones invariantes. Una copia de esta licencia esta incluida en el apéndice titulado «GNU Free Documentation License».

Muchos de los nombres usados por las compañías para diferenciar sus productos y servicios son reclamados como marcas registradas. Allí donde estos nombres aparezcan en este documento, y cuando la persona autora haya sido informada de esas marcas registradas, los nombres estarán escritos en mayúsculas o como nombres propios.

TRIBUNAL:

Presidencia:

Vocal:

Secretaría:

FECHA DE DEFENSA:

CALIFICACIÓN:

PRESIDENCIA

VOCAL

SECRETARÍA

Fdo.:

Fdo.:

Fdo.:

Declaración de Autoría

Yo, **Ángel Luis Lara Martín**, con DNI 03221367X, declaro que soy el único autor del trabajo fin de grado titulado «Entangle: Mapeo, Análisis y Visualización del Ecosistema de Computación Cuántica en GitHub» y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores.

Talavera de la Reina, a 13 de julio de 2026.

Firma (firma digital):



Fdo.: Ángel Luis Lara Martín

Resumen

La computación cuántica se ha consolidado como una tecnología con potencial transformador, impulsada por avances sostenidos en hardware, por el desarrollo de *frameworks* de software abiertos como Qiskit, Cirq, PennyLane o Amazon Braket, y por una inversión acumulada que supera los 42.000 millones de dólares a nivel mundial. Esta dinámica ha derivado en una proliferación acelerada de proyectos de desarrollo cuántico e híbrido cuyo reflejo más visible se encuentra en GitHub, con más de 1.500 repositorios, 27.000 colaboradores y 400 organizaciones activas. Sin embargo, el ecosistema resultante se distribuye en silos débilmente conectados, lo que dificulta responder a preguntas estratégicas: qué repositorios son relevantes, quiénes son los desarrolladores clave en cada tecnología u organización y cómo colaboran los distintos expertos en proyectos compartidos.

Este Trabajo Fin de Grado presenta **Entangle**, una plataforma de extremo a extremo que cartografía, analiza y visualiza el ecosistema de computación cuántica en GitHub. La solución integra un *pipeline* de ingesta *bottom-up* sobre la API GraphQL de GitHub, un módulo de análisis de comunidades basado en el algoritmo de *Louvain* y métricas de centralidad, un cuadro de mando interactivo con indicadores clave y filtros globales, un universo 3D que distribuye espacialmente las entidades con varias *lentes analíticas*, y un agente de IA conversacional que permite consultar los datos en lenguaje natural mediante *function calling*. Todo el sistema se despliega en Azure con infraestructura como código y entrega continua automatizada.

El proyecto aporta como resultados de ingeniería la arquitectura completa del sistema, el diseño UML de sus módulos, el código de los algoritmos más relevantes, los planes y resultados de pruebas, los indicadores de calidad obtenidos con SonarQube y la estimación de costes reales de la infraestructura en producción. La validación del sistema sobre el ecosistema cuántico real confirma la utilidad de Entangle para identificar tendencias, concentraciones organizativas, distribución disciplinar y puntos críticos de resiliencia (*bus factor*), ofreciendo una base cuantitativa para la toma de decisiones estratégicas en investigación y desarrollo.

Abstract

Quantum computing has consolidated as a technology with transformative potential, driven by sustained hardware progress, by the development of open-source frameworks such as Qiskit, Cirq, PennyLane or Amazon Braket, and by a cumulative global investment that exceeds USD 42 billion. This momentum has led to a rapid proliferation of quantum and hybrid software projects whose clearest expression is found on GitHub, with more than 1,500 repositories, 27,000 contributors and 400 active organizations. However, the resulting ecosystem is fragmented into weakly connected silos, which hinders answering strategic questions: which repositories are relevant, who are the key developers for each technology or organization, and how do different experts collaborate on shared projects.

This Bachelor's Thesis presents **Entangle**, an end-to-end platform that maps, analyses and visualises the quantum computing ecosystem on GitHub. The solution integrates a *bottom-up* ingestion pipeline over GitHub's GraphQL API, a community analysis module based on the *Louvain* algorithm and centrality metrics, an interactive dashboard with key indicators and global filters, a 3D universe that spatially distributes entities with several *analytical lenses*, and a conversational AI agent that allows the data to be queried in natural language via *function calling*. The whole system is deployed on Azure with infrastructure as code and fully automated continuous delivery.

The project delivers, as engineering outcomes, the complete system architecture, UML designs of its modules, code for the most relevant algorithms, test plans and results, quality indicators obtained with SonarQube, and real-world cost estimation of the production infrastructure. System validation on the real quantum ecosystem confirms Entangle's usefulness for identifying trends, organizational concentrations, disciplinary distribution and resilience bottlenecks (*bus factor*), providing a quantitative basis for strategic decision-making in research and development.

Agradecimientos

Escribir estas líneas supone echar la vista atrás a todos estos años de carrera. Un camino que no habría sido posible recorrer, ni mucho menos disfrutar, en solitario.

En primer lugar, quiero dar las gracias a mi tutor, Ricardo, por ser toda una inspiración y tener una cultura del esfuerzo admirable. Su exigencia hace que ir a sus clases sea verdaderamente enriquecedor, y es el claro ejemplo de que un profesor estricto no siempre es algo malo. Quiero extender este agradecimiento no solo a Ricardo, sino a todos los profesores de la UCLM por su acompañamiento y paciencia en sus clases.

A mis compañeros y amigos, con quienes he vivido todos los buenos y malos momentos en equipo desde el primer día; sin vosotros nada habría sido posible. Gracias por todas las risas en clase y las noches interminables donde el tiempo se detenía. En especial, me gustaría mencionar a Fernando, por haber sido mi compañero de equipo de principio a fin, con el que he peleado hasta el final cada examen y cada trabajo. Por muy agobiados que estuviésemos, los momentos llorando de risa siempre estuvieron ahí, sin duda marcando la diferencia.

A mis padres y a mi hermana, por haberme dado la oportunidad de cumplir este sueño. Gracias por el esfuerzo y el apoyo que han hecho posible que hoy esté aquí, asumiendo el sacrificio que supuso tener que dejar mi hogar para formarme. Este trabajo es también el resultado de esa inversión a lo largo de estos años.

A mi pareja, por haber estado siempre ahí la noche antes de cada examen tranquilizándome y ayudándome a confiar en mí mismo, por secarme cada lágrima cuando las cosas no salían como esperaba, por celebrar conmigo cada logro y sujetarme a cada paso del camino. Sin él, nada habría sido posible.

Un recuerdo especial a Noa, mi perra, que nos dejó durante estos años de carrera, por todas las horas de estudio y madrugadas en las que fue mi compañía más fiel y silenciosa; este logro también lleva una parte de su cariño incondicional.

Finalmente, agradecer al Ángel del pasado, que nunca se rindió, luchó hasta el final y no hizo caso a aquellos que intentaron apagarle, menospreciarle y hundirle diciendo que no valía. Esto es por él: sigue esforzándote, al final lo vas a conseguir, te lo aseguro. Eres el mejor ejemplo de que, con esfuerzo y creyendo en uno mismo, se puede llegar exactamente a donde uno se proponga.

Todos y cada uno de vosotros me habéis cambiado la vida y habéis hecho posible que me convierta en la persona que soy hoy, alguien de quien estar orgulloso. La vida sigue y toca avanzar, pero recordaré cada momento, cada risa, cada logro y también cada caída, siempre con cariño y una sonrisa. Gracias por todos estos años.

Ángel Luis Lara Martín

A Nacho y Raquel, a Sara, a Rafa, a Noa y al Ángel que nunca se rindió.

Índice general

Resumen	VII
Abstract	IX
Agradecimientos	XI
Índice general	XV
Índice de tablas	XXV
Índice de figuras	XXVII
Índice de listados	XXXI
Listado de acrónimos	XXXIII
1. Introducción	1
2. Objetivos	3
2.1. Objetivo Principal y Subobjetivos	3
3. Antecedentes	5
3.1. Conceptos Fundamentales sobre Computación Cuántica	5
3.2. Desarrollo de Software Cuántico	6
3.2.1. Principales Frameworks	7
3.3. Minería de Repositorios de Software	8
3.3.1. GitHub como Fuente de Datos	8
3.3.2. API GraphQL y REST	8
3.3.3. Técnicas de Ingesta Incremental y Rate Limits	9

0. ÍNDICE GENERAL

3.4.	Análisis de Redes de Colaboración	10
3.4.1.	Grafos de Colaboración	10
3.4.2.	Métricas de Centralidad	10
3.4.3.	Detección de Comunidades con Louvain	11
3.4.4.	Caminos Mínimos: Algoritmo de Dijkstra	12
3.4.5.	Bridge Users	13
3.5.	Visualización de Datos	13
3.5.1.	Principios de Diseño de Dashboards	13
3.5.2.	Visualización 3D de Grafos	14
3.5.3.	Técnicas de Distribución Espacial	14
3.6.	Agentes de IA Conversacional	15
3.6.1.	Grandes Modelos de Lenguaje	15
3.6.2.	IA Agéntica y Sistemas Multi-Agente	15
3.6.3.	Arquitecturas Router-Worker	16
3.6.4.	Function Calling y Acceso a Datos Externos	16
3.7.	Trabajos Relacionados	16
3.7.1.	GHTorrent	17
3.7.2.	GH Archive	17
3.7.3.	Libraries.io	17
3.7.4.	Open Source Insights	17
3.7.5.	Estudios empíricos en MSR	17
3.7.6.	Minería de Stack Overflow sobre computación cuántica	18
3.7.7.	Posicionamiento de Entangle	18
4.	Metodología	19
4.1.	Scrumban como Metodología de Gestión de Proyectos	19
4.2.	Vibe Coding como Metodología de Desarrollo de Software	19
4.3.	Aplicación de las Metodologías al Proyecto Entangle	20
4.3.1.	Roles y Aplicación de Scrumban y Vibe Coding	20
4.3.2.	Planificación por Fases y Sprints	21
4.3.3.	Iteraciones y Sprints	22

4.4. Stack Tecnológico del Proyecto	23
5. Resultados	25
5.1. Arquitectura General del Sistema Entangle	25
5.1.1. Visión Global y Flujo de Datos	25
5.1.2. Comunicación entre Capas	28
5.2. Diseño del Sistema	28
5.2.1. Diagrama de Componentes	28
5.2.2. Diagrama de Clases del Backend	29
5.2.3. Diagrama de Secuencia: Flujo de Consulta IA	34
5.3. Pipeline de Datos de Entangle	35
5.3.1. Fases del Pipeline	35
5.4. Modelo de Datos en MongoDB	36
5.4.1. Colecciones Principales	37
5.4.2. Colección de Métricas	37
5.4.3. Relaciones entre Colecciones	38
5.5. Backend API REST con FastAPI	38
5.5.1. Visión General de la API	39
5.5.2. Endpoints de la API	39
5.5.3. Filtros, Compresión y Caché	40
5.6. Agente de IA Conversacional	40
5.6.1. Arquitectura Orientada a Configuración	40
5.6.2. Workers Especializados y Capacidades	41
5.6.3. Canalización de RAG y Búsqueda Semántica	42
5.6.4. Streaming de Razonamiento y Seguridad	43
5.7. Frontend: Dashboard 2D Interactivo en React	43
5.7.1. Tarjetas de Indicadores Clave de Rendimiento	43
5.7.2. Charts Section	43
5.7.3. Network Graph	44
5.7.4. Otros Componentes del Cuadro de Mando	45
5.8. Universo 3D: Visualización Tridimensional Inmersiva del Ecosistema Cuántico	46

0. ÍNDICE GENERAL

5.8.1. Tecnología Base	46
5.8.2. Metáfora Cuántica: Entidades como Objetos Físicos	47
5.8.3. Vínculos y Canales	47
5.8.4. Algoritmo de Distribución	47
5.8.5. Zonas de Densidad	48
5.8.6. Capa Analítica y Narrativa	48
5.9. Métricas e Indicadores Clave del Proyecto Entangle	49
5.9.1. Métricas de Usuarios	49
5.9.2. Métricas de Organizaciones	49
5.9.3. Análisis de Sensibilidad de los Pesos	50
5.9.4. Métricas de Repositorios	50
5.9.5. Métricas de Red de Colaboración	50
5.10. Infraestructura y Despliegue en Azure	51
5.10.1. Servicios Azure	51
5.10.2. Docker, Bicep y Entorno Local	52
5.10.3. Seguridad	52
5.11. Gestión de la Configuración del Software	52
5.11.1. Control de Versiones	53
5.11.2. Organización del Repositorio	53
5.12. Análisis de Datos del Ecosistema Cuántico	54
5.12.1. Volumen y Composición del Ecosistema	55
5.12.2. Lenguajes Predominantes	55
5.12.3. Organizaciones y Disciplinas	56
5.12.4. Análisis de Comunidades	57
5.13. Análisis de Comunidades mediante Universo 3D	59
5.13.1. Distribución Espacial	59
5.13.2. Insights por Lente Analítica	60
5.13.3. Tour Cinemático	61
5.14. Evaluación del Agente de IA Conversacional	61
5.14.1. Clasificación y Procesamiento de Consultas	61

5.14.2. Consultas sobre Datos	62
5.14.3. Conocimiento, Análisis Cruzado, Memoria e Investigación	62
5.14.4. Evaluación Cuantitativa y Limitaciones	63
5.15. Rendimiento y Escalabilidad	64
5.15.1. Rendimiento del Pipeline de Ingesta	64
5.15.2. Latencia de la API y Sistema de Caché	65
5.15.3. Transmisión de Datos y Renderizado 3D	66
5.15.4. Conclusión sobre el Rendimiento	66
5.16. Pruebas	66
5.16.1. Suite de Pruebas del Backend	66
5.16.2. Suite de Pruebas del Frontend	67
5.17. Calidad del Código basada en SonarQube	67
5.17.1. Visión Agregada del Proyecto	68
5.17.2. Conclusión sobre la Calidad del Código	70
5.18. Estimación de Costes del Proyecto	70
5.18.1. Costes de Personal y Servicios de IA	70
5.18.2. Costes de Puesta en Producción	71
5.18.3. Coste Total del Proyecto	72
6. Conclusiones y trabajo futuro	73
6.1. Revisión de Objetivos	74
6.2. Limitaciones del Estudio	76
6.3. Trabajo Futuro	77
6.4. Competencias del Grado Aplicadas	78
6.5. Desafíos Encontrados durante el Desarrollo	79
6.6. Valoración Personal	80
Referencias	81
A. Manual de Instalación y Despliegue	93
A.1. Prerrequisitos	93
A.2. Ejecución Local	93

0. ÍNDICE GENERAL

A.2.1. Backend	93
A.2.2. Frontend	94
A.3. Despliegue con Docker	94
A.4. Despliegue a Azure	94
A.4.1. Backend: Azure Container Apps	95
A.4.2. Frontend: Azure Static Web Apps	95
B. Palabras Clave del Pipeline de Ingesta	97
B.1. Palabras clave de búsqueda	97
B.2. Palabras clave de filtrado	97
C. Estructura del Repositorio	99
C.1. Entangle-Core (backend)	99
C.2. Entangle-Visualizer (frontend)	101
D. Catálogo de Endpoints de la API	103
D.1. Router Principal (43 endpoints)	103
D.2. Router de Administración (11 endpoints)	104
D.3. Router de Chat IA (5 endpoints)	105
E. Variables de Entorno y Dependencias	107
E.1. Variables de Entorno del Backend	107
E.2. Variables de Entorno del Frontend	108
E.3. Dependencias Principales	108
E.3.1. Backend	108
E.3.2. Frontend	109
E.4. Colecciones de la Base de Datos	109
F. Modelo de Datos Detallado	111
F.1. Colección de Repositorios	111
F.2. Colección de Usuarios	112
F.3. Colección de Organizaciones	113

G. Análisis de Sensibilidad de Métricas	115
G.1. Sensibilidad del Quantum Expertise Score	115
G.2. Sensibilidad del Quantum Focus Score	115
G.3. Conclusiones del Análisis	116
G.4. Ejemplos Numéricos de Cálculo	116
H. Configuración de Infraestructura	117
H.1. Dockerfile del Backend	117
H.2. Infraestructura como Código con Bicep	117
H.3. Entorno de Desarrollo Local	118
H.4. Catálogo Detallado de Servicios Azure	119
H.5. Detalle de la Capa de Seguridad	119
I. Vocabulario Físico: Definiciones y Correspondencia Visual	121
I.1. Mecánica Cuántica	121
I.2. Astrofísica y Física Atómica	122
J. Galería del Universo 3D	123
J.1. Correspondencia entre Conceptos Físicos y Metáforas Visuales	123
J.2. Distribución Espacial	124
J.3. Modos de Interacción y Detalle de Entidades	125
J.3.1. Modo Focus sobre una Organización	125
J.3.2. Cluster de Bridge Users	126
J.3.3. Detalle de un Cúbit (Repositorio)	126
J.4. Galería de Lentes Analíticas	127
J.4.1. Lente de Comunidades	127
J.4.2. Lente de Centralidad	127
J.4.3. Lente de Resiliencia	128
J.4.4. Lente de Intensidad	128
J.4.5. Lente de Disciplinas	129
J.4.6. Túnel Cuántico	129
J.5. Animaciones Cinemáticas de Entrada y Salida, y Tour Cósmico	130

0. ÍNDICE GENERAL

J.5.1.	Quantum Genesis (Big Bang): Animación de Entrada	130
J.5.2.	Black Hole Collapse: Animación de Salida	131
J.5.3.	Tour Cósmico: Guion Narrativo Opcional	132
J.6.	Efectos Visuales y Fenómenos Cuánticos: Parámetros Detallados	135
K.	Galería del Agente de IA Conversacional	137
K.1.	Catálogo de Workers	137
K.2.	Consultas al Agente DATA	138
K.3.	Razonamiento de los Agentes y Acciones Automáticas	139
K.4.	Detalle de la Canalización de RAG	141
K.5.	Conocimiento Cualitativo: Worker KNOWLEDGE	141
K.6.	Análisis Cruzado: Worker INSIGHTS	141
K.7.	Investigación Externa: Worker DEEP_RESEARCH	142
K.8.	Memoria Conversacional	142
L.	Galería del Dashboard 2D Interactivo	145
L.1.	Vista General de la Página Principal y Tarjetas KPI	145
L.2.	Panel de <i>Bridge Profiles</i>	146
L.3.	Grafo de Colaboración	146
L.4.	Cabecera y Pie de Página	147
M.	Detalle de los resultados de rendimiento	149
M.1.	Detalle del Renderizado del Universo 3D	149
M.2.	Detalle de Base de Datos y Consultas	150
M.3.	Detalle de la Caché <i>Chunked</i> en MongoDB	151
N.	Detalle del análisis de calidad de código (SonarQube)	153
N.1.	Configuración de SonarQube	153
N.2.	Detalle del Backend	153
N.3.	Detalle del Frontend	155
N.4.	<i>Dashboards</i> de SonarQube por Repositorio	156
N.5.	Integración Continua con SonarCloud	157

N.5.1. Workflow del Backend	157
N.5.2. Workflow del Frontend	158
N.5.3. Configuración del Proyecto	159
N.5.4. Quality Gate <i>Sonar Way</i> y resultados	159
N.5.5. Dashboards y Resultados Visuales en SonarCloud	159
N.5.6. Análisis local vs. CI continuo: complementariedad	161
Ñ. Fragmentos de código adicionales	163
Ñ.1. Ejecución Segura de Agregaciones MongoDB	163
Ñ.2. Clasificación por Jenks Natural Breaks	164
Ñ.3. Camino Mínimo entre Entidades (Quantum Tunnel)	165
Ñ.4. Cálculo del Bus Factor	166
Ñ.5. Gestión del Rate Limit de GitHub	167
O. Detalle de la suite de pruebas	169
O.1. Estrategia de Aislamiento del Backend	169
O.2. Pruebas de Resiliencia frente a <i>Throttling</i>	169
O.3. Pruebas de la API REST	170
O.4. Idempotencia y Consistencia del Pipeline	170
P. Declaración de Uso Responsable de la Inteligencia Artificial	171
P.1. Ámbito del uso de la inteligencia artificial	171
P.2. Compromisos asumidos	171
P.3. Firma	172

Índice de tablas

3.1. Comparación de capacidades entre trabajos relacionados y Entangle.	18
4.1. Fases de desarrollo del proyecto Entangle e hitos principales de cada iteración.	22
4.2. Sprints del proyecto Entangle e incrementos verificables al cierre de cada iteración.	23
5.1. Resumen de colecciones MongoDB del sistema Entangle.	37
5.2. Endpoints principales de la API REST de Entangle (muestra representativa).	39
5.3. Cifras clave del ecosistema cuántico tras la última ejecución del <i>pipeline</i> . .	55
5.4. Rendimiento del <i>pipeline</i> de ingesta antes y después de la migración a Cosmos DB vCore M30.	64
5.5. Tiempos de respuesta de los <i>endpoints</i> principales con y sin caché activa. .	65
5.6. Distribución de los 919 tests del backend por área funcional.	67
5.7. Distribución de los 168 tests del frontend por fichero.	68
5.8. Métricas agregadas de SonarQube para el proyecto Entangle.	68
5.9. Estimación de costes de personal del proyecto Entangle.	70
5.10. Desglose de costes mensuales de Azure para Entangle (mayo 2026).	71
5.11. Coste total estimado del proyecto Entangle.	72
6.1. Evaluación del cumplimiento de los objetivos del proyecto Entangle.	75
B.1. Lista completa de palabras clave de filtrado.	98
D.1. Endpoints del router principal.	103
D.2. Endpoints del router de administración.	104
D.3. Endpoints del router de chat.	105
E.1. Variables de entorno del backend.	107

0. ÍNDICE DE TABLAS

E.2. Variables de entorno de los scripts de ingesta.	108
E.3. Variables de entorno del frontend.	108
E.4. Dependencias principales del backend.	109
E.5. Dependencias principales del frontend.	109
E.6. Colecciones de la base de datos.	109
J.1. Correspondencia entre conceptos físicos y metáforas visuales de Entangle. .	123
K.1. Workers especializados del agente de IA conversacional.	137
M.1. <i>Draw calls</i> estimados sin y con InstancedMesh (IM).	149
N.1. Condiciones de la <i>Quality Gate</i> «Entangle».	153
N.2. Métricas de SonarQube para el backend (entangle-backend).	154
N.3. Incidencias corregidas en el backend tras el primer análisis de SonarQube. .	155
N.4. Métricas de SonarQube para el frontend (entangle-frontend).	156
N.5. Condiciones de la <i>Quality Gate Sonar Way</i> y resultados sobre código nuevo en SonarCloud.	159

Índice de figuras

3.1. Algoritmo de teletransporte cuántico: circuito y código Qiskit	7
3.2. REST vs GraphQL: misma consulta sobre la API de GitHub	9
5.1. Arquitectura general del sistema Entangle.	27
5.2. Diagrama de componentes UML del sistema Entangle.	29
5.3. Diagrama de clases UML del backend (parte 1 de 4)	30
5.4. Diagrama de clases UML del backend (parte 2 de 4)	31
5.5. Diagrama de clases UML del backend (parte 3 de 4)	32
5.6. Diagrama de clases UML del backend (parte 4 de 4)	33
5.7. Diagrama de secuencia UML del flujo de consulta al agente de IA (tipo DATA).	34
5.8. Modelo conceptual de relaciones entre las colecciones principales de MongoDB.	38
5.9. <i>Donut</i> de Distribución por Disciplina y panel de <i>Puentes Interdisciplinares</i> del cuadro de mando.	56
5.10. Gráficos analíticos del cuadro de mando interactivo.	57
5.11. Mini-universo de colaboración embebido en el cuadro de mando.	58
5.12. Vista general del universo 3D de Entangle.	59
5.13. Lente de comunidades del universo 3D.	60
5.14. <i>Waypoint</i> intermedio del tour cinemático.	61
5.15. Respuesta del <i>worker</i> KNOWLEDGE: comparación de Qiskit y Cirq vía Retrieval- Augmented Generation (RAG).	62
5.16. Respuesta del <i>worker</i> INSIGHTS: comparativa de repositorios con <i>compare_repos</i>	63
5.17. Tiempos del <i>pipeline</i> de ingesta: Cosmos DB RU frente a vCore M30.	65
5.18. Estado consolidado de la <i>Quality Gate</i> «Entangle» en SonarQube.	69
5.19. Desglose del coste mensual de Azure por servicio (mayo 2026).	72
J.1. Fronteras zonales del universo 3D: núcleo, zona intermedia y periferia.	124

O. ÍNDICE DE FIGURAS

J.2.	Vista panorámica inmersiva del universo 3D desde el interior.	125
J.3.	Modo <i>focus</i> sobre una organización.	125
J.4.	Cluster de <i>bridge users</i> (puntos dorados) entre dos organizaciones.	126
J.5.	Detalle micro de un cúbit (repositorio) con <i>Quantum Bonds</i>	126
J.6.	Lente de comunidades: colores asignados por ángulo áureo.	127
J.7.	Lente de centralidad: gradiente por <i>betweenness centrality</i>	127
J.8.	Lente de resiliencia: niveles de <i>bus factor</i> codificados por color.	128
J.9.	Lente de intensidad: gradiente por <i>degree centrality</i>	128
J.10.	Lente de disciplinas: seis categorías temáticas codificadas por color.	129
J.11.	Túnel Cuántico: camino más corto animado entre dos entidades.	129
J.12.	<i>Quantum Genesis (Big Bang)</i> : fase de ignición de la animación de entrada. .	131
J.13.	<i>Black Hole Collapse</i> : animación de salida durante la fase de <i>Event Horizon</i> . .	132
J.14.	<i>Waypoint Genesis</i> : inicio del recorrido del tour.	134
J.15.	<i>Waypoint Entrelazamiento</i> : canales inter-organizacionales con pulsos de <i>tunneling</i>	134
J.16.	<i>Waypoint Panorámica Final</i>	135
K.1.	Consulta al agente DATA con datos reales de la base de datos.	138
K.2.	Consulta con agregación al agente DATA.	139
K.3.	Respuesta contextual del agente DASHBOARD.	139
K.4.	Panel de razonamiento en vivo del agente DATA (<i>Analista</i>).	140
K.5.	Acción automática CREATE_VIEW ejecutada por el agente DASHBOARD.	140
K.6.	Consulta al <i>worker</i> DEEP_RESEARCH: recuperación de publicaciones reales de arXiv con título, autores y enlace.	142
K.7.	Memoria conversacional: respuesta del agente a una pregunta anafórica («¿y de esos...?») que solo es resoluble recuperando el contexto del turno anterior desde la sesión persistida.	143
L.1.	Vista panorámica de la página principal de Entangle.	145
L.2.	Tarjeta KPI tras la interacción del usuario: colapso de la función de onda. .	146
L.3.	Detalle del donut de distribución por disciplina y panel de puentes interdisciplinarios.	146

L.4. Grafo de colaboración con <i>layout</i> circular jerárquico y aristas diferenciadas por tipo.	147
L.5. Cabecera del cuadro de mando.	147
L.6. Pie de página FooterExtra del cuadro de mando.	148
N.1. Dashboard de SonarQube del proyecto entangle-backend.	156
N.2. Dashboard de SonarQube del proyecto entangle-frontend.	157
N.3. Vista consolidada de la organización angel-tfg-uclm en SonarCloud.	160
N.4. Dashboard de SonarCloud del proyecto Angel-TFG-UCLM-Entangle-Core (backend).	160
N.5. Dashboard de SonarCloud del proyecto Angel-TFG-UCLM-Entangle-Visualizer (frontend).	161
N.6. Comentario automático del bot SonarCloud en un <i>pull request</i>	161

Índice de listados

5.1. Router de clasificación de intención con gpt-5-mini.	41
5.2. Detección de comunidades mediante Louvain.	51
5.3. Árbol de directorios de los repositorios Entangle-Core y Entangle-Visualizer.	54
F.1. Ejemplo de documento en la colección repositories	111
F.2. Ejemplo de documento en la colección users	112
F.3. Ejemplo de documento en la colección organizations	113
H.1. Fragmento del Dockerfile para el backend	117
H.2. Fragmento del archivo Bicep para la infraestructura	118
N.1. Extracto de .github/workflows/sonarcloud.yml (Backend).	158
N.2. Extracto de .github/workflows/sonarcloud.yml (Frontend).	158
Ñ.1. Ejecución segura de pipelines de agregación.	163
Ñ.2. Algoritmo de Jenks Natural Breaks.	164
Ñ.3. Camino mínimo con filtrado de bots (Dijkstra).	165
Ñ.4. Cálculo del bus factor por repositorio.	166
Ñ.5. Decorador con reintentos ante rate limit.	167

Listado de acrónimos

ACR	Azure Container Registry
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BSON	Binary JSON
CI/CD	Continuous Integration / Continuous Deployment
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSP	Content Security Policy
DOM	Document Object Model
GLSL	OpenGL Shading Language
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
GZip	GNU Zip
HTTP	HyperText Transfer Protocol
IA	Inteligencia Artificial
IAC	Infrastructure as Code
IVF	Inverted File Index
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LLM	Large Language Model

0. LISTADO DE ACRÓNIMOS

MSR	Mining Software Repositories
NISQ	Noisy Intermediate-Scale Quantum
OIDC	OpenID Connect
OWASP	Open Worldwide Application Security Project
PRNG	Pseudorandom Number Generator
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RGPD	Reglamento General de Protección de Datos
SCA	Software Composition Analysis
SDK	Software Development Kit
SI	Sistemas de Información
SPA	Single Page Application
SSAO	Screen Space Ambient Occlusion
SSE	Server-Sent Events
SVG	Scalable Vector Graphics
SWA	Static Web Apps
TFG	Trabajo Fin de Grado
TLS	Transport Layer Security
TTL	Time To Live
UCLM	Universidad de Castilla-La Mancha
UI	User Interface
UUID	Universally Unique Identifier
VQE	Variational Quantum Eigensolver
XSS	Cross-Site Scripting

Capítulo 1

Introducción

La computación cuántica aprovecha principios de la mecánica cuántica (superposición, entrelazamiento) para resolver problemas intratables para los computadores clásicos [NC10]. Desde que Shor [Sho94] y Grover [Gro96] propusieron los primeros algoritmos con ventaja cuántica demostrable, el campo ha experimentado un crecimiento acelerado. Empresas como IBM, Google, Amazon y Xanadu han desplegado procesadores con centenares de cúbits, han abierto plataformas de acceso en la nube y han impulsado frameworks de software de código abierto que permiten programar y simular circuitos cuánticos [Pre18, IBM24, Goo24b, Xan24, Ama24]. Según McKinsey, la inversión acumulada en tecnologías cuánticas superó los 42.000 millones de dólares en 2024, con una financiación anual récord por parte tanto de fondos públicos como de capital privado [McK24]. IDC estima que el gasto mundial en computación cuántica alcanzará los 7.600 millones de dólares en 2027, con una tasa de crecimiento anual compuesto del 48 % [Int23].

Toda esta dinámica ha hecho proliferar numerosos proyectos de soluciones cuánticas e híbridas, cuyo reflejo más visible es la plataforma de control de versiones Git [CS14]. Solo en GitHub se identifican más de 1.500 repositorios de computación cuántica, con cerca de 27.000 colaboradores y más de 400 organizaciones. Sin embargo, este crecimiento ha generado una fragmentación significativa: los repositorios, desarrolladores y organizaciones se distribuyen en silos con escasa colaboración cruzada. Más allá de conocer el ecosistema, el reto consiste en explorar las conexiones no evidentes y descubrir patrones de colaboración entre proyectos, equipos y disciplinas que mejoren el conocimiento global de la comunidad de software cuántico. Estas conexiones latentes determinan qué repositorios son realmente influyentes, quiénes actúan como puentes entre comunidades distantes y cómo se propaga el conocimiento entre organizaciones aparentemente independientes. Estudios recientes confirman esta necesidad: Tavassoli Sabzevari y Khan [TSK26] analizan 1.404 publicaciones de Stack Overflow sobre computación cuántica y concluyen que los retos prácticos de adopción de herramientas dominan las discusiones de los desarrolladores. Estas preguntas son de interés estratégico para empresas, grupos de investigación y organismos públicos que necesitan comprender el ecosistema para tomar decisiones informadas sobre inversión, contratación o alianzas tecnológicas.

1. INTRODUCCIÓN

Este Trabajo Fin de Grado presenta **Entangle**, una plataforma que aborda esta problemática mediante la integración, el análisis y la visualización del ecosistema de computación cuántica en GitHub. El nombre deriva del fenómeno del entrelazamiento cuántico (*quantum entanglement*), una propiedad donde partículas separadas espacialmente mantienen correlaciones instantáneas [NC10]. La metáfora resulta adecuada: al igual que el entrelazamiento conecta partículas distantes, Entangle descubre y mapea las conexiones ocultas entre los actores del ecosistema cuántico.

La plataforma opera como un sistema de extremo a extremo organizado en cinco capas:

1. Un *pipeline* de ingesta que extrae datos de la Application Programming Interface (API) GraphQL de GitHub [Git24b] y los almacena en una base de datos documental [Mon24a].
2. Un módulo de análisis de redes que construye un grafo de colaboración con NetworkX [HSS24] y aplica análisis de comunidades y métricas de centralidad [BGLL08, Fre77].
3. Un cuadro de mando interactivo con indicadores clave y grafos de colaboración [Met24, Rec24].
4. Un universo 3D basado en Three.js [Cab24] y React Three Fiber [pmn24a] que distribuye espacialmente las entidades del ecosistema.
5. Un agente de Inteligencia Artificial (IA) conversacional basado en Large Language Model (LLM) [BMR⁺20] con *function calling* [Ope23a] y arquitectura agéntica [WMF⁺24] que permite consultar los datos en lenguaje natural.

El sistema se despliega en Azure [Mic24d] mediante contenedores Docker [Doc24], infraestructura como código con Bicep [Mic24f] y un flujo de integración y despliegue continuos con GitHub Actions [Git24a].

Las implicaciones más importantes del proyecto son la disponibilidad, por primera vez en formato unificado, de una visión cuantitativa del ecosistema cuántico en GitHub que permite identificar concentraciones organizativas y disciplinares, evidenciar puntos críticos de resiliencia mediante el *bus factor* y descubrir comunidades y puentes entre actores hasta ahora invisibles. Más allá de los resultados puntuales, Entangle aporta una metodología y herramientas reutilizables (pipeline de ingesta, modelo de datos, lentes analíticas y agente conversacional) adaptables a otros dominios emergentes en los que la rápida evolución del ecosistema dificulta los estudios estratégicos manuales.

El resto del documento se organiza como sigue: el Capítulo 2 formaliza los objetivos; el Capítulo 3, los antecedentes teóricos y tecnológicos; el Capítulo 4, la metodología de desarrollo; el Capítulo 5, los resultados de ingeniería (diseño, implementación y validación con datos reales), y el Capítulo 6, las conclusiones y el trabajo futuro. Los anexos complementan el documento con material de soporte.

Capítulo 2

Objetivos

Este capítulo formaliza los objetivos que guían el desarrollo de Entangle. A continuación se enuncian, en primer lugar, el *objetivo principal* (OP) que articula la finalidad global del proyecto y, después, los seis *subobjetivos* (SO1–SO6) que descomponen el problema en metas específicas y verificables. Cada subobjetivo incluye criterios de aceptación que permitirán evaluar su cumplimiento en el Capítulo 6.

2.1 Objetivo Principal y Subobjetivos

El objetivo principal (OP) de este TFG es el siguiente:

OP: Diseñar e implementar una plataforma de extremo a extremo que permita cartografiar, analizar y visualizar el ecosistema de computación cuántica de código abierto en GitHub.

Este objetivo principal trata de facilitar la comprensión de las dinámicas de colaboración del ecosistema cuántico a investigadores y desarrolladores, integrando ingesta automatizada, análisis de redes, visualización 2D y 3D, y un agente conversacional sobre los datos del ecosistema.

La consecución del objetivo principal del proyecto se pretende realizar mediante la satisfacción de la siguiente lista de subobjetivos:

SO1: Cartografiar e ingestar el ecosistema cuántico en GitHub. Construir un proceso automatizado que descubra y almacene repositorios, colaboradores y organizaciones del ecosistema cuántico a partir de la plataforma GitHub, obteniendo una representación estructurada del ecosistema que sirva de base a todos los análisis posteriores.

- *Criterio de aceptación:* el sistema identifica y almacena al menos 1.000 repositorios de computación cuántica junto con sus colaboradores y organizaciones asociadas.

SO2: Analizar estadísticamente las relaciones de colaboración. Caracterizar de forma cuantitativa la estructura del ecosistema a partir del grafo de colaboración, identificando actores relevantes, comunidades y puntos críticos de resiliencia.

2. OBJETIVOS

- *Criterio de aceptación:* el sistema ofrece métricas de centralidad, una descomposición del ecosistema en comunidades y un análisis de resiliencia basado en el *bus factor*.

SO3: Analizar y visualizar los resultados estadísticos de colaboración. Proporcionar una interfaz 2D interactiva que combine el análisis estadístico con su presentación gráfica, integrando indicadores clave, gráficos analíticos y vistas de grafo con filtros globales para la exploración tabular y gráfica de los datos del ecosistema.

- *Criterio de aceptación:* el cuadro de mando integra KPIs, series temporales, distribuciones y un grafo de colaboración filtrable por tipo de entidad, disciplina y rango temporal.

SO4: Analizar y visualizar el ecosistema mediante un universo 3D. Diseñar una representación espacial tridimensional del ecosistema que distribuya las entidades en el espacio y ofrezca varias lentes analíticas para inspeccionar métricas en contexto, complementando al cuadro de mando estadístico.

- *Criterio de aceptación:* el universo 3D distribuye espacialmente repositorios, usuarios y organizaciones, y ofrece al menos tres lentes analíticas junto con un recorrido guiado.

SO5: Diseñar e implementar un agente de IA conversacional. Incorporar un agente conversacional que permita consultar el ecosistema en lenguaje natural y ejecutar acciones sobre la propia interfaz, acercando la plataforma a usuarios no técnicos.

- *Criterio de aceptación:* el agente interpreta preguntas del usuario, genera consultas válidas sobre los datos del ecosistema y responde de forma incremental sobre la interfaz de usuario.

SO6: Identificar tendencias y patrones del ecosistema cuántico. Extraer conclusiones cuantitativas que ayuden a comprender la evolución y la estructura del ecosistema: lenguajes predominantes, concentración en organizaciones, distribución disciplinar y resiliencia global.

- *Criterio de aceptación:* se presentan resultados sobre la distribución de lenguajes predominantes, las organizaciones clave, la distribución disciplinar de los colaboradores y el análisis de *bus factor* del ecosistema.

Capítulo 3

Antecedentes

Este capítulo revisa los fundamentos teóricos y tecnológicos sobre los que se construye Entangle. Se parte de la computación cuántica como disciplina (Sección 3.1) y del ecosistema de desarrollo de software cuántico (Sección 3.2). A continuación, se introduce la minería de repositorios de software como metodología de obtención de datos (Sección 3.3), y se explican las técnicas de análisis de redes (Sección 3.4) y visualización (Sección 3.5) que permiten interpretar la información extraída. Se describen los agentes de IA conversacional basados en LLM (Sección 3.6). El capítulo cierra con una revisión de trabajos relacionados y el posicionamiento de Entangle frente a ellos (Sección 3.7).

3.1 Conceptos Fundamentales sobre Computación Cuántica

La computación cuántica se sustenta sobre dos principios contraintuitivos de la mecánica cuántica que carecen de análogo clásico: la **superposición** y el **entrelazamiento** [NC10]. La superposición permite que un sistema cuántico se encuentre simultáneamente en una combinación lineal de estados, mientras que el entrelazamiento establece correlaciones entre partículas de tal forma que el estado de una determina instantáneamente el de la otra, independientemente de la distancia que las separe [Sch05].

La unidad básica de computación es el **cúbit** (del inglés *qubit*), que utiliza los principios descritos anteriormente. A diferencia de un bit clásico, que adopta el valor 0 o 1, un cúbit puede encontrarse en un estado de superposición y entrelazarse con otros, lo que confiere a los algoritmos cuánticos una ventaja computacional para ciertos problemas [Sho94, Gro96].

Las **puertas cuánticas** son operaciones unitarias que manipulan cúbits, análogas a las puertas lógicas clásicas. Sin embargo, la **decoherencia** (pérdida de coherencia cuántica por interacción con el entorno a lo largo del tiempo) constituye un desafío crítico, ya que los cúbits son extremadamente sensibles a perturbaciones externas [Zur03].

Vocabulario físico adoptado como lenguaje visual

Uno de los rasgos distintivos de Entangle es la traducción de fenómenos de la física cuántica, la astrofísica y la física atómica en metáforas visuales interactivas dentro del

3. ANTECEDENTES

universo 3D y el dashboard. Los conceptos adoptados abarcan dos dominios: *mecánica cuántica* (notación de Dirac, esfera de Bloch, colapso de la función de onda, principio de incertidumbre de Heisenberg, nube de probabilidad, efecto túnel cuántico, espuma cuántica y vacío cuántico) [NC10, GS18, Whe57] y *astrofísica y física atómica* (esfera de Dyson [Dys60], rayos cósmicos [Lon11] y modelo atómico de Bohr [Boh13]). Cada concepto se mapea a un elemento visual concreto de la plataforma; la correspondencia completa se detalla en la Tabla J.1 del Capítulo 5, y las definiciones formales con sus fórmulas y su implementación en Entangle se recogen en el Anexo I.

En el contexto de la computación cuántica actual, se habla de la era Noisy Intermediate-Scale Quantum (NISQ) [Pre18], caracterizada por dispositivos con un número limitado de cúbits que aún no son completamente fiables. La corrección de errores cuánticos es fundamental para mitigar los efectos de la decoherencia y mejorar la precisión de los cálculos. Algoritmos como el Variational Quantum Eigensolver (VQE) son ejemplos de técnicas que aprovechan las capacidades actuales de los dispositivos NISQ para resolver problemas de química cuántica y optimización [Pre18].

3.2 Desarrollo de Software Cuántico

A diferencia del software clásico, el software cuántico es **intrínsecamente no determinista**: las operaciones de medida que cierran cualquier algoritmo son probabilísticas, por lo que una misma ejecución del mismo circuito puede producir resultados distintos, y solo tras miles de repeticiones (*shots*) se obtiene una distribución estadística estable [NC10]. Esta naturaleza estadística obliga a estructurar los algoritmos como **circuitos cuánticos**: secuencias temporales de puertas aplicadas sobre los cúbits seguidas de mediciones finales que colapsan el estado cuántico a un resultado clásico. La Figura 3.1 ilustra esta correspondencia con el protocolo canónico de teletransporte cuántico: el circuito (a) parte del estado $|\psi\rangle$ en el cúbit q_0 de Alice, establece un par de Bell entre q_1 y q_2 , aplica dos operaciones locales de Alice (CNOT y H) seguidas de dos mediciones, y termina con las correcciones condicionales de Bob (X^{c_1} y Z^{c_0}) que reconstruyen $|\psi\rangle$ en q_2 ; el código (b) es la implementación literal del mismo circuito en Qiskit. Este avance ha dado lugar a un ecosistema creciente de *frameworks* de software de código abierto que materializan dicho modelo (Sección 3.2.1) y cierra con su reflejo en GitHub, contexto directo del análisis realizado por Entangle.

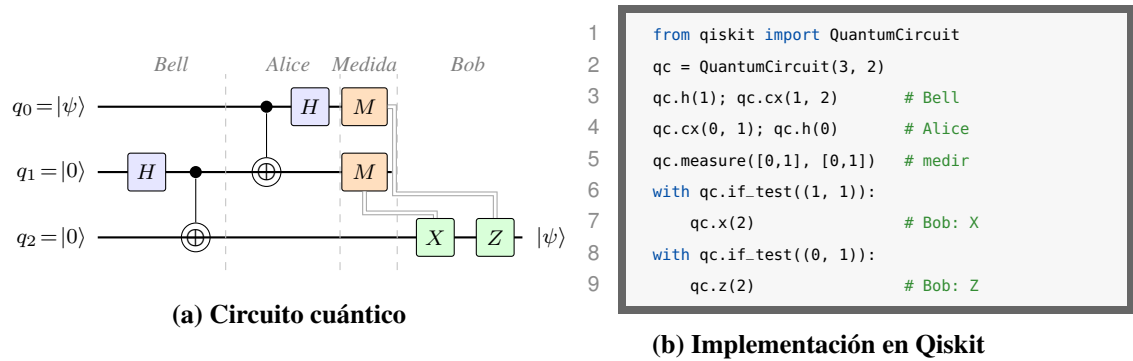


Figura 3.1: Algoritmo de teletransporte cuántico: (a) circuito abstracto, (b) implementación en Qiskit.

3.2.1 Principales Frameworks

El ecosistema de computación cuántica está impulsado por varios frameworks desarrollados por empresas líderes. Qiskit, desarrollado por IBM en 2017, es un framework de código abierto que utiliza Python como lenguaje base. Se centra en la programación de circuitos cuánticos y ofrece herramientas para simular y ejecutar algoritmos en hardware cuántico real [IBM24]¹. Su integración con IBM Quantum Experience lo distingue al permitir el acceso a procesadores cuánticos a través de la nube; el fragmento (b) de la Figura 3.1 es un ejemplo representativo de su API declarativa.

Cirq, lanzado por Google en 2018, también utiliza Python y está diseñado para optimizar circuitos cuánticos en dispositivos NISQ. Cirq es conocido por su enfoque en la investigación y experimentación con algoritmos cuánticos, facilitando la implementación de experimentos personalizados [Goo24b]². PennyLane, desarrollado por Xanadu, se centra en la computación cuántica diferenciable, lo que permite integrar algoritmos cuánticos con técnicas de aprendizaje automático [Xan24]³.

Amazon Braket, lanzado por Amazon Web Services, ofrece una plataforma unificada para desarrollar, probar y ejecutar algoritmos cuánticos. Braket soporta múltiples frameworks cuánticos, proporcionando flexibilidad y acceso a diferentes tipos de hardware cuántico [Ama24]⁴. OpenQASM, por su parte, es un lenguaje de ensamblaje cuántico que permite describir circuitos cuánticos de manera detallada, siendo ampliamente utilizado en simuladores y hardware cuántico [CBSG17].

Reflejo en GitHub. El conjunto de estos *frameworks* y de las iniciativas asociadas se manifiesta en GitHub como un ecosistema de aproximadamente 1.500 repositorios cuánticos, cerca de 27.000 usuarios activos y más de 400 organizaciones. Su mapeo es explorable a partir

¹<https://github.com/Qiskit/qiskit>

²<https://github.com/quantumlib/Cirq>

³<https://github.com/PennyLaneAI/pennylane>

⁴<https://github.com/amazon-braket/amazon-braket-sdk-python>

3. ANTECEDENTES

de palabras clave específicas que identifican tecnologías y conceptos relevantes [Git24b]; las más destacadas son qiskit, cirq, pennylane, braket, quantum computing, qubit, openqasm, quantum circuit, quantum error correction y quantum machine learning.

El análisis de estos repositorios permite identificar tendencias y patrones de colaboración, proporcionando una visión integral del desarrollo cuántico en la plataforma. La detección de comunidades dentro de este ecosistema se realiza mediante el algoritmo Louvain, que permite identificar grupos de interés y colaboración, facilitando el entendimiento de las dinámicas internas [BGLL08]. Además, el uso de la API de GitHub para la extracción de datos permite mantener actualizada la información sobre el ecosistema, asegurando que el proyecto Entangle refleje fielmente el estado actual de la computación cuántica en GitHub.

3.3 Minería de Repositorios de Software

Esta sección revisa las técnicas de minería de repositorios de software (Mining Software Repositories (MSR)) que fundamentan la recolección de datos de Entangle, desde la elección de GitHub como fuente primaria hasta las herramientas de extracción y análisis de grafos de colaboración.

3.3.1 GitHub como Fuente de Datos

GitHub se ha consolidado como una plataforma esencial para el desarrollo de software, albergando millones de repositorios y usuarios activos. Su relevancia en el ámbito de la minería de repositorios de software (MSR) es indiscutible, dado que proporciona acceso a una vasta cantidad de datos sobre el desarrollo de software, incluyendo código fuente, historial de commits, issues, pull requests y más [Gou13].

A diferencia de otras plataformas como GitLab y Bitbucket, GitHub ofrece una mayor visibilidad y comunidad, lo que se traduce en un volumen de datos significativamente superior. Con 71 palabras clave específicas para el ámbito cuántico, GitHub se convierte en una fuente inigualable para el proyecto Entangle, permitiendo identificar repositorios relevantes y mapear el ecosistema de computación cuántica de manera efectiva [Git24b]. La amplia adopción de GitHub en la comunidad de desarrolladores también facilita la integración de herramientas y servicios que enriquecen el análisis de datos.

Además, GitHub proporciona una infraestructura robusta para la extracción de datos mediante sus APIs, tanto Representational State Transfer (REST) como GraphQL, lo que permite acceder a datos jerárquicos y estructurados de manera eficiente.

3.3.2 API GraphQL y REST

Las APIs de GitHub, tanto GraphQL como REST, ofrecen capacidades distintas que son aprovechadas en el proyecto Entangle para la extracción de datos. La API REST es conocida por su simplicidad y facilidad de uso, proporcionando endpoints específicos para acceder

a diferentes recursos [Git24c]. Sin embargo, GraphQL ofrece una ventaja significativa al permitir la recuperación de datos jerárquicos en una sola consulta, lo que es especialmente útil para obtener información compleja y relacionada [Git24b].

GraphQL permite especificar exactamente qué datos se necesitan, lo que reduce la sobrecarga de datos y mejora la eficiencia de las consultas. Por ejemplo, una consulta GraphQL puede recuperar información detallada de un repositorio y sus colaboradores en una sola llamada, en lugar de realizar múltiples llamadas como sería necesario con la API REST.

El manejo de los *rate limits* es un aspecto crítico al utilizar estas APIs. GraphQL y REST tienen diferentes límites de tasa, y Entangle implementa estrategias de segmentación y batching para optimizar el uso de las consultas. La paginación es otro aspecto importante, especialmente en GraphQL, donde se utilizan cursores para navegar entre páginas de resultados, asegurando que se puedan procesar grandes conjuntos de datos sin exceder los límites de tasa. Conviene recordar que estos *rate limits* no son una restricción arbitraria: permiten a GitHub mantener un acceso sostenible y regulado a su infraestructura, contener el uso abusivo y articular su oferta comercial (cuotas ampliadas en planes de pago para empresas y servicios productivos) [Git24c].

La Figura 3.2 ilustra esta diferencia con un ejemplo concreto: obtener el nombre, las estrellas, los lenguajes principales y los primeros colaboradores de un repositorio. REST requiere tres llamadas independientes que devuelven, cada una, un documento completo con muchos campos no necesarios; la consulta GraphQL declara explícitamente los campos requeridos y los obtiene en una sola petición.

(a) REST: 3 llamadas	(b) GraphQL: una única consulta
<pre>GET /repos/IBM/qiskit GET /repos/IBM/qiskit/contributors ?per_page=100 GET /repos/IBM/qiskit/languages # Cada llamada devuelve un # documento completo, con muchos # campos no utilizados.</pre>	<pre>query { repository(owner: "IBM", name: "qiskit") { stargazerCount languages(first: 5) { nodes { name } } collaborators(first: 100) { nodes { login } } } }</pre>

Figura 3.2: Comparativa REST (a) vs GraphQL (b) para una misma consulta sobre la API de GitHub.

3.3.3 Técnicas de Ingesta Incremental y Rate Limits

El estudio del ecosistema de GitHub a gran escala requiere gestionar de forma explícita los *rate limits* de la plataforma, la paginación de resultados y la posible duplicación de entidades entre consultas. Trabajos previos sobre minería de repositorios a gran escala [Gou13, KGB⁺14] recogen estrategias bien conocidas para ello: la **segmentación** de búsquedas por

3. ANTECEDENTES

fecha o número de estrellas, que fragmenta el problema en subconsultas capaces de sortear límites como el tope de 1.000 resultados de GitHub; el **batching** de consultas GraphQL, que reduce el número total de peticiones a la API; el uso de **backoff exponencial** ante errores de tasa; y la **deduplicación** basada en identificadores estables para evitar procesar entidades ya recuperadas. La aplicación concreta de estas técnicas al pipeline de Entangle se detalla en la Sección 5.3 del Capítulo 5.

3.4 Análisis de Redes de Colaboración

Esta sección presenta los fundamentos teóricos del análisis de redes sociales aplicados al estudio de la colaboración en ecosistemas de software. Se describe la construcción de grafos de colaboración (Sección 3.4.1), las métricas de centralidad empleadas para caracterizar la importancia de los actores (Sección 3.4.2) y el algoritmo de detección de comunidades utilizado (Sección 3.4.3).

3.4.1 Grafos de Colaboración

Un grafo es una estructura matemática que representa relaciones entre objetos. Se compone de nodos (o vértices) y aristas (o enlaces) que conectan pares de nodos. En el contexto de Entangle, se utiliza un grafo heterogéneo para modelar las interacciones entre usuarios, repositorios y organizaciones en GitHub.

El grafo heterogéneo en Entangle incluye tres tipos de nodos: `org_login`, `repo_full_name` y `user_login`. Las aristas representan relaciones como `owns`, `contributed_to` y `collab`, lo que permite modelar la propiedad de repositorios, las contribuciones de usuarios y las colaboraciones entre entidades.

Para construir y analizar este grafo, se utiliza la biblioteca `NetworkX`, que ofrece herramientas avanzadas para el manejo de grafos complejos [HSS24]. `NetworkX` permite la creación y manipulación de grafos, el cálculo de métricas de red y la aplicación de algoritmos de detección de comunidades, como el algoritmo Louvain [BGLL08].

3.4.2 Métricas de Centralidad

Las métricas de centralidad resultan clave para comprender la importancia de los nodos dentro de un grafo [Fre77]. En Entangle, se calculan varias métricas de centralidad, incluyendo el grado, la centralidad de intermediación y la cercanía, cada una con sus propias fórmulas y aplicaciones.

La centralidad de grado mide el número de aristas incidentes en un nodo. Se calcula como:

$$C_D(v) = \deg(v) \quad (3.1)$$

donde $\deg(v)$ es el número de conexiones de v . Esta métrica es útil para identificar nodos altamente conectados, que pueden actuar como hubs en la red.

La centralidad de intermediación, o *betweenness centrality*, mide la frecuencia con la que un nodo aparece en las rutas más cortas entre otros nodos. Su fórmula es:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (3.2)$$

donde σ_{st} es el número total de rutas más cortas de s a t , y $\sigma_{st}(v)$ es el número de esas rutas que pasan por v . En un ecosistema de software, un nodo con alta *betweenness* actúa como cuello de botella en la difusión de conocimiento: si un desarrollador conecta dos comunidades que de otro modo no interactuarían, su desaparición fragmentaría el flujo de información entre ellas. En el contexto cuántico, donde los equipos suelen trabajar en silos organizacionales (Qiskit, Cirq, PennyLane), identificar estos conectores es especialmente relevante porque revela qué individuos facilitan la transferencia de ideas entre plataformas competidoras.

La centralidad de cercanía se define como el inverso de la suma de las distancias más cortas desde un nodo a todos los demás nodos:

$$C_C(v) = \frac{1}{\sum_t d(v, t)} \quad (3.3)$$

donde $d(v, t)$ es la distancia más corta entre v y t . Esta métrica es útil para identificar nodos que pueden difundir información de manera eficiente a través de la red.

3.4.3 Detección de Comunidades con Louvain

El algoritmo Louvain es ampliamente utilizado para la detección de comunidades en grafos debido a su capacidad para maximizar la modularidad de manera eficiente. La modularidad Q se calcula como:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (3.4)$$

donde A_{ij} es la matriz de adyacencia, k_i y k_j son los grados de los nodos i y j , m es el número total de aristas, y δ es la función delta de Kronecker [For10].

El algoritmo Louvain opera en dos fases principales: primero, asigna cada nodo a su propia comunidad y optimiza localmente la modularidad, y luego agrupa nodos en supernodos y repite el proceso [BGLL08]. Su complejidad empírica es aproximadamente $O(n \log n)$ con n el número de nodos del grafo, lo que lo hace especialmente adecuado para redes de gran tamaño como las de colaboración en GitHub. La modularidad resultante permite evaluar si el ecosistema se organiza en grupos cohesivos (valor alto) o si la colaboración se distribuye

3. ANTECEDENTES

de forma uniforme sin fronteras claras (valor bajo). En el caso del software cuántico, una modularidad alta indica que los desarrolladores colaboran predominantemente dentro de su organización o framework, lo que puede ser una barrera para la estandarización del campo pero, al mismo tiempo, refleja la especialización técnica inherente a cada plataforma de hardware cuántico.

Comparación con el algoritmo de Leiden. Vincent Traag y sus colaboradores propusieron en 2019 el algoritmo de Leiden [TWvE19] como evolución de Louvain. Su principal mejora consiste en garantizar que todas las comunidades resultantes están bien conectadas internamente, algo que Louvain no puede asegurar: tras las fusiones sucesivas, pueden aparecer comunidades con nodos desconectados entre sí. Leiden introduce una fase de refinamiento que detecta y corrige estas anomalías, manteniendo una eficiencia computacional comparable.

Para Entangle se ha optado por Louvain por tres razones prácticas: (i) la implementación de referencia en NetworkX [HSS24] proporciona integración directa con el resto del análisis de red, (ii) el tamaño del grafo (~ 31.000 nodos, ~ 101.000 aristas) no alcanza la escala donde las diferencias entre ambos algoritmos son significativas, y (iii) la modularidad obtenida (0,9585) es suficientemente alta como para que las posibles mejoras de Leiden resulten marginales en este contexto. En ecosistemas de mayor tamaño o con estructuras comunitarias más ambiguas, la migración a Leiden sería recomendable.

3.4.4 Caminos Mínimos: Algoritmo de Dijkstra

El algoritmo de Dijkstra [Dij59] resuelve el problema del camino más corto desde un nodo origen a todos los demás nodos de un grafo con pesos no negativos. Dado un grafo $G = (V, E)$ con función de peso $w : E \rightarrow \mathbb{R}_{\geq 0}$, el algoritmo calcula la distancia mínima $d(s, v)$ desde el nodo origen s a cada nodo $v \in V$:

$$d(s, v) = \min_{p \in P_{s,v}} \sum_{(u,w) \in p} w(u, w) \quad (3.5)$$

donde $P_{s,v}$ es el conjunto de todos los caminos posibles de s a v .

El algoritmo mantiene un conjunto de nodos visitados y una cola de prioridad con las distancias tentativas. En cada iteración, extrae el nodo con menor distancia, lo marca como visitado y relaja las aristas adyacentes. Su complejidad temporal es $O((|V| + |E|) \log |V|)$ cuando se implementa con una cola de prioridad binaria, lo que lo hace adecuado para grafos dispersos como los de colaboración.

En Entangle, Dijkstra se utiliza en la funcionalidad de *Túnel Cuántico* del universo 3D para calcular y animar el camino más corto entre dos entidades cualesquiera del grafo, revelando cadenas de colaboración indirectas que atraviesan múltiples organizaciones. Aplicado sobre

un grafo de 30.970 nodos y 101.190 aristas, el algoritmo resuelve cada consulta en pocos milisegundos gracias a su complejidad logarítmica. No obstante, el coste dominante de la operación no reside en el algoritmo sino en la construcción previa del grafo en memoria a partir de la base de datos, que en arranque en frío puede requerir cerca de un minuto al recuperar y ensamblar todos los nodos y aristas desde MongoDB. Una vez cacheado el grafo, las consultas sucesivas son prácticamente instantáneas.

3.4.5 Bridge Users

Un *bridge user* se define formalmente como un usuario que contribuye a repositorios pertenecientes a dos o más organizaciones independientes (no consideradas hermanas entre sí). En la literatura de minería de repositorios, este tipo de actor recibe distintas denominaciones (*broker*, *cross-cutting contributor*, *cross-project developer*) y se asocia tanto a procesos de difusión de conocimiento entre comunidades como a posiciones de alta intermediación en el grafo de colaboración [Gou13, KGB⁺14]. La aplicación concreta de este concepto al ecosistema cuántico de GitHub, que incluye los criterios específicos de exclusión de cuentas *bot*, la noción complementaria de *bridge disciplinar* y los resultados cuantitativos sobre Entangle, se desarrolla en el Capítulo 5 (Sección 5.12.4).

3.5 Visualización de Datos

Esta sección examina los principios de diseño de dashboards y las técnicas de visualización interactiva que han guiado el diseño de la interfaz de Entangle, así como las herramientas de renderizado 3D empleadas para la representación del ecosistema.

3.5.1 Principios de Diseño de Dashboards

El diseño de dashboards efectivos se basa en varios principios fundamentales que aseguran su usabilidad y eficacia. La usabilidad es un aspecto crucial, ya que un dashboard debe ser intuitivo y fácil de navegar para que los usuarios puedan extraer información valiosa sin esfuerzo. Esto se logra mediante una disposición clara y lógica de los elementos, el uso de iconografía comprensible y una paleta de colores que facilite la distinción entre diferentes tipos de información [Tuf01].

Otro principio esencial es la información en capas, que permite a los usuarios explorar los datos a diferentes niveles de detalle. Los dashboards deben proporcionar una vista general clara, con la opción de profundizar en áreas específicas a través de interacciones como el clic o el hover.

La interactividad y el filtrado progresivo son también componentes clave de un dashboard bien diseñado. La capacidad de interactuar con los datos, como ajustar filtros o seleccionar diferentes vistas, permite a los usuarios personalizar su experiencia y obtener insights más precisos. Además, la accesibilidad visual debe ser considerada, asegurando que el dashboard

3. ANTECEDENTES

sea utilizable por personas con diversas capacidades visuales, mediante el uso de contrastes adecuados y opciones de accesibilidad.

3.5.2 Visualización 3D de Grafos

La visualización 3D de grafos ofrece una perspectiva enriquecida para el análisis de datos complejos, permitiendo a los usuarios explorar relaciones y patrones que podrían no ser evidentes en representaciones bidimensionales. WebGL es una tecnología fundamental en este ámbito, proporcionando la capacidad de renderizar gráficos 3D en navegadores web sin necesidad de plugins adicionales. Esto facilita la creación de visualizaciones interactivas y de alto rendimiento.

Three.js [Cab24] es una biblioteca ampliamente utilizada para desarrollar visualizaciones 3D en la web. Ofrece un conjunto de herramientas que simplifican la creación de escenas tridimensionales, permitiendo a los desarrolladores implementar metáforas visuales que representan datos complejos de manera intuitiva. Estas metáforas pueden incluir nodos como esferas y conexiones como líneas, lo que ayuda a los usuarios a conceptualizar las relaciones entre entidades.

Los layouts dirigidos por fuerzas, o force-directed layouts, son una técnica común para organizar grafos en el espacio tridimensional. Este enfoque utiliza algoritmos que simulan fuerzas físicas, como la atracción y la repulsión, para posicionar los nodos de manera que las conexiones sean claras y el grafo sea fácil de interpretar.

3.5.3 Técnicas de Distribución Espacial

Las técnicas de distribución espacial son fundamentales para organizar datos en visualizaciones de manera que sean comprensibles y estéticamente agradables. Un Pseudorandom Number Generator (PRNG) determinista, como el método `seededRandom(42)`, se utiliza para asegurar que las posiciones de los elementos sean reproducibles, lo cual es esencial para mantener la consistencia visual en múltiples sesiones o entre diferentes usuarios.

La distribución por métricas, como la centralidad, permite posicionar los nodos de acuerdo a su importancia relativa en el grafo. Esto asegura que los nodos más relevantes sean más prominentes y fácilmente identificables por los usuarios. La centralidad puede calcularse mediante diversas métricas, como la centralidad de grado o de intermediación, cada una ofreciendo una perspectiva diferente sobre la importancia de un nodo [Fre77].

El clústering natural, como el algoritmo de Jenks Natural Breaks, se utiliza para identificar zonas de densidad dentro de los datos. Este método clasifica los datos en grupos, minimizando la varianza dentro de cada grupo y maximizando la diferencia entre grupos. Esto es particularmente útil para identificar regiones de alta, media y baja densidad en visualizaciones espaciales, permitiendo a los usuarios centrarse en áreas de interés [Jen67].

3.6 Agentes de IA Conversacional

Esta sección revisa los fundamentos teóricos que sustentan el agente de IA conversacional de Entangle. Se introducen primero los Large Language Model (LLM) y la arquitectura *transformer* en la que se basan (Sección 3.6.1). A continuación se presenta el paradigma de IA agéntica y los sistemas multi-agente (Sección 3.6.2). Finalmente se describen dos patrones arquitectónicos relevantes: las arquitecturas Router-Worker para enrutamiento por intención (Sección 3.6.3) y el mecanismo de *function calling* para el acceso controlado a datos externos (Sección 3.6.4).

3.6.1 Grandes Modelos de Lenguaje

Los LLM han transformado el procesamiento del lenguaje natural al permitir la generación y comprensión de texto a gran escala. Estos modelos se basan en la arquitectura *transformer* [VSP⁺17], cuyo mecanismo de autoatención (*self-attention*) asigna pesos diferenciados a cada posición de la secuencia de entrada, capturando dependencias a largo plazo sin recurrir a la recurrencia. A medida que se incrementa el número de capas y parámetros, la capacidad de los modelos para generar texto coherente y resolver tareas complejas mejora significativamente [BMR⁺20].

El escalado ha sido un factor determinante en el éxito de los LLM. Modelos como Generative Pre-trained Transformer (GPT)-4o han demostrado capacidades avanzadas en generación de texto, traducción automática y respuesta a preguntas [Ope23b]. No obstante, este escalado conlleva desafíos: el coste computacional de entrenamiento e inferencia crece de forma sustancial, y se requieren grandes volúmenes de datos curados [RWC⁺19, KMH⁺20].

A pesar de su potencia, los LLM presentan limitaciones inherentes: propensión a generar respuestas sesgadas o factualmente incorrectas (*hallucinations*) [JLF⁺23], y una interpretabilidad limitada que dificulta su uso en contextos donde se exige transparencia. La investigación activa aborda estos problemas mediante técnicas de alineamiento ético [OWJ⁺22], *fine-tuning* supervisado y modelos especializados de menor escala.

3.6.2 IA Agéntica y Sistemas Multi-Agente

La IA agéntica representa un cambio de paradigma en el uso de los LLM: en lugar de limitar el modelo a responder preguntas de forma pasiva, se le dota de capacidad para planificar acciones, utilizar herramientas externas y perseguir objetivos de forma autónoma [WMF⁺24, YZY⁺23]. Un agente basado en LLM percibe su entorno (a través de *prompts* y contexto), razona sobre la tarea y ejecuta acciones (llamadas a APIs, consultas a bases de datos o modificaciones de interfaz) en un bucle de retroalimentación [XCG⁺23, SDYD⁺23]. En el dominio del desarrollo de software, este paradigma ha dado lugar a asistentes capaces de generar código, depurar errores y operar sobre el repositorio del usuario, como demuestran los trabajos sobre *benchmarks* para agentes de programación [LYZ⁺24, JYW⁺24].

3. ANTECEDENTES

Este enfoque ha dado lugar a arquitecturas multi-agente donde varios agentes especializados colaboran para resolver tareas complejas [WBZ⁺23]. Un componente coordinador delega subtareas a agentes expertos en dominios concretos, mejorando la calidad de las respuestas y la escalabilidad del sistema.

3.6.3 Arquitecturas Router-Worker

Las arquitecturas Router-Worker constituyen un patrón de diseño para sistemas multi-agente en el que un componente *router* clasifica la intención de cada consulta entrante y la delega al agente *worker* más adecuado [WMF⁺24]. A diferencia de las arquitecturas monolíticas, donde un único modelo gestiona todas las solicitudes, este patrón permite la especialización: cada *worker* está optimizado para un conjunto específico de funciones, lo que mejora la precisión y reduce la latencia de respuesta.

El *routing* por intención es el componente central del patrón. Puede implementarse mediante clasificadores basados en LLM, modelos ligeros de clasificación o heurísticas definidas por reglas. La ventaja principal reside en la flexibilidad: nuevos *workers* pueden incorporarse sin alterar el flujo general, lo que facilita la evolución del sistema ante requisitos cambiantes.

3.6.4 Function Calling y Acceso a Datos Externos

El mecanismo de *function calling* permite que un LLM invoque funciones externas de forma estructurada, siguiendo un esquema (*schema*) que define los parámetros de entrada y el formato de salida esperado [Ope23a]. En lugar de generar texto de forma libre para responder preguntas factuales, el modelo selecciona la función adecuada, genera los argumentos a partir del contexto conversacional y delega la ejecución al sistema anfitrión. Esto fundamenta el acceso controlado a bases de datos y APIs sin exponer directamente las credenciales al modelo.

La seguridad es un aspecto central de este patrón: las funciones suelen configurarse como de solo lectura, lo que minimiza el riesgo de modificaciones no autorizadas y simplifica el cumplimiento de normativas de privacidad [Mon24a]. El *streaming* de respuestas mediante Server-Sent Events (SSE) complementa este enfoque al permitir la transmisión continua del resultado al cliente sobre una conexión HyperText Transfer Protocol (HTTP) unidireccional, reduciendo la latencia percibida en comparación con el *polling* tradicional [Moz24c, Hic15].

3.7 Trabajos Relacionados

El análisis de ecosistemas en plataformas de desarrollo colaborativo ha generado diversas herramientas y estudios académicos. Esta sección revisa los más relevantes para contextualizar las aportaciones de Entangle.

3.7.1 GHTorrent

GHTorrent [Gou13] construye un espejo offline de los datos públicos de GitHub, monitorizando su API REST en tiempo real y almacenando los eventos en MySQL (datos relacionales) y MongoDB (datos crudos en JavaScript Object Notation (JSON)). El dataset resultante ha sido utilizado en más de 600 estudios empíricos según sus autores. Su principal contribución es proporcionar acceso reproducible a datos históricos de GitHub sin depender del rate limit de la API en cada estudio individual. Sin embargo, GHTorrent se limita a la recopilación y almacenamiento de datos: no incorpora análisis de redes, visualización ni herramientas de exploración interactiva.

3.7.2 GH Archive

GH Archive [Gri15] registra la línea temporal de eventos públicos de GitHub en ficheros JSON comprimidos por hora, disponibles para su consulta a través de Google BigQuery. Este enfoque permite análisis temporales a escala masiva (miles de millones de eventos) pero, al igual que GHTorrent, se trata de una fuente de datos sin capacidades analíticas propias. El investigador debe construir su propio pipeline de procesamiento sobre los datos descargados.

3.7.3 Libraries.io

Libraries.io [Lib24] indexa más de 30 gestores de paquetes (npm, PyPI, Maven, etc.) y rastrea las relaciones de dependencia entre proyectos de software libre. Su enfoque se centra en el grafo de dependencias técnicas (qué paquete depende de cuál) y proporciona métricas como SourceRank para evaluar la salud de un proyecto. No obstante, este grafo refleja dependencias entre artefactos de software, no relaciones de colaboración entre personas, por lo que no aborda el análisis de redes sociales de desarrolladores.

3.7.4 Open Source Insights

Open Source Insights (deps.dev) [Goo24a], desarrollado por Google, realiza análisis de seguridad, licencias y dependencias transitivas sobre paquetes de código abierto. Su valor reside en la identificación de vulnerabilidades conocidas (CVE) a lo largo de la cadena de dependencias. Al igual que Libraries.io, se centra en el grafo de artefactos y no en las dinámicas de colaboración humana.

3.7.5 Estudios empíricos en MSR

La conferencia International Conference on Mining Software Repositories (MSR) ha producido investigaciones directamente relevantes para este trabajo. Kalliamvakou et al. [KGB⁺14] analizan las limitaciones metodológicas de usar GitHub como fuente de datos, identificando sesgos como repositorios inactivos y cuentas duplicadas que deben tenerse en cuenta al

3. ANTECEDENTES

interpretar resultados. Cohen y Consens [CC18] estudian patrones de co-commit en los repositorios más activos de GitHub, construyendo grafos de colaboración a partir de contribuciones conjuntas. Estos trabajos aportan técnicas y consideraciones metodológicas que informan el diseño del pipeline de Entangle, pero se presentan como estudios individuales sin una plataforma integrada de exploración.

3.7.6 Minería de Stack Overflow sobre computación cuántica

Tavassoli Sabzevari y Khan [TSK26] presentan un estudio empírico reciente que mina 1.404 publicaciones de Stack Overflow relacionadas con la computación cuántica. Mediante *topic modeling* con LDA, identifican siete temas principales de discusión entre desarrolladores, siendo la computación híbrida cuántico-clásica y la implementación de circuitos los más prevalentes. El trabajo también cuantifica las herramientas más referenciadas (Qiskit domina con el 31 % de las menciones, seguido de Q#) y los algoritmos más discutidos (Grover y Shor), además de evaluar la dificultad de las preguntas mediante métricas de respuestas aceptadas y tiempo de resolución. Sus hallazgos confirman que los retos prácticos de adopción (instalación, configuración, integración híbrida) dominan las discusiones frente a los aspectos teóricos. La principal diferencia con Entangle radica en la fuente de datos y en el enfoque: mientras que este estudio analiza preguntas de un foro de Q&A para identificar temas y dificultades, Entangle mina directamente la actividad de desarrollo en GitHub para construir un grafo de colaboración y ofrecer herramientas de exploración visual e interactiva.

3.7.7 Posicionamiento de Entangle

La Tabla 3.1 resume las capacidades de cada trabajo. Las herramientas anteriores abordan aspectos específicos del problema (recopilación, dependencias, seguridad, análisis puntual de colaboración o minería de foros) pero ninguna los integra en una plataforma unificada. Entangle combina un pipeline de ingesta sobre la API de GitHub, análisis de redes con detección de comunidades y bridge users, un dashboard 2D con gráficos y grafos interactivos, un universo 3D inmersivo, y un agente de IA conversacional con acceso directo a los datos.

Trabajo	Pipeline	Red social	Dashboard	3D	IA
GHTorrent [Gou13]	✓	×	×	×	×
GH Archive [Gri15]	✓	×	×	×	×
Libraries.io [Lib24]	✓	×	Parcial	×	×
Open Source Insights [Goo24a]	✓	×	Parcial	×	×
Estudios MSR [KGB ⁺ 14, CC18]	Varía	✓	×	×	×
Tavassoli y Khan [TSK26]	✓	×	×	×	×
Entangle	✓	✓	✓	✓	✓

Tabla 3.1: Comparación de capacidades entre trabajos relacionados y Entangle.

Capítulo 4

Metodología

Este capítulo describe la metodología seguida en el desarrollo del proyecto Entangle. Se presentan en primer lugar las dos metodologías que han guiado el trabajo: *Scrumban* como metodología de gestión del proyecto (Sección 4.1) y *Vibe Coding* como metodología de desarrollo del software (Sección 4.2). A continuación se detalla cómo se han aplicado ambas (Sección 4.3), incluyendo roles, planificación por fases y sprints; el capítulo cierra con el *stack* tecnológico adoptado para cada capa del sistema (Sección 4.4).

4.1 Scrumban como Metodología de Gestión de Proyectos

Scrumban [Lad08, NKMS12] es una metodología híbrida de gestión de proyectos software que combina los principios de Scrum [SS20, Sut14] con las prácticas de Kanban [And10, Rei09], aplicando los valores del *Manifiesto Ágil* [BBvB⁺01]. De Scrum hereda la planificación por iteraciones acotadas (*sprints*) con un objetivo concreto y un hito de cierre verificable, así como los principios de inspección y adaptación continua del producto. De Kanban toma el tablero de tareas con columnas de estado, la limitación explícita del trabajo en curso (*Work In Progress*, WIP) [Rei09] y la priorización dinámica del *backlog*.

La combinación aporta una flexibilidad que el Scrum puro no admite: el equipo mantiene la disciplina del sprint pero sin las penalizaciones rituales que se producen cuando aparecen *spikes* técnicos, defectos críticos o tareas no planificadas. Esto la hace especialmente adecuada para equipos pequeños, proyectos de I+D y desarrollos donde el alcance se concreta progresivamente, escenarios en los que los ceremoniales de Scrum (reuniones diarias, revisiones con múltiples participantes) resultan desproporcionados respecto al beneficio que aportan [NKMS12].

4.2 Vibe Coding como Metodología de Desarrollo de Software

Vibe Coding [Kar25, Sar25] es un estilo emergente de desarrollo de software asistido por modelos de lenguaje conversacionales (Large Language Model (LLM)) entrenados específicamente sobre código fuente [CTJ⁺21] en el que el desarrollador describe la intención

4. METODOLOGÍA

en lenguaje natural y delega en el modelo la generación de fragmentos de código, refactorizaciones, pruebas y documentación, validando y dirigiendo iterativamente los resultados hacia la solución deseada. El término fue acuñado por Andrej Karpathy en 2025 [Kar25] y rápidamente popularizado en el ámbito del desarrollo asistido por IA.

A diferencia del par-programming clásico, donde dos personas comparten un único editor, en *vibe coding* el rol del segundo programador lo asume un LLM (por ejemplo, GitHub Copilot, ChatGPT/o3, Claude o Cursor) que sugiere implementaciones a partir del contexto del repositorio y del diálogo en lenguaje natural. Estudios empíricos sobre el uso de estas herramientas estiman ganancias significativas de productividad: en una evaluación a gran escala de GitHub Copilot, Ziegler et al. [ZKL⁺24] reportan reducciones del tiempo de tarea de hasta el 55 % y un incremento sostenido de la sensación de fluidez en el desarrollador. El desarrollador conserva en todo momento la responsabilidad arquitectónica, la revisión crítica y los criterios de aceptación; el modelo actúa como un *pair-programmer* acelerador, no como autor autónomo. Esta práctica reduce el tiempo dedicado a *boilerplate*, refactorización mecánica y redacción de pruebas, pero exige una disciplina explícita en la verificación del código generado: Pearce et al. [PAT⁺22] demostraron que un porcentaje no despreciable de las sugerencias contiene patrones inseguros, por lo que la revisión humana y el análisis estático automatizado siguen siendo imprescindibles [Sar25].

4.3 Aplicación de las Metodologías al Proyecto Entangle

El desarrollo de Entangle ha sido realizado por un único desarrollador bajo la dirección de un tutor académico, siguiendo un modelo iterativo e incremental adaptado a las características de un Trabajo Fin de Grado (TFG) individual. Sobre este armazón se han aplicado las dos metodologías presentadas en las secciones anteriores: *Scrumban* como metodología de gestión del proyecto y *Vibe Coding* como metodología de desarrollo de software. El proyecto adopta además las prácticas de *entrega continua* [HF10] y los principios DevOps que la literatura asocia a equipos de alto rendimiento [FHK18].

4.3.1 Roles y Aplicación de Scrumban y Vibe Coding

El equipo está compuesto por dos personas con roles diferenciados que se aproximan a los de Scrum sin reproducirlos literalmente. El **tutor académico** actúa como *Product Owner*, fijando la dirección del proyecto, validando incrementos y priorizando el *backlog* mediante revisiones periódicas; el **estudiante** desempeña simultáneamente los roles de *Developer* y *Scrum Master*, ejecutando el desarrollo y velando por la cadencia. Las reuniones diarias se sustituyen por un registro continuo en GitHub Issues y el propio historial de *commits*, y las revisiones de sprint se concretan en sesiones con el tutor al cierre de cada hito relevante. El tablero Kanban de Scrumban se materializa en GitHub Projects, con columnas *Backlog*, *To Do*, *In Progress*, *Review* y *Done* y un límite explícito de tareas concurrentes para evitar la dispersión propia del trabajo individual.

A partir de la fase 2 del proyecto, una parte sustancial de la implementación, refactorización y documentación se ha realizado mediante *Vibe Coding*. Las herramientas utilizadas han sido *GitHub Copilot* integrado en el editor, *ChatGPT/o3* para diseño de *prompts* y revisiones arquitectónicas, y *Claude* para generación de código y documentación extensa. El enfoque se ha aplicado de forma especialmente intensiva en el desarrollo del agente conversacional (Sec. 5.6), en la generación de los *prompts* del propio sistema y en la batería de pruebas del backend y del frontend.

4.3.2 Planificación por Fases y Sprints

A nivel macroscópico, el desarrollo se ha estructurado en tres fases delimitadas por incrementos funcionales acumulativos:

1. **Fase 1: Construcción del núcleo de ingesta y análisis (oct. 2025 – ene. 2026).** Diseño e implementación del *pipeline* de ingesta sobre la API GraphQL de GitHub, modelo de datos en MongoDB, primera versión del análisis de redes con NetworkX y un dashboard 2D inicial. El sistema operaba sobre infraestructura Azure de cuenta de estudiante y un script de despliegue manual.
2. **Fase 2: Endurecimiento, escalado y nuevas capas visibles (feb. 2026).** Profesionalización del *pipeline* (idempotencia, resiliencia ante errores HTTP 429, reintentos con *backoff*), primera versión del universo 3D inmersivo, rediseño del cuadro de mando con métricas precalculadas, panel de colaboración y sistema de favoritos. La migración de Azure Cosmos DB del modelo *Request Units* al clúster *vCore M30*, ejecutada como una iteración mayor en este periodo, multiplicó el rendimiento de la ingesta y se relata en detalle en la Sec. 5.15; aquí se menciona como hito técnico, no como motor de las decisiones funcionales.
3. **Fase 3: Producto cerrado, despliegue continuo y métricas (mar.– jun. 2026).** Agente conversacional *config-driven* con *streaming* SSE (RAG sobre README, memoria conversacional e investigación externa), lentes analíticas avanzadas del universo 3D (soporte multidisciplinar, detección de *bridge users* y filtrado de bots), internacionalización, consolidación de la batería de tests, despliegue continuo automatizado en Azure Static Web Apps y Container Apps, y campaña de medición de rendimiento, calidad de código y validación con datos reales.

Cada fase agrupa varias iteraciones menores centradas en funcionalidades concretas. La Tabla 4.1 esquematiza esta evolución temporal junto con los hitos verificables de cada fase.

4. METODOLOGÍA

Tabla 4.1: Fases de desarrollo del proyecto Entangle e hitos principales de cada iteración.

Hito	Fase 1: Núcleo	Fase 2: Endurecimiento	Fase 3: Producto cerrado
Período	Oct 2025 – Ene 2026	Feb 2026	Mar – Jun 2026
Funcionales	Pipeline v1 Modelo de datos Análisis de red v1 Dashboard 2D base	Universo 3D v1 Dashboard cuántico Lentes de colaboración v1 Sistema de favoritos	Agente IA config-driven + RAG Multidisciplina + bots i18n Métricas + validación
Infraestructura y Operaciones	Cosmos DB RU + deploy manual	Migración a vCore M30	SWA + Container Apps + CI/CD completo

La tabla refleja que en cada fase coexisten hitos funcionales y de infraestructura/operaciones. Los rangos temporales se han reconstruido directamente a partir del historial de *commits* de los dos repositorios públicos del proyecto: Entangle-Core (188 *commits* entre el 9 de octubre de 2025 y el 17 de junio de 2026) y Entangle-Visualizer (94 *commits* en el mismo intervalo). Tanto las fechas como los hitos enumerados son verificables sobre la rama *main* de cada repositorio.

4.3.3 Iteraciones y Sprints

Dentro del modelo iterativo descrito, el desarrollo se organizó en sprints de duración variable (en su mayoría de dos a cuatro semanas, con un sprint final más extenso dedicado a la consolidación y a la evolución del agente conversacional), cada uno orientado a un incremento funcional concreto y cerrado con un incremento verificable. La planificación no siguió una cadencia rígida sino que se adaptó a las dependencias funcionales entre componentes (por ejemplo, la disponibilidad de un endpoint estable era condición previa para construir las pantallas que lo consumen) y a los compromisos académicos paralelos. La Tabla 4.2 sintetiza los siete sprints en los que se ha estructurado el desarrollo.

Tabla 4.2: Sprints del proyecto Entangle e incrementos verificables al cierre de cada iteración.

Sprint	Periodo	Foco principal	Incremento conseguido
S1	9–15 oct 2025	Bootstrap del proyecto, configuración de entorno, primer cliente de la API de GitHub, modelos Pydantic y módulo de persistencia MongoDB.	Primera ingesta funcional persistida.
S2	29 oct–11 nov 2025	Enriquecimiento de repositorios y colaboradores, filtrado inteligente por lenguajes y mejora de logs.	Ingesta segmentada con cobertura ampliada.
S3	18–28 nov 2025	Gestión del <i>rate limit</i> de la API de GitHub, refresco condicional cada 7 días, nuevos <i>end-points</i> y migración de la API a la base de datos en Azure.	API operativa contra Cosmos DB en Azure.
S4	1–31 dic 2025	Refactorización del enriquecimiento de usuarios y organizaciones, sistema de reintentos sobre Cosmos DB, configuración de CI/CD y primera batería de tests unitarios.	Pipeline CI/CD del backend en producción.
S5	1 ene–4 feb 2026	Base del frontend, integración con Azure Static Web Apps, ajustes de CORS y <i>auto-deploy</i> , y endurecimiento del backend ante errores transitorios de la base de datos.	Frontend desplegado en SWA + backend con rendimiento mejorado tras la migración a vCore.
S6	5–27 feb 2026	Dashboard con métricas precalculadas y filtros temporales, primera versión del universo cuántico 3D, panel de colaboración, sistema de favoritos y caché de métricas.	Universo Cuántico v1 + dashboard funcional.
S7	mar–jun 2026	Soporte multidisciplinar y filtrado contextual; agente conversacional <i>config-driven</i> con <i>streaming</i> SSE, RAG sobre README, memoria conversacional e investigación externa; internacionalización, filtrado de bots y refinamiento de la UX.	Agente IA conversacional (<i>config-driven</i> , RAG, memoria) integrado y producto cerrado funcionalmente.

Como puede observarse, todos los sprints están alineados con los objetivos específicos definidos en el Capítulo 2: S1–S3 concentran el *pipeline* de datos (SO1), S4–S5 cubren el despliegue y la primera transición de infraestructura, y S6–S7 materializan las capas visibles para el usuario: dashboard (SO3), universo 3D (SO4) y agente IA (SO5).

4.4 Stack Tecnológico del Proyecto

Esta sección presenta de forma concisa las tecnologías seleccionadas para cada capa del sistema, justificando las decisiones frente a las alternativas consideradas. Se ubica al final del capítulo de Metodología porque el conjunto se concibe como un *stack* de soporte al

4. METODOLOGÍA

desarrollo, no como un fundamento conceptual. La organización sigue el orden *backend* → persistencia → *frontend* → visualización → interfaz → IA → infraestructura → *testing*. La organización en carpetas y módulos del repositorio que materializa este *stack* se documenta en el Anexo C.

- **Backend** (Sección 4.4): Python 3.11 con FastAPI 0.115 sobre Uvicorn 0.32. FastAPI [Ram24] se eligió por su orientación a APIs, su soporte nativo Asynchronous Server Gateway Interface (ASGI) y la generación automática de documentación OpenAPI. Frente a un *full-stack* (Django, Flask con extensiones), un microframework asíncrono encaja mejor con un servicio puramente API.
- **Base de datos** (Sección 4.4): MongoDB sobre Azure Cosmos DB en su modalidad vCore M30. Los datos de la API de GitHub llegan como documentos JSON con esquema variable, lo que hace natural un modelo orientado a documentos [Mon24a, Mic24c]. La modalidad vCore aporta costes predecibles y ausencia de *throttling* HTTP 429 (Sección 5.15).
- **Frontend** (Sección 4.4): React 19 con Vite 7 como *bundler* (*Hot Module Replacement* para iteración rápida) y Zustand para la gestión de estado. Zustand ofrece una API minimalista sin *providers* ni *boilerplate*, adecuada a la escala del proyecto [Met24, You24].
- **Visualización** (Sección 4.4): Three.js con React Three Fiber para el universo 3D, Recharts para gráficos 2D interactivos y react-force-graph-2d para el grafo de colaboración [Cab24, Rec24]. La combinación cubre los tres modos de visualización del proyecto con APIs declarativas integradas en el ecosistema React.
- **Interfaz de usuario** (Sección 4.4): react-markdown y KaTeX para Markdown y fórmulas matemáticas; Lucide React y React Icons para iconografía Scalable Vector Graphics (SVG) *tree-shakeable* que reduce el tamaño del *bundle* [rem24, Luc24].
- **Inteligencia artificial** (Sección 4.4): Azure AI Foundry con gpt-5-mini, integrado mediante azure-identity y DefaultAzureCredential [Mic24a]. Al estar toda la infraestructura desplegada en Azure, su propio servicio de IA simplifica la autenticación, la facturación y la gestión de credenciales bajo un único proveedor.
- **Infraestructura** (Sección 4.4): Azure Container Apps para el *backend*, Azure Static Web Apps (SWA) para el *frontend*, Bicep como Infrastructure as Code (IAC) y Azure Developer CLI (azd) para automatizar el despliegue; Docker contenedoriza el *backend* [Mic24b, Doc24, Mic24f]. Container Apps abstrae la orquestación con escalado automático sin el coste operativo de un clúster completo.
- **Testing** (Sección 4.4): pytest con pytest-asyncio (alineado con la arquitectura *async* del *backend*) y pytest-cov para cobertura, integrados en la Continuous Integration / Continuous Deployment (CI/CD) [pyt24].

Capítulo 5

Resultados

Este capítulo presenta los resultados del proyecto Entangle desde una perspectiva de ingeniería del software. Se parte de la arquitectura del sistema (Sección 5.1), seguida del diseño detallado mediante diagramas UML (Sección 5.2), la implementación de cada componente (pipeline de datos, modelo, backend, agente de IA, frontend, universo 3D, métricas e infraestructura), y finalmente las pruebas, la validación con datos reales y la estimación de costes.

5.1 Arquitectura General del Sistema Entangle

Esta sección describe la arquitectura general del sistema y la organización de sus cinco capas, proporcionando una visión de alto nivel antes de abordar cada componente en las secciones posteriores. Conviene recordar, como ya se introdujo en el Capítulo 4, que Entangle se ha desarrollado como **dos subsistemas independientes**: Entangle-Core (backend Python/FastAPI) y Entangle-Visualizer (frontend React/Vite), cada uno con su propio ciclo de desarrollo y despliegue. Esta separación se refleja a nivel arquitectónico en el desacoplamiento entre las capas de servicio y presentación, comunicadas exclusivamente vía HTTP/REST y SSE.

5.1.1 Visión Global y Flujo de Datos

La arquitectura del sistema Entangle se organiza en cinco capas que reflejan el flujo de datos desde la extracción inicial hasta la presentación final al usuario. La Figura 5.1 ilustra esta estructura, agrupando dentro del recuadro punteado los componentes que constituyen Entangle-Core (subsistema *backend*) y representando aparte Entangle-Visualizer (subsistema *frontend*).

La primera capa, la **API de GitHub**, actúa como fuente primaria y se consume mediante dos clientes complementarios con control de *rate-limit*: el cliente **GraphQL** resuelve las consultas de búsqueda y descarga masiva, mientras que el cliente **REST** recupera los detalles individuales de cada entidad. Esta separación permite balancear cobertura y profundidad respetando los límites de tasa de la plataforma.

5. RESULTADOS

La información obtenida alimenta el ***pipeline de datos en Python***, organizado en tres bloques funcionales visibles en el diagrama: *Ingesta* (repositorios, usuarios y organizaciones identificados a partir de un vocabulario de más de 72 palabras clave cuánticas), *Enriquecimiento* (contribuidores, lenguajes, *topics* y READMEs por entidad) y *Análisis* (métricas con NetworkX [New03], detección de comunidades, clasificador disciplinar y detección de organizaciones hermanas). Las seis fases secuenciales que materializan este *pipeline* se describen en la Sección 5.3.

Los datos procesados se almacenan en **Azure Cosmos DB for MongoDB (vCore)**. La base de datos comprende ocho colecciones de dominio: cuatro principales (*repositories*, *users*, *organizations* y *metrics*), representadas en el diagrama, y cuatro operativas (*admin_config*, *ingestion_metadata*, *operation_history* y *user_preferences*); a ellas, el agente de IA suma dos colecciones de soporte (*chat_sessions* y *repos_chunks*). Todas ellas se detallan en la Sección 5.4.

El **servicio FastAPI** expone los datos al *frontend* mediante una API REST con soporte para eventos enviados por servidor (SSE), una caché permanente de métricas, documentación OpenAPI auto-generada, CORS y compresión GNU Zip (GZIP). A su lado, el **Asistente IA** aparece como bloque independiente dentro del Core: implementado sobre Azure AI Foundry con un agente de *tool-calling*, recibe peticiones a través del servicio FastAPI y consulta indirectamente la base de datos mediante las herramientas que el propio servicio le expone.

Finalmente, el ***frontend Entangle Visualizer*** (SPA en React 19 y Vite 7) presenta los datos al usuario a través del cuadro de mando 2D (Recharts), el universo 3D inmersivo (Three.js y React Three Fiber) y el chat de IA, todo ello con internacionalización a cinco idiomas. La comunicación con el *backend* se realiza siempre sobre HTTPS, transportando JSON sobre REST o eventos sobre SSE. A nivel operativo, el *backend* se ejecuta sobre Azure Container Apps y el *frontend* sobre Azure Static Web Apps, con infraestructura como código en Bicep orquestada por *azd* y un ciclo CI/CD automatizado con GitHub Actions; los detalles de despliegue se recogen en la Sección 5.11.

EXTERNO · FUENTES Y USUARIO

GitHub API
Fuente única de datos

- GraphQL + REST
- Super-query enriquecimiento
- 71 palabras clave cuánticas
- Rate-limit adaptativo + backoff

github/queries.py · rate_limit.py

USUARIO FINAL · NAVEGADOR

Investigador
explora · consulta · pregunta

Desarrollador

PIPELINE DE INGESTA Y ENRIQUECIMIENTO · CONTAINER APP JOB

1 Ingesta
repositorios · usuarios · orgs

- Discovery 71 keywords
- Pattern Repository-First
- Batch size 500 (vCore)
- Bulk upserts idempotentes

github/ingestion.py

2 Enriquecimiento
super-query GraphQL

- Contribuidores · lenguajes
- Topics · READMEs · estrellas
- Métricas de actividad
- time.sleep(0.5) · API limit

github/enrichment.py

3 Azure Cosmos DB
MongoDB API · vCore M30

- 8 colecciones de dominio (repositories · users · organizations · metrics) + operativas
- Almacén vectorial RAG (embeddings 1536d · IVF + coseno) · memoria de chat (TTL 30d) · chunked cache

MOTOR ANALÍTICO · ON-DEMAND

4 Análisis de Red
NetworkX + clasificador

- Louvain · 752 comunidades
- Betweenness · bus factor
- 6 disciplinas · bridge users
- Jenks Natural Breaks

analysis/network.py · classifier.py

5 Dashboard 2D
React 19 + Vite 7 + Recharts

- KPIs cuánticos animados
- Rankings + donuts + bridges
- Gráfico de colaboración 2D
- Filtrado cruzado global

6 Universo 3D
Three.js · React Three Fiber

- ~28,000 entidades GPU
- 6 lentes analíticas
- Tour cósmico narrativo
- InstanceMesh · shaders GLSL

components/quantum-universe/

AGENTE CONVERSACIONAL
Config-driven · Azure AI Foundry · gpt-5-mini

- Router config-driven: 5 intenciones enrutables + fallback
- 6 workers · RAG sobre READMEs · memoria conversacional
- Function-calling read-only · investigación externa · SSE

aiagent.py · aiutils.functions.py

Servicio FastAPI
Azure Container Apps · Python 3.11

- 59 endpoints REST agrupados en 4 familias
- SSE streaming · GZip (-87 % payload) · CORS · OpenAPI
- Caché 3 niveles · Async <100 ms /grafo <1 s

api/routes.py · cache.py · main.py

Chat IA
Streaming SSE en tiempo real

- Respuestas con RAG y citas reales
- Multilingüe (5 idiomas)
- Acciones: OPEN_UNIVERSE · CREATE_VIEW

AZURE STATIC WEB APPS
Despliegue automático desde GitHub

staticwebapp.config.json

DEVOPS · INFRAESTRUCTURA COMO CÓDIGO · CALIDAD CONTINUA

Bicep · IaC

Container Apps · Cosmos · SWA · AI

infra/azure.yaml (local)

GitHub Actions

C/CD continuo a Azure

5 workflows · ACA · SWA · QA

github/workflows/

SonarQube Cloud · QG

Cobertura back/front · 0 vulnerabilidades

sonar-project.properties

Suite de Pruebas

pytest + Viteest · integradas en CI

919 tests backend · 168 frontend

tests/ · tests.py | -spec.js

Backend Python/FastAPI · Frontend React/Vite · Persistencia Cosmos DB/MongoDB · vCore · IA Azure AI Foundry · gpt-5-mini · IaC Bicep · C/CD GitHub Actions · Calidad SonarQube Cloud

5.1.2 Comunicación entre Capas

La comunicación entre capas combina varios protocolos y formatos estándar. HTTP se utiliza para las operaciones de API REST entre *frontend* y *backend*, gestionado por FastAPI con un marco asíncrono que sostiene múltiples conexiones simultáneas. Para el *streaming* de datos en tiempo real, como el chat de IA, se emplea el protocolo SSE, que entrega información actualizada sin necesidad de refrescar la página. La compresión GZIP reduce significativamente el tamaño de las respuestas HTTP y los datos se transfieren en formato JSON, ampliamente compatible y fácil de manipular en ambos extremos.

5.2 Diseño del Sistema

Antes de abordar la implementación de cada componente, esta sección presenta el diseño del sistema Entangle mediante diagramas UML que formalizan su estructura y comportamiento. Los diagramas se basan en el código fuente real del proyecto.

5.2.1 Diagrama de Componentes

La Figura 5.2 muestra la descomposición del sistema en sus componentes principales y las dependencias entre ellos. El sistema se organiza en cinco capas:

1. **Fuente de datos** (GitHub API): la API GraphQL de GitHub constituye la única fuente externa de datos.
2. **Capa de ingesta** (github): seis motores especializados (tres de ingesta: `IngestionEngine`, `UserIngestionEngine`, `OrganizationIngestionEngine`; y tres de enriquecimiento: `EnrichmentEngine`, `UserEnrichmentEngine`, `OrganizationEnrichmentEngine`) extraen y enriquecen datos mediante `GitHubGraphQLClient`, supervisados por `RateLimitMonitor`.
3. **Capa de persistencia** (core): el patrón `MongoRepository` proporciona operaciones CRUD genéricas sobre MongoDB, complementado por el módulo `chunked_cache` para documentos que superan el límite de 16 MB.
4. **Capa de lógica** (analysis + ai): la clase `CollaborationNetworkAnalyzer` calcula métricas de red con `NetworkX` y el módulo `discipline_classifier` clasifica usuarios por disciplina, mientras que el módulo `agent.py` implementa la arquitectura orientada a configuración (*config-driven*) con seis *workers* especializados.
5. **Capa de presentación** (api + Frontend): FastAPI expone 59 endpoints REST agrupados en tres routers (`routes`, `admin_routes`, `chat_routes`), y el frontend React consume estos endpoints para renderizar el dashboard 2D y el universo 3D.

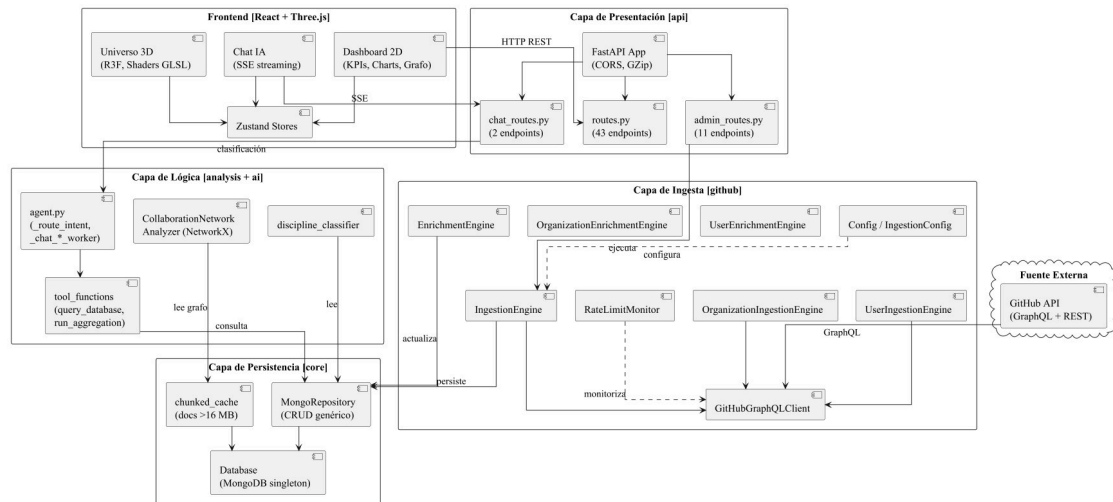


Figura 5.2: Diagrama de componentes UML del sistema Entangle.

5.2.2 Diagrama de Clases del Backend

Las Figuras 5.3, 5.4, 5.5 y 5.6 presentan las clases y módulos del *backend* organizados en seis capas, repartidas por legibilidad en cuatro hojas apaisadas: presentación, lógica IA y análisis (Figura 5.3); ingesta y enriquecimiento (Figura 5.4); persistencia y configuración (Figura 5.5); y modelos de dominio Pydantic (Figura 5.6). Los **paquetes** del *backend* (api, ai, analysis, github, core, models) emplean la notación UML estándar de paquete; las cajas con borde discontinuo y estereotipo «a Figura X» o «de Figura X» son representaciones de continuidad de una misma clase entre figuras, y el estereotipo «module» identifica módulos Python con funciones de nivel superior.

El diseño se articula en torno a tres modelos Pydantic (Repository, User, Organization) reforzados con el objeto valor EnrichmentStatus; un repositorio genérico (MongoRepository) que abstrae las operaciones sobre las diez colecciones MongoDB; y dos clases singleton (Database y Config) que gestionan el pool de conexiones y la carga de variables de entorno desde .env o Azure Key Vault.

En la capa de ingesta, seis motores especializados (IngestionEngine, UserIngestionEngine, OrganizationIngestionEngine y sus tres homólogos de enriquecimiento) componen sobre MongoRepository y GitHubGraphQLClient, supervisado por RateLimitMonitor (*backoff* exponencial con margen de 5 s al *reset*). La parametrización vive en el objeto valor IngestionConfig, apoyada en el módulo queries y en RepositoryFilters.

En la capa de lógica, agent.py clasifica las intenciones (_route_intent) y despacha los *workers* declarados en config/workers.yaml, que invocan herramientas de solo lectura registradas en tool_registry (*function calling*). La clase CollaborationNetworkAnalyzer construye el grafo NetworkX y calcula centralidad, *bus factor*, comunidades (Louvain, $Q = 0,9585$) y caminos mínimos (Dijkstra), cacheando con chunked_cache los resultados que superan los 16 MB de BSON.

Figura 5.3: Diagrama de clases UML del backend: paquetes de presentación, lógica IA y análisis.

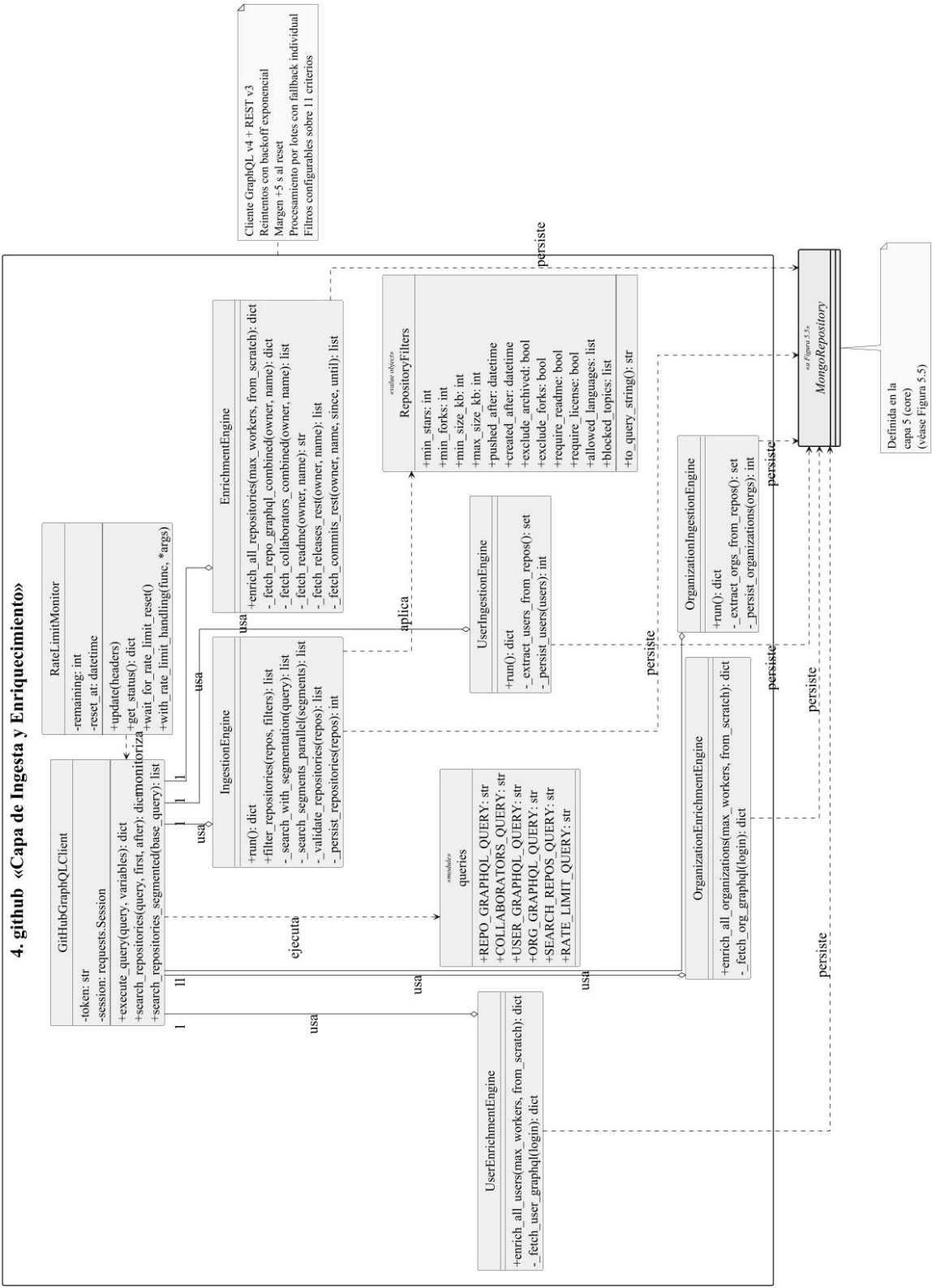


Figura 5.4: Diagrama de clases UML del backend: paquete de ingesta y enriquecimiento.

5. RESULTADOS

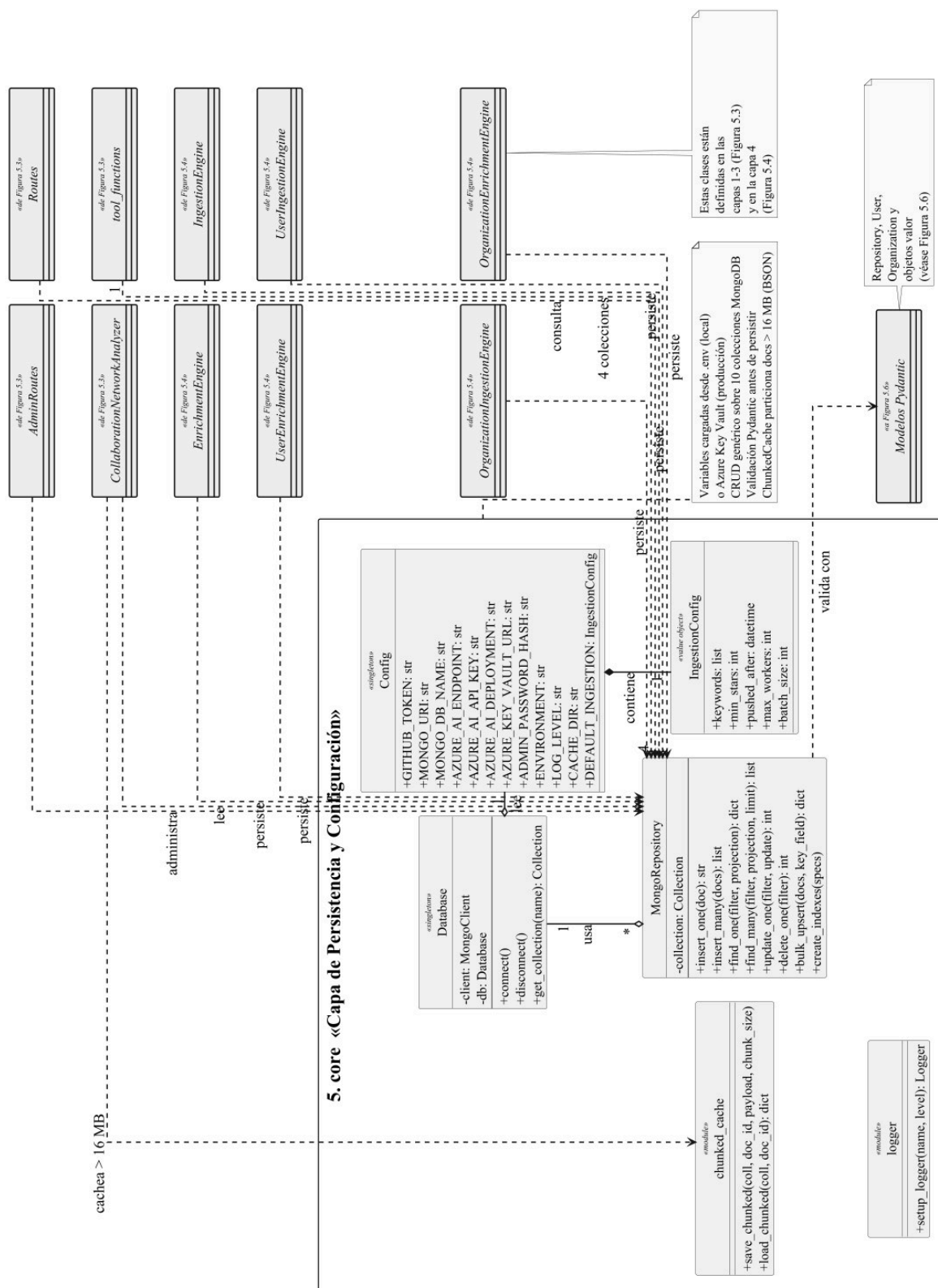


Figura 5.5: Diagrama de clases UML del backend: paquete de persistencia y configuración.

5. RESULTADOS

5.2.3 Diagrama de Secuencia: Flujo de Consulta IA

La Figura 5.7 ilustra el flujo de una consulta al agente de IA cuando el usuario envía un mensaje de tipo DATA (por ejemplo, “¿Qué porcentaje de repositorios usan Python?”). La secuencia es:

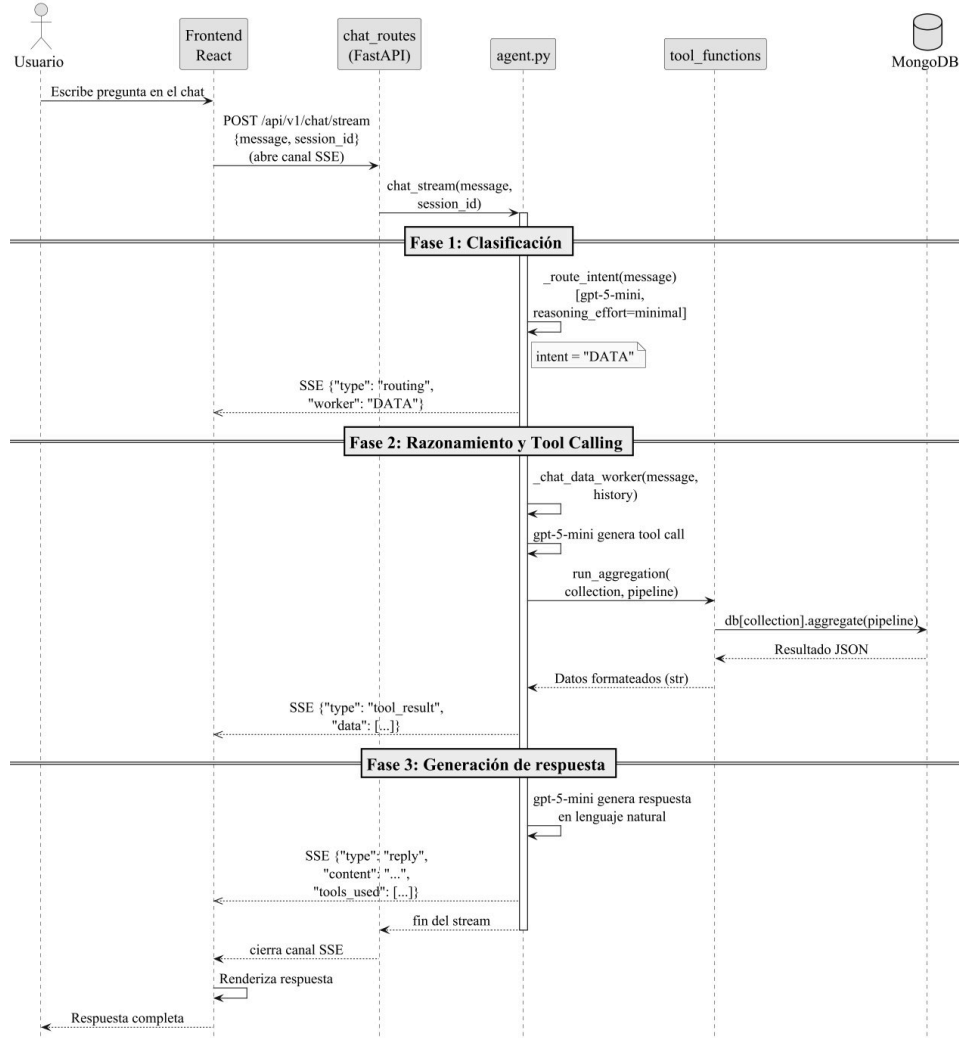


Figura 5.7: Diagrama de secuencia UML del flujo de consulta al agente de IA (tipo DATA).

1. El **Frontend** envía una petición `POST /api/v1/chat/stream` con el mensaje y el identificador de sesión.
2. El módulo **chat_routes** (FastAPI) invoca `chat_stream()` del módulo `agent.py`.
3. La función `_route_intent()` (`gpt-5-mini`, `reasoning_effort=minimal`) clasifica la intención como `DATA`.
4. El *worker* `DATA`, despachado desde la configuración (`gpt-5-mini` con *tools*), razona sobre la consulta y genera una llamada a `run_aggregation()`.
5. El módulo `tool_functions` ejecuta el *pipeline* de agregación sobre MongoDB y devuelve los resultados como cadena formateada.

6. gpt-5-mini genera la respuesta final en lenguaje natural, que se transmite al Frontend token a token mediante SSE.

5.3 Pipeline de Datos de Entangle

El *pipeline* de datos de Entangle transforma los datos crudos de GitHub en un ecosistema estructurado y analizable a lo largo de seis fases secuenciales de extracción, enriquecimiento y análisis. La arquitectura es modular: cada fase tiene un propósito específico, depende de las anteriores y puede ejecutarse de forma independiente. La orquestación se realiza desde el panel de administración o mediante *scripts* de línea de comandos, y el archivo `config/pipeline_config.json` centraliza los parámetros globales (intervalos de actualización, criterios de selección, número de *workers*).

5.3.1 Fases del Pipeline

- **Fase 1: Ingesta de Repositorios.** Búsqueda exhaustiva sobre 71 palabras clave del dominio cuántico (qiskit, cirq, pennylane, braket, quantum computing, qubit, openqasm, quantum entanglement, quantum cryptography, quantum machine learning, quantum error correction...; el listado completo se reproduce en el Anexo B), segmentando por rangos de estrellas y año de creación para superar el límite de 1.000 resultados de la *Search* API de GitHub. Cada segmento se ejecuta en paralelo con un `ThreadPoolExecutor` de cuatro *workers* y los resultados se deduplican por `nameWithOwner`. Como las palabras clave son genéricas, el *pipeline* captura inevitablemente falsos positivos (Firefox Quantum, videojuegos, librerías CSS); para mitigarlo, los *scripts* `clean_dataset.py` y `audit_dataset.py` aplican filtros heurísticos sobre *topics*, descripción y README. La ingesta es incremental: solo se incorporan repositorios nuevos o modificados desde la última ejecución.
- **Fase 2: Enriquecimiento de Repositorios.** Combinación de *super-queries* GraphQL (que obtienen lenguajes, *topics*, *issues* y *pull requests* en una sola llamada) con peticiones REST paralelas (`ThreadPoolExecutor` de 6–7 tareas simultáneas) para *commits*, *contributors*, *releases*, *branches*, *tags* y *collaborators*. La reutilización de sesiones HTTP minimiza el sobrecoste del *handshake* Transport Layer Security (TLS). Como resultado se generan métricas derivadas: *pull requests* fusionados, número de colaboradores y detalle completo de las *releases*.
- **Fase 3: Ingesta de Usuarios y Organizaciones.** Ambas entidades se extraen exclusivamente a partir de los repositorios ya enriquecidos. Los usuarios provienen del campo *collaborators* y se deduplican por su *id* de GitHub; las organizaciones se descubren mediante una agregación MongoDB que filtra repositorios cuyo *owner.type* es *Organization* y agrupa por *owner.login*. Las dos ingestas se lanzan simultáneamente (orquestador con `ThreadPoolExecutor`) y dentro de cada una las consultas GraphQL se

5. RESULTADOS

agrupan en lotes (25 usuarios o 5 organizaciones por *query*) servidos por tres *workers* concurrentes.

- **Fase 4: Enriquecimiento de Usuarios y Organizaciones.** Para los usuarios se calculan contribuciones anuales, lenguajes más utilizados y el `quantum_expertise_score` (Sección 5.9.1); la clasificación disciplinar combina cinco señales (biografía, compañía, organizaciones, *topics* de los repositorios y lenguajes utilizados). Para las organizaciones se calcula el `quantum_focus_score` (Sección 5.9.2), que pondera el porcentaje de repositorios cuánticos, la presencia de la palabra «quantum» en el nombre/descripción y el estado de verificación.
- **Fase 5: Análisis de Red y Comunidades.** NetworkX construye el grafo heterogéneo (usuarios, repositorios y organizaciones; aristas de contribución y pertenencia). Sobre él se realiza el *análisis de comunidades* mediante el algoritmo de Louvain [BGLL08], que identifica 752 comunidades distintas (Sección 3.4.3). En paralelo se calculan métricas de centralidad (intermediación y grado [New03], Sección 5.9.5), se identifican los *bridge users* (usuarios que contribuyen a repositorios de dos o más organizaciones independientes excluyendo pares hermanos, Sección 3.4.5) y los *bridge disciplinares*, cuyas contribuciones abarcan al menos dos disciplinas técnicas distintas.
- **Fase 6: Caché de Métricas.** Las estadísticas del cuadro de mando se precálculan y almacenan en una colección de métricas, lo que permite tiempos de respuesta de ~ 100 ms para las consultas más comunes. La invalidación se gestiona mediante un mecanismo de `force_refresh` que actualiza las métricas cuando se detectan cambios significativos en los datos subyacentes.

5.4 Modelo de Datos en MongoDB

El modelo de datos se implementa en MongoDB siguiendo la estrategia *Repository-First*. La Tabla 5.1 resume las ocho colecciones de dominio del sistema, agrupadas en cuatro principales y cuatro operativas; el agente de IA emplea además dos colecciones de soporte (`chat_sessions`, para la memoria conversacional, y `repos_chunks`, con los *embeddings* del RAG), recogidas en el Anexo E. Los ejemplos completos de documentos JSON para cada colección principal se encuentran en el Anexo F.

Tabla 5.1: Resumen de colecciones MongoDB del sistema Entangle.

Colección	Tipo	Propósito
repositories	Principal	Entidad raíz: identificación, lenguajes, colaboradores, commits, estado de enriquecimiento
users	Principal	Desarrolladores: contribuciones, expertise cuántica, organizaciones, perfil GitHub
organizations	Principal	Entidades organizacionales: enfoque cuántico, repositorios, lenguajes, verificación
metrics	Principal	Caché pre-calculado de KPIs, gráficos y grafo de colaboración
admin_config	Operativa	Hash de contraseña del panel de administración
ingestion_metadata	Operativa	Estadísticas de la última ejecución de cada ingesta
operation_history	Operativa	Historial de auditoría de operaciones con estado y duración
user_preferences	Operativa	Favoritos y vistas personalizadas del dashboard

5.4.1 Colecciones Principales

La colección `repositories` constituye la entidad raíz del modelo. Cada documento integra la identificación del proyecto (nombre, propietario, descripción), sus lenguajes de programación, colaboradores con contadores de contribuciones, commits recientes y un objeto `enrichment_status` que registra el progreso del enriquecimiento.

La colección `users` almacena información sobre los desarrolladores: identificación, métricas de presencia en GitHub, contribuciones desglosadas por tipo (commits, PRs, issues, reviews), repositorios cuánticos en los que participa y un `quantum_expertise_score` (0–100) que cuantifica su especialización. El campo `extracted_from` refleja la estrategia *Repository-First*.

La colección `organizations` captura las entidades organizacionales con su `quantum_focus_score` (porcentaje de repositorios cuánticos), perfil tecnológico (`top_languages`) y verificación por GitHub. El campo `discovered_from_repos` traza su origen a los repositorios.

5.4.2 Colección de Métricas

La colección `metrics` almacena el caché pre-calculado del dashboard, incluyendo Key Performance Indicator (KPI)s, gráficos, tablas y el grafo del análisis de red. El grafo de colaboración requiere un tratamiento especial: con decenas de miles de nodos, puede alcanzar 37–45 MB, superando el límite de 16 MB por documento BSON. El sistema implementa un mecanismo de *caching* chunked mediante `save_chunked()` y `load_chunked()`, que dividen los arrays grandes en fragmentos de aproximadamente 1,5 MB almacenados como documentos independientes con identificador secuencial. En la práctica, un grafo de ~31.000 nodos se

5. RESULTADOS

distribuye en 25–30 documentos. La caché del grafo se invalida manualmente tras cada ejecución del pipeline, mientras que las métricas restantes se actualizan periódicamente mediante `force_refresh`.

5.4.3 Relaciones entre Colecciones

Los repositorios actúan como punto de partida para descubrir usuarios (a través de sus colaboradores) y organizaciones (a través de sus propietarios), garantizando que todas las entidades indexadas sean relevantes para el ecosistema cuántico.

Aunque MongoDB es una base de datos documental sin restricciones de integridad referencial, las cuatro colecciones principales mantienen relaciones conceptuales que el código del *backend* aplica de forma explícita. La Figura 5.8 representa este modelo conceptual: la colección *repositories* es la entidad raíz, alimenta a *users* (a través del array `collaborators`) y a *organizations* (a través del campo `owner.login` / `owner.id`); a su vez, *users* y *organizations* se cruzan en una relación *muchos-a-muchos* embebida (un usuario puede pertenecer a varias organizaciones, y una organización agrupa a múltiples usuarios), y la colección *metrics* se construye agregando información de las tres anteriores. Las relaciones se mantienen mediante *identificadores numéricos de GitHub* (no por `login`, sujeto a renombrados) y se materializan en consultas `find`, `aggregate` con `$lookup` y operaciones idempotentes de *bulk upsert*. En el diagrama, las flechas continuas representan relaciones de origen (*Repository-First*) con sus cardinalidades, y las líneas discontinuas con rombo abierto en *metrics* expresan agregación UML.

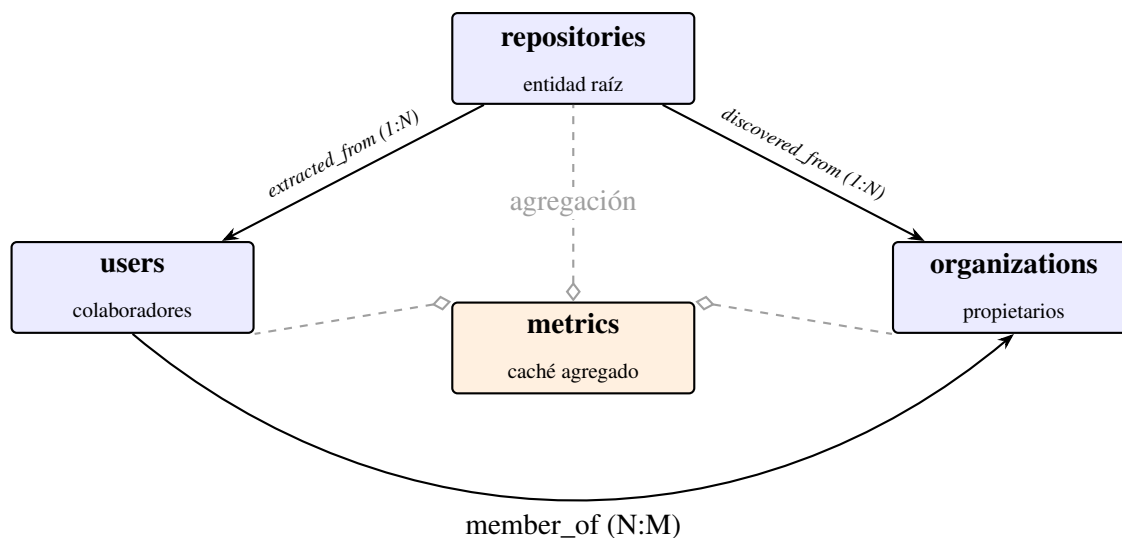


Figura 5.8: Modelo conceptual de relaciones entre las colecciones principales de MongoDB.

5.5 Backend API REST con FastAPI

Esta sección presenta la API REST desarrollada con FastAPI, describiendo su estructura modular, los 56 *endpoints* expuestos y los patrones de diseño empleados para garantizar eficiencia y mantenibilidad. El catálogo completo de *endpoints* se reproduce en el Anexo D.

5.5.1 Visión General de la API

El *backend* se desarrolla con FastAPI [Ram24], un *framework* moderno de alto rendimiento que aprovecha el modelo asíncrono ASGI y genera automáticamente documentación OpenAPI. La estructura modular se organiza en directorios con responsabilidades específicas: *api/* (rutas y *endpoints*: *main.py*, *routes.py*, *chat_routes.py*), *ai/* (lógica del agente: *agent.py*, *prompts.py*, *tool_functions.py*) y *core/* (configuración, conexión a base de datos, *logging* y repositorio de MongoDB).

5.5.2 Endpoints de la API

Los 59 *endpoints* de la API se agrupan en cuatro familias funcionales: *endpoints* generales (información, estado y conteos del sistema), de entidades (organizaciones, repositorios y usuarios con sus respectivos listados y detalles), de colaboración (grafo, análisis de red y métricas globales) y de IA (chat síncrono y en *streaming*, investigación externa y gestión de la memoria de sesión). La Tabla 5.2 recoge una muestra representativa de los *endpoints* más significativos; el catálogo completo se reproduce en el Anexo D.

Tabla 5.2: Endpoints principales de la API REST de Entangle (muestra representativa).

Familia	Método y ruta	Propósito
General	GET /	Información básica
General	GET /health	Verificación de estado
General	GET /api/v1/stats	Conteos globales
Entidades	GET /api/v1/organizations	Listado de organizaciones
Entidades	GET /api/v1/organization/{login}	Detalle de una organización
Entidades	GET /api/v1/repositories	Listado de repositorios
Entidades	GET /api/v1/repositories/{owner}/{name}	Detalle de un repositorio
Entidades	GET /api/v1/users	Listado de usuarios
Entidades	GET /api/v1/users/{login}	Detalle de un usuario
Colaboración	GET /api/v1/discover	Grafo de colaboración
Colaboración	POST /api/v1/collaboration/analyze	Análisis de red profundo
Colaboración	GET /api/v1/network/metrics	Métricas globales de la red
IA	POST /api/v1/chat	Respuesta síncrona
IA	POST /api/v1/chat/stream	Respuesta <i>streaming</i> (SSE)
IA	POST /api/v1/chat/research	Investigación externa (<i>worker</i> DEEP_RESEARCH)

Todos los *endpoints* de entidades admiten filtros (lenguaje, disciplina, enfoque cuántico...) y paginación. La arquitectura orientada a configuración que sostiene los *endpoints* de IA se describe en la Sección 5.6.

5.5.3 Filtros, Compresión y Caché

El *sistema de filtros* se materializa mediante *query parameters* (`org`, `language`, `repo`, `discipline`, `include_bots`, `collab_type`...) que pueden aplicarse de forma combinada para refinar las consultas con un mínimo acoplamiento. La *compresión* GZIP se inyecta vía `GZipMiddleware` de FastAPI sobre toda respuesta superior a 1 KB; el grafo de colaboración, que ronda los 22 MB sin comprimir, se reduce a 2–3 MB, lo que mejora notablemente los tiempos de carga y el consumo de ancho de banda. Finalmente, la *caché* sobre MongoDB sostiene tiempos de acceso de ~ 100 ms para las consultas más frecuentes; las invalidaciones se gestionan mediante el parámetro `force_refresh=true`, que provoca el recálculo (hasta 35 s en operaciones de red completas) garantizando la precisión de los datos.

5.6 Agente de IA Conversacional

Esta sección describe el diseño e implementación del agente de IA conversacional, que evoluciona el patrón Router-Worker clásico [WMF⁺24] hacia una arquitectura orientada a configuración (*config-driven*) con cinco intenciones enrutables, RAG, memoria conversacional persistente y acceso controlado a fuentes externas. Se apoya en Azure AI Foundry con el modelo `gpt-5-mini` y transmite las respuestas mediante SSE. El detalle técnico de cada capacidad y las capturas reales se recogen en el Anexo K.

5.6.1 Arquitectura Orientada a Configuración

El agente conserva la separación entre un *router*, que clasifica la *intención* del mensaje, y un conjunto de *workers* especializados, que la resuelven. Conviene distinguir ambos conceptos: la *intención* es la categoría que predice el *router*, mientras que el *worker* es el componente que la ejecuta. La diferencia esencial respecto a un enfoque *hardcoded* es que los *workers* no se codifican individualmente en el módulo de enrutado, sino que se declaran en un único fichero de configuración (`config/workers.yaml`): cada uno especifica su *prompt*, sus herramientas, su *temperature* y su esfuerzo de razonamiento (`reasoning_effort`). Las herramientas se registran mediante un decorador `@tool` en un registro central (`tool_registry.py`), de modo que añadir un nuevo *worker* o una capacidad no requiere modificar el motor de enrutado: basta con editar la configuración. Esto reduce el acoplamiento y evita el coste por ejecución y la dependencia de proveedor de los servicios gestionados de orquestación de agentes: la orquestación (clasificación y enrutado) se ejecuta en el propio *backend* de Entangle, y solo la inferencia del modelo se delega en Azure AI Foundry. El *router* emplea `gpt-5-mini` con `reasoning_effort=minimal` para clasificar cada mensaje en una de **cinco intenciones** enrutables, cada una asociada a un *worker* homónimo; un **sexto** *worker*, `DEEP_RESEARCH`, no es enrutable y solo se activa mediante una acción explícita del usuario (Sección 5.6.2), de modo que el agente dispone de seis *workers* en total. El *fallback* a `DATA` resuelve los casos ambiguos.

El Listado 5.1 recoge la función de clasificación de intención del *router*: a partir de la configuración declarativa, invoca gpt-5-mini con un límite de salida reducido para acotar la respuesta a una sola palabra y normaliza el resultado contra el conjunto de intenciones válidas, con *fallback* determinista a DATA ante cualquier error o respuesta no reconocida.

```

1  def _route_intent(user_message: str) -> str:
2      router = _agent_config().router # configuración declarativa
3      payload = _normalize_payload({
4          "messages": [
5              {"role": "system", "content": router.prompt},
6              {"role": "user", "content": user_message},
7          ],
8          "max_completion_tokens": router.max_completion_tokens,
9          "reasoning_effort": router.reasoning_effort, # "minimal"
10     })
11     try:
12         data = _api_call_with_retry(_build_api_url(), payload)
13         raw = (
14             data.get("choices", [{}])[0]
15             .get("message", {})
16             .get("content", "")
17             .strip().upper()
18         )
19         return raw if raw in router.intents else "DATA"
20     except Exception:
21         return "DATA" # fallback determinista

```

Listado 5.1: Router de clasificación de intención con gpt-5-mini.

5.6.2 Workers Especializados y Capacidades

El agente integra seis *workers*, cuya función y herramientas se resumen en la tabla del Anexo K. Tres constituyen el núcleo de consulta y guía de la interfaz, y los tres restantes incorporan las capacidades avanzadas del sistema:

- **DATA.** Único *worker* con acceso de consulta **de propósito general** a MongoDB: mediante *function calling*, el modelo formula directamente consultas *find* y *aggregation pipelines*, siempre con herramientas estrictamente de solo lectura (véase la Figura K.1).
- **DASHBOARD y UNIVERSE.** Responden a partir del conocimiento embebido en sus *prompts* y pueden emitir acciones que el *frontend* ejecuta automáticamente (OPEN_UNIVERSE, CREATE_VIEW).
- **KNOWLEDGE.** Responde preguntas cualitativas («¿en qué se diferencian Qiskit y Cirq?») mediante una canalización de RAG [LPP⁺20] sobre los ficheros README, detallada en la Sección 5.6.3, de modo que responde **citando fragmentos reales** (Figura 5.15).
- **INSIGHTS.** Reutiliza esa infraestructura vectorial para análisis cruzados: comparativas, similitud y solapamiento de contribuidores mediante el índice de Jaccard.

5. RESULTADOS

- **DEEP_RESEARCH.** Accesible solo mediante una acción explícita del usuario, consulta arXiv y la web (vía Tavily) bajo una política estricta de honestidad de fuentes.

A estas capacidades se suma una **memoria conversacional** persistente: cada sesión se identifica con un Universally Unique Identifier (UUID) y se almacena como documento independiente en la colección `chat_sessions` de Cosmos DB, con un índice Time To Live (TTL) que expira las sesiones inactivas a los 30 días; el identificador se conserva en el *localStorage* del navegador, de modo que el contexto sobrevive a recargas de página y permite resolver referencias anafóricas (Figura K.7). El detalle técnico de la memoria y de la investigación externa, junto con capturas reales, se documenta en el Anexo K.

5.6.3 Canalización de RAG y Búsqueda Semántica

El *worker* KNOWLEDGE fundamenta sus respuestas en una canalización de RAG [LPP⁺20], que mitiga las alucinaciones características de los LLM [JLF⁺23] al obligar al modelo a responder a partir de fragmentos reales de la documentación en lugar de apoyarse únicamente en el conocimiento paramétrico adquirido durante su preentrenamiento [BMR⁺20]. El indexado se ejecuta de forma idempotente sobre los README de los 1.565 repositorios del *dataset* y comprende cuatro fases:

1. **Fragmentación.** Cada README se divide en bloques de tamaño acotado respetando la estructura *Markdown* (encabezados, listas y bloques de código), de modo que cada fragmento conserve un contexto semántico coherente.
2. **Vectorización.** Cada fragmento se proyecta en un vector de 1536 dimensiones mediante el modelo `text-embedding-3-small` de Azure AI Foundry, basado en la arquitectura *transformer* [VSP⁺17], que codifica el significado del texto en un espacio continuo.
3. **Almacenamiento.** Los vectores se persisten en una colección dedicada de Cosmos DB, indexada con el algoritmo Inverted File Index (IVF) y distancia coseno. Reutilizar el clúster existente como almacén vectorial, en lugar de un servicio de búsqueda dedicado, evita infraestructura y coste adicionales.
4. **Búsqueda semántica.** En tiempo de consulta, la pregunta del usuario se vectoriza con el mismo modelo y se recuperan por similitud coseno los fragmentos más afines, que se inyectan en el contexto del modelo para fundamentar una respuesta **con citas a las fuentes**.

El corpus resultante asciende a unos 11.000 fragmentos. El *worker* INSIGHTS reutiliza esta misma infraestructura vectorial para sus análisis cruzados (similitud entre repositorios y comparativas), combinándola con consultas estructuradas sobre MongoDB. Los aspectos de implementación se detallan en el Anexo K.

5.6.4 Streaming de Razonamiento y Seguridad

La transmisión mediante SSE (*Server-Sent Events*) permite una experiencia de chat fluida: el sistema emite en tiempo real los hitos del ciclo de vida de cada consulta (clasificación del *router*, selección del *worker*, invocación de herramientas y resultados) y, a continuación, transmite el texto de la respuesta **token a token** conforme gpt-5-mini lo genera. El *frontend* de React + Vite renderiza las respuestas con react-markdown, las fórmulas con KaTeX y los bloques de código con resaltado de sintaxis. La seguridad del agente se ha abordado por diseño, siguiendo el principio de mínimo privilegio: las herramientas del *worker* DATA son estrictamente de solo lectura y run_aggregation bloquea de forma explícita los operadores capaces de modificar datos (\$out, \$merge) antes de ejecutar el *pipeline*, lo que constituye una defensa frente a ataques de *prompt injection* [OWA25]; además, los *prompts* de todos los *workers* prohíben revelar el *system prompt* e inyectar instrucciones, y el acceso a fuentes externas se aísla en un worker no enrutable. La capa de seguridad de la infraestructura se amplía en la Sección 5.10.3.

5.7 Frontend: Dashboard 2D Interactivo en React

Esta sección detalla la implementación del dashboard 2D interactivo desarrollado en React, describiendo cada componente visual y las interacciones que proporciona al usuario para explorar el ecosistema cuántico. **Material visual:** galería completa de capturas en el Anexo L; aplicación analítica de estas capturas en la Sección 5.12.

5.7.1 Tarjetas de Indicadores Clave de Rendimiento

La sección de Key Performance Indicator (KPI)s del dashboard presenta tres tarjetas interactivas que muestran métricas clave relacionadas con el ecosistema cuántico: repositorios, colaboradores y organizaciones. Cada tarjeta está diseñada con una estética cuántica que incluye esferas de Bloch animadas, reflejando conceptos de la mecánica cuántica. Al pasar el cursor sobre las tarjetas, se activa un efecto de «wave function collapse», proporcionando una experiencia visual dinámica que simula el colapso de una función de onda (este efecto se ilustra más adelante en la Figura L.2).

Estas tarjetas presentan datos estáticos. Además, incorporan conteos animados que utilizan un efecto de ease-out para suavizar la transición de los números. Además, cuando se aplican filtros globales, las tarjetas indican el estado filtrado mediante un badge `|FILTERED>`, conectando visualmente las tarjetas con líneas de entrelazamiento SVG por las que viajan partículas animadas, lo que mejora la interacción del usuario al mostrar claramente cómo los filtros afectan los datos visualizados.

5.7.2 Charts Section

La sección de gráficos utiliza Recharts para proporcionar cinco visualizaciones interactivas que cubren las distintas dimensiones del ecosistema cuántico:

5. RESULTADOS

- **Top Organizaciones** (BarChart horizontal): permite seleccionar entre cinco métricas mediante un dropdown: quantum focus score, número de repositorios cuánticos, estrellas totales, colaboradores únicos y usuarios compartidos entre organizaciones.
- **Top Repositorios** (BarChart horizontal): ordena los repositorios por estrellas, forks, colaboradores o colaboradores compartidos. Cuando el usuario selecciona un repositorio concreto, el gráfico de lenguajes queda oculto y se muestra el lenguaje principal del repositorio seleccionado directamente en su tarjeta.
- **Top Usuarios** (BarChart vertical): ofrece doble filtrado. La métrica principal alterna entre score de colaboración y contribución multi-repositorio, mientras que un segundo selector limita por tipo de colaborador (todos, con commits o reviewers). Un toggle adicional permite excluir cuentas de bots (dependabot, github-actions, etc.).
- **Distribución de Lenguajes** (PieChart donut): muestra los seis lenguajes más utilizados con colores diferenciados, agrupando el resto bajo la categoría «Otros». Al hacer clic en «Otros», un panel expandible muestra el detalle completo de todos los lenguajes menores.
- **Distribución por Disciplina** (PieChart donut + panel de bridge profiles): clasifica a los usuarios en seis categorías disciplinarias (Quantum Software, Quantum Physics, Quantum Hardware, Dev Tooling, Education & Research y Multidisciplinary), cada una con un color y emoji identificativo. Junto al donut se muestra un panel de bridge profiles: usuarios que cruzan múltiples disciplinas, ordenados por ranking y con tags indicando sus repositorios por disciplina (véase la Figura L.3).

Todos los gráficos se renderizan con animaciones de entrada controladas por un Intersection Observer (threshold del 30 %): la animación de Recharts (`isAnimationActive`) se activa solo cuando el gráfico es visible en el *viewport*, con duraciones de 600 ms para barras y 800 ms para *donuts*. Los *tooltips* se personalizan según el tipo de entidad (avatar, descripción, contadores y desglose de contribuciones según corresponda) y la interactividad se estructura en tres niveles de clic: clic simple activa el filtrado cruzado global, Shift+clic abre un panel lateral de detalle y Ctrl+clic habilita la multi-selección comparativa. Todos los gráficos están integrados con el *store* de Zustand para que cualquier selección actualice dinámicamente el resto de componentes del cuadro de mando.

5.7.3 Network Graph

El grafo de red 2D visualiza las relaciones de colaboración entre organizaciones, repositorios y usuarios mediante SVG puro, sin librerías de simulación de fuerzas. El componente emplea un *layout* circular jerárquico por sectores donde cada organización seleccionada recibe un sector angular proporcional. Dentro de cada sector se distribuyen tres anillos concéntricos: **anillo exterior** con las organizaciones (cian, círculos de 11 px); **anillo medio** con los repositorios de cada organización (morado, hexágonos escalados por $\sqrt{\text{stars}}$ entre

3–9 px, ordenados por número de contribuidores); **anillo interior** con los usuarios de una sola organización (verde, círculos de 4–9 px escalados por centralidad de colaboración); y **centro** con los *bridge users* (ámbar, círculos con halo discontinuo, 5–12 px escalados por centralidad y distribuidos en tres anillos concéntricos internos). Las aristas se diferencian visualmente por tipo: las contribuciones (usuario → repositorio) son curvas Bézier cuadráticas en verde tenue; las relaciones de propiedad (organización → repositorio) en morado tenue; y los vínculos inter-organizacionales (*entangled_with*) líneas rectas discontinuas en cian brillante con animación de brillo pulsante de 4 s. El resultado de este *layout* circular jerárquico se observa más adelante en la Figura L.4.

El grafo requiere seleccionar al menos dos organizaciones mediante un *dropdown* multi-selector con búsqueda *fuzzy* que recomienda inicialmente las cinco con mayor *score* de colaboración. Una vez renderizado, el usuario dispone de tres niveles de detalle (compacto, normal, detallado) que controlan el número máximo de repositorios (4/7/12), usuarios (3/5/8) y *bridge users* (6/12/20) visibles por organización. La interacción ofrece dos modos: el *hover* despliega un *tooltip* HTML con contenido adaptado al tipo de entidad (avatar, centralidad, repositorios activos y riesgo de *bus factor* para usuarios; lenguaje principal, estrellas y *forks* para repositorios; avatar, *badges*, número de *bridge users*, *quantum focus* y lenguajes principales para organizaciones); el clic activa el modo *focus* (el nodo crece, los no conectados se atenúan a opacidad 0,12 y se disparan pulsos animados a lo largo de las aristas conectadas), con una barra inferior que indica el nodo enfocado y su número de conexiones. Sobre las aristas se simula un flujo de energía mediante pulsos: ráfagas iniciales de 12 pulsos cada 100 ms, otros 8 adicionales y posteriormente pulsos ambientales cada 2,5–6,5 s, donde cada pulso es un punto de 2,5 px con estela animada heredando el color del nodo origen. El centro del grafo presenta un modelo atómico decorativo con tres electrones orbitando en trayectorias elípticas (12, 16 y 20 s) que refuerza la estética cuántica del cuadro de mando.

5.7.4 Otros Componentes del Cuadro de Mando

Los componentes restantes del cuadro de mando se sintetizan a continuación; su estructura es paralela a la de los componentes anteriores y comparte el mismo *store* de Zustand.

- **Filtros globales y favoritos.** Filtros combinables por organización, lenguaje, repositorio y disciplina, activables desde selectores dedicados o haciendo clic en elementos de los gráficos, con actualización propagada a todo el cuadro de mando. La funcionalidad de favoritos permite marcar entidades y agruparlas en *vistas* personalizadas; estas se persisten en la colección *user_preferences* de MongoDB a través de *endpoints* REST dedicados, con caché local en *localStorage*.
- **Estética y sistema de diseño cuántico.** Sistema de diseño centralizado mediante variables CSS en `:root`: paleta con fondo #0f1419, acentos cian (#00D4E4) y púrpura

5. RESULTADOS

(#9D6FDB), tipografía Inter/JetBrains Mono y *glassmorphism* con `backdrop-filter` (4–24 px). Varios componentes incorporan metáforas cuánticas: esferas de Bloch animadas en las KPI, modelo atómico central en el grafo y un `QuantumBackground` con 60 partículas flotantes conectadas por líneas de entrelazamiento.

- **Herramientas de administración y desarrollo.** Dos paneles protegidos por contraseña `bcrypt` con 12 rondas de *salt*: el *Panel de Administración* (`Ctrl+Shift+A`) orquesta el *pipeline* de datos (ingestas, enriquecimientos, ejecución completa), monitoriza operaciones activas con barra de progreso y terminal de *logs* en *streaming*, consulta el historial y gestiona la configuración de la base de datos; el *Menú Dev* (`Ctrl+Shift+D`) alterna individualmente los 19 componentes del cuadro de mando organizados en cinco categorías (con activación por componente, categoría o bloque), y persiste el estado en `localStorage` con migración automática entre versiones.
- **Internacionalización.** Interfaz internacionalizada mediante `i18next` [Moz24b] con cinco idiomas (*en*, *es*, *fr*, *de*, *pt*) y paridad total de $\sim 1\,086$ claves por idioma. La detección sigue el orden `localStorage` $\rightarrow$ idioma del navegador $\rightarrow$ inglés por defecto. Las claves se organizan en espacios de nombres lógicos (`app`, `nav`, `kpi`, `charts`, `admin`, `chat`, `universe...`) y emplean interpolación de variables; la cobertura alcanza veinte componentes React.

5.8 Universo 3D: Visualización Tridimensional Inmersiva del Ecosistema Cuántico

Esta sección describe la implementación del Universo 3D inmersivo: la tecnología base, la metáfora cuántica que guía la representación visual de cada entidad y los algoritmos que sustentan la distribución espacial y la interacción. **Material visual:** efectos visuales detallados, guion del Tour Cósmico y galería de capturas en el Anexo J; aplicación analítica en la Sección 5.13.

5.8.1 Tecnología Base

El Universo 3D se construye sobre `Three.js` integrado con `React Three Fiber`, un binding declarativo que permite definir la escena como componentes React [Cab24]. La librería `@react-three/drei` aporta utilidades como `OrbitControls` para la navegación orbital y `HTML` para superponer elementos Document Object Model (DOM) sobre la escena [pmn24a]. Todo el cálculo de posiciones se delega a un `Web Worker` dedicado que ejecuta el algoritmo de distribución fuera del hilo principal, evitando bloqueos durante la carga inicial de las ~ 28.000 entidades del subgrafo conectado. La escena se renderiza íntegramente mediante `InstancedMesh` y `ShaderMaterial` con shaders OpenGL Shading Language (GLSL) personalizados: gracias a la instanciación, las más de 27.000 partículas de usuarios, 1.500 cúbits de repositorios y más de 400 procesadores de organizaciones se dibujan en apenas cuatro *draw calls*.

5.8.2 Metáfora Cuántica: Entidades como Objetos Físicos

Cada tipo de entidad del ecosistema se representa como un análogo de la física cuántica (las definiciones formales de los conceptos físicos invocados se recogen en el Anexo I):

- **Procesadores cuánticos** (organizaciones): toros concéntricos en cian (#00f7ff) que envuelven una esfera central, emulando campos toroidales de energía confinada.
- **Cúbits** (repositorios): octaedros violeta cuyo tamaño escala con $\text{stars}/800$, rodeados de diez partículas que orbitan en trayectorias esféricas simulando **nubes de probabilidad** $\psi(r, \theta, \varphi)$ (Figura J.5).
- **Partículas cuánticas** (usuarios): puntos verdes (#00ff9f) o dorados (#ffbd00) para los *bridge users*, renderizados en la Graphics Processing Unit (GPU) y sometidos a una **vibración de incertidumbre de Heisenberg** (oscilaciones sinusoidales independientes en los tres ejes).

La correspondencia completa entre concepto físico y elemento visual se recoge en la Tabla J.1 (Anexo J, Sección J.1).

5.8.3 Vínculos y Canales

Las relaciones entre entidades se materializan mediante tres tipos de conexiones: **Quantum Bonds** (dobles hélices estilo ADN que conectan cada cúbit con sus contribuyentes), **canales de entrelazamiento** (ondas sinusoidales púrpura entre organizaciones colaboradoras) y **arcos de entrelazamiento organizacional** (curvas de Bézier cuadráticas con degradado cian-púrpura y un pulso viajero smoothstep). Sobre los canales de entrelazamiento viajan **pulsos de tunneling**: hasta 25 esferas que se desplazan describiendo una onda transversal y se desvanecen suavemente, apareciendo solo tras un retardo de 7 s para no saturar la carga inicial.

5.8.4 Algoritmo de Distribución

El algoritmo de distribución se basa en un generador pseudo-aleatorio determinista con semilla fija (`seededRandom(42)`) que garantiza una disposición reproducible. El proceso se divide en cinco fases:

1. Construcción del grafo de colaboración a partir de los datos de la API, estableciendo conexiones entre organizaciones, repositorios y usuarios.
2. Cálculo de la *betweenness centrality* de cada nodo para identificar las entidades más influyentes [Fre77].
3. Posicionamiento de las organizaciones a distancias logarítmicas del centro del universo según su importancia relativa.
4. Distribución de los repositorios en órbitas toroidales alrededor de su organización matriz.

5. RESULTADOS

5. Colocación de los usuarios en sub-órbitas alrededor de los repositorios, reflejando su nivel de contribución.

5.8.5 Zonas de Densidad

La distribución espacial de las organizaciones genera zonas de concentración variable que el sistema clasifica automáticamente mediante el algoritmo de *Jenks Natural Breaks* [Jen67]. Este método de clasificación cartográfica minimiza la varianza intra-clase y maximiza la inter-clase, encontrando puntos de corte naturales sin umbrales arbitrarios; su complejidad $O(n^2 \cdot k)$ es trivial para las ≈ 400 organizaciones y $k = 3$ clases. El algoritmo recibe el vector de radios objetivo y produce dos fronteras que dividen el espacio en tres zonas concéntricas: **Core** (alta densidad, cerca del centro), **Mid** (densidad media) y **Peripheral** (baja densidad, en la frontera del universo). Cuando hay menos de seis organizaciones colaborativas se aplica un *fallback* de proporciones fijas (25 % y 60 % del radio máximo). El mismo algoritmo se reutiliza en el *detail worker* sobre las métricas `collab centrality raw` y `collab connectivity raw` para clasificar entidades en niveles *high/mid/low* cuya combinación matricial 3×3 determina el rol de red de cada nodo.

5.8.6 Capa Analítica y Narrativa

Sobre la distribución base se superponen tres capas de carácter analítico y narrativo:

- **Lentes analíticas.** Filtros visuales que resaltan dimensiones específicas del ecosistema. Se han implementado seis: *Comunidades* (752 grupos detectados por Louvain [BGLL08], coloreados con el ángulo dorado HSL); *Centralidad* (gradiente azul-cian según la *betweenness centrality*); *Resiliencia* (cuatro niveles de color basados en el *bus factor* de cada repositorio); *Intensidad* (gradiente rojo-amarillo según la *degree centrality*); *Disciplinas* (seis categorías con colores fijos); y *Quantum Tunneling*, que en lugar de colorear el universo calcula el camino más corto entre dos nodos mediante el algoritmo de Dijkstra [Dij59] (Sección 3.4.4) y lo anima sobre los arcos de entrelazamiento, revelando cadenas indirectas de colaboración (Figura J.11). Las capturas y la galería completa de fronteras de densidad se incluyen en el Anexo J.
- **Cinematografía y Tour Cósmico.** Dos animaciones intrínsecas a la vista, automáticas, y un recorrido narrativo opcional. El *Quantum Genesis* (*Big Bang*, 4 s) ejecuta la entrada con datos reales: los nodos del universo parten de la singularidad e interpolan su posición final con `easeOutCubic` sobre un destello cinematográfico y cinco anillos de onda de choque (Figura J.12). El *Black Hole Collapse* (5,5 s) ejecuta la salida con un colapso gravitacional (*photon ring*, *accretion disk* y `clip-path` con oscilación amortiguada hasta el cierre). Independientemente, el **Tour Cósmico** es un recorrido opcional de 12–16 *waypoints* generados a partir de los datos reales del ecosistema (12–18 s por *waypoint*, controlado con $\rightarrow$ /Espacio, $\leftarrow$ o un botón de salto rápido). Los detalles cinemáticos y el guion completo se recogen en el Anexo J, Sección J.5.

- **Efectos visuales y fenómenos cuánticos.** Capas adicionales que refuerzan la metáfora cuántica: *vacío cuántico* (rejilla *lattice* con 400 partículas de fluctuación virtual), *campo de interferencia* (600 partículas formando ondas estacionarias), *Quantum Foam* (dos armónicos superpuestos), *ondas de decoherencia* (anillos expansivos cada 8 ± 6 s desde los procesadores), *rayos cósmicos ribbon*, *anillos de energía* toroidales y una *Dyson Shell* (icosaedro subdividido ≈ 1.280 triángulos, radio 3.500 u) que envuelve todo el universo conduciendo 12 pulsos de energía por sus aristas. Sobre la escena se aplican efectos de post-procesado (*bloom* y *depth of field*). Los parámetros numéricos exactos y las capturas se documentan en el Anexo J, Sección J.6.

5.9 Métricas e Indicadores Clave del Proyecto Entangle

Esta sección formaliza las métricas y fórmulas que cuantifican la especialización cuántica de usuarios, repositorios y organizaciones, así como las propiedades de red derivadas del grafo de colaboración. Los ejemplos numéricos completos y el análisis de sensibilidad detallado se presentan en el Anexo G.

5.9.1 Métricas de Usuarios

La métrica `quantum_expertise_score` evalúa la especialización cuántica de cada usuario combinando propiedad, colaboración, reconocimiento (estrellas) y pertenencia institucional. Sea R_o el número de repositorios cuánticos propios del usuario, R_c los repositorios cuánticos en los que colabora, S_q las estrellas acumuladas en su portfolio cuántico, C_q las contribuciones cuánticas registradas y O_q las organizaciones cuánticas a las que pertenece. La puntuación se define como:

$$Score = 5 \cdot R_o + 2 \cdot R_c + \min(0,1 \cdot S_q, 50) + \min(0,05 \cdot C_q, 25) + 10 \cdot O_q \quad (5.1)$$

El resultado se acota a 100. Los pesos son heurísticas calibradas empíricamente sobre el dataset de Entangle: la propiedad (R_o , peso 5) prevalece sobre la colaboración (R_c , peso 2); las estrellas S_q se penalizan con un factor de 0,1 y un techo de 50 puntos para evitar que un único repositorio viral distorsione el ranking; y la pertenencia a organizaciones cuánticas (O_q , peso 10) se premia para reflejar integración institucional. La calibración produce una distribución que discrimina entre contribuidores ocasionales ($score < 20$) y figuras centrales del ecosistema ($score > 70$), un enfoque consistente con la construcción de métricas de reputación en plataformas colaborativas [APHV16].

5.9.2 Métricas de Organizaciones

El `quantum_focus_score` mide el enfoque cuántico de las organizaciones. Sea R_q el número de repositorios cuánticos de la organización, R_t el total de sus repositorios, $\mathbb{K}_q$ una variable

5. RESULTADOS

indicadora que vale 1 si la palabra «quantum» aparece en el nombre o descripción y 0 en caso contrario, y $\mathbb{K}_v$ una variable indicadora que vale 1 si la organización está verificada por GitHub y 0 en caso contrario. La fórmula es:

$$Score = \left[\min \left(\frac{R_q}{R_t} \cdot 100 + 10 \cdot \mathbb{K}_q, 100 \right) \right] \cdot (1 + 0,2 \cdot \mathbb{K}_v) \quad (5.2)$$

Como en el caso anterior, los pesos son heurísticas calibradas: el ratio R_q/R_t proporciona la base proporcional, $\mathbb{K}_q$ aporta un bono de 10 puntos por presencia léxica explícita y $\mathbb{K}_v$ aplica un multiplicador de 1,2 a las organizaciones verificadas. El resultado intermedio se acota a 100 antes de aplicar el multiplicador.

5.9.3 Análisis de Sensibilidad de los Pesos

Para evaluar la robustez de ambas heurísticas se realizó un análisis de sensibilidad perturbando cada peso un ± 20 % sobre siete perfiles arquetípicos por métrica (32 combinaciones extremas). El `quantum_expertise_score` mantuvo el top-3 en el 84,4 % de los casos (estabilidad global del 81,2 %), y el `quantum_focus_score` en el 100 % de las combinaciones. El detalle, ejemplos numéricos y un caso límite con perturbaciones del 400 % se presentan en el Anexo G.

5.9.4 Métricas de Repositorios

El `bus_factor` evalúa la dependencia de un repositorio en pocos desarrolladores. Se define como el mínimo número de contribuidores necesarios para acumular al menos el 50 % de las contribuciones; un valor 1 indica un único responsable y, por tanto, riesgo elevado.

5.9.5 Métricas de Red de Colaboración

Esta última categoría cuantifica la estructura del grafo de colaboración del ecosistema, esto es, la red de relaciones entre usuarios, repositorios y organizaciones: mide tanto su conectividad global como su organización interna en comunidades. La `network_density` mide la conectividad del grafo de colaboración. Sea $|E|$ el número de aristas (edges) y $|V|$ el número de nodos. La densidad se calcula como:

$$density = \frac{|E|}{|V| \cdot (|V| - 1) / 2} \quad (5.3)$$

Valores cercanos a 0 indican un ecosistema fragmentado y cercanos a 1 un entorno altamente colaborativo. La `modularity` se calcula con el algoritmo de Louvain [BGLL08] y mide la calidad de la partición en comunidades; valores superiores a 0,3 indican una estructura de comunidades bien definida.

El Listado 5.2 muestra la implementación de la detección de comunidades con Louvain sobre el grafo de colaboración construido con NetworkX. Si Louvain falla, el sistema recurre

a las componentes conexas como *fallback*; las comunidades se ordenan por tamaño y se identifican sus miembros clave por grado de centralidad.

```

1  def detect_communities(self):
2      if self.G.number_of_nodes() < 2:
3          return [], {}
4      try:
5          communities_gen = nx.community.louvain_communities(
6              self.G, weight='weight', resolution=1.0, seed=42
7          )
8          communities = [list(c) for c in communities_gen]
9      except Exception as e:
10         logger.warning(f"Louvain failed: {e}, using components")
11         communities = [list(c) for c in nx.connected_components(self.G)]
12
13     communities.sort(key=len, reverse=True)
14     node_community = {}
15     for idx, members in enumerate(communities):
16         member_degrees = [
17             (m, self.G.degree(m)) for m in members
18         ]
19         member_degrees.sort(key=lambda x: x[1], reverse=True)
20         key_members = [m for m, _ in member_degrees[:3]]

```

Listado 5.2: Detección de comunidades mediante Louvain.

El Anexo Ñ reproduce otros fragmentos de código representativos de los algoritmos clásicos del proyecto.

5.10 Infraestructura y Despliegue en Azure

El despliegue de Entangle combina varios servicios de Azure que separan claramente *frontend*, *backend*, datos, inferencia y operación. La descripción exhaustiva de cada servicio, el detalle del *Dockerfile*, los módulos de Bicep y la configuración del entorno local se presentan en el Anexo H.

5.10.1 Servicios Azure

En producción intervienen los siguientes servicios:

- **Azure Container Apps** (Spain Central): alojamiento del *backend* FastAPI; el **Container Apps Environment** aporta el plano de ejecución, escalado y observabilidad.
- **Azure Static Web Apps** (Global): publicación del *frontend* React con distribución global.
- **Azure Cosmos DB for MongoDB (vCore)** (Spain Central): base de datos documental principal.
- **Azure AI Foundry**: inferencia del agente conversacional con gpt-5-mini y generación de *embeddings* para el RAG con text-embedding-3-small.

5. RESULTADOS

- **Azure Container Registry** (Spain Central): almacenamiento de imágenes Docker del *backend*.
- **Azure Log Analytics** (Spain Central): centralización de *logs* y soporte al diagnóstico operativo.

La separación entre *frontend* estático global y *backend* regional reduce la latencia de entrega de la interfaz sin replicar la lógica de negocio.

5.10.2 Docker, Bicep y Entorno Local

El *backend* se contenedoriza con Docker (imagen base python:3.11-slim, usuario no privilegiado y *health check*) y la infraestructura se gestiona mediante Bicep con un diseño modular a nivel de suscripción que separa la provisión de cada recurso. El flujo `azd up` orquesta provisión, construcción, publicación y despliegue en un único comando. Los listados completos del Dockerfile, los módulos de Bicep y la configuración del entorno local de desarrollo se encuentran en el Anexo H; el procedimiento paso a paso para reproducir el despliegue se documenta en el Anexo A y el catálogo completo de variables de entorno requeridas por *backend* y *frontend* en el Anexo E.

5.10.3 Seguridad

La seguridad se articula en capas complementarias: el *backend* prioriza System-Assigned Managed Identity con `DefaultAzureCredential` para consumir Azure AI Foundry, evitando claves embebidas; los secretos sensibles (`MONGO_URI`, `GITHUB_TOKEN`) se declaran como parámetros `@secure()` en Bicep e inyectan vía `secretRef` en la *Container App*; Cross-Origin Resource Sharing (CORS) restringe los orígenes permitidos a la URL del *frontend* en producción; Azure Static Web Apps añade cabeceras de endurecimiento (`Content-Security-Policy`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `X-XSS-Protection`) recomendadas por el proyecto Open Worldwide Application Security Project (OWASP) [Mic24g]; y el agente conversacional limita sus capacidades sobre MongoDB a operaciones de lectura, bloqueando explícitamente los operadores `$out` y `$merge` en `run_aggregation`. En conjunto, no descansa en una única medida sino en la combinación de identidad gestionada, secretos fuera del código, control de orígenes, cabeceras endurecidas y *tools* acotadas a operaciones observacionales [Mic24e, Moz24a].

5.11 Gestión de la Configuración del Software

Esta sección recoge la gestión de la configuración del software del proyecto Entangle, presentando los dos repositorios independientes que componen el sistema y la organización interna de cada uno. La separación en dos subsistemas (*backend* y *frontend*) responde tanto al desacoplamiento natural del sistema como a la conveniencia de aplicar a cada parte su propia configuración de despliegue continuo.

5.11.1 Control de Versiones

El control de versiones se gestiona con Git, utilizando la organización Angel-TFG-UCLM en GitHub como plataforma central. El proyecto se ha desarrollado como dos subsistemas independientes, materializados en dos repositorios separados: **Entangle-Core** (backend Python/FastAPI) y **Entangle-Visualizer** (frontend React/Vite), cada uno con su propia configuración de despliegue continuo. Esta separación permite que cada subsistema evolucione a su propio ritmo, mantenga su propio ciclo de vida de *commits* y reciba su propio pipeline de CI/CD.

La estrategia de ramas es directa: la rama *main* constituye la rama de producción en ambos repositorios. Tanto el backend como el frontend disponen además de una rama *develop* vinculada a un entorno de *staging* (Azure Container Apps en el backend, Azure Static Web Apps en el frontend), lo que permite verificar cambios antes de promoverlos a producción. Los *commits* se realizan de forma frecuente y con mensajes descriptivos, dado que el historial de Git constituye el principal registro de progreso del proyecto al tratarse de un desarrollo individual.

El despliegue continuo se implementa mediante cuatro workflows de GitHub Actions:

- **Frontend (producción):** cada push a *main* dispara el despliegue automático a Azure Static Web Apps.
- **Frontend (staging):** cada push a *develop* despliega en un entorno de *staging* de Azure Static Web Apps para validación previa.
- **Backend (producción):** un *trigger* de autodespliegue publica la imagen Docker en Azure Container Apps al fusionar cambios en *main*.
- **Backend (staging):** un workflow análogo despliega la rama *develop* en el entorno de *staging* de Azure Container Apps.

La Infrastructure as Code (IAC) se gestiona mediante plantillas Bicep almacenadas en el directorio *infra/* del backend, lo que permite reproducir el entorno de producción de forma determinista. La configuración de Azure Developer CLI (*azure.yaml*) complementa este flujo, integrando el aprovisionamiento de recursos y el despliegue en un único comando.

5.11.2 Organización del Repositorio

La organización de los repositorios es fundamental para un desarrollo eficiente y ordenado. La estructura interna de **Entangle-Core** y **Entangle-Visualizer** está diseñada para separar claramente las diferentes responsabilidades del sistema, lo que facilita el desarrollo modular y escalable.

5. RESULTADOS

```
1  Angel-TFG-UCLM/
2  |-- Entangle-Core/
3  | |-- src/
4  | | |-- ai/ # Agente IA (config-driven) + RAG + prompts + tools
5  | | |-- analysis/ # Análisis de red (NetworkX) + clasificador disciplinas
6  | | |-- api/ # Endpoints FastAPI (routes, admin, chat)
7  | | |-- core/ # Config, DB, logging, caché, repositorio genérico
8  | | |-- github/ # Ingesta, enriquecimiento, GraphQL, filtros, rate limit
9  | | |-- models/ # Modelos Pydantic (Repository, User, Organization)
10 | | |-- utils/ # Utilidades comunes
11 | |-- scripts/ # Pipelines ejecutables y utilidades
12 | |-- tests/ # Tests pytest
13 | |-- config/ # ingestion_config.json, pipeline_config.json, workers.yaml
14 | |-- infra/ # Bicep (IaC) para Azure
15 | |-- Dockerfile # Imagen Docker optimizada
16 | |-- azure.yaml # Configuración Azure Developer CLI
17 | |-- requirements.txt # Dependencias Python
18 |-- Entangle-Visualizer/
19 | |-- src/
20 | | |-- components/ # Componentes de la interfaz
21 | | |-- store/ # Gestión de estado (Zustand)
22 | | |-- services/ # Servicios de API (Axios + SSE)
23 | | |-- data/ # Datos de prueba
24 | | |-- assets/ # Recursos estáticos
25 | |-- package.json # Configuración del proyecto
26 | |-- vite.config.js # Configuración de Vite
27 | |-- staticwebapp.config.json # Configuración de la aplicación web estática
```

Listado 5.3: Árbol de directorios de los repositorios Entangle-Core y Entangle-Visualizer.

Cada directorio y archivo dentro del repositorio tiene un propósito específico. Por ejemplo, el directorio `api/` contiene los archivos `main.py`, `routes.py` y `chat_routes.py`, que definen los endpoints de la API utilizando FastAPI. El directorio `ai/` alberga el agente de inteligencia artificial y las funciones relacionadas, mientras que `core/` gestiona la configuración básica y el manejo de la base de datos.

5.12 Análisis de Datos del Ecosistema Cuántico

Esta sección presenta los hallazgos del análisis sobre el ecosistema cuántico indexado por Entangle, organizando el discurso por **fenómeno analizado** (volumen y composición, lenguajes predominantes, organizaciones y disciplinas, comunidades) y mostrando para cada uno la cifra agregada y la visualización del cuadro de mando interactivo que la sostiene. Los componentes individuales del cuadro de mando 2D (KPIs, gráficos analíticos, grafo de colaboración) se describen en detalle como herramienta en el Capítulo 4, Sección 5.7; aquí se utilizan como vehículo de los *insights* sobre los datos.

El *pipeline* ejecuta de forma autónoma seis fases (descubrimiento, ingesta de repositorios, extracción de colaboradores, resolución de organizaciones, enriquecimiento con métricas de red y clasificación disciplinar) partiendo de 71 palabras clave del dominio cuántico. Su diseño modular permite ejecutar cada fase de forma independiente y reintentar automáticamente ante errores de la API (códigos 429 y 502), lo que garantiza la completitud de la extracción. El resultado es un *dataset* integrado que conecta repositorios, colaboradores y organizaciones con métricas de centralidad, comunidades y disciplinas.

5.12.1 Volumen y Composición del Ecosistema

El *pipeline* ha producido el conjunto de datos resumido en la Tabla 5.3. Las cifras reflejan el estado de la base de datos tras la última ejecución completa.

Tabla 5.3: Cifras clave del ecosistema cuántico tras la última ejecución del *pipeline*.

Métrica	Valor
Repositorios cuánticos	>1.500
Usuarios/colaboradores	>27.000
Organizaciones	>400
Comunidades analizadas	752
Disciplinas clasificadas	6

El volumen obtenido confirma la existencia de un ecosistema cuántico activo y diverso en GitHub: más de 1.500 repositorios mantenidos por aproximadamente 27.000 colaboradores y agrupados en más de 400 organizaciones. La relación de aproximadamente 18 colaboradores por repositorio sugiere un ecosistema con un grado de participación elevado, aunque distribuido de forma desigual, como se analiza en las secciones siguientes.

La sección de KPIs del cuadro de mando interactivo (Sección 5.7.1) permite verificar de un vistazo este volumen: tres tarjetas presentan los conteos animados de repositorios, colaboradores y organizaciones, con un estado inicial *cuántico* (onda de probabilidad y vector de Bloch en superposición) que colapsa al estado clásico tras la interacción del usuario. La galería de capturas (estado inicial y estado tras el colapso) se incluye en el Anexo L, Sección L.1.

5.12.2 Lenguajes Predominantes

Python es el lenguaje predominante del ecosistema, presente en la mayoría de los repositorios cuánticos gracias a los principales SDKs del sector (Qiskit, Cirq, PennyLane) [IBM24]. Jupyter Notebook ocupa el segundo lugar, lo que refleja la importancia de los entornos interactivos para la experimentación y la enseñanza en computación cuántica.

C++ y Julia aparecen como los lenguajes compilados con mayor presencia. C++ se asocia principalmente a simuladores de alto rendimiento y *backends* de hardware, mientras que Julia se emplea en proyectos de simulación numérica que aprovechan su capacidad de cómputo. La convivencia de estos lenguajes con Python refleja la naturaleza multidisciplinar del campo: el prototipado rápido convive con la necesidad de rendimiento en producción.

El *donut* de lenguajes del cuadro de mando interactivo (incluido en la Figura 5.10) confirma visualmente esta hegemonía y permite observar la cola larga de lenguajes secundarios.

5. RESULTADOS

5.12.3 Organizaciones y Disciplinas

Entre las más de 400 organizaciones descubiertas, cuatro concentran el mayor volumen de repositorios y colaboradores: Qiskit (IBM) [IBM24], Cirq (Google Quantum AI) [Goo24b], PennyLane (Xanadu) [Xan24] y Braket (Amazon) [Ama24]. Estas organizaciones actúan como ejes centrales del grafo de colaboración y aparecen de forma recurrente como nodos de alta centralidad en los análisis de red posteriores (Sección 5.12.4).

El `quantum_focus_score` (definido en la Sección 5.9.2) permite distinguir entre organizaciones dedicadas exclusivamente a computación cuántica y aquellas que participan de forma tangencial. Las organizaciones con puntuación superior al 30 % representan el núcleo del ecosistema, mientras que las restantes son típicamente organizaciones de software clásico con algún proyecto cuántico aislado.

En cuanto a la clasificación disciplinar de usuarios (Sección 5.9.1), la distribución resultante muestra que `quantum_software` y `dev_tooling` concentran conjuntamente más de la mitad de los usuarios, lo que evidencia el peso del desarrollo de software frente a la investigación teórica. `quantum_hardware` agrupa a la comunidad más reducida pero altamente especializada, coherente con la barrera de entrada que supone el acceso a *hardware* cuántico físico. El *donut* de distribución por disciplina del cuadro de mando interactivo (Figura 5.9) cuantifica visualmente este reparto, con predominio multidisciplinar y de herramientas de desarrollo sobre el *software* y el *hardware* cuánticos puros, y lista junto al donut los *Puentes Interdisciplinares* (usuarios cuyas contribuciones cruzan dos o más disciplinas distintas).

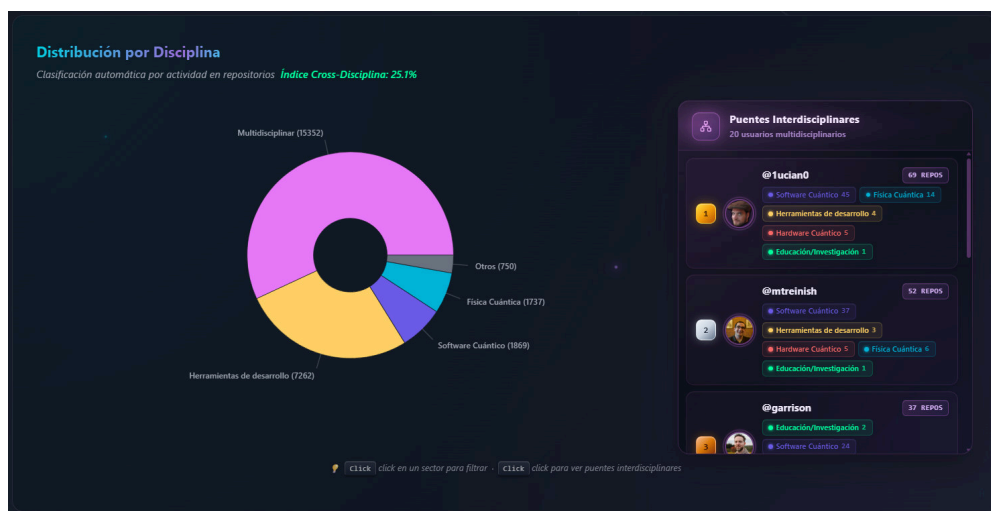


Figura 5.9: *Donut* de Distribución por Disciplina y panel de *Puentes Interdisciplinares* del cuadro de mando.

La Figura 5.10 reúne los cuatro gráficos analíticos del cuadro de mando interactivo, que materializan los resultados anteriores: los rankings de *Top Organizaciones*, *Top Repositorios* y *Top Usuarios*, y el *donut* de *Distribución de Lenguajes*. Su filtrado cruzado permite aislar una

organización o usuario y observar cómo se reconfiguran todos los gráficos simultáneamente, revelando patrones que no serían visibles de forma independiente.



Figura 5.10: Gráficos analíticos del cuadro de mando interactivo.

5.12.4 Análisis de Comunidades

El grafo de colaboración construido por Entangle contiene 30.970 nodos y 101.190 aristas. Sobre esta estructura, el algoritmo de Louvain (Sección 3.4.3) identifica 752 comunidades con una modularidad de 0,9585. Este valor, cercano a 1, indica una partición de alta calidad donde las conexiones intra-comunidad superan ampliamente a las inter-comunidad.

La mayoría de comunidades agrupan una organización con sus repositorios y colaboradores directos, lo que confirma que la estructura de colaboración del ecosistema está fuertemente organizada en torno a las organizaciones. Sin embargo, los 2.387 *bridge users* identificados (Sección 3.4.5) rompen este patrón al contribuir a repositorios de dos o más organizaciones independientes, generando las conexiones inter-comunidad que el análisis revela.

Para evitar falsos positivos en la detección, el sistema aplica dos criterios de exclusión: las organizaciones consideradas hermanas (por ejemplo qiskit y qiskit-community) se tratan como una única entidad y no cuentan como puente; y las cuentas *bot* (detectadas tanto en backend como en frontend) se excluyen del análisis. De forma complementaria, Entangle implementa la noción de *bridge disciplinar* sobre el clasificador de seis disciplinas (Sección 5.7): un usuario cuyas contribuciones abarcan al menos dos disciplinas distintas ($\text{disciplines_spanned} \geq 2$) se clasifica como tal. Aunque ambos conceptos comparten la palabra *bridge*, miden dimensiones diferentes: los *bridge users* capturan conexiones inter-organizacionales,

5. RESULTADOS

mientras que los *bridge disciplinares* reflejan transferencias de conocimiento entre campos técnicos.

Un examen más detallado de las comunidades permite distinguir dos perfiles. Las comunidades de perfil industrial se nuclean en torno a las grandes organizaciones (Qiskit/IBM, Cirq/Google, Braket/Amazon) y se caracterizan por repositorios con alta frecuencia de *releases*, documentación exhaustiva y numerosos colaboradores externos. Las comunidades de perfil académico, por el contrario, presentan repositorios con menos estrellas y contribuidores, pero mayor presencia de Jupyter Notebooks y publicaciones asociadas, coherente con su función de soporte a la investigación y la docencia. Los *bridge users* que conectan ambos perfiles, contribuyendo por ejemplo a Qiskit y simultáneamente a repositorios de investigación universitaria, representan un vector de transferencia tecnológica entre la industria y la academia que resultaría invisible sin el análisis de red.

Las métricas de centralidad calculadas sobre el grafo permiten cuantificar la influencia de cada nodo. Los nodos con mayor *betweenness centrality* coinciden predominantemente con *bridge users*, lo que valida su papel como conectores estructurales del ecosistema. El mini-universo de colaboración (Figura 5.11) permite analizar a bajo nivel un subconjunto de organizaciones seleccionadas: *clusters* densos conectados por aristas cian inter-organizacionales y nodos ámbar centrales que actúan como puentes.

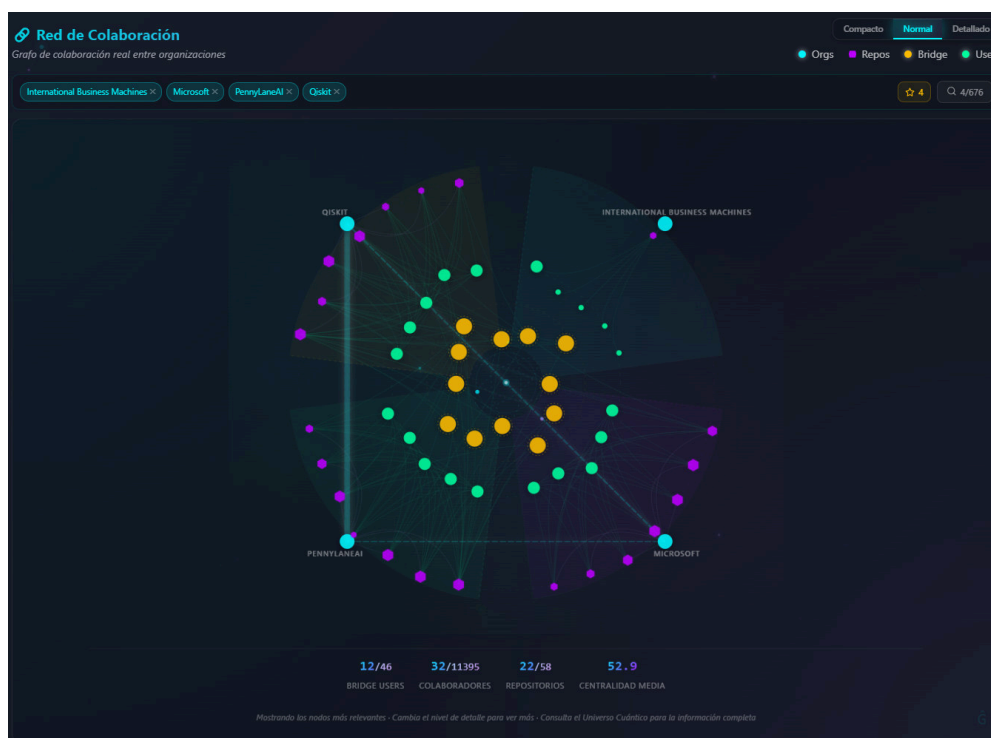


Figura 5.11: Mini-universo de colaboración embebido en el cuadro de mando.

5.13 Análisis de Comunidades mediante Universo 3D

Esta sección presenta los *insights* sobre el ecosistema cuántico que aporta el universo 3D de Entangle: la lectura visual de la distribución espacial de las entidades (Véase Sec. 5.13.1), los hallazgos extraídos por las cinco lentes analíticas (Véase Sec. 5.13.2) y la narrativa cronológica del tour cinemático (Véase Sec. 5.13.3). La correspondencia detallada entre conceptos físicos y metáforas visuales (Tabla J.1) y la galería completa con todas las lentes, la herramienta de *pathfinding* y el tour completo se reúnen en el Anexo J; aquí se incluyen únicamente las capturas mínimas necesarias para sostener cada *insight*.

5.13.1 Distribución Espacial

El algoritmo de distribución (Sección 5.8.4) sitúa las organizaciones más influyentes en el centro del universo y las menos conectadas en la periferia. Tres zonas de densidad generadas por Jenks Natural Breaks (Sección 5.8.5) estructuran el espacio: el núcleo concentra las organizaciones con mayor centralidad y enfoque cuántico rodeadas por anillos orbitales densos, la zona intermedia alberga organizaciones con colaboración moderada y la periferia muestra proyectos aislados. La Figura 5.12 ofrece la vista general resultante, con los efectos ambientales de fondo (Sección 5.8.6); las tres zonas se delimitan en la captura con elipses superpuestas en **verde** (núcleo), **amarillo** (zona intermedia) y **rojo** (periferia), y los anillos azules visibles dentro del universo son *ondas de decoherencia* generadas dinámicamente desde los procesadores cuánticos como efecto ambiental (Véase Sec. J.6). Esta distribución confirma que el ecosistema se organiza en torno a un núcleo reducido de organizaciones industriales que atraen la mayor actividad colaborativa.

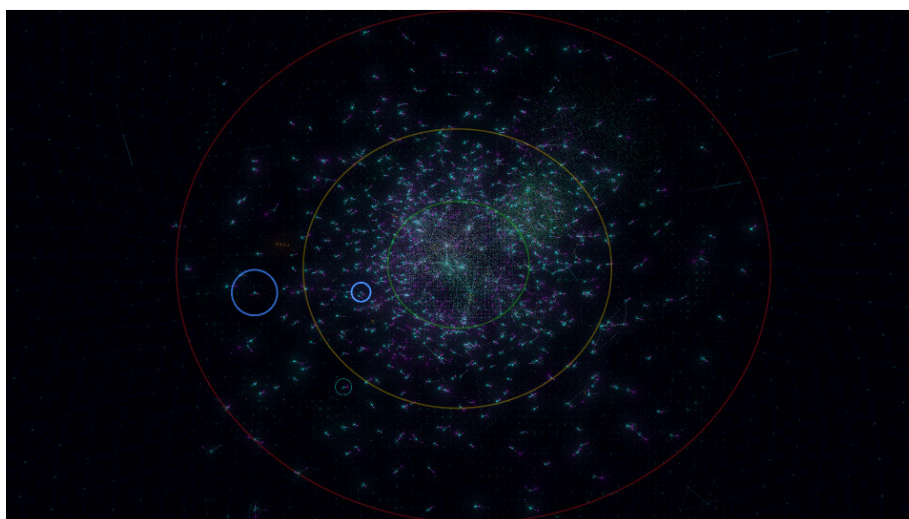


Figura 5.12: Vista general del universo 3D de Entangle.

5.13.2 Insights por Lente Analítica

Las cinco lentes analíticas y la herramienta de *pathfinding* (Sección 5.8.6) permiten observar el ecosistema desde perspectivas complementarias. La Figura 5.13 muestra la lente de comunidades como vista representativa; el resto de las lentes se incluyen en el Anexo J.

- **Comunidades.** Colores asignados por ángulo áureo sobre las ~ 752 comunidades analizadas por Louvain. La mayoría son compactas y autocontenidas, organizadas en torno a una organización principal; las regiones donde los colores se mezclan corresponden a zonas con alta actividad de *bridge users*.
- **Centralidad.** Gradiente de azul oscuro a cian brillante según la *betweenness centrality*: los nodos puente más luminosos se concentran en el núcleo, ocupando posiciones centrales tanto topológica como espacialmente.
- **Resiliencia.** Cuatro niveles cromáticos por *bus factor* [AVH15] (rojo = 1, naranja = 2, amarillo 3–4, verde ≥ 5). El **81,7 %** de los repositorios tiene *bus factor* = 1, evidenciando una vulnerabilidad sistémica del ecosistema cuántico.
- **Intensidad.** Gradiente rojo–amarillo por *degree centrality*, identificando las áreas con mayor actividad colaborativa bruta, que no necesariamente coinciden con las más centrales topológicamente.
- **Disciplinas.** Color por categoría temática del usuario: `quantum_software` y `dev_tooling` dominan, y los multidisciplinares se sitúan con frecuencia en intersecciones, lo que sugiere que la interdisciplinariedad se correlaciona con posiciones de puente en el grafo.
- **Túnel Cuántico.** Camino más corto animado mediante Dijkstra (Sección 3.4.4), que revela cadenas de colaboración que atraviesan múltiples organizaciones.

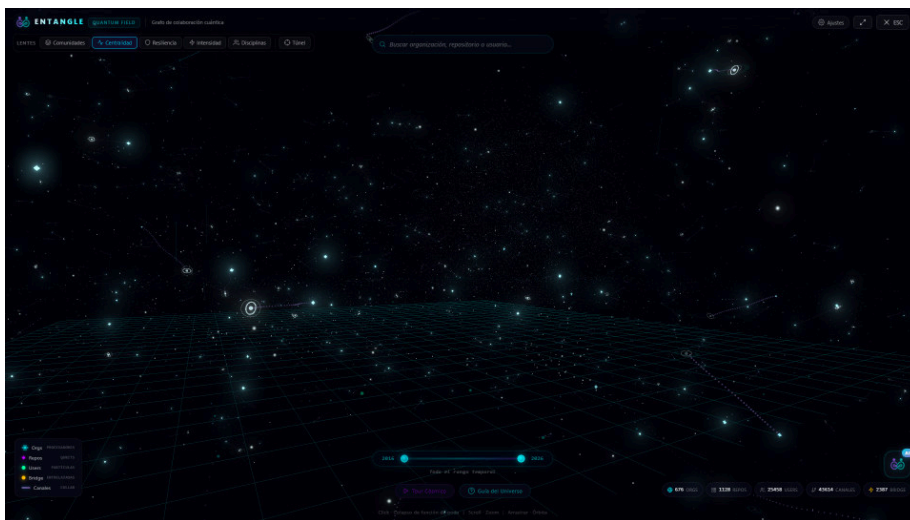


Figura 5.13: Lente de comunidades del universo 3D.

5.13.3 Tour Cinemático

El tour cinemático (Sección 5.8.6) construye una narrativa cronológica del ecosistema. Cada *waypoint* interpola valores de la base de datos, de modo que el recorrido refleja siempre el estado actual; un *slider* temporal filtra los nodos por año de creación y el universo crece ante el usuario a medida que avanza cronológicamente. La Figura 5.14 muestra un *waypoint* intermedio con encuadre cinematográfico; el resto de capturas del tour, junto con los detalles cinemáticos del *Quantum Genesis* y el *Black Hole Collapse*, se reúnen en el Anexo J.



Figura 5.14: *Waypoint* intermedio del tour cinemático.

5.14 Evaluación del Agente de IA Conversacional

Esta sección evalúa el funcionamiento del agente de IA conversacional: la precisión de la clasificación, la calidad de las respuestas de cada *worker* y el comportamiento de las capacidades incorporadas (recuperación aumentada, memoria conversacional e investigación externa). La galería completa de capturas de interacción con cada *worker* y con las acciones automáticas se recoge en el Anexo K.

5.14.1 Clasificación y Procesamiento de Consultas

La arquitectura orientada a configuración descrita en la Sección 5.6.1 clasifica cada mensaje en una de las cinco intenciones enrutables antes de procesarlo, asignándolo a su *worker* correspondiente. Tras la clasificación, la interfaz presenta un *badge* con el *worker* activo y los pasos de razonamiento, transmitidos en tiempo real mediante SSE (Sección 5.6.4). En la práctica, cada categoría se deriva a su *worker* homónimo (consultas cuantitativas a DATA, cualitativas a KNOWLEDGE, comparativas a INSIGHTS, y guía de interfaz a DASHBOARD o UNIVERSE); cuando la intención es ambigua, el *fallback* a DATA resulta la opción más segura, al ser el *worker* con acceso directo a la base de datos.

5. RESULTADOS

5.14.2 Consultas sobre Datos

El *worker* DATA (Sección 5.6.2) es el único con acceso de consulta **de propósito general** a MongoDB: el modelo formula directamente las *aggregation pipelines* mediante *function calling*, con tres herramientas de solo lectura; `run_aggregation` bloquea los operadores `$out` y `$merge` antes de ejecutar el *pipeline*, como defensa frente a *prompt injection* [OWA25]. Resuelve tanto *rankings* (con `$sort` y `$limit`) como agregaciones (por ejemplo, el 42,05 % de repositorios que usan Python), presentando siempre datos reales. Los resultados se truncan a 8.000 caracteres para respetar la ventana de contexto, reduciendo los documentos retornados (de 50 a 5, y a 2 si persiste). Las capturas de ambos casos se recogen en el Anexo K.

5.14.3 Conocimiento, Análisis Cruzado, Memoria e Investigación

El *worker* KNOWLEDGE responde preguntas cualitativas mediante RAG (Sección 5.6.2): ante «¿En qué se diferencian Qiskit y Cirq?», recupera los pasajes afines de los README y elabora una comparación fundamentada en ellos (Figura 5.15); si el repositorio no está en el corpus, lo admite en lugar de fabricar datos. El *worker* INSIGHTS aporta análisis cruzados: compara varios repositorios por sus métricas reales (estrellas, lenguaje, contribuidores y temas) con una síntesis de diferencias (Figura 5.16), y mide el solapamiento de contribuidores entre organizaciones; entre `qiskit` y `qiskit-community`, por ejemplo, cuantifica 260 compartidos (índice de Jaccard de 0,19). La memoria conversacional (Sección 5.6.2) resuelve referencias anafóricas entre turnos, y el *worker* DEEP_RESEARCH (Sección 5.6.2) recupera publicaciones de arXiv y la web con declaración explícita de la fuente. El resto de capturas se recoge en el Anexo K.

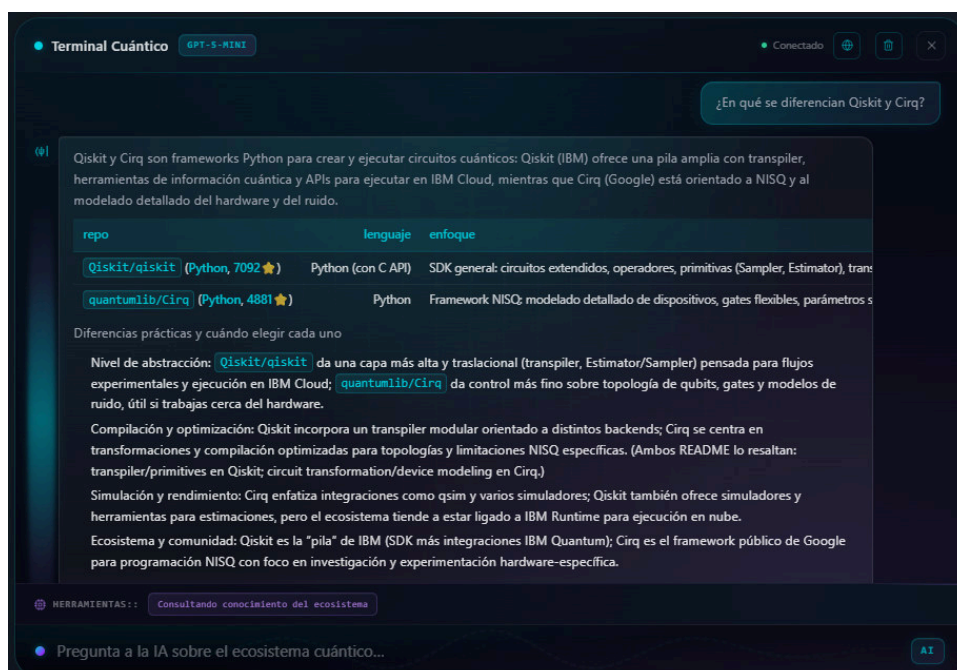


Figura 5.15: Respuesta del *worker* KNOWLEDGE: comparación de Qiskit y Cirq vía RAG.



Figura 5.16: Respuesta del *worker* INSIGHTS: comparativa de repositorios con `compare_repos`.

5.14.4 Evaluación Cuantitativa y Limitaciones

Para medir la fiabilidad del sistema más allá de ejemplos individuales, se diseñó un banco de 30 consultas de prueba que cubre los cinco *workers* enrutables, la investigación externa y las dos acciones de interfaz, combinando representatividad (al menos cuatro consultas por *worker*), dificultad técnica progresiva y casos de borde (consultas ambiguas entre *workers*, referencias anafóricas que exigen memoria y entidades mal escritas que requieren *fuzzy matching*). Cada consulta se evaluó manualmente según tres criterios, con los siguientes resultados consolidados:

- **Clasificación del *router*: 96,7 % (29/30).** El único error, en una consulta ambigua entre DATA e INSIGHTS, lo resolvió de forma satisfactoria el *fallback* a DATA.
- **Validez técnica de la operación generada: 93,3 % (28/30).** Las dos restantes correspondieron a búsquedas que no hallaron el repositorio exacto solicitado, ante lo cual el agente lo admitió sin fabricar datos.
- **Utilidad de la respuesta: 90,0 % (27/30)** completamente satisfactorias; las tres parciales corresponden a los casos anteriores.

Aunque el tamaño del banco no permite extraer intervalos de confianza estadísticos, los resultados son consistentes con el diseño conservador del sistema: vocabulario restringido del *router*, *fallback* determinista, herramientas de solo lectura y política de honestidad de fuentes.

Del uso del agente se desprenden tres limitaciones principales: el **truncamiento de resultados por diseño** (8.000 caracteres y 50 documentos por consulta), un *trade-off* necesario para

5. RESULTADOS

no desbordar la ventana de contexto, común a todos los sistemas basados en LLM; la **dependencia de fuentes externas**, ya que la búsqueda de *papers* está sujeta a los límites de tasa de la API pública de arXiv, mitigados mediante reintentos con espera incremental y un *fallback* honesto a la búsqueda web; y la **cobertura del corpus RAG**, pues la calidad de las respuestas de KNOWLEDGE depende de la exhaustividad de cada README, quedando infrarrepresentados los proyectos con documentación escasa.

5.15 Rendimiento y Escalabilidad

Esta sección sintetiza los resultados de rendimiento observados en los componentes críticos de Entangle: el pipeline de ingesta, la API REST con su sistema de caché, la transmisión de datos y el renderizado del universo 3D. El detalle de medidas, configuraciones y comparativas se encuentra en el Anexo M; aquí se presentan únicamente las cifras necesarias para sostener las conclusiones del proyecto.

5.15.1 Rendimiento del Pipeline de Ingesta

El *pipeline* (Sección 5.3) se ejecuta de forma asíncrona en un Container App Job con varias *phases*: ingesta y enriquecimiento de repositorios, ingesta y enriquecimiento de usuarios y organizaciones, y consolidación. La Tabla 5.4 resume los tiempos observados en ejecuciones reales antes y después de la migración de Cosmos DB del modelo *Request Units* al clúster vCore M30 (Sección 4.4), y la Figura 5.17 los representa gráficamente en escala logarítmica. Los rangos se han medido sobre el *dataset* completo (>1.500 repositorios, >27.000 usuarios y >400 organizaciones); los tiempos altos del modelo RU corresponden a ejecuciones con *throttling* HTTP 429 sostenido.

Tabla 5.4: Rendimiento del *pipeline* de ingesta antes y después de la migración a Cosmos DB vCore M30.

Fase	RU (HTTP 429)	vCore M30	Mejora (nº de veces)
Ingesta de repositorios	1–2 h	5–10 min	10–15
Enriquecimiento de repositorios	4–6 h	15–25 min	15–20
Ingesta de usuarios + organizaciones	1–2 h	5–10 min	10–15
Enriquecimiento de usuarios	hasta 3 días	hasta 10 h	7–8
Enriquecimiento de organizaciones	1–3 h	5–10 min	15–25
Consolidación y caché	30–60 min	1–2 min	20–30
Tiempo total extremo a extremo	>72 h (peor caso)	10–12 h	>6–8

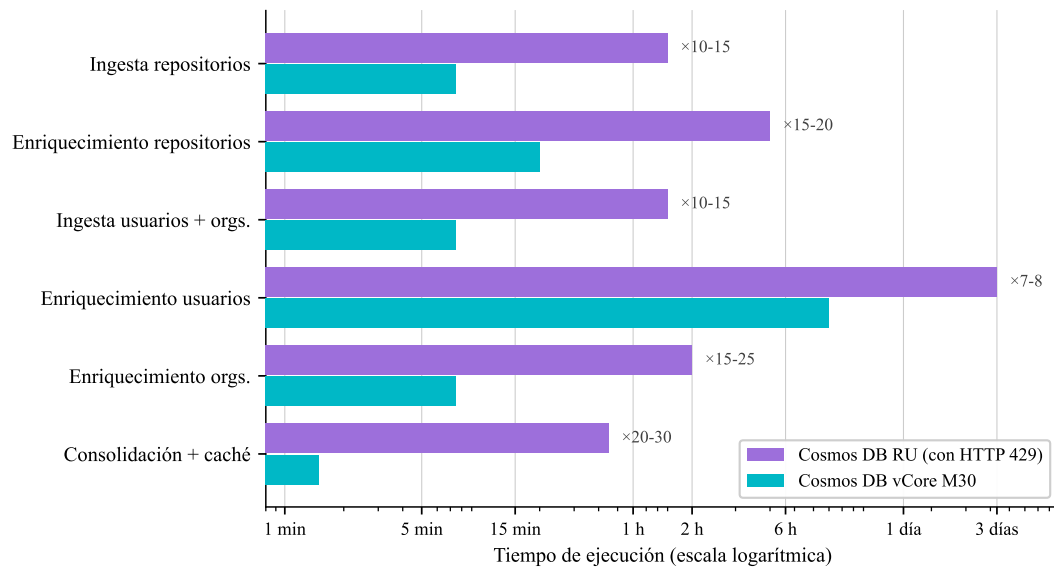


Figura 5.17: Tiempos del *pipeline* de ingesta: Cosmos DB RU frente a vCore M30.

El factor de mejora se descompone en dos contribuciones. En las fases dominadas por escritura masiva en MongoDB (ingesta y enriquecimiento de repositorios y organizaciones), la migración a vCore elimina el *throttling* HTTP 429 y los *sleeps* defensivos asociados, eleva el *batch_size* de 5–50 a 100–500 documentos y aumenta el *throughput* de la base de datos de ~5.000 a ~75.000–100.000 docs/h (×15–30). El enriquecimiento de usuarios, en cambio, está limitado por la API GraphQL de GitHub: cada perfil exige una consulta independiente sujeta al techo de 5.000 *points/h* y a un *time.sleep(0.5)* obligatorio entre llamadas, que se mantiene tras la migración. Esta fase, que con RU acumulaba reintentos por *throttling* combinado y podía superar los **tres días** de ejecución continua, queda acotada con vCore a un peor caso de ~10 h, dominadas ya únicamente por el *rate limit* de la API de GitHub.

5.15.2 Latencia de la API y Sistema de Caché

La API REST (Sección 5.5) sirve datos pre-agregados desde el sistema de caché de tres niveles descrito en la Sección 5.5.3 (memoria Python, documentos MongoDB y recomputación completa) complementado por la serialización binaria de *orjson* [Lei24], ~10× más rápida que la librería estándar *json*.

Tabla 5.5: Tiempos de respuesta de los *endpoints* principales con y sin caché activa.

Endpoint	Caché activa	Sin caché	Notas
/stats	<100 ms	5 s	Conteos simples
/dashboard/stats	<100 ms	35 s	Métricas pre-agregadas
/dashboard/stats (filtros)	50–100 ms	N/A	Cálculo en tiempo real
/collaboration/network-metrics	<1 s	60–90 s	Grafo completo
/collaboration/discover	3,3 s	7,2 s	Organizaciones + <i>scoring</i>

5. RESULTADOS

El caso más significativo es el del grafo de colaboración (*network-metrics*): su construcción desde MongoDB con NetworkX [HSS24] requiere entre 60 y 90 segundos en frío, y se resuelve en menos de un segundo desde la caché *chunked*, incluso tras un reinicio del servidor.

5.15.3 Transmisión de Datos y Renderizado 3D

El grafo de colaboración (27.887 nodos y 98.557 enlaces) genera un JSON de ~ 22 MB. Dos mecanismos minimizan su impacto: la compresión **GZip** en tránsito reduce el *payload* en torno al 87 % (a $\sim 2\text{--}3$ MB), y la **caché chunked** en MongoDB sorteja el límite de 16 MB por documento BSON fragmentando los *arrays* en bloques de 1,5 MB recuperados en una sola consulta `find({$in})`.

En el frontend, el universo 3D renderiza simultáneamente ~ 28.000 entidades y ~ 98.000 conexiones en la GPU. Las cuatro técnicas que sostienen la fluidez (consolidación de *draw calls* con InstancedMesh, *shaders* GLSL personalizados para los ~ 27.000 usuarios, montaje progresivo en 9 etapas y delegación de los cálculos de *layout* a Web Workers dedicados [Moz24d]) se cuantifican una a una en el Anexo M.1.

5.15.4 Conclusión sobre el Rendimiento

Los resultados anteriores avalan los objetivos no funcionales del proyecto: el *pipeline* se completa en minutos (frente a horas antes de la migración), la API responde en menos de un segundo en sus rutas de mayor coste y el universo 3D mantiene una experiencia fluida con decenas de miles de entidades. La sección de detalle (Anexo M) incluye las tablas extendidas de *draw calls*, post-procesado, configuración del *pool* de conexiones y comportamiento del cálculo de centralidad de intermediación con muestreo.

5.16 Pruebas

Esta sección presenta la estrategia de pruebas automatizadas del proyecto. El *backend* dispone de **919 tests** ejecutados con pytest [pyt24] (Sección 5.16.1), mientras que el *frontend* cuenta con **168 tests** ejecutados con Vitest (Sección 5.16.2). Ambas suites alcanzan coberturas superiores al 60 % y superan la *Quality Gate* definida en SonarQube (Sección 5.17). Los detalles de la estrategia de aislamiento, las pruebas de resiliencia frente a *throttling* y las propiedades de idempotencia del *pipeline* se recogen en el Anexo O.

5.16.1 Suite de Pruebas del Backend

El *backend* dispone de una suite de **919 funciones de test** distribuidas en 44 ficheros dentro del directorio `tests/`, ejecutadas con pytest [pyt24]. La cobertura global medida con `pytest-cov` es del **61,0 %** sobre el código fuente de `src/`. La Tabla 5.6 agrupa los tests por área funcional.

Tabla 5.6: Distribución de los 919 tests del backend por área funcional.

Área funcional	Tests	Ficheros representativos
Endpoints REST y rutas	163	test_routes_*.py, test_api.py
Ingesta y enriquecimiento	234	test_repos_enrichment_*.py, test_org_ingestion_*.py
Modelos y filtros	66	test_filters.py, test_repositories.py
Base de datos y caché	54	test_database.py, test_chunked_cache.py, test_mongo_*.py
Administración y autenticación	91	test_admin_*.py
Métricas de red y grafos	48	test_network_metrics*.py
API GraphQL de GitHub	52	test_graphql_*.py
Clasificadores e IA	51	test_discipline_classifier.py, test_chat_extra_v6.py
Configuración y utilidades	53	test_config.py, test_extract.py, test_engines.py
Usuarios y organizaciones	107	test_users.py, test_organizations.py, test_user_*.py
Total	919	44 ficheros

La estrategia se apoya en *mocks* y *fixtures* definidos en `conftest.py` (cliente GraphQL simulado, base de datos en memoria y repositorio MongoDB), lo que permite ejecutar la suite completa sin depender de servicios externos. Los tests de la API REST utilizan el `TestClient` de FastAPI [Ram24] y cubren *endpoints* de salud, listado y obtención por ID, filtros, paginación, lanzamiento del *pipeline*, administración y búsqueda. Existen además pruebas específicas para el mecanismo de reintento ante errores HTTP 429 de Cosmos DB [Mic24c] y verificaciones de idempotencia que garantizan que la re-ejecución del *pipeline* no duplica datos ni corrompe registros previos ((Véase Cap. O)).

5.16.2 Suite de Pruebas del Frontend

El *frontend* dispone de una suite de **168 tests** distribuidos en 8 ficheros, ejecutados con Vitest [Vit24] y `@testing-library/react` [Ken24]. La cobertura global medida con el proveedor V8 es del **62,2 %**. La Tabla 5.7 resume la distribución.

Las pruebas se centran en la lógica de negocio: los *stores* de Zustand [pmn24b] se testean de forma aislada mediante *mocks* de la capa HTTP, verificando transiciones de estado, gestión de errores y efectos secundarios. Los componentes 3D basados en Three.js y React Three Fiber se excluyen de la cobertura porque su correcta visualización se valida mediante inspección manual; la configuración de Vitest emplea `jsdom` como entorno de navegador simulado.

5.17 Calidad del Código basada en SonarQube

Esta sección complementa la suite de pruebas (Sección 5.16) con un análisis estático del código fuente realizado con SonarQube [Son24b] sobre los dos repositorios del proyecto. Para no fragmentar el discurso entre back y front (ambos forman parte del mismo sistema y persiguen el mismo umbral de calidad) el análisis se presenta de forma **agregada**; el detalle de configuración, incidencias corregidas y desgloses por proyecto se traslada al Anexo N.

5. RESULTADOS

Tabla 5.7: Distribución de los 168 tests del frontend por fichero.

Fichero	Tests	Alcance principal
api.test.js	53	Capa de comunicación HTTP con el backend
dashboardStore.test.js	32	Store Zustand: carga de datos, filtros, métricas de red, colaboración
favoritesStore.test.js	31	Store de favoritos: persistencia, sincronización
adminStore.test.js	19	Store de administración: pipeline, logs, configuración
devStore.test.js	13	Store de desarrollo: caché, paginación, scroll infinito
computeTemporalVisibility.test.js	10	Algoritmo de visibilidad temporal del universo 3D
useEnrichedData.test.js	6	Hook de enriquecimiento de datos para visualización
ErrorBoundary.test.jsx	4	Componente React de captura de errores
Total	168	8 ficheros

5.17.1 Visión Agregada del Proyecto

Se desplegó una instancia local de SonarQube Community Edition sobre Docker y se definió una *Quality Gate* personalizada denominada «Entangle» con condiciones equivalentes para ambos repositorios (umbrales detallados en el Anexo N.1). La cobertura del backend se genera con `pytest-cov` en formato Cobertura XML; la del frontend, con el proveedor V8 de Vitest. SonarScanner [Son24c] importa ambos informes.

Tabla 5.8: Métricas agregadas de SonarQube para el proyecto Entangle.

Métrica	Backend	Frontend	Total / Media
Líneas de código (<i>ncloc</i>)	14 687	14 040	28 727
Ficheros analizados	35	34	69
Tests automatizados	919	168	1 087
Cobertura de tests	61,0 %	62,2 %	61,6 %
Líneas duplicadas	2,1 %	3,9 %	3,0 %
Bugs	0	2	2
Vulnerabilidades	0	0	0
<i>Security Hotspots</i>	0 (100 %)	0 (100 %)	0 (100 %)
<i>Code Smells</i>	294	734	1 028
Deuda técnica	72 h 4 min	70 h	142 h 4 min
Fiabilidad / Seguridad / Mant.	A / A / A	B / A / A	N/A
Quality Gate	OK	OK	OK

Los dos repositorios superan la *Quality Gate* personalizada. El proyecto acumula **1 087 tests** sobre **28 727 líneas de código**, sin vulnerabilidades ni *security hotspots* abiertos. La calificación A en mantenibilidad y seguridad es común a ambos; la fiabilidad del backend (A)

es ligeramente superior a la del frontend (B), motivada por dos *bugs* de severidad baja en componentes React que no comprometen la operación del sistema.

La Figura 5.18 muestra el estado de la *Quality Gate* «Entangle» simultáneamente para ambos proyectos en el panel global de SonarQube. Esta vista única sustituye a los *dashboards* individuales de back y front, que se incluyen en el Anexo N.4 junto con la configuración detallada y la batería de incidencias corregidas en el primer escaneo.



Figura 5.18: Estado consolidado de la *Quality Gate* «Entangle» en SonarQube.

Cobertura global frente a cobertura de código nuevo. SonarQube distingue entre *coverage* (cobertura sobre el código completo del proyecto) y *new_coverage* (cobertura sobre las líneas modificadas desde el análisis de referencia). En ambos repositorios la métrica global supera holgadamente el umbral del 60 %; la métrica sobre código nuevo refleja un valor inferior porque el volumen de líneas modificadas respecto al *baseline* es reducido y la condición se evalúa sobre una muestra muy pequeña. La cobertura representativa para juzgar la calidad del proyecto es por tanto la global.

Deuda técnica y *code smells*. La deuda técnica acumulada (142 h) se concentra en los 1 028 *code smells* repartidos entre ambos repositorios. En el backend predominan funciones de complejidad cognitiva elevada en los módulos de ingesta y enriquecimiento, aceptables dado el carácter procedural del procesamiento de datos. En el frontend, el volumen es proporcional al elevado número de funciones (1 353 entre componentes, *hooks* y *handlers* React/JSX); la duplicación se mantiene por debajo del umbral del 5 % y los componentes Three.js se excluyen deliberadamente de la cobertura por requerir validación visual.

Integración continua con SonarCloud. A las mediciones puntuales en local se añade un **análisis automático en cada *push* y *pull request*** mediante dos *workflows* de GitHub Actions [Git24a] (`sonarcloud.yml` en cada repositorio) que delegan en **SonarCloud** [Son24a] con la *Quality Gate* por defecto *Sonar Way*, más estricta que la *Entangle* local al exigir cobertura ≥ 80 % y cero incidencias *en código nuevo*. Los detalles del *pipeline* y de las

5. RESULTADOS

condiciones evaluadas se recogen en el Anexo N.5; en el momento de redactar esta memoria, ambos repositorios pasan la *gate* en estado **OK**. Esta integración convierte el análisis estático en un *control continuo* que acompaña a cada cambio, en lugar de una fotografía puntual al cierre del proyecto.

5.17.2 Conclusión sobre la Calidad del Código

El análisis estático confirma que el proyecto cumple con los criterios objetivos de calidad establecidos: cero vulnerabilidades, cero *security hotspots* abiertos, cobertura por encima del umbral exigido y *Quality Gate* en estado **OK** para los dos repositorios. La deuda técnica restante se documenta como *code smells* de severidad media y constituye trabajo futuro abordable mediante refactorizaciones puntuales que no alteran el comportamiento observable del sistema.

5.18 Estimación de Costes del Proyecto

Esta sección desglosa los costes asociados al desarrollo y operación de Entangle, diferenciando entre costes de personal (estimados a partir del esfuerzo dedicado), costes de herramientas y servicios de IA utilizados durante el desarrollo, costes de infraestructura en producción (reales, basados en la facturación de Azure) y coste total agregado del proyecto.

5.18.1 Costes de Personal y Servicios de IA

El desarrollo ha sido realizado por un único ingeniero a tiempo parcial durante aproximadamente 9 meses (de octubre de 2025 a junio de 2026). Tomando como referencia el salario medio de un ingeniero informático junior en España (~25.000 €/año brutos, equivalente a ~13 €/hora), la Tabla 5.9 estima las horas dedicadas a cada bloque funcional. La estimación contempla un esfuerzo medio sostenido de 4 a 5 h/día durante los días lectivos y jornadas más intensas en vacaciones de Navidad y Semana Santa.

Tabla 5.9: Estimación de costes de personal del proyecto Entangle.

Bloque funcional	Horas estimadas	Coste (€)
Análisis, diseño y arquitectura	80	1.040
Pipeline de datos + Backend API	310	4.030
Agente de IA conversacional	100	1.300
Frontend (Dashboard 2D y Universo 3D)	280	3.640
Infraestructura, DevOps y CI/CD	80	1.040
Pruebas, validación y documentación	200	2.600
Total	1.050	13.650

El proyecto se ha desarrollado siguiendo la metodología *Vibe Coding* (Sección 4.2), lo que implica el uso intensivo de modelos de lenguaje conversacionales como asistentes de programación. La herramienta principal utilizada ha sido *GitHub Copilot* (incluyendo acceso a

sus modelos premium como GPT-5, Claude y Gemini a través del propio entorno de Copilot), al que se ha accedido sin coste adicional gracias a la condición de *becario* del autor en *Microsoft*, que da derecho a una suscripción ilimitada a la plataforma. No se han contratado otras suscripciones de pago, por lo que el coste directo de herramientas y servicios de IA es de **0 €**. A título orientativo, replicar el mismo entorno contratando los planes premium de mayor capacidad de cada proveedor (*Copilot Pro+* a 39 USD/mes, *ChatGPT Pro* a 200 USD/mes y *Claude Max 20x* a 200 USD/mes) supondría unos **3.650 € adicionales** a lo largo de los 9 meses de desarrollo.

5.18.2 Costes de Puesta en Producción

La Tabla 5.10 desglosa el coste mensual de la infraestructura de producción de Entangle en la suscripción Azure actual, basado en datos reales extraídos de la API de Azure Cost Management para el periodo de producción (de febrero a junio de 2026), tomando mayo como mes completo representativo.

Tabla 5.10: Desglose de costes mensuales de Azure para Entangle (mayo 2026).

Servicio	Coste mensual (€)
Azure Cosmos DB (vCore M30)	139
Azure App Service (SWA)	14
Azure Container Apps (backend)	13
Storage	5
Container Registry	5
Virtual Network	3
Azure Monitor + Log Analytics	<1
Foundry Models (gpt-5-mini)	<1
Otros (Defender, Bandwidth)	<1
Total proyecto	181

El principal vector de coste es Azure Cosmos DB (~ 77 % del total), cuyo clúster vCore M30 opera de forma continua con capacidad reservada, como ilustra la Figura 5.19. El segundo componente es Azure App Service para el frontend estático (~ 8 %). Por el contrario, los servicios de cómputo y el modelo de IA (gpt-5-mini vía Foundry Models) operan bajo un modelo de consumo y su coste es marginal: la inferencia del agente se sitúa muy por debajo de 1 €/mes dada la escala de uso actual, y con un precio aproximadamente $20\times$ inferior al de gpt-4o ($\sim 0,23$ €/M *tokens* de entrada y $\sim 1,85$ €/M *tokens* de salida) deja amplio margen para crecer en peticiones sin impacto presupuestario. Proyectado sobre los 9 meses de desarrollo, el coste acumulado de infraestructura asciende a unos 1.629 €. El coste total mensual de ~ 181 € se sitúa muy por debajo del crédito mensual disponible en la suscripción de cuenta empresarial utilizada en producción (~ 1.380 €/mes), lo que deja un amplio margen para escalar la infraestructura. Conviene recordar que durante la Fase 1 del proyecto se trabajó sobre la cuenta de estudiante de Azure (~ 92 €/año), cuyas restricciones forzaron decisiones

5. RESULTADOS

de diseño concretas (modelo *Request Units* en Cosmos DB, despliegue manual del frontend) que se relajaron en cuanto se dispuso de la suscripción completa.

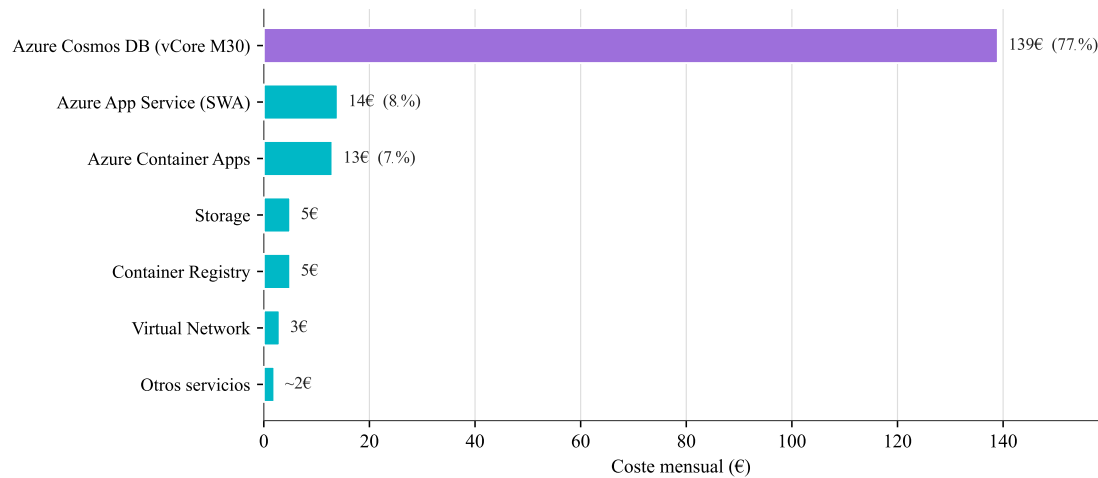


Figura 5.19: Desglose del coste mensual de Azure por servicio (mayo 2026).

5.18.3 Coste Total del Proyecto

La Tabla 5.11 resume el coste total estimado del proyecto, agregando personal, herramientas de IA e infraestructura. Las licencias de software no se contabilizan porque todas las dependencias utilizadas son de código abierto y, por tanto, su coste es cero.

Tabla 5.11: Coste total estimado del proyecto Entangle.

Concepto	Coste (€)
Personal (1.050 h · 13 €/h)	13.650
Herramientas y servicios de IA (becario Microsoft)	0
Infraestructura Azure (9 meses)	1.629
Total	15.279

Capítulo 6

Conclusiones y trabajo futuro

Entangle ha materializado una plataforma que integra la recopilación, el análisis y la visualización del ecosistema de computación cuántica en GitHub. El *pipeline* de datos ha indexado más de 1.500 repositorios, ~27.000 colaboradores y más de 400 organizaciones (Véase Sec. 5.12), conformando un grafo de colaboración de 30.970 nodos y 101.190 aristas sobre el que el algoritmo de Louvain ha identificado 752 comunidades con una modularidad de 0,9585 (Véase Sec. 5.12.4). Estos resultados revelan un ecosistema con estructura comunitaria marcadamente modular, donde los desarrolladores se agrupan en torno a frameworks concretos (Qiskit, Cirq, PennyLane, Braket) con poca interacción entre grupos; los 2.387 *bridge users* identificados constituyen el tejido conectivo minoritario que posibilita la difusión de ideas entre plataformas, lo que justifica su detección y visibilización dentro del sistema.

Las dos capas de visualización se complementan: el cuadro de mando interactivo (Véase Sec. 5.12) ofrece KPIs, gráficos analíticos y un grafo de colaboración con filtros globales, mientras que el universo 3D (Véase Sec. 5.13) renderiza las ~28.000 entidades del subgrafo conectado en una escena con cinco lentes analíticas y un tour cinemático guiado. El agente de IA conversacional, con arquitectura *config-driven* y *function calling* (Véase Sec. 5.14), permite consultar la base de datos en lenguaje natural y ejecutar acciones sobre la propia interfaz. La fiabilidad del sistema se sustenta en una suite de 1.087 pruebas automatizadas (919 *backend* + 168 *frontend*) con cobertura $> 60\%$ (Véase Sec. 5.16).

Un hallazgo especialmente revelador es la distribución del *bus factor*: el **81,7 %** de los repositorios cuánticos depende de un único desarrollador principal (Véase Sec. 5.13.2). Combinada con la modularidad de 0,9585, esta cifra dibuja un ecosistema técnicamente productivo pero estructuralmente frágil. Este panorama sitúa a la computación cuántica en un estadio análogo al de los primeros años de la informática clásica: pluralidad de plataformas competidoras, ausencia de estándares transversales, dependencia de un núcleo reducido de personas y *bridge users* como único mecanismo orgánico de difusión.

Antes de Entangle, un investigador que quisiera entender la estructura de este ecosistema tenía que extraer datos de la API, construir el grafo, ejecutar algoritmos de comunidades e interpretar los resultados sin apoyo visual. Este trabajo condensa todo ese proceso en una plataforma operativa de extremo a extremo que, además, generaliza más allá del dominio

cuántico: sustituyendo las palabras clave del descubrimiento y los filtros de ingesta, la misma metodología puede aplicarse a otros ecosistemas emergentes (*machine learning*, bioinformática, computación neuromórfica). En este sentido, la contribución concreta del proyecto es triple: (i) el dataset resultante, hasta donde sabemos la cartografía abierta más extensa del software cuántico de código abierto; (ii) dos capas de visualización complementarias que hacen accesible ese volumen a perfiles no técnicos; y (iii) una arquitectura modular reutilizable que demuestra que es posible transformar el ruido de miles de repositorios dispersos en una representación estructurada y navegable de una comunidad tecnológica.

6.1 Revisión de Objetivos

La evaluación del proyecto Entangle parte de los objetivos formalizados en el Capítulo 2: un objetivo principal (OP) que delimita el alcance global y seis subobjetivos (SO1–SO6) que descomponen el OP en hitos verificables. Para cada uno se fijó en su día un criterio de aceptación cuantificable; la consecución se ha medido contra los datos reales extraídos por el *pipeline* en producción y contra las funcionalidades efectivamente desplegadas, cruzando cada criterio con la sección concreta del Capítulo 5 en la que figura la evidencia.

La verificación arroja que los seis subobjetivos se han alcanzado, varios de ellos superando ampliamente el umbral inicial. La cartografía del ecosistema (SO1) supera los 1.500 repositorios indexados frente al criterio de ≥ 1.000 ; el análisis estructural del grafo de colaboración (SO2) identifica 752 comunidades con una modularidad de 0,9585 y 2.387 *bridge users*, demostrando la utilidad de las técnicas de análisis de redes; las dos capas de visualización (SO3 y SO4) materializan tanto el cuadro de mando 2D con filtrado cruzado como el universo 3D inmersivo con cinco lentes analíticas y un tour cinemático guiado; el agente conversacional *config-driven* (SO5) integra la base de datos a través de *function calling* sobre MongoDB y ejecuta acciones automáticas sobre la propia interfaz; y la identificación de tendencias (SO6) revela un ecosistema dominado por Python, concentrado en torno a Qiskit, Cirq, PennyLane y Braket, con un hallazgo especialmente relevante para la sostenibilidad del software cuántico: un *bus factor* de 1 en el 81,7 % de los repositorios analizados.

La Tabla 6.1 sintetiza visualmente esta evaluación, integrando en una única vista el identificador, el enunciado del objetivo, el estado de cumplimiento y la referencia explícita a la sección del Capítulo 5 donde se recoge la evidencia detallada.

Dado que todos los subobjetivos SO1–SO6 se han cumplido sobre el dataset real descrito en el Capítulo 5 y que cada uno satisface (y en varios casos supera ampliamente) su criterio de aceptación inicial, se considera que el **objetivo principal (OP) del proyecto ha quedado plenamente satisfecho**: el sistema Entangle es una plataforma operativa de extremo a extremo que cartografía, analiza y visualiza el ecosistema de computación cuántica de código abierto en GitHub.

Tabla 6.1: Evaluación del cumplimiento de los objetivos del proyecto Entangle.

ID	Objetivo	Cumpl.	Evidencia
OP	Plataforma de extremo a extremo para el ecosistema cuántico	✓	Pipeline + cuadro de mando + universo 3D + agente IA operativos en producción (Véase Cap. 5); los seis SO descomponen y verifican el OP.
SO1	Cartografiar e ingestar el ecosistema cuántico en GitHub	✓	>1.500 repos, >27.000 usuarios, >400 orgs descubiertos <i>bottom-up</i> a partir de 71 palabras clave (Véase Sec. 5.12); supera ampliamente el criterio de ≥ 1.000 repositorios.
SO2	Analizar estadísticamente las relaciones de colaboración	✓	Grafo de 30.970 nodos / 101.190 aristas; Louvain identifica 752 comunidades con modularidad 0,9585 y 2.387 <i>bridge users</i> ; centralidad y <i>bus factor</i> computados sobre todo el grafo (Véase Sec. 5.12.4).
SO3	Visualizar resultados estadísticos de colaboración	✓	Cuadro de mando 2D con KPIs, series temporales, distribuciones y grafo de colaboración filtrable por entidad, disciplina y rango temporal (Véase Sec. 5.12).
SO4	Analizar y visualizar mediante un universo 3D	✓	Universo 3D con ~ 28.000 entidades distribuidas espacialmente (Jenks Natural Breaks), 5 lentes analíticas y tour cinemático guiado (Véase Sec. 5.13).
SO5	Agente de IA conversacional	✓	Arquitectura orientada a configuración (gpt-5-mini) con seis <i>workers</i> , <i>function calling</i> sobre MongoDB, RAG, memoria conversacional, <i>streaming</i> SSE y acciones OPEN - UNIVERSE/CREATE_VIEW sobre la User Interface (UI) (Véase Sec. 5.14).
SO6	Identificar tendencias y patrones del ecosistema	✓	Predominio de Python; concentración en Qiskit/Cirq/PennyLane/Braket; disciplinas quantum - software y dev_tooling agrupan >50 % del talento; <i>bus factor</i> = 1 en el 81,7 % de los repos (Véase Sec. 5.12).

6.2 Limitaciones del Estudio

A pesar de los resultados obtenidos, este trabajo presenta limitaciones que deben considerarse al interpretar las conclusiones.

Representatividad de la fuente de datos. El ecosistema cuántico analizado se limita a GitHub. Plataformas como GitLab, Bitbucket o repositorios institucionales internos albergan proyectos que no aparecen en este estudio. Además, el sesgo inherente hacia el software de código abierto excluye desarrollos propietarios de empresas como IBM, Google o Amazon que, aunque publican parte de sus herramientas, mantienen componentes significativos fuera del alcance público. Por tanto, los resultados reflejan la fracción abierta y colaborativa del ecosistema, no su totalidad.

Cobertura del pipeline de ingesta. La estrategia de descubrimiento basada en 71 palabras clave (Anexo B) puede introducir dos tipos de sesgo. Por un lado, repositorios cuánticos que no incluyan ninguna de estas palabras en su nombre, descripción o *topics* quedarían fuera del dataset (*falsos negativos*). Por otro, repositorios tangencialmente relacionados podrían pasar los filtros y añadir ruido (*falsos positivos*). Los scripts de auditoría mitigan este segundo efecto, pero no lo eliminan por completo.

Naturaleza estática del análisis. Los resultados presentados corresponden a una captura puntual del ecosistema. La estructura de comunidades, las métricas de centralidad y la distribución disciplinar son fotografías del momento de la ingesta. Un ecosistema en rápida evolución como el de la computación cuántica puede experimentar cambios significativos en períodos relativamente cortos, lo que limita la vigencia temporal de las conclusiones.

Algoritmo de detección de comunidades. El algoritmo de Louvain [BGLL08] maximiza la modularidad de forma *greedy* y no determinista: distintas ejecuciones pueden producir particiones ligeramente diferentes. Además, la modularidad como función objetivo tiende a no detectar comunidades por debajo de un cierto tamaño (*resolution limit* [FB07]). Las 752 comunidades identificadas representan una partición plausible del grafo, pero no la única posible ni necesariamente la óptima. Dicho esto, como se argumenta en la Sección 3.4.3, el tamaño del grafo (30.970 nodos) y la alta modularidad obtenida (0,9585) hacen que la mejora práctica de migrar al algoritmo de Leiden [TWvE19] sea marginal en este contexto: las ventajas teóricas de Leiden (comunidades garantizadamente conexas, determinismo) se manifiestan de forma significativa en grafos de mayor escala o con estructuras comunitarias más ambiguas.

Agente de IA. Como se detalla en la Sección 5.14.4, el repertorio de acciones del agente sobre la interfaz se limita a dos comandos y la calidad de las respuestas del *worker* KNOWLEDGE está condicionada por la cobertura del corpus de RAG y por la disponibilidad de los servicios externos de investigación. Además, depende del modelo gpt-5-mini subyacente, cuyo comportamiento puede variar entre versiones del servicio de Azure AI Foundry.

6.3 Trabajo Futuro

A partir del estado actual del proyecto y de las limitaciones identificadas en la Sección 6.2, se proponen las siguientes líneas de evolución, alineadas con cada grupo de limitaciones detectadas:

- **Ampliación del pipeline a otras plataformas.** Incorporar las APIs de GitLab [Git24d] y Bitbucket (y, en su caso, repositorios institucionales internos) ampliaría la representatividad del estudio. La arquitectura de conectores del pipeline (Véase Sec. 5.3.1) facilita esta extensión: cada plataforma aportaría su propio módulo de ingesta y reutilizaría las fases de enriquecimiento, análisis de red y caché. Un mecanismo de descubrimiento adaptativo, que combinara las palabras clave actuales con expansión automática a partir de los *topics* ya indexados, reduciría además los falsos negativos.
- **Análisis temporal y predicción de tendencias.** El dataset ya almacena metadatos temporales (fecha de creación, último *push*, evolución de estrellas) que permitirían construir series temporales del ecosistema, revelar la evolución de los lenguajes predominantes, la aparición de nuevas organizaciones y la dinámica de formación de comunidades a lo largo del tiempo, complementando la fotografía estática actual.
- **Migración a Leiden si el grafo crece.** Aunque a la escala actual la mejora respecto a Louvain es modesta (Véase Sec. 3.4.3), sustituir Louvain por Leiden [TWvE19] resultaría pertinente al incorporar nuevas plataformas: garantizaría comunidades internamente conexas y aportaría determinismo a las ejecuciones sucesivas.
- **Gestión y conmutación de sesiones.** La infraestructura multi-sesión ya persiste cada conversación de forma independiente en el servidor (Véase Sec. 5.6.2); resta exponer en la interfaz un panel de historial para navegar, renombrar o retomar conversaciones pasadas, e incorporar autenticación para asociar el historial a una identidad concreta.
- **Repertorio de acciones del agente.** Más allá de `OPEN_UNIVERSE` y `CREATE_VIEW`, el agente podría aplicar filtros del dashboard, navegar a entidades concretas en el universo 3D o activar lentes específicas desde el chat.
- **Ampliación del corpus de conocimiento.** La canalización de RAG (Véase Sec. 5.6.2) indexa hoy los README de los repositorios; extenderla a documentación adicional (*wikis*, guías de contribución) o a publicaciones académicas vectorizadas reduciría la dependencia de servicios externos.
- **Extensibilidad del cuadro de mando vía *feature flags*.** Los 19 *flags* del devStore (Véase Sec. 5.7.4), concebidos como herramienta interna, constituyen también una infraestructura para añadir nuevas visualizaciones, hacer despliegues graduales o experimentar con perfiles de usuario distintos sin tocar el resto de la aplicación.
- **Endurecimiento del control de calidad continuo.** Con la *Quality Gate Sonar Way* ya activa en ambos repositorios (Véase Sec. N.5), los siguientes pasos serían habilitar

branch protection en *main* para bloquear cualquier *merge* cuya *gate* no esté en estado OK, añadir un análisis Software Composition Analysis (SCA) sobre las dependencias y extender la cobertura a los componentes Three.js mediante pruebas de *snapshot*.

6.4 Competencias del Grado Aplicadas

Esta sección justifica la adquisición de las competencias de la tecnología específica de Sistemas de Información (SI) cursada por el estudiante. Las seis competencias específicas (SI01–SI06) se recogen literalmente de las guías docentes oficiales de la mención SI del Grado en Ingeniería Informática de la Universidad de Castilla-La Mancha (UCLM) en el Campus de Talavera de la Reina [Fac24, Uni20, Sec09].

- SI01.** *Capacidad de integrar soluciones de TIC y procesos empresariales para satisfacer las necesidades de información de las organizaciones.* Entangle integra una pila tecnológica heterogénea (pipeline de ingesta, BBDD documental, API REST, agente IA y dos frontales 2D/3D, Sección 5.1) al servicio de un proceso analítico concreto: cartografiar el ecosistema cuántico de GitHub para investigadores y desarrolladores.
- SI02.** *Capacidad para determinar los requisitos atendiendo a aspectos de seguridad y cumplimiento de la normativa.* Los requisitos se formalizan en seis subobjetivos verificables (Capítulo 2) y la capa de seguridad incluye *Managed Identity*, secretos parametrizados en Bicep, CORS restringido, cabeceras Content Security Policy (CSP) y mínimo privilegio sobre Cosmos DB [Mic24e], alineados con el Reglamento General de Protección de Datos (RGPD) aplicable a datos públicos de GitHub.
- SI03.** *Capacidad para participar activamente en especificación, diseño, implementación y mantenimiento.* El TFG cubre el ciclo completo: especificación (Capítulo 2), diseño UML (Sección 5.2), implementación (Secciones 5.3–5.8) y mantenimiento mediante CI/CD con GitHub Actions, IaC con Bicep y observabilidad con Azure Monitor (Sección 5.10).
- SI04.** *Capacidad para ejercer como enlace entre las comunidades técnica y de gestión y participar en la formación de los usuarios.* La documentación, el cuadro de mando interactivo y el agente conversacional (Sección 5.14) están diseñados explícitamente como puente entre el dato técnico y el usuario no técnico (investigadores, gestores de comunidad cuántica).
- SI05.** *Capacidad para comprender y aplicar los principios de la evaluación de riesgos.* Se han identificado y documentado los riesgos técnicos del proyecto (*throttling* de la API de GitHub, límites de Cosmos DB en *Request Units*, coste cloud, dependencia de pocos mantenedores analizada como *bus factor*) y se describe el plan de mitigación ejecutado (migración a vCore, *backoff* adaptativo, presupuesto Azure controlado, Capítulo 6).

SI06. *Capacidad para aplicar técnicas de gestión de la calidad y de la innovación tecnológica.* La calidad del software se gestiona con SonarCloud (*Quality Gate Sonar Way*, Sección 5.17) y con una suite de 1.087 tests integrada en CI/CD. La innovación tecnológica se materializa en la adopción de un agente conversacional *config-driven* con *streaming* SSE y de una visualización 3D inmersiva sobre Three.js.

Competencias generales del Grado. Complementariamente, se han ejercitado de forma directa contenidos de las asignaturas troncales: **Fundamentos de Programación y Estructura de Datos** (backend Python y frontend JS/TS, grafos y diccionarios); **Metodología de la Programación** (Dijkstra, Louvain, Jenks, centralidades y *bus factor* [BGLL08, Dij59, Jen67, Fre77, APHV16]); **Bases de Datos** (MongoDB documental, migración RU → vCore [Mic24c]); **Ingeniería del Software I/II** (patrones, pruebas y SonarQube [PM20, Son24b]); **Sistemas Inteligentes** (LLMs y *function calling* [WMF⁺24, Ope23a]); **IPO I** (visualización 2D/3D [Met24, Mar24, Cab24]); **Redes y Sistemas Distribuidos** (HTTP, REST, SSE, CORS, contenedores [Moz24a, Doc24]); y **Aspectos Profesionales** (planificación, deontología y RGPD).

6.5 Desafíos Encontrados durante el Desarrollo

A lo largo del proyecto han aparecido obstáculos no anticipados que han condicionado la planificación y forzado decisiones técnicas relevantes. Se enumeran a continuación de forma breve, sin ánimo de ser exhaustivos:

- **Crédito limitado de la cuenta de estudiante de Azure (año).** La Fase 1 del proyecto se desarrolló sobre una cuenta con cuotas restrictivas que no permitían, por ejemplo, Azure Static Web Apps. Esto obligó a sostener temporalmente un *deploy* manual del frontend y a operar Cosmos DB en modo *Request Units*, con el consiguiente *throttling* HTTP 429 que limitaba el rendimiento. La concesión posterior de una suscripción completa permitió migrar a vCore y automatizar el despliegue.
- **Curva de aprendizaje del desarrollo de agentes de IA.** Construir el agente conversacional *config-driven* exigió familiarizarse con *prompt engineering*, *function calling* [Ope23a], RAG y búsqueda vectorial, los modelos de la familia GPT-5 sobre Azure AI Foundry, *streaming* vía SSE y patrones agénticos [WMF⁺24], todos ellos contenidos no cubiertos directamente por el plan de estudios y con buenas prácticas todavía en rápida evolución.
- **Curva de aprendizaje del despliegue cloud y la IaC.** Aunque la asignatura *Desarrollo y Gestión de SI* introduce DevOps, llevar a producción un sistema multicomponente con Bicep, Azure Developer CLI, Container Apps, Static Web Apps, redes virtuales, *Managed Identity* y secretos parametrizados ha requerido un esfuerzo de aprendizaje muy superior al inicialmente previsto.

- **Optimización del universo 3D.** Renderizar varios miles de entidades sin penalizar la fluidez obligó a trabajar de forma directa con Three.js: consolidación con InstancedMesh, montaje progresivo en 9 etapas, *shaders* personalizados para los efectos cuánticos y lógica de visibilidad temporal con *frustum culling*, técnicas que requirieron varios ciclos de prueba y medición.
- **Migración de Cosmos DB a vCore (11 ficheros, sin downtime).** La migración de la base de datos del modelo *Request Units* al clúster vCore M30 con API nativa de MongoDB afectó a 11 ficheros del backend y obligó a eliminar los *sleeps* defensivos y la lógica de reintentos, manteniendo el sistema operativo durante la transición.
- **Idempotencia y consistencia del pipeline.** Garantizar que la re-ejecución parcial del *pipeline* no duplicara datos ni dejara documentos huérfanos requirió diseñar la deduplicación por id numérico de GitHub (no por login, sujeto a renombrados) y *bulk upserts* idempotentes (Véase Sec. 5.16.1).
- **Diversidad de tecnologías integradas en un TFG individual.** Coordinar Python/FastAPI, MongoDB, NetworkX, React/Vite, Three.js, GitHub Actions, Bicep, Azure y un agente de IA dentro de un mismo proyecto, manteniendo una calidad mínima en cada capa, ha sido posiblemente el desafío más transversal de todos.

6.6 Valoración Personal

El desarrollo de Entangle ha supuesto un reto técnico y personal de primer orden. El carácter multidisciplinar del proyecto, que abarca desde la ingesta de datos vía API hasta la visualización 3D con *shaders* y el diseño de agentes de IA, ha exigido adquirir y aplicar conocimientos que exceden ampliamente el currículo ordinario del Grado. La necesidad de tomar decisiones en solitario sobre arquitectura, tecnologías y prioridades ha sido, sin duda, una experiencia formativa difícilmente replicable en un entorno académico convencional.

Uno de los aprendizajes más significativos ha sido la gestión de la complejidad en un sistema real desplegado en la nube: los problemas de *throttling* de Cosmos DB, la migración de modelo de facturación, la automatización del despliegue con Bicep y GitHub Actions, y la optimización del renderizado 3D para decenas de miles de objetos son retos que solo emergen cuando el sistema opera en producción, no en ejercicios de laboratorio. Más allá del producto entregado, lo que mejor representa esta experiencia es haber comprobado que es posible llevar de forma individual una idea desde el primer *commit* hasta una plataforma operativa, observable y mantenible, asumiendo a la vez los roles de arquitecto, desarrollador, encargado de despliegue y revisor de calidad. El empleo de herramientas de inteligencia artificial como apoyo durante el desarrollo se ha regido en todo momento por criterios de integridad académica y supervisión humana, según se detalla en el Anexo P.

Referencias

- [Ama24] Amazon Web Services. Amazon Braket — Explore and Experiment with Quantum Computing, 2024. Consultado: 2026. url: <https://aws.amazon.com/braket/>.
- [And10] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, Sequim, WA, USA, 2010.
- [APHV16] Guilherme Avelino, Leonardo Passos, Andre Hora, y Marco Tulio Valente. A Novel Approach for Estimating Truck Factors. En *Proceedings of the 24th International Conference on Program Comprehension (ICPC)*, páginas 1–10, 2016.
- [AVH15] Guilherme Avelino, Marco Tulio Valente, y Andre Hora. What is the Truck Factor of Popular GitHub Applications? A First Assessment. *PeerJ PrePrints*, 2015.
- [BBvB⁺01] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, y Dave Thomas. Manifesto for Agile Software Development. <https://agilemanifesto.org/>, 2001.
- [BGLL08] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, y Etienne Lefebvre. Fast Unfolding of Communities in Large Networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008, 2008.
- [BMR⁺20] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, et al. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020. url: <https://arxiv.org/abs/2005.14165>.
- [Boh13] Niels Bohr. On the Constitution of Atoms and Molecules. *Philosophical Magazine*, 26(151):1–25, 1913.

- [Cab24] Ricardo Cabello. Three.js — JavaScript 3D Library, 2024. Consultado: 2026. url: <https://threejs.org/>.
- [CBSG17] Andrew W. Cross, Lev S. Bishop, John A. Smolin, y Jay M. Gambetta. Open Quantum Assembly Language. *arXiv preprint arXiv:1707.03429*, 2017. url: <https://arxiv.org/abs/1707.03429>.
- [CC18] Eldan Cohen y Mariano P. Consens. Large-Scale Analysis of the Co-Commit Patterns of the Active Developers in GitHub’s Top Repositories. En *Proceedings of the 15th International Conference on Mining Software Repositories (MSR)*, páginas 426–436, 2018.
- [CS14] Scott Chacon y Ben Straub. *Pro Git*. Apress, edición 2, 2014. Acceso libre en <https://git-scm.com>. url: <https://git-scm.com/book/en/v2>.
- [CTJ⁺21] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021. url: <https://arxiv.org/abs/2107.03374>.
- [Dij59] Edsger W. Dijkstra. A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1(1):269–271, 1959.
- [Doc24] Docker, Inc. Docker Documentation, 2024. Consultado: 2026. url: <https://docs.docker.com/>.
- [Dys60] Freeman J. Dyson. Search for Artificial Stellar Sources of Infrared Radiation. *Science*, 131(3414):1667–1668, 1960.
- [Fac24] Facultad de Ciencias Sociales y Tecnologías de la Información, Universidad de Castilla-La Mancha. Guías docentes oficiales de las asignaturas de la mención *Sistemas de Información* del Grado en Ingeniería Informática (plan 405, Campus de Talavera de la Reina). Repositorio institucional UCLM, 2024. Códigos SI01–SI06 publicados literalmente en las guías docentes de las asignaturas de intensificación 42401–42407. url: <https://guias.apps.uclm.es/2023-24/405/42401/GuiaES.pdf>.
- [FB07] Santo Fortunato y Marc Barthélemy. Resolution Limit in Community Detection. *Proceedings of the National Academy of Sciences*, 104(1):36–41, 2007.
- [FHK18] Nicole Forsgren, Jez Humble, y Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, Portland, OR, 2018.
- [For10] Santo Fortunato. Community Detection in Graphs. *Physics Reports*, 486(3–5):75–174, 2010.

- [Fre77] Linton C. Freeman. A Set of Measures of Centrality Based on Betweenness. *Sociometry*, 40(1):35–41, 1977.
- [Git24a] GitHub, Inc. GitHub Actions Documentation, 2024. Consultado: 2026. url: <https://docs.github.com/en/actions>.
- [Git24b] GitHub, Inc. GitHub GraphQL API Documentation, 2024. Consultado: 2026. url: <https://docs.github.com/en/graphql>.
- [Git24c] GitHub, Inc. GitHub REST API Documentation, 2024. Consultado: 2026. url: <https://docs.github.com/en/rest>.
- [Git24d] GitLab B.V. GitLab API Documentation, 2024. Consultado: 2026. url: <https://docs.gitlab.com/ee/api/>.
- [Goo24a] Google. Open Source Insights (deps.dev), 2024. Consultado: 2025. url: <https://deps.dev/>.
- [Goo24b] Google Quantum AI. Cirq — A Python Framework for Creating, Editing, and Invoking Quantum Circuits, 2024. Consultado: 2026. url: <https://quantumai.google/cirq>.
- [Gou13] Georgios Gousios. The GHTorrent Dataset and Tool Suite. *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*, páginas 233–236, 2013.
- [Gri15] Ilya Grigorik. GH Archive — A Project to Record the Public GitHub Timeline, 2015. Consultado: 2026. url: <https://github.com/igrigorik/gharchive.org>.
- [Gro96] Lov K. Grover. A Fast Quantum Mechanical Algorithm for Database Search. En *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, páginas 212–219. ACM, 1996.
- [GS18] David J. Griffiths y Darrell F. Schroeter. *Introduction to Quantum Mechanics*. Cambridge University Press, edición 3rd, 2018.
- [HF10] Jez Humble y David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010.
- [Hic15] Ian Hickson. Server-Sent Events. W3C Recommendation, 2015. url: <https://www.w3.org/TR/eventsource/>.
- [HSS24] Aric A. Hagberg, Daniel A. Schult, y Pieter J. Swart. NetworkX — Network Analysis in Python, 2024. Consultado: 2026. url: <https://networkx.org/>.

- [IBM24] IBM Quantum. Qiskit — Open-Source Quantum Development, 2024. Consultado: 2026. url: <https://qiskit.org/>.
- [Int23] International Data Corporation. Worldwide Quantum Computing Forecast, 2023–2027, 2023. Informe IDC #US51310523. url: <https://www.idc.com/getdoc.jsp?containerId=US51310523>.
- [Jen67] George F. Jenks. The Data Model Concept in Statistical Mapping. *International Yearbook of Cartography*, 7:186–190, 1967.
- [JLF⁺23] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Jin Bang, Andrea Madotto, y Pascale Fung. Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [JYW⁺24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, y Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? En *International Conference on Learning Representations (ICLR)*, 2024. url: <https://arxiv.org/abs/2310.06770>.
- [Kar25] Andrej Karpathy. There’s a new kind of coding I call “vibe coding”. Tweet on X, Febrero 2025. url: <https://x.com/karpathy/status/1886192184808149383>.
- [Ken24] Kent C. Dodds and contributors. Testing Library — Simple and Complete Testing Utilities, 2024. Consultado: 2026. url: <https://testing-library.com/>.
- [KGB⁺14] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, y Daniela Damian. The Promises and Perils of Mining GitHub. *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, páginas 92–101, 2014.
- [KMH⁺20] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, y Dario Amodei. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*, 2020. url: <https://arxiv.org/abs/2001.08361>.
- [Lad08] Corey Ladas. Scrumban: Essays on Kanban Systems for Lean Software Development, 2008.
- [Lei24] Ian James Leitch. orjson — Fast, Correct Python JSON Library, 2024. Consultado: 2026. url: <https://github.com/ijl/orjson>.
- [Lib24] Libraries.io contributors. Libraries.io — Open Source Discovery Service, 2024. Consultado: 2025. url: <https://libraries.io/>.

- [Lon11] Malcolm S. Longair. *High Energy Astrophysics*. Cambridge University Press, edición 3rd, 2011.
- [LPP⁺20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020. url: <https://arxiv.org/abs/2005.11401>.
- [Luc24] Lucide contributors. Lucide React — Beautiful and Consistent Icon Toolkit for React, 2024. Consultado: 2026. url: <https://lucide.dev/>.
- [LYZ⁺24] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as Agents. En *International Conference on Learning Representations (ICLR)*, 2024. url: <https://arxiv.org/abs/2308.03688>.
- [Mar24] Vasco Mart  nez. react-force-graph — Force-Directed Graph Component for React, 2024. Consultado: 2026. url: <https://github.com/vasturiano/react-force-graph>.
- [McK24] McKinsey & Company. Quantum Technology Monitor — April 2024, 2024. Consultado: 2026. url: <https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/steady-progress-in-approaching-the-quantum-advantage>.
- [Met24] Meta Platforms, Inc. React — A JavaScript Library for Building User Interfaces, 2024. Consultado: 2026. url: <https://react.dev/>.
- [Mic24a] Microsoft. Azure AI Services Overview, 2024. Consultado: 2026. url: <https://azure.microsoft.com/en-us/products/ai-services>.
- [Mic24b] Microsoft. Azure Container Apps Documentation, 2024. Consultado: 2026. url: <https://learn.microsoft.com/en-us/azure/container-apps/>.
- [Mic24c] Microsoft. Azure Cosmos DB Documentation, 2024. Consultado: 2026. url: <https://learn.microsoft.com/en-us/azure/cosmos-db/>.
- [Mic24d] Microsoft. Azure Documentation, 2024. Consultado: 2026. url: <https://learn.microsoft.com/en-us/azure/>.
- [Mic24e] Microsoft. Azure Security Best Practices and Patterns, 2024. Consultado: 2026. url: <https://learn.microsoft.com/en-us/azure/security/fundamentals/best-practices-and-patterns>.

- [Mic24f] Microsoft. Bicep — Domain-Specific Language for Azure Resource Deployment, 2024. Consultado: 2026. url: <https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/>.
- [Mic24g] Microsoft. Configure Azure Static Web Apps, 2024. Consultado: 2026. url: <https://learn.microsoft.com/en-us/azure/static-web-apps/configuration>.
- [Mon24a] MongoDB, Inc. MongoDB Documentation, 2024. Consultado: 2026. url: <https://www.mongodb.com/docs/>.
- [Mon24b] MongoDB, Inc. PyMongo — Python Driver for MongoDB, 2024. Consultado: 2026. url: <https://pymongo.readthedocs.io/en/stable/>.
- [Moz24a] Mozilla Developer Network. Cross-Origin Resource Sharing (CORS), 2024. Consultado: 2026. url: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>.
- [Moz24b] Mozilla Developer Network. Internationalization (i18n), 2024. Consultado: 2026. url: <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Internationalization>.
- [Moz24c] Mozilla Developer Network. Server-Sent Events (SSE), 2024. Consultado: 2026. url: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events.
- [Moz24d] Mozilla Developer Network. Using Web Workers, 2024. Consultado: 2026. url: https://developer.mozilla.org/en-US/docs/Web/API/WebWorkersAPI/Using_web_workers.
- [NC10] Michael A. Nielsen y Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, edición 10th Anniversary, 2010.
- [New03] Mark E. J. Newman. The Structure and Function of Complex Networks. *SIAM Review*, 45(2):167–256, 2003.
- [NKMS12] Natalja Nikitina, Mira Kajko-Mattsson, y Magnus Straåle. From Scrum to Scrumban: A Case Study of a Process Transition. En *Proceedings of the International Conference on Software and System Process (ICSSP 2012)*, páginas 140–149. IEEE, 2012.
- [Ope23a] OpenAI. Function Calling and Other API Updates. <https://openai.com/index/function-calling-and-other-api-updates/>, 2023. Consultado: 2025-01-15.
- [Ope23b] OpenAI. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*, 2023. url: <https://arxiv.org/abs/2303.08774>.

- [OWA25] OWASP Foundation. OWASP Top 10 for Large Language Model Applications, 2025. Consultado en marzo de 2025. url: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [OWJ⁺22] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training Language Models to Follow Instructions with Human Feedback. En *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, páginas 27730–27744, 2022.
- [PAT⁺22] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, y Ramesh Karri. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. En *2022 IEEE Symposium on Security and Privacy (SP)*, páginas 754–768. IEEE, 2022.
- [PM20] Roger S. Pressman y Bruce R. Maxim. *Software Engineering: A Practitioner’s Approach*. McGraw-Hill, edición 9th, 2020.
- [pmn24a] pmndrs. React Three Fiber — A React Renderer for Three.js, 2024. Consultado: 2026. url: <https://r3f.docs.pmnd.rs/>.
- [pmn24b] pmndrs. Zustand — Bear Necessities for State Management in React, 2024. Consultado: 2026. url: <https://zustand-demo.pmnd.rs/>.
- [Pre18] John Preskill. Quantum Computing in the NISQ Era and Beyond. *Quantum*, 2:79, 2018.
- [pyt24] pytest contributors. pytest — Full-Featured Python Testing Tool, 2024. Consultado: 2026. url: <https://docs.pytest.org/en/stable/>.
- [Ram24] Sebastián Ramírez. FastAPI — Modern, Fast Web Framework for Building APIs with Python, 2024. Consultado: 2026. url: <https://fastapi.tiangolo.com/>.
- [Rec24] Recharts contributors. Recharts — A Composable Charting Library Built on React Components, 2024. Consultado: 2026. url: <https://recharts.org/>.
- [Rei09] Donald G. Reinertsen. *The Principles of Product Development Flow: Second Generation Lean Product Development*. Celeritas Publishing, Redondo Beach, CA, 2009.
- [rem24] remarkjs contributors. react-markdown — Markdown Component for React, 2024. Consultado: 2026. url: <https://github.com/remarkjs/react-markdown>.

- [RWC⁺19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, y Ilya Sutskever. Language Models are Unsupervised Multitask Learners, 2019. OpenAI Technical Report. url: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
- [Sar25] Advait Sarkar. Vibe Coding: Programming through Conversation with Artificial Intelligence. *arXiv preprint arXiv:2506.23253*, 2025. url: <https://arxiv.org/abs/2506.23253>.
- [Sch05] Maximilian Schlosshauer. Decoherence, the measurement problem, and interpretations of quantum mechanics. *Reviews of Modern Physics*, 76(4):1267–1305, 2005.
- [SDYD⁺23] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, y Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [Sec09] Secretaría General de Universidades, Ministerio de Ciencia e Innovación. Resolución de 8 de junio de 2009, de la Secretaría General de Universidades, por la que se da publicidad al Acuerdo del Consejo de Universidades, por el que se establecen recomendaciones para la propuesta por las universidades de memorias de solicitud de títulos oficiales en los ámbitos de la Ingeniería Informática, Ingeniería Técnica Informática e Ingeniería Química. Boletín Oficial del Estado, núm. 187, de 4 de agosto de 2009, BOE-A-2009-12977, 2009. url: <https://www.boe.es/buscar/doc.php?id=BOE-A-2009-12977>.
- [Sho94] Peter W. Shor. Algorithms for Quantum Computation: Discrete Logarithms and Factoring. En *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, páginas 124–134. IEEE, 1994.
- [Son24a] SonarSource SA. SonarCloud — Cloud-based Code Quality Service, 2024. Consultado: 2026. url: <https://www.sonarsource.com/products/sonarcloud/>.
- [Son24b] SonarSource SA. SonarQube — Code Quality and Code Security, 2024. Consultado: 2026. url: <https://www.sonarsource.com/products/sonarqube/>.
- [Son24c] SonarSource SA. SonarScanner — Scanner for SonarQube, 2024. Consultado: 2026. url: <https://docs.sonarsource.com/sonarqube/latest/analyzing-source-code/scanners/sonarscanner/>.
- [SS20] Ken Schwaber y Jeff Sutherland. *The Scrum Guide*. Scrum.org, 2020. url: <https://scrumguides.org/scrum-guide.html>.

- [Sut14] Jeff Sutherland. *Scrum: The Art of Doing Twice the Work in Half the Time*. Crown Business, New York, 2014.
- [TSK26] Maryam Tavassoli Sabzevari y Arif Ali Khan. Empirical Investigation of Quantum Computing Toolchains and Algorithms: Mining Stack Overflow Repository. En *Companion Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE Companion 2026)*. ACM, 2026. arXiv:2604.15512 [cs.SE].
- [Tuf01] Edward R. Tufte. *The Visual Display of Quantitative Information*. Graphics Press, edición 2nd, 2001.
- [TWvE19] Vincent A. Traag, Ludo Waltman, y Nees Jan van Eck. From Louvain to Leiden: Guaranteeing Well-Connected Communities. *Scientific Reports*, 9(1):5233, 2019.
- [Uni20] Universidad de Castilla-La Mancha. Resolución de 2 de octubre de 2020, de la Universidad de Castilla-La Mancha, por la que se publica la modificación del plan de estudios de Graduado o Graduada en Ingeniería Informática. Boletín Oficial del Estado, núm. 282, de 27 de octubre de 2020, BOE-A-2020-13016, 2020. url: https://www.boe.es/diario_boe/txt.php?id=BOE-A-2020-13016.
- [Vit24] Vitest contributors. Vitest — Next Generation Testing Framework, 2024. Consultado: 2026. url: <https://vitest.dev/>.
- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, y Illia Polosukhin. Attention is All You Need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [WBZ⁺23] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, y Chi Wang. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*, 2023. url: <https://arxiv.org/abs/2308.08155>.
- [Whe57] John A. Wheeler. On the Nature of Quantum Geometrodynamics. *Annals of Physics*, 2(6):604–614, 1957.
- [WMF⁺24] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, y Ji-Rong Wen. A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

- [Xan24] Xanadu. PennyLane — A Cross-Platform Python Library for Quantum Computing, 2024. Consultado: 2026. url: <https://pennylane.ai/>.
- [XCG⁺23] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The Rise and Potential of Large Language Model Based Agents: A Survey. *arXiv preprint arXiv:2309.07864*, 2023. url: <https://arxiv.org/abs/2309.07864>.
- [You24] Evan You. Vite — Next Generation Frontend Tooling, 2024. Consultado: 2026. url: <https://vite.dev/>.
- [YZY⁺23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, y Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. En *International Conference on Learning Representations (ICLR)*, 2023. url: <https://arxiv.org/abs/2210.03629>.
- [ZKL⁺24] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, y Edward Aftandilian. Measuring GitHub Copilot’s Impact on Productivity. *Communications of the ACM*, 67(3):54–63, 2024.
- [Zur03] Wojciech H. Zurek. Decoherence, Einselection, and the Quantum Origins of the Classical. *Reviews of Modern Physics*, 75(3):715–775, 2003.

ANEXOS

Anexo A

Manual de Instalación y Despliegue

Este anexo describe los pasos necesarios para instalar, configurar y desplegar la plataforma Entangle tanto en un entorno de desarrollo local como en producción sobre Azure.

A.1 Prerrequisitos

- **Backend:** Python 3.11, pip.
- **Frontend:** Node.js $\geq 18.x$, npm $\geq 9.x$.
- **Base de datos:** MongoDB (local o Azure Cosmos DB for MongoDB vCore).
- **Docker:** necesario para el despliegue en contenedores.
- **Azure CLI** (az): necesario para el despliegue a Azure.
- **Azure Developer CLI** (azd): opcional, simplifica el despliegue con infraestructura Bicep.

A.2 Ejecución Local

Se describen los pasos para ejecutar cada componente del sistema en un entorno de desarrollo local: el backend Python (Sección siguiente) y el frontend React.

A.2.1 Backend

1. Clonar el repositorio e instalar las dependencias:

```
1 cd Entangle-Core
2 pip install -r requirements.txt
```

2. Copiar el fichero de variables de entorno y configurar los valores (véase el Anexo E):

```
1 cp .env.example .env
```

3. Arrancar el servidor de desarrollo:

```
1 | uvicorn src.api.main:app --host 0.0.0.0 --port 8000 --reload
```

La API queda accesible en <http://localhost:8000/api/v1>.

A.2.2 Frontend

1. Instalar las dependencias y configurar la URL del backend:

```
1 | cd Entangle-Visualizer
2 | npm install
3 | cp .env.example .env
```

En el fichero `.env`, ajustar la variable `VITE_API_URL` a la dirección del backend (por defecto <http://localhost:8000/api/v1>).

2. Arrancar el servidor de desarrollo:

```
1 | npm run dev
```

La aplicación queda accesible en <http://localhost:5173>.

3. Para generar la versión de producción:

```
1 | npm run build
```

Los artefactos se generan en el directorio `dist/`.

A.3 Despliegue con Docker

El backend incluye un `Dockerfile` basado en `python:3.11-slim` que ejecuta la aplicación con un usuario no privilegiado (`appuser`):

```
1 | docker build -t entangle-api .
2 | docker run -p 8000:8000 --env-file .env entangle-api
```

El health check del contenedor se realiza contra el endpoint `GET /api/v1/health`.

A.4 Despliegue a Azure

Se detallan los tres componentes que conforman el despliegue en la nube: el backend como contenedor en Azure Container Apps, el frontend como Azure Static Web App, y la puesta en marcha mediante Azure Developer CLI.

A.4.1 Backend: Azure Container Apps

Existen tres métodos de despliegue:

1. **Azure Developer CLI (azd)**: el fichero `azure.yaml` define el servicio `api` con host `containerapp` e infraestructura Bicep en `./infra`:

```
1 | azd up
```

Este comando aprovisiona el grupo de recursos, el registro de contenedores (ACR), el entorno de Container Apps y despliega la imagen.

2. **Script manual** (`deploy.ps1`): ejecuta cuatro pasos secuenciales: autenticación en ACR, construcción de la imagen Docker, push al registro y actualización de la Container App.
3. **GitHub Actions**: el workflow de despliegue automático se dispara en cada push a la rama `main`. Utiliza la acción `azure/container-apps-deploy-action@v2` con el registro `crentangle.azurecr.io`.

La infraestructura Bicep (`infra/main.bicep`) aprovisiona: grupo de recursos, Log Analytics, Container Registry, entorno de Container Apps y la propia Container App con 1.0 vCPU, 2.0 GiB de memoria y autoescalado de 0 a 3 réplicas.

A.4.2 Frontend: Azure Static Web Apps

El frontend se despliega automáticamente a Azure Static Web Apps mediante un workflow de GitHub Actions que utiliza autenticación OpenID Connect (OIDC). El fichero `staticwebapp.config.json` configura el fallback Single Page Application (SPA) para el enrutamiento del lado del cliente y las cabeceras de seguridad (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options).

Anexo B

Palabras Clave del Pipeline de Ingesta

El pipeline de ingesta utiliza la API de búsqueda de GitHub para localizar repositorios relacionados con la computación cuántica. La configuración del fichero `ingestion_config.json` define dos conjuntos de palabras clave con distintas funciones.

B.1 Palabras clave de búsqueda

Las siguientes 8 palabras clave se utilizan como términos directos en las consultas a la API de búsqueda de GitHub. Representan los marcos de trabajo y estándares más relevantes del ecosistema cuántico:

1. quantum
2. qiskit
3. cirq
4. pennylane
5. braket
6. pyquil
7. openqasm
8. projectq

B.2 Palabras clave de filtrado

Las siguientes 71 palabras clave se utilizan como filtro posterior para verificar que los repositorios encontrados están efectivamente relacionados con la computación cuántica. Un repositorio debe contener al menos una de estas palabras clave en su nombre, descripción o topics para ser incluido en el dataset. Las primeras 8 palabras clave de búsqueda están incluidas también en esta lista de filtrado, por lo que actúan tanto como términos de búsqueda como de validación.

Tabla B.1: Lista completa de palabras clave de filtrado.

#	Palabra clave	#	Palabra clave
1	quantum	37	projectq
2	qiskit	38	quantum inspire
3	braket	39	quantum development kit
4	cirq	40	forest
5	pennylane	41	rigetti
6	quantum computing	42	ionq
7	quantum algorithms	43	d-wave
8	quantum machine learning	44	ibm quantum
9	qml	45	google quantum
10	qubit	46	azure quantum
11	quantum simulator	47	aws quantum
12	quantum circuit	48	quantum supremacy
13	quantum annealing	49	quantum advantage
14	quantum optimization	50	nisq
15	quantum cryptography	51	fault-tolerant
16	quantum teleportation	52	topological quantum
17	quantum entanglement	53	adiabatic quantum
18	quantum superposition	54	variational quantum
19	quantum gates	55	hybrid quantum
20	quantum error correction	56	quantum neural network
21	quantum chemistry	57	qnn
22	quantum mechanics	58	quantum walk
23	quantum physics	59	quantum fourier transform
24	quantum programming	60	grover algorithm
25	quantum software	61	shor algorithm
26	quantum hardware	62	quantum key distribution
27	quantum processor	63	qkd
28	quantum computer	64	bell state
29	qpu	65	ghz state
30	qaoa	66	bloch sphere
31	vqe	67	density matrix
32	quil	68	quantum state
33	openqasm	69	quantum measurement
34	qasm	70	quantum decoherence
35	pyquil	71	quantum noise
36	strawberry fields		

Anexo C

Estructura del Repositorio

Este anexo presenta la organización de directorios de los dos repositorios del proyecto: Entangle-Core, que aloja el backend (API y *pipeline* de ingesta), y Entangle-Visualizer, que aloja el frontend (interfaz de usuario).

C.1 Entangle-Core (backend)

```
1  Entangle-Core/
2    .github/workflows/          # CI/CD: despliegue a Azure Container Apps
3    config/
4      ingestion_config.json     # Keywords, filtros, segmentacion
5      pipeline_config.json      # Parametros del pipeline
6      workers.yaml              # Definicion config-driven de workers
7    infra/
8      main.bicep                 # IaC principal
9      main.parameters.json
10   core/
11     host/                      # Bicep: container-app, ACR, env
12     monitor/                   # Bicep: Log Analytics
13   scripts/
14     run_full_pipeline.py       # Pipeline completo
15     run_repositories_ingestion.py
16     run_repositories_enrichment.py
17     run_user_ingestion.py
18     run_user_enrichment.py
19     run_organization_ingestion.py
20     run_organization_enrichment.py
21     audit_dataset.py           # Auditoria del dataset
22     clean_dataset.py           # Limpieza de datos
23     analyze_radar_data.py      # Analisis radar organizaciones
24   src/
25     ai/                         # Agente IA config-driven
```

```

26     agent.py                # Orquestador (router + workers)
27     workers.py             # Carga de config/workers.yaml
28     prompts.py             # System prompts de los workers
29     tool_registry.py        # Registro de tools (@tool)
30     tool_functions.py       # Tools de datos (worker DATA)
31     rag_tool.py             # Tool RAG (worker KNOWLEDGE)
32     insights_tools.py       # Tools de analisis (INSIGHTS)
33     research_tools.py       # Tools externas (DEEP_RESEARCH)
34     chunker.py              # Fragmentacion de READMEs
35     embedder.py             # Generacion de embeddings
36     indexer.py              # Indexado vectorial idempotente
37 analysis/
38     discipline_classifier.py # Clasificador de disciplinas
39     network_metrics.py      # Metricas de red (NetworkX)
40 api/
41     main.py                 # App FastAPI + CORS + lifespan
42     routes.py               # Router principal (43 endpoints)
43     admin_routes.py         # Panel admin (11 endpoints)
44     chat_routes.py          # Chat IA (5 endpoints)
45 core/
46     config.py               # Variables de entorno
47     db.py                   # Conexion MongoDB
48     mongo_repository.py     # Patron Repository
49     chunked_cache.py        # Cache chunked en memoria
50     session_memory.py       # Memoria conversacional (chat_sessions)
51     vector_search.py        # Búsqueda vectorial (IVF + coseno)
52     logger.py               # Configuración logging
53 github/
54     graphql_client.py       # Cliente GitHub GraphQL
55     queries.py               # Consultas GraphQL
56     extract.py               # Extracción de datos
57     filters.py               # Filtros de repositorios
58     rate_limit.py           # Gestión rate limit
59     repositories_ingestion.py
60     repositories_enrichment.py
61     user_ingestion.py
62     user_enrichment.py
63     organization_ingestion.py
64     organization_enrichment.py
65 models/
66     repository.py            # Modelo Repository

```

```

67     user.py                # Modelo User
68     organization.py        # Modelo Organization
69     utils/
70     tests/                  # 919 tests pytest
71     Dockerfile              # python:3.11-slim
72     requirements.txt
73     azure.yaml              # Configuracion azd
74     deploy.ps1              # Script despliegue manual

```

C.2 Entangle-Visualizer (frontend)

```

1  Entangle-Visualizer/
2    .github/workflows/      # CI/CD: Azure Static Web Apps
3    src/
4      components/
5        Dashboard/          # 35+ componentes del dashboard
6          AdminPanel.jsx
7          ChartsSection.jsx
8          CollaborationPanel.jsx
9          NetworkGraph.jsx
10         KPISection.jsx
11         DetailTable.jsx
12         FloatingChat.jsx
13         QuantumChat.jsx
14         FavoritesPanel.jsx
15         BridgeUsersTable.jsx
16         OrgComparisonRadar.jsx
17         TechStackMap.jsx
18         ContributorSankey.jsx
19         ViewBar.jsx
20     Universe/               # Visualizacion 3D
21       UniverseView.jsx
22       BigBangEntry.jsx
23       BlackHoleExit.jsx
24       computeLayout.worker.js
25       computeDetailData.worker.js
26     BlochSphere.jsx
27     EntanglementLines.jsx
28     QuantumBackground.jsx

```

```
29     QuantumDivider.jsx
30     WavefunctionCollapse.jsx
31     ErrorBoundary.jsx
32 services/
33     api.js                # Cliente HTTP (Axios)
34 store/                   # Estado global (Zustand)
35     dashboardStore.js
36     adminStore.js
37     favoritesStore.js
38     devStore.js
39 hooks/
40     useEnrichedData.js
41 i18n/                    # Internacionalizacion (i18next)
42     index.js
43     locales/             # en, es, de, fr, pt
44 App.jsx
45 main.jsx
46 staticwebapp.config.json # Config Azure SWA
47 vite.config.js
48 package.json
49 .env.example
```

Anexo D

Catálogo de Endpoints de la API

La API REST de Entangle expone 59 endpoints organizados en tres routers, todos montados bajo el prefijo `/api/v1`. Este anexo documenta la lista completa.

D.1 Router Principal (43 endpoints)

Tabla D.1: Endpoints del router principal.

Método	Ruta	Descripción
GET	/	Información de la API
GET	/health	Health check
GET	/stats	Estadísticas generales
GET	/dashboard/stats	Estadísticas del dashboard
POST	/dashboard/refresh-metrics	Refrescar métricas
GET	/collaboration/discover	Descubrir red de colaboración
POST	/collaboration/discover/invalidate	Invalidar caché de discovery
POST	/collaboration/analyze	Analizar colaboración
GET	/collaboration/user/{login}	Perfil de colaboración de un usuario
GET	/collaboration/network-metrics	Métricas de red
GET	/collaboration/quantum-tunneling	Análisis quantum tunneling
GET	/rate-limit	Estado del rate limit de GitHub
GET	/organizations/github/{org}	Organización desde GitHub API
GET	/repositories/github/{owner}/{name}	Repositorio desde GitHub API
GET	/users/github/{login}	Usuario desde GitHub API
GET	/search/repositories	Buscar repositorios en GitHub
GET	/repositories	Listar repositorios (BD)
GET	/repositories/db/{id}	Repositorio por ID (BD)
GET	/users	Listar usuarios (BD)
GET	/users/db/{id}	Usuario por ID (BD)
GET	/users/profile/{login}	Perfil de usuario
GET	/organizations	Listar organizaciones (BD)

Método	Ruta	Descripción
GET	/organizations/db/{id}	Organización por ID (BD)
POST	/ingestion/repositories	Iniciar ingesta de repositorios
POST	/ingestion/users	Iniciar ingesta de usuarios
POST	/ingestion/organizations	Iniciar ingesta de organizaciones
POST	/enrichment/repositories	Enriquecer repositorios
POST	/enrichment/users	Enriquecer usuarios
POST	/enrichment/organizations	Enriquecer organizaciones
GET	/ingestion/status/{task_id}	Estado de tarea de ingesta
GET	/enrichment/status/{task_id}	Estado de tarea de enriquecimiento
GET	/tasks	Listar tareas
POST	/pipeline/run-all	Ejecutar pipeline completo
GET	/favorites	Listar favoritos
POST	/favorites	Añadir favorito
DELETE	/favorites/{entity_id}	Eliminar favorito
GET	/favorites/{entity_id}/children	Hijos de un favorito
GET	/views	Listar vistas guardadas
POST	/views	Crear vista
DELETE	/views/{view_id}	Eliminar vista
POST	/views/{view_id}/data	Datos de una vista
GET	/search/entity/{entity_id}	Buscar entidad por ID
GET	/search/entities	Búsqueda general de entidades

D.2 Router de Administración (11 endpoints)

Todos los endpoints de este router se montan bajo el prefijo `/api/v1/admin`.

Tabla D.2: Endpoints del router de administración.

Método	Ruta	Descripción
POST	/admin/auth	Autenticación del administrador
POST	/admin/setup-password	Configurar contraseña
GET	/admin/has-password	Comprobar si existe contraseña
POST	/admin/operations/run	Ejecutar operación
GET	/admin/operations/active	Operaciones activas
GET	/admin/operations/{id}	Detalle de una operación
POST	/admin/operations/{id}/cancel	Cancelar operación
GET	/admin/operations/{id}/logs	Logs de una operación
GET	/admin/history	Historial de operaciones

Método	Ruta	Descripción
DELETE	/admin/history	Limpiar historial
GET	/admin/db-stats	Estadísticas de la base de datos

D.3 Router de Chat IA (5 endpoints)

Tabla D.3: Endpoints del router de chat.

Método	Ruta	Descripción
POST	/chat	Enviar mensaje al agente IA
POST	/chat/stream	Chat con respuesta en streaming
POST	/chat/research	Investigación externa (worker DEEP_RESEARCH)
GET	/chat/session/{id}/stats	Estadísticas de una sesión de chat
DELETE	/chat/session/{id}	Reiniciar la memoria de una sesión de chat

Anexo E

Variables de Entorno y Dependencias

Este anexo detalla las variables de entorno necesarias para configurar ambos componentes de la plataforma, así como las dependencias principales de cada uno.

E.1 Variables de Entorno del Backend

Tabla E.1: Variables de entorno del backend.

Variable	Defecto	Descripción
GITHUB_TOKEN	(requerido)	Token de acceso personal de GitHub con permisos repo, read:org y read:user.
MONGO_URI	mongodb://localhost:27017/	URI de conexión a MongoDB o Azure Cosmos DB for MongoDB vCore.
MONGO_DB_NAME	quantum_github	Nombre de la base de datos.
ENVIRONMENT	development	Entorno de ejecución: development, staging o production.
DEBUG	True	Activa el modo de depuración.
API_HOST	0.0.0.0	Host del servidor.
API_PORT	8000	Puerto del servidor.
PORT	8000	Puerto (override para Azure Container Apps).
LOG_LEVEL	INFO	Nivel de logging.
FRONTEND_URL	(vacío)	URL del frontend para la configuración CORS.
AZURE_AI_ENDPOINT	(vacío)	Endpoint de Azure AI Foundry.
AZURE_AI_PROJECT	(vacío)	Nombre del proyecto en AI Foundry.
AZURE_AI_API_KEY	(vacío)	Clave API de AI Foundry.

Variable	Defecto	Descripción
AZURE_AI_DEPLOYMENT	gpt-5-mini	Nombre del deployment del modelo.
TAVILY_API_KEY	(vacío)	Clave de la API de Tavily para la búsqueda web del <i>worker</i> DEEP-RESEARCH. Si está vacía, la herramienta devuelve un error controlado.

Adicionalmente, los scripts del pipeline de ingesta aceptan las siguientes variables opcionales:

Tabla E.2: Variables de entorno de los scripts de ingesta.

Variable	Descripción
INGESTION_MODE	Modo de ingesta: incremental o full.
MAX_WORKERS	Número de workers paralelos (por defecto 4).
AUTO_CONFIRM	Saltar confirmaciones interactivas (true/false).
ENRICHMENT_LIMIT	Límite de entidades a enriquecer por ejecución.
BATCH_SIZE	Tamaño de lote para el enriquecimiento.
FORCE_REENRICHMENT	Forzar re-enriquecimiento de entidades ya procesadas (true/false).

E.2 Variables de Entorno del Frontend

Tabla E.3: Variables de entorno del frontend.

Variable	Defecto	Descripción
VITE_API_URL	http://localhost:8000/api/v1	URL base del backend.
VITE_DEBUG	true	Activa logs adicionales en la consola del navegador.

E.3 Dependencias Principales

E.3.1 Backend

Versiones extraídas de `requirements.txt`:

Tabla E.4: Dependencias principales del backend.

Paquete	Versión
FastAPI	0.115.0
Uvicorn	0.32.0
PyMongo	4.10.1
Pydantic	2.9.2
NetworkX	≥ 3.0
orjson	(última)
python-dotenv	(última)
azure-identity	(última)
bcrypt	(última)

E.3.2 Frontend

Versiones extraídas de `package.json`:

Tabla E.5: Dependencias principales del frontend.

Paquete	Versión
React	19.2
Three.js	0.182
React Three Fiber	(última)
Drei	(última)
react-force-graph-2d	(última)
Recharts	(última)
Zustand	(última)
Axios	(última)
i18next	(última)
react-router-dom	(última)
Vite	7.2

E.4 Colecciones de la Base de Datos

La base de datos MongoDB (nombre por defecto: `quantum_github`) utiliza las siguientes colecciones:

Tabla E.6: Colecciones de la base de datos.

Colección	Descripción
<code>repositories</code>	Repositorios de software cuántico ingestados y enriquecidos.

Colección	Descripción
users	Contribuidores y desarrolladores.
organizations	Organizaciones de GitHub.
metrics	Métricas pre-calculadas del dashboard.
ingestion_metadata	Metadatos de los procesos de ingesta (timestamps, conteos).
user_preferences	Favoritos y vistas guardadas del usuario.
admin_config	Configuración del panel de administración.
operation_history	Historial de operaciones ejecutadas desde el panel admin.
chat_sessions	Memoria conversacional del agente de IA: turnos por sesión, con índice TTL de 30 días.
repos_chunks	Fragmentos vectorizados de los README (embeddings 1536d) para la búsqueda semántica del RAG.

Anexo F

Modelo de Datos Detallado

Este anexo amplía la descripción resumida del modelo de datos presentada en la Sección 5.4, detallando todos los campos de cada colección MongoDB.

A continuación se presentan ejemplos representativos de documentos para cada colección principal del modelo de datos de Entangle. Estos listados complementan la descripción resumida de la Sección 5.4.

F.1 Colección de Repositorios

Cada documento incluye campos de identificación (`id`, `name`, `full_name`, `name_with_owner`), una `description`, y un objeto `owner` con `id`, `login` y `type`. El campo `primary_language` indica el lenguaje principal, complementado por `languages` (array con `name` y `size`). Los `repository_topics` enumeran los temas asociados, mientras que `stargazer_count` y `fork_count` cuantifican el interés y la replicación.

Los `collaborators` documentan la participación mediante `login`, `contributions` y `has_commits`. Los `recent_commits` almacenan `oid`, `message`, `committed_date` y `author_login`. Los campos `created_at`, `pushed_at`, `is_archived` e `is_fork` registran fechas y estado. Cada documento incluye `license_info` y un objeto `enrichment_status`.

```
1 {
2   "id": "MDEwOlJlcG9zaXRvcnk4MzAz",
3   "name": "qiskit",
4   "full_name": "Qiskit/qiskit",
5   "description": "Qiskit is an open-source SDK for working
6     with quantum computers",
7   "owner": {
8     "id": "MDEyOk9yZ2FuaXphdGlvbG9jaWNTI3MDE1",
9     "login": "Qiskit",
10    "type": "Organization"
11  },
12  "primary_language": "Python",
13  "languages": [
14    {"name": "Python", "size": 2850000},
15    {"name": "C++", "size": 350000}
16  ],
17  "repository_topics": ["quantum-computing", "qiskit"],
18  "stargazer_count": 5384,
19  "fork_count": 2387,
20  "collaborators": [
21    {"login": "mtreinish", "contributions": 3421,
22      "has_commits": true},
```

```

23     {"login": "jakelishman", "contributions": 2876,
24       "has_commits": true}
25   ],
26   "recent_commits": [
27     {"oid": "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6",
28       "message": "Fix transpiler pass ordering",
29       "committed_date": "2026-01-22T13:15:00Z",
30       "author_login": "mtreinish"}
31   ],
32   "license_info": {
33     "key": "apache-2.0",
34     "name": "Apache License 2.0"
35   },
36   "created_at": "2017-03-05T09:00:00.000Z",
37   "pushed_at": "2026-01-22T13:15:00.000Z",
38   "ingested_at": "2026-03-02T12:22:18.992Z",
39   "is_archived": false,
40   "is_fork": false,
41   "enrichment_status": {
42     "is_complete": true,
43     "total_fields_enriched": 19
44   }
45 }

```

Listado F.1: Ejemplo de documento en la colección repositories

F.2 Colección de Usuarios

Cada documento incluye id, login y name. Campos como bio, company y location proporcionan contexto, mientras que followers_count, following_count y public_repos_count cuantifican la presencia en GitHub. Las contribuciones se detallan en total_commit_contributions, total_pr_contributions, total_issue_contributions y total_pr_review_contributions. El campo quantum_repositories registra la participación detallada en cada repositorio cuántico. El quantum_expertise_score (0–100) cuantifica la expertise cuántica, y extracted_from refleja la estrategia *Repository-First*.

```

1  {
2      "id": "MDQ6VXNlcjE2OTY4Mg==",
3      "login": "mtreinish",
4      "name": "Matthew Treinish",
5      "bio": "Qiskit core developer",
6      "company": "IBM Quantum",
7      "location": "USA",
8      "followers_count": 250,
9      "following_count": 100,
10     "public_repos_count": 42,
11     "total_commit_contributions": 3421,
12     "total_pr_contributions": 567,
13     "total_issue_contributions": 234,
14     "total_pr_review_contributions": 1203,
15     "organizations": [
16         {
17             "id": "MDEyOk9yZ2FuaXphdGlvbji4NTI3MDE1",
18             "login": "Qiskit", "name": "Qiskit"
19         }
20     ],
21     "top_languages": ["Python", "Rust", "C++"],
22     "quantum_repositories": [
23         {
24             "id": "MDEwOlJlcG9zaXRvcnk4MzAz",
25             "name": "qiskit", "owner": "Qiskit",
26             "contributions": 3421, "has_commits": true
27         }
28     ],
29     "quantum_expertise_score": 97.5,

```



```

26     "is_quantum_contributor": true,
27     "is_bot": false,
28     "extracted_from": [
29         {"repository_id": "MDEwOlJlcG9zaXRvcnk4MzAz",
30          "repository_name": "qiskit",
31          "extraction_type": "contributor"}
32     ],
33     "ingested_at": "2026-03-02T12:30:45.123Z",
34     "enrichment_status": {"is_complete": true}
35 }

```

Listado F.2: Ejemplo de documento en la colección users

F.3 Colección de Organizaciones

Cada organización se identifica mediante id, login y name, con description e is_verified. El quantum_focus_score (0–100) mide el enfoque cuántico, y quantum_repositories_count contabiliza los repositorios cuánticos. El campo top_languages proporciona un perfil tecnológico con name, percentage y repo_count. El campo discovered_from_repos refleja la estrategia *Repository-First*.

```

1  {
2    "id": "MDEyOk9yZ2FuaXphdGlvbG9jaWNTI3MDE1",
3    "login": "Qiskit",
4    "name": "Qiskit",
5    "description": "An open-source SDK for quantum computers",
6    "is_verified": true,
7    "members_count": 45,
8    "public_repos_count": 87,
9    "quantum_focus_score": 72.4,
10   "is_quantum_focused": true,
11   "quantum_repositories_count": 63,
12   "total_stars": 18500,
13   "top_quantum_contributors": [
14     {"id": "MDQ6VXNlcjE=", "login": "mtreinish"},
15     {"id": "MDQ6VXNlcjI=", "login": "jakelishman"}
16   ],
17   "top_languages": [
18     {"name": "Python", "percentage": 78.5,
19      "repo_count": 49},
20     {"name": "Jupyter Notebook",
21      "percentage": 12.3, "repo_count": 8}
22   ],
23   "discovered_from_repos": [
24     {"id": "MDEwOlJlcG9zaXRvcnk0NTc=",
25      "name": "qiskit"}
26   ],
27   "ingested_at": "2026-03-02T12:35:12.456Z",
28   "enrichment_status": {"is_complete": true}
29 }

```

Listado F.3: Ejemplo de documento en la colección organizations

Anexo G

Análisis de Sensibilidad de Métricas

Este anexo amplía el análisis de sensibilidad presentado en la Sección 5.9, detallando la robustez de las heurísticas empleadas en las métricas de Entangle.

Para evaluar la robustez de las heurísticas, se realizó un análisis de sensibilidad perturbando cada peso un ± 20 % respecto a su valor base. Se definieron siete perfiles arquetípicos para el `quantum_expertise_score` (desde contribuidor ocasional hasta mantenedor principal) y siete para el `quantum_focus_score` (desde falso positivo hasta organización puramente cuántica).

G.1 Sensibilidad del Quantum Expertise Score

En el `quantum_expertise_score`, ninguna perturbación individual de un solo peso produjo cambios en el orden de ranking. Incluso bajo perturbación combinada simultánea de los cinco pesos (32 combinaciones extremas), el top-3 se mantuvo idéntico en el 84,4 % de los casos, y la estabilidad global del ranking fue del 81,2 %. Los perfiles con scores extremos (mantenedores principales y contribuidores ocasionales) resultaron insensibles a las variaciones, mientras que las mayores fluctuaciones (hasta $\pm 14,7$ puntos) se concentraron en perfiles intermedios, cuyo orden relativo no afecta a la interpretación del ecosistema.

A modo de ejemplo concreto, si el factor de estrellas se quintuplicase de 0,1 a 0,5 (un cambio del 400 %, muy superior al ± 20 % del análisis sistemático), un repositorio con 5.000 estrellas pasaría de aportar el máximo de 50 puntos a un hipotético 2.500 (igualmente limitado a 50 por el techo). El efecto real es nulo para repositorios populares porque el cap de 50 puntos absorbe la perturbación; y para repositorios modestos (100 estrellas), la diferencia entre 10 y 50 puntos no es suficiente para superar el peso combinado de la propiedad de repositorios y la pertenencia a organizaciones. Esta es precisamente la razón de diseño de los techos: desacoplar el ranking de valores atípicos en una sola dimensión.

G.2 Sensibilidad del Quantum Focus Score

En el `quantum_focus_score`, el top-3 (organizaciones puramente cuánticas) se mantuvo estable en el 100 % de las combinaciones. El único cambio de ranking observado se produjo al reducir el multiplicador de verificación un 20 %, afectando exclusivamente a posiciones intermedias entre organizaciones con valores de enfoque similares.

G.3 Conclusiones del Análisis

Estos resultados confirman que los pesos elegidos son suficientemente robustos: las conclusiones del análisis del ecosistema (quién lidera, quién es periférico, quién es falso positivo) no dependen de la elección exacta de los coeficientes, sino de la estructura cualitativa de la fórmula.

G.4 Ejemplos Numéricos de Cálculo

A continuación se ilustran los cálculos con un par de casos representativos.

Quantum Expertise Score. Considérese un usuario propietario de 3 repositorios cuánticos, colaborador en 5, con 400 estrellas acumuladas en repositorios cuánticos, 100 contribuciones y pertenencia a 2 organizaciones cuánticas. Aplicando la fórmula de la Sección 5.9.1:

$$(3 \times 5) + (5 \times 2) + \min(400 \times 0,1, 50) \\ + \min(100 \times 0,05, 25) + (2 \times 10) = 15 + 10 + 40 + 5 + 20 = 90$$

Quantum Focus Score. Para una organización con 20 de sus 100 repositorios dedicados a computación cuántica:

$$\text{base} = (20/100) \times 100 = 20$$

Si la palabra «quantum» aparece en su nombre se suma un bono de 10 ($\rightarrow 30$); si la organización está verificada por GitHub se aplica el multiplicador de 1,2 y el resultado final es $30 \times 1,2 = 36$.

Network Density. Para un grafo con 100 nodos y 450 aristas la densidad es $\frac{450}{100 \times 99/2} = 0,091$, valor consistente con un ecosistema parcialmente fragmentado.

Bus Factor. Si en un repositorio con 10 contribuidores únicamente 2 acumulan el 80 % de las contribuciones, el bus_factor es 2: dependencia significativa pero no crítica.

Anexo H

Configuración de Infraestructura

Este anexo presenta los listados completos de configuración de la infraestructura de Entangle, complementando la descripción resumida de la Sección 5.10.

H.1 Dockerfile del Backend

La contenedorización del backend se realiza mediante Docker, utilizando una imagen base de `python:3.11-slim`. El contenedor añade dependencias del sistema, copia selectiva del código, ejecución con usuario no privilegiado, variables de entorno y un *health check*.

```
1 FROM python:3.11-slim
2 WORKDIR /app
3 RUN apt-get update && apt-get install -y --no-install-recommends gcc \
4     && rm -rf /var/lib/apt/lists/*
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir --upgrade pip && \
7     pip install --no-cache-dir -r requirements.txt
8 COPY src/ ./src/
9 COPY config/ ./config/
10 RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
11 USER appuser
12 ENV PORT=8000
13 EXPOSE 8000
14 HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
15     CMD python -c "import requests; \
16     requests.get('http://localhost:8000/api/v1/health')\" \
17     || exit 1
18 CMD uvicorn src.api.main:app --host 0.0.0.0 --port ${PORT:-8000}
```

Listado H.1: Fragmento del Dockerfile para el backend

La instalación explícita de `gcc` resuelve dependencias nativas de paquetes Python, mientras que la creación de `appuser` evita ejecutar el proceso como *root*. El *health check* contra `/api/v1/health` facilita la supervisión del contenedor en Azure Container Apps [Doc24].

H.2 Infraestructura como Código con Bicep

El archivo principal `main.bicep` orquesta el despliegue a nivel de suscripción (`targetScope = 'subscription'`), creando el grupo de recursos y encadenando los módulos necesarios. El diseño modular separa explícitamente la provisión del *resource group*, el Log Analytics

Workspace, el Container Registry, el Container Apps Environment y la Container App del backend.

```
1 targetScope = 'subscription'
2
3 resource rg 'Microsoft.Resources/resourceGroups@2021-04-01' = {
4   name: 'rg-entangle-prod'
5   location: location
6 }
7
8 module logAnalytics './core/monitor/loganalytics.bicep' = {
9   name: 'log-analytics'
10  scope: rg
11  params: { name: 'log-entangle' }
12 }
13
14 module containerRegistry './core/host/container-registry.bicep' = {
15   name: 'container-registry'
16   scope: rg
17   params: { name: 'crentangle' }
18 }
19
20 module api './core/host/container-app.bicep' = {
21   name: 'api'
22   scope: rg
23   params: {
24     containerRegistryName: containerRegistry.outputs.name
25     targetPort: 8000
26   }
27 }
```

Listado H.2: Fragmento del archivo Bicep para la infraestructura

Este enfoque permite una gestión consistente y reproducible de la infraestructura, evita configuraciones manuales y favorece la trazabilidad de cambios. Sobre esta definición se apoya azd up, que provisiona, construye, publica y despliega en un único flujo automatizado [Mic24f].

H.3 Entorno de Desarrollo Local

El entorno local replica la separación de responsabilidades de producción: backend FastAPI, frontend Vite/React y base de datos MongoDB o Azure Cosmos DB for MongoDB (vCore). Las dependencias se instalan desde requirements.txt (backend) y npm install (frontend).

El backend se ejecuta con uvicorn src.api.main:app -reload para recarga automática al detectar cambios. El frontend se inicia con npm run dev, donde .env.development define VITE_API_URL=http://localhost:8000/api/v1.

La configuración de entorno se separa por capa: el backend usa un archivo .env con variables como GITHUB_TOKEN, MONGO_URI, AZURE_AI_ENDPOINT y AZURE_AI_DEPLOYMENT; el frontend expone únicamente variables prefijadas con VITE_, evitando trasladar secretos al cliente.

H.4 Catálogo Detallado de Servicios Azure

El *backend* se ejecuta en Azure Container Apps, concretamente en la *Container App* entangle-api. Este servicio proporciona una abstracción *serverless* sobre contenedores, con escalado automático y despliegue directo desde la imagen construida para producción. Debajo de esta capa se provisiona un Container Apps Environment, recurso que agrupa la configuración de red, observabilidad y ciclo de vida de la aplicación contenedorizada.

El *frontend* se publica en Azure Static Web Apps, con distribución global y despliegue automatizado mediante GitHub Actions. Esta separación entre *frontend* estático global y *backend* regional reduce la latencia de entrega de la interfaz sin obligar a replicar la lógica de negocio.

La persistencia se resuelve con Azure Cosmos DB for MongoDB (vCore) en Spain Central, manteniendo compatibilidad completa con el ecosistema MongoDB usado por el proyecto. Sobre esta base documental se almacenan las colecciones operativas, métricas y configuraciones descritas en las Secciones 5.4.1 y 5.4.

La capa de IA se implementa mediante Azure AI Foundry, consumida desde el *backend* a través de un *endpoint* dedicado con dos *deployments*: gpt-5-mini para la inferencia conversacional y text-embedding-3-small para la generación de *embeddings* de la canalización de RAG. Estos *embeddings* se persisten e indexan en la colección repos_chunks de Cosmos DB mediante un índice vectorial IVF con distancia coseno. Esta separación permite que la lógica conversacional permanezca desacoplada del *frontend* y reutilice la autenticación con identidades gestionadas descrita en la Sección 5.10.3.

Finalmente, la infraestructura incorpora dos servicios de soporte que resultan clave aunque no sean visibles para el usuario final: Azure Container Registry, donde se almacenan las imágenes Docker que luego consume Azure Container Apps, y Azure Log Analytics, al que se envían los *logs* del entorno de contenedores para monitorización, diagnóstico y análisis operativo.

H.5 Detalle de la Capa de Seguridad

En la capa de autenticación, el *backend* prioriza System-Assigned Managed Identity para consumir Azure AI Foundry. La implementación utiliza DefaultAzureCredential y solicita un token *bearer* para el ámbito de *Cognitive Services*, evitando depender de claves embebidas en el código. La API key de Azure AI existe únicamente como mecanismo de compatibilidad opcional, no como ruta principal de producción. Esto reduce la exposición de credenciales y alinea la autenticación con el modelo nativo de Azure [Mic24e].

La gestión de secretos sigue el mismo principio de minimización de exposición. En la infraestructura declarada con Bicep, valores sensibles como MONGO_URI y GITHUB_TOKEN se definen como parámetros @secure() y se inyectan en la Container App mediante secretRef.

Por tanto, el código de aplicación consume secretos desde variables de entorno, mientras que su provisión queda centralizada en el flujo de despliegue. Es importante distinguir este punto: en el estado actual del proyecto, la autenticación hacia la base de datos no se resuelve con Managed Identity, sino mediante una cadena de conexión protegida como secreto.

En la capa de exposición web, el *backend* configura CORS dinámicamente. La lista de orígenes permitidos combina direcciones locales de desarrollo con la URL del *frontend* en producción, recibida a través de `FRONTEND_URL`. De este modo, la API solo acepta peticiones desde clientes previstos, reduciendo la superficie de abuso desde orígenes arbitrarios [Moz24a].

En el *frontend* desplegado sobre Azure Static Web Apps, la configuración de `staticwebapp.config.json` añade varias cabeceras de endurecimiento recomendadas por el proyecto OWASP [Mic24g]: `Content-Security-Policy`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` y `X-XSS-Protection`. La CSP limita los orígenes válidos para *scripts*, estilos, imágenes y conexiones de red, mitigando vectores típicos de Cross-Site Scripting (XSS) e inyección de contenido externo. El bloqueo de *frame embedding* y la desactivación del *MIME sniffing* refuerzan aún más la protección en cliente.

Por último, el agente conversacional incorpora una capa de seguridad propia basada en restricción de capacidades. Solo el agente DATA dispone de herramientas contra MongoDB, y dichas herramientas son estrictamente de lectura: `query_database`, `run_aggregation` y `get_collection_schema`. En particular, `run_aggregation` bloquea de forma explícita los operadores `$out` y `$merge`, impidiendo escrituras desde los *pipelines* del modelo. Esta decisión es relevante porque evita que el LLM actúe como un cliente con permisos de modificación sobre la base de datos.

Anexo I

Vocabulario Físico: Definiciones y Correspondencia Visual

Este anexo recoge las definiciones formales de los conceptos de la física cuántica, la astrofísica y la física atómica que Entangle traduce en metáforas visuales interactivas. El resumen condensado de estos conceptos se encuentra en la Sección 3.1; la correspondencia completa entre conceptos y elementos visuales se detalla en la Tabla J.1 del Capítulo 5.

I.1 Mecánica Cuántica

- **Notación de Dirac.** El estado de un sistema cuántico se expresa mediante *kets*: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, donde α y β son amplitudes complejas cuyo módulo al cuadrado indica la probabilidad de medir cada estado base [NC10]. En Entangle aparece en el badge `|FILTERED>` del dashboard y en etiquetas del universo 3D.
- **Esfera de Bloch.** Cualquier estado puro de un cúbit se representa como un punto en la superficie de una esfera unitaria parametrizada por θ y φ : $|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\varphi}\sin(\theta/2)|1\rangle$. El polo norte corresponde a $|0\rangle$, el polo sur a $|1\rangle$ y el ecuador a superposiciones equilibradas [NC10]. En Entangle, los cúbits del universo 3D muestran ejes verticales que evocan esta esfera, y las tarjetas KPI incluyen esferas de Bloch animadas cuyo vector colapsa a $|0\rangle$ al interactuar.
- **Colapso de la función de onda.** Al medir un cúbit en superposición, su estado colapsa de forma probabilística a uno de los estados base [GS18]. En Entangle se traduce como el efecto de *wave function collapse*: la onda que decora las tarjetas KPI se contrae a un delta de Dirac al pasar el cursor, simulando el acto de observación.
- **Principio de incertidumbre de Heisenberg.** Establece que no es posible conocer simultáneamente con precisión arbitraria la posición y el momento de una partícula: $\Delta x \cdot \Delta p \geq \hbar/2$ [NC10]. En Entangle, las ~ 27.000 partículas que representan usuarios experimentan una vibración tridimensional con fases desacopladas en cada eje, impidiendo predecir su posición exacta en un instante dado.
- **Nube de probabilidad.** La función de onda $\psi(r, \theta, \varphi)$ de un electrón define una distribución de probabilidad de presencia alrededor del núcleo, formando regiones difusas denominadas orbitales [GS18]. En el universo 3D, cada cúbit (repositorio) está rodeado

por diez partículas GPU que orbitan en trayectorias esféricas con ángulos variables, simulando estas nubes.

- **Efecto túnel cuántico.** Una partícula cuántica puede atravesar una barrera de potencial que sería impenetrable según la mecánica clásica, gracias a la naturaleza ondulatoria de su función de onda [GS18]. Entangle emplea esta metáfora en la funcionalidad *Quantum Tunneling*: dados dos nodos cualesquiera, calcula el camino más corto (Dijkstra) y lo anima como pulsos que “atraviesan” las barreras organizacionales del grafo.
- **Espuma cuántica (*Quantum Foam*).** A escalas de Planck ($\sim 10^{-35}$ m), las fluctuaciones cuánticas del espacio-tiempo generan una estructura espumosa de geometría variable [Whe57]. En el universo 3D se simula mediante partículas con ciclos de vida basados en armónicos superpuestos que aparecen y desaparecen.
- **Vacío cuántico e interferencia.** La teoría cuántica de campos predice que el vacío no está vacío: pares virtuales de partícula-antipartícula se crean y aniquilan continuamente [GS18]. Asimismo, las partículas cuánticas exhiben patrones de interferencia al superponer sus funciones de onda. Entangle representa ambos fenómenos con una rejilla de fluctuaciones del vacío (400 partículas) y un plano de 600 partículas cuyas ondas estacionarias reproducen franjas de interferencia.

1.2 Astrofísica y Física Atómica

- **Esfera de Dyson.** Estructura teórica propuesta por Dyson que envolvería completamente una estrella para capturar su energía [Dys60]. En Entangle, la *Dyson Shell* es un icosaedro geodésico de 3.500 unidades de radio (≈ 1.280 triángulos) que encierra todo el universo, con pulsos de energía recorriendo sus aristas.
- **Rayos cósmicos.** Partículas subatómicas de alta energía que viajan por el espacio interestelar a velocidades cercanas a la de la luz [Lon11]. En el universo 3D se representan como *ribbons* triangulares multicolor que cruzan la escena e impactan contra la Dyson Shell generando ondas de propagación.
- **Modelo atómico de Bohr.** Modelo semiclásico en el que los electrones orbitan el núcleo en niveles de energía discretos [Boh13]. En el dashboard, el centro del grafo de colaboración presenta un átomo decorativo con tres electrones en trayectorias elípticas (12 s, 16 s y 20 s), reforzando la estética cuántica de la interfaz.

Anexo J

Galería del Universo 3D

Este anexo recoge la correspondencia detallada entre conceptos físicos y metáforas visuales (Sección J.1) y la galería completa de capturas del universo 3D inmersivo de Entangle. El análisis funcional condensado se encuentra en la Sección 5.13 del Capítulo 5; aquí se reúnen las capturas detalladas de cada una de las cinco lentes analíticas, la herramienta de *pathfinding* y los hitos del tour cinemático guiado.

J.1 Correspondencia entre Conceptos Físicos y Metáforas Visuales

La Tabla J.1 resume la correspondencia entre conceptos de la física cuántica, la astrofísica y la física atómica y las metáforas visuales implementadas en Entangle. Las definiciones formales y fórmulas asociadas se recogen en el Anexo I.

Tabla J.1: Correspondencia entre conceptos físicos y metáforas visuales de Entangle.

Concepto físico	Elemento visual	Ubicación
Notación de Dirac	Badge <code> FILTERED></code>	Dashboard
Esfera de Bloch	Esferas animadas, ejes de cúbits	KPI / Universo 3D
Colapso de función de onda	Onda $\rightarrow$ delta de Dirac	Tarjetas KPI
Incertidumbre de Heisenberg	Vibración tridimensional	Partículas usuario
Nube de probabilidad	Partículas orbitales GPU	Cúbits (repos)
Efecto túnel cuántico	Pathfinding animado	Túnel Cuántico
Espuma cuántica	Partículas con armónicos	Fondo Universo 3D
Vacío cuántico	Rejilla + pares virtuales	Plano XZ
Interferencia cuántica	Ondas estacionarias	Plano de fondo
Decoherencia	Anillos expansivos	Procesadores cuánt.
Entrelazamiento	Canales sinusoidales, SVG	Arcos / Dashboard
Superposición	Animación <code>quantumShift</code>	Logo, nav bar
Esfera de Dyson	Icosaedro geodésico	Shell exterior
Rayos cósmicos	Ribbons multicolor	Escena 3D
Modelo atómico (Bohr)	Electrones orbitando	Centro del grafo

J.2 Distribución Espacial

La Figura J.1 muestra las tres zonas de densidad generadas por Jenks Natural Breaks (núcleo, zona intermedia y periferia), que sustentan la distribución espacial descrita en la Sección 5.13.1.

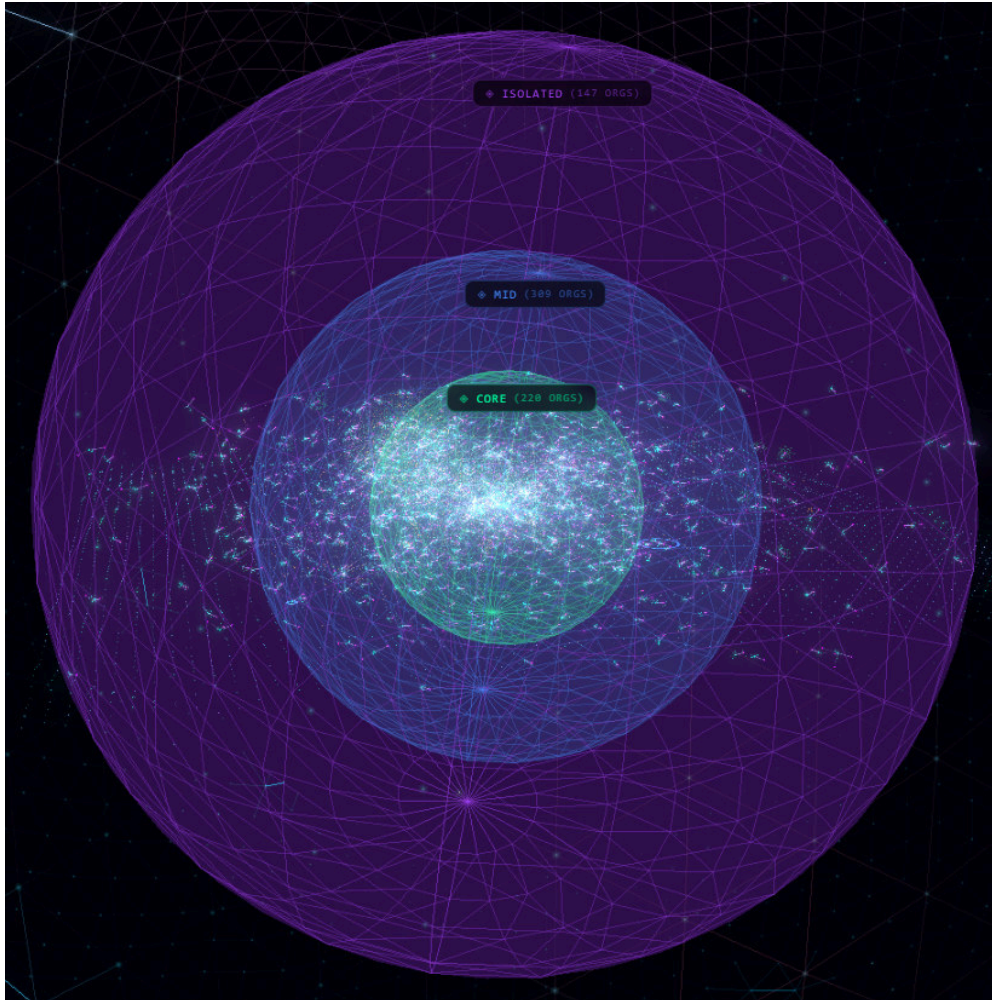


Figura J.1: Fronteras zonales del universo 3D: núcleo, zona intermedia y periferia.

La Figura J.2 ofrece una vista panorámica inmersiva desde el interior del universo 3D, con la rejilla *lattice* del vacío cuántico desplegándose en perspectiva hacia el horizonte. Sobre esta misma escena se aprecian los nodos distribuidos en profundidad, los *ribbons* de los rayos cósmicos cruzando el espacio y la atmósfera de partículas y entrelazamientos que envuelve el ecosistema.

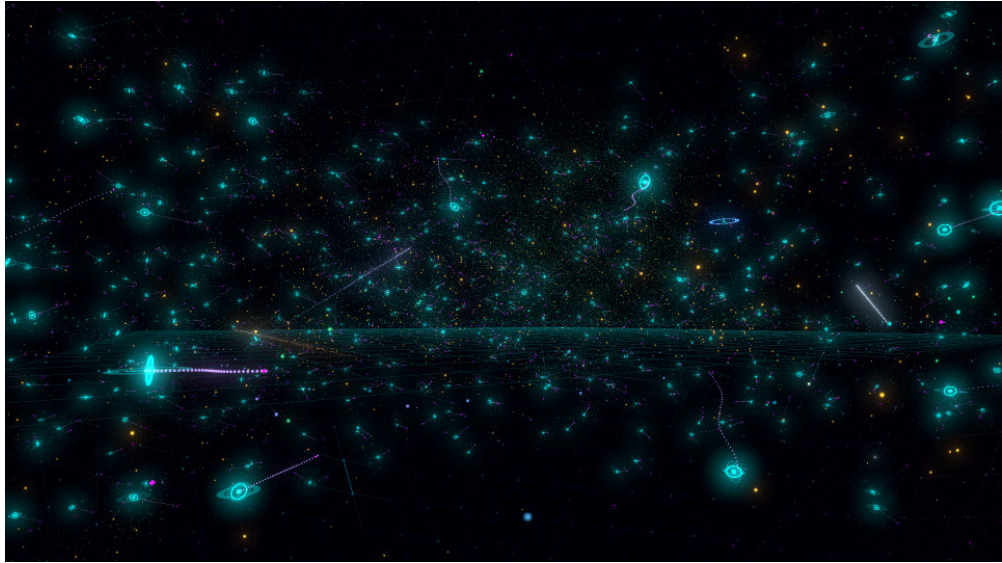


Figura J.2: Vista panorámica inmersiva del universo 3D desde el interior.

J.3 Modos de Interacción y Detalle de Entidades

Esta sección recopila capturas de las principales modalidades de exploración del universo descritas en la Sección 5.8: el modo *focus* sobre una organización, la inspección del subgrafo de *bridge users* y el detalle micro de un cúbit (repositorio) con su nube de probabilidad orbital.

J.3.1 Modo Focus sobre una Organización

Al hacer clic sobre un procesador cuántico (organización), el resto del universo se atenúa y se resaltan únicamente sus repositorios asociados, sus colaboradores y los enlaces con organizaciones relacionadas (Figura J.3). Esta vista facilita el análisis aislado de una organización dentro del ecosistema.

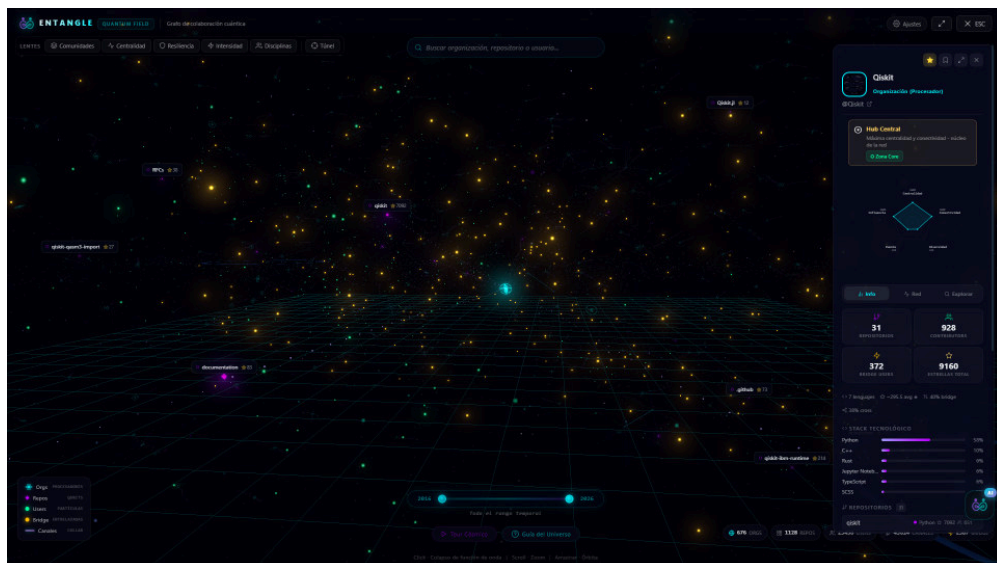


Figura J.3: Modo *focus* sobre una organización.

J.3.2 Cluster de Bridge Users

Los *bridge users* se distinguen visualmente por su color dorado (#ffbd00) frente al verde estándar (#00ff9f) de los usuarios convencionales. La Figura J.4 muestra un *cluster* concreto donde dos organizaciones colaboran a través de varios *bridge users*, con los canales de entrelazamiento y los pulsos de *tunneling* resaltados.

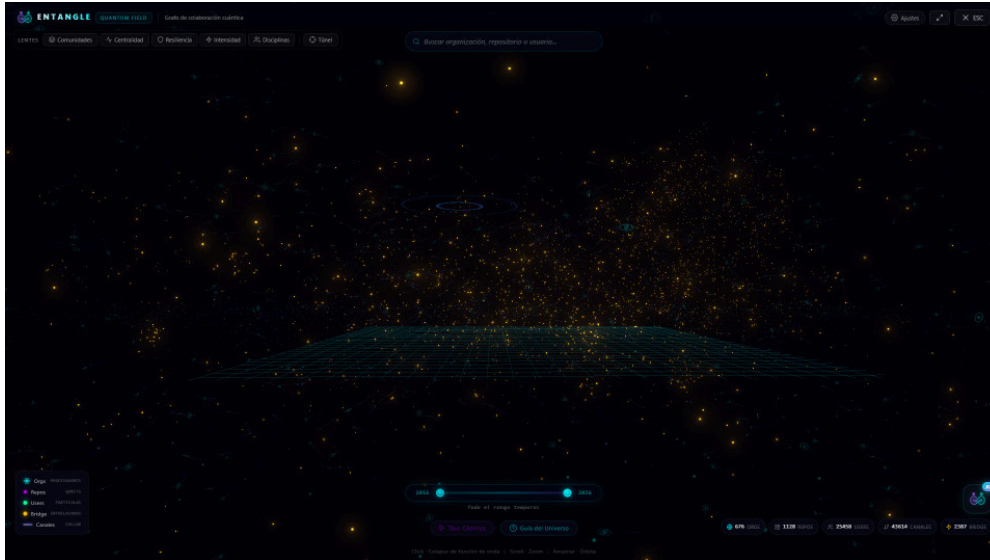


Figura J.4: Cluster de *bridge users* (puntos dorados) entre dos organizaciones.

J.3.3 Detalle de un Cúbit (Repositorio)

Al acercar la cámara a un repositorio individual se aprecia el octaedro violeta y las diez partículas que orbitan en trayectorias esféricas, simulando la nube de probabilidad cuántica $\psi(r, \theta, \varphi)$ descrita en la Sección 5.8.2. La Figura J.5 captura este detalle con los *Quantum Bonds* (dobles hélices) que conectan el cúbit con sus contribuyentes.

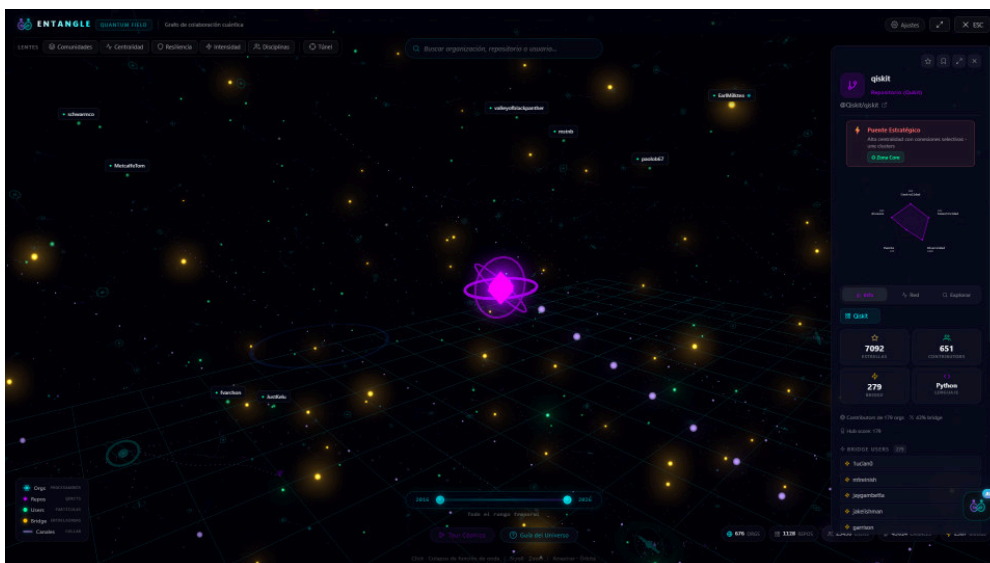


Figura J.5: Detalle micro de un cúbit (repositorio) con *Quantum Bonds*.

J.4 Galería de Lentes Analíticas

J.4.1 Lente de Comunidades

Las ~ 752 comunidades detectadas por Louvain se distinguen mediante colores generados por el ángulo áureo. La mayoría son compactas y autocontenidas, organizadas en torno a una organización principal; las regiones donde los colores se mezclan corresponden a zonas con alta actividad de *bridge users*.

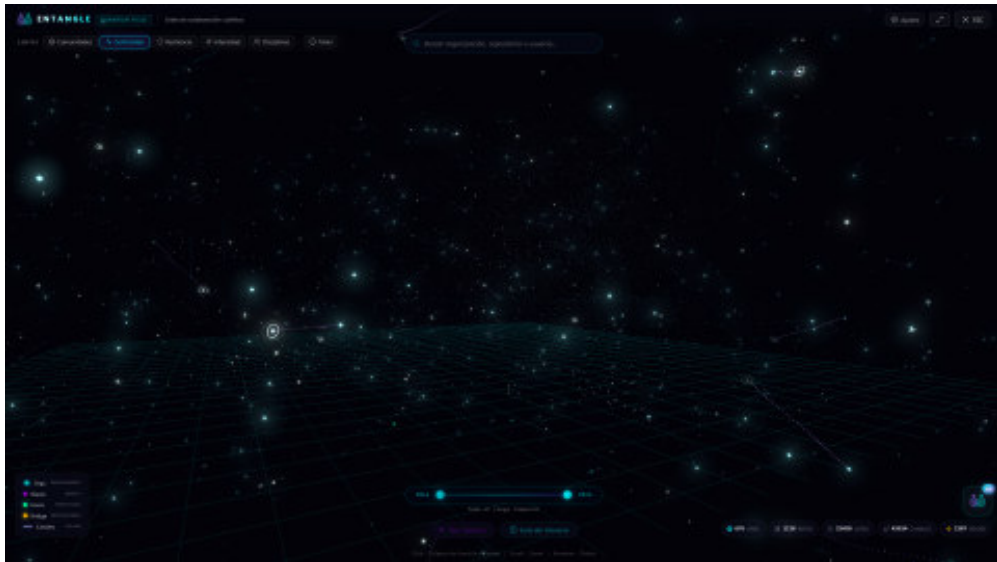


Figura J.6: Lente de comunidades: colores asignados por ángulo áureo.

J.4.2 Lente de Centralidad

Un gradiente de azul oscuro a cian brillante según la *betweenness centrality* confirma que los nodos puente más luminosos se concentran en el núcleo del universo, ocupando posiciones centrales tanto topológica como espacialmente.

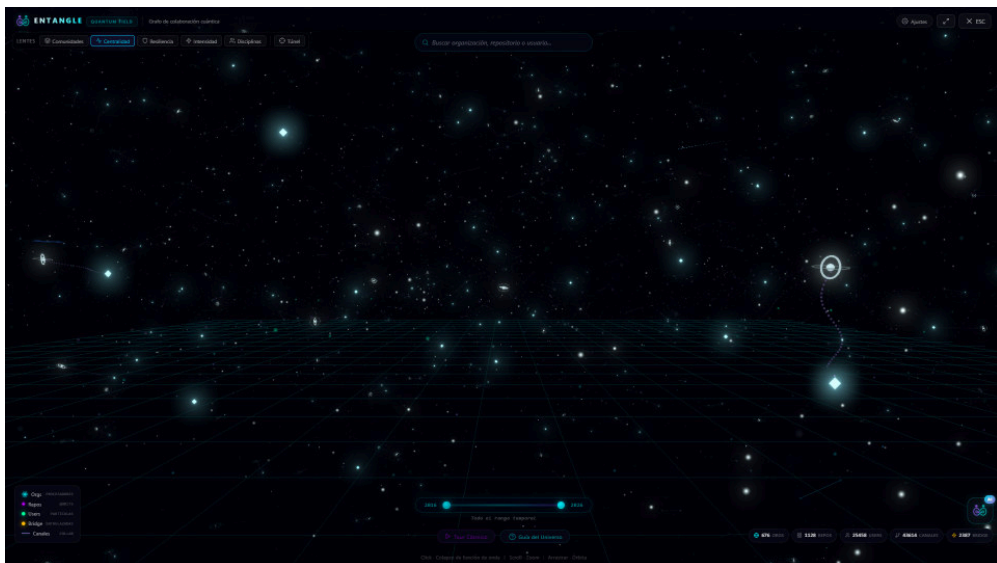


Figura J.7: Lente de centralidad: gradiente por *betweenness centrality*.

J.4.3 Lente de Resiliencia

La lente de resiliencia evalúa el *bus factor* [AVH15] mediante cuatro niveles cromáticos: rojo (= 1), naranja (= 2), amarillo (3–4) y verde (≥ 5). El **81,7 %** de los repositorios tiene *bus factor* = 1, evidenciando una vulnerabilidad sistémica del ecosistema cuántico.



Figura J.8: Lente de resiliencia: niveles de *bus factor* codificados por color.

J.4.4 Lente de Intensidad

La lente de intensidad visualiza la *degree centrality* con un gradiente rojo-amarillo, identificando las áreas con mayor actividad colaborativa bruta, que no necesariamente coinciden con las más centrales topológicamente.

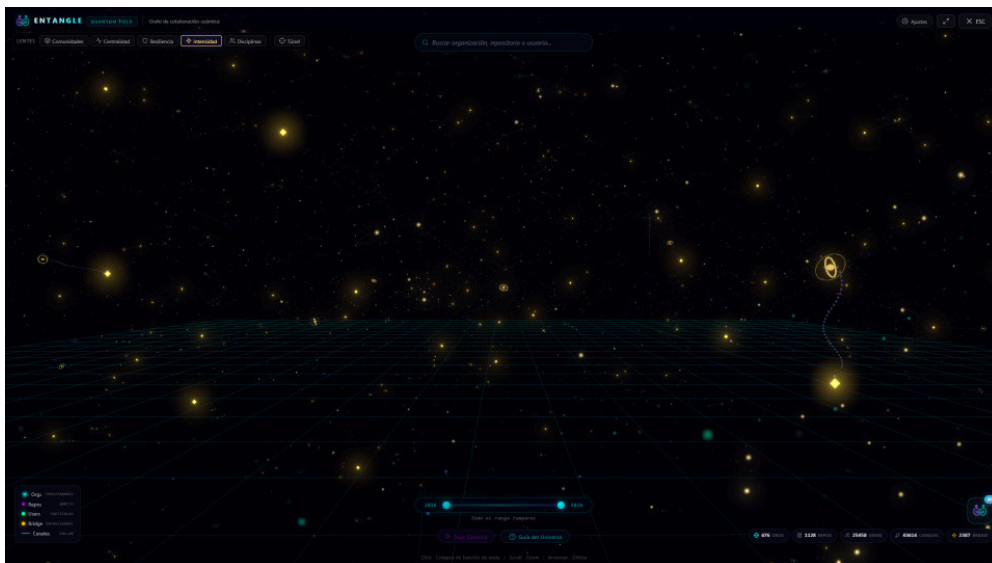


Figura J.9: Lente de intensidad: gradiente por *degree centrality*.

J.4.5 Lente de Disciplinas

La lente de disciplinas colorea cada usuario según su categoría temática. Los *clusters* de quantum_software y dev_tooling dominan el universo, y los usuarios multidisciplinares se sitúan con frecuencia en las regiones de intersección entre clusters, lo que sugiere que la interdisciplinariedad se correlaciona con posiciones de puente en el grafo.

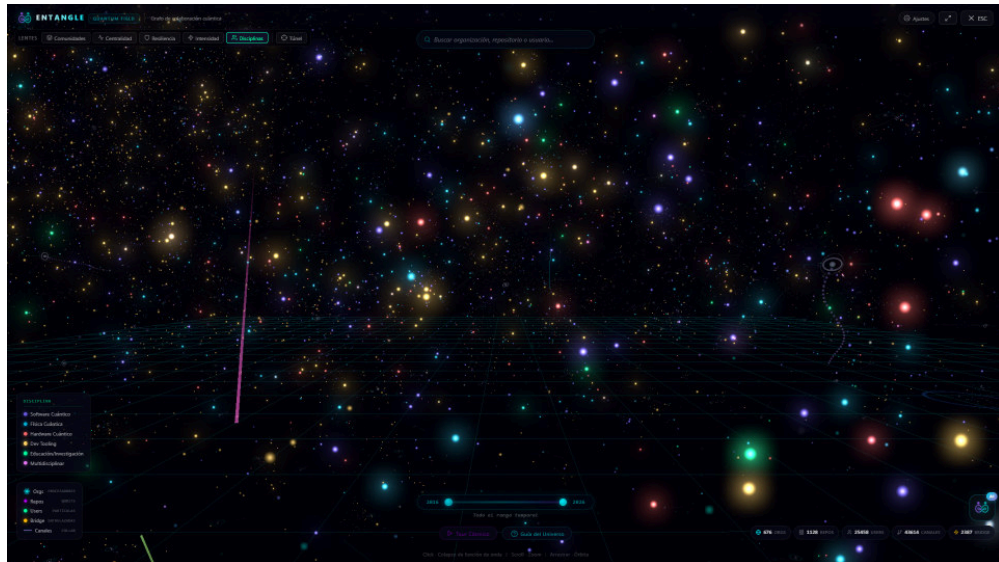


Figura J.10: Lente de disciplinas: seis categorías temáticas codificadas por color.

J.4.6 Túnel Cuántico

El Túnel Cuántico calcula el camino más corto mediante Dijkstra, revelando cadenas de colaboración que atraviesan múltiples organizaciones.

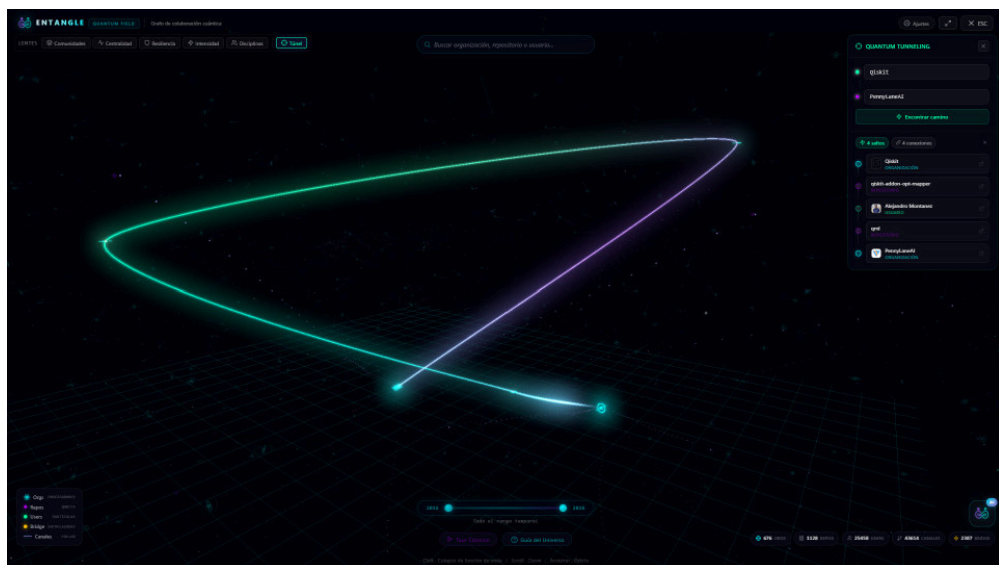


Figura J.11: Túnel Cuántico: camino más corto animado entre dos entidades.

J.5 Animaciones Cinemáticas de Entrada y Salida, y Tour Cósmico

Esta sección documenta los tres elementos cinemáticos del Universo 3D, que son funcionalmente independientes entre sí. Por un lado, dos animaciones intrínsecas a la vista que se ejecutan **siempre** al transitar entre el cuadro de mando y el universo: el *Quantum Genesis* (*Big Bang*) al entrar y el *Black Hole Collapse* al salir. Por otro lado, el **Tour Cósmico** es una experiencia narrada opcional, activable por el usuario, cuya lógica es completamente independiente de las animaciones anteriores y que aporta una capa narrativa sobre el universo ya materializado.

J.5.1 Quantum Genesis (Big Bang): Animación de Entrada

La animación de entrada materializa el universo con **datos reales**: los nodos que el usuario ve aparecer no son partículas decorativas, sino las propias organizaciones, repositorios y usuarios obtenidos por el pipeline de datos (Véase Sec. 5.3) y posicionados por el algoritmo de distribución (Véase Sec. 5.8.4). Cada entidad parte desde la singularidad (origen de coordenadas), donde queda comprimido todo el universo, e interpola su posición hasta su destino final mediante *easeOutCubic* con un retardo escalonado, produciendo una expansión por capas que respeta la jerarquía radial del ecosistema.

La animación dura 4 s y se compone de cinco fases solapadas que combinan un *overlay* Canvas2D transparente con manipulaciones de cámara y de filtros CSS sobre el *wrapper* del universo, de modo que el efecto cinematográfico acompaña al movimiento real de los nodos en lugar de superponerse a una escena estática:

1. **Singularidad** (0–0,4 s): un punto luminoso blanco-cian pulsa en el centro como semilla del universo, sobre un fondo con viñeta máxima que oculta toda la escena.
2. **Ignición** (0,4–0,9 s): destello blanco-cian masivo que cubre la pantalla, seguido de un *flare* anamórfico horizontal cinematográfico. Filtros CSS de *brightness* y *saturate* sobre el *wrapper* refuerzan el *peak* del destello, y la viñeta se desvanece liberando la visión.
3. **Onda de choque** (0,8–1,9 s): cinco anillos concéntricos escalonados (blanco, cian, púrpura, verde y dorado) se expanden con *easeOutQuart*, acompañando visualmente la expansión real de los nodos que en este intervalo están viajando desde la singularidad hacia su posición final.
4. **Cosmic Web** (1,9–3,2 s): los nodos reales del universo (procesadores, cúbits, usuarios) completan su trayectoria y la escena 3D queda plenamente visible. El *overlay* se atenúa para ceder el protagonismo al contenido real.
5. **Settle** (3,2–4,0 s): un *afterglow* central residual se desvanece suavemente, marcando el cierre del nacimiento.

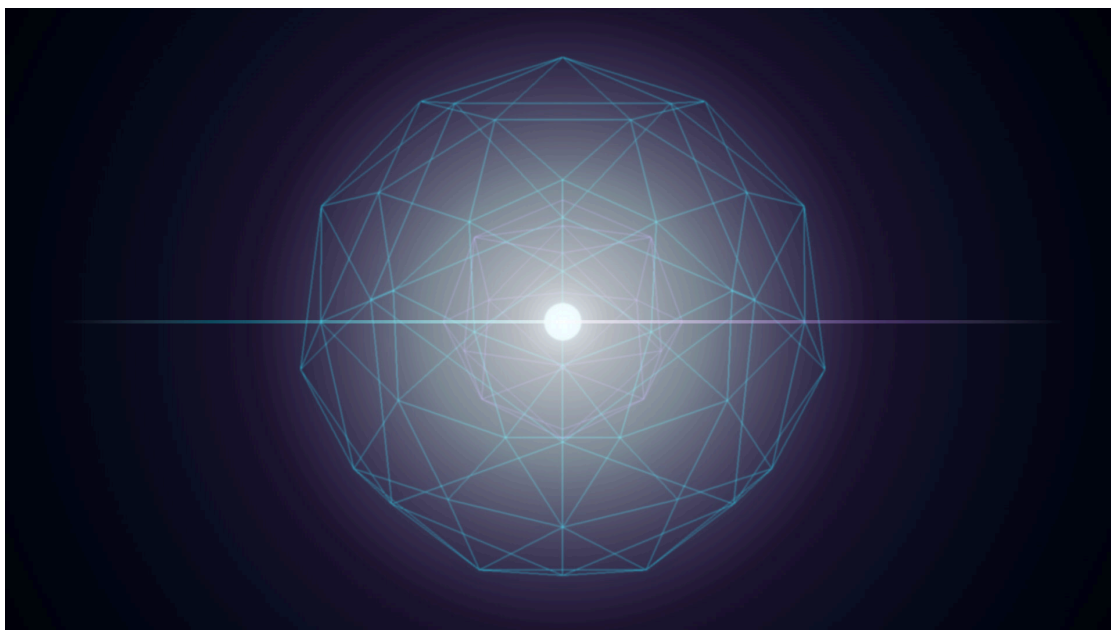


Figura J.12: *Quantum Genesis (Big Bang)*: fase de ignición de la animación de entrada.

Una cámara cinematográfica acompaña la secuencia: durante la anticipación el *wrapper* se mantiene con escala 1,04 (zoom sutil sobre el punto de nacimiento) y, tras el destello, retrocede con `easeOutCubic` hasta 1,0, revelando el universo completo. El *canvas* se mantiene transparente durante toda la animación y se auto-limpia al terminar, devolviendo el *wrapper* a su estado original sin residuos de filtros ni de transformaciones.

J.5.2 Black Hole Collapse: Animación de Salida

Como contraparte de la entrada, la animación de salida del universo 3D se ejecuta **siempre** que el usuario abandona la vista para volver al cuadro de mando. Está inspirada en la imaginaria de los agujeros negros y comprende cinco fases solapadas (con una duración total de 5,5 s) que combinan un *overlay* Canvas2D transparente con manipulaciones del `clip-path` y de las transformaciones CSS del *wrapper* del universo, de modo que el colapso afecta visualmente a la escena 3D real y no a una capa decorativa independiente:

1. **Premonición** (0–0,5 s): desaturación progresiva, micro-*glitch* cromático y leve temblor sinusoidal sobre el contenedor del universo.
2. **Gathering** (0,5–1,7 s): el contenedor comienza a curvarse hacia el centro y aparece un sutil halo de fondo.
3. **Event Horizon** (1,7–2,9 s): se materializan el *photon ring* y el *accretion disk*, y arranca el `clip-path` circular que recorta progresivamente la escena.
4. **Spaghettification** (2,9–3,9 s): el contenido se estira radialmente y la lente gravitacional intensifica el efecto de distorsión sobre los nodos próximos al horizonte.

5. **Total Collapse** (3,9–5,5 s): el clip-path se contrae hacia cero siguiendo una curva analítica con oscilación amortiguada (`computeClipFactor`) que produce *bounces* suaves antes de que el agujero se cierre por completo. Una vez consumida la escena, persiste durante $\sim 0,5$ s un remanente luminoso (*remnant*) que ejecuta un destello final (*blink*) y se apaga, marcando el cierre cinematográfico antes de devolver el control al cuadro de mando.

El *photon ring* se dibuja siempre en el borde exacto del clip-path (en lugar de en una posición independiente), lo que evita el efecto de salto entre los *bounces* del cierre y refuerza la coherencia con la imaginería astrofísica de los agujeros negros. El *canvas* se mantiene transparente durante toda la animación: el fondo cuántico de la aplicación queda visible a través del agujero, garantizando una transición continua entre el universo 3D y el cuadro de mando 2D.

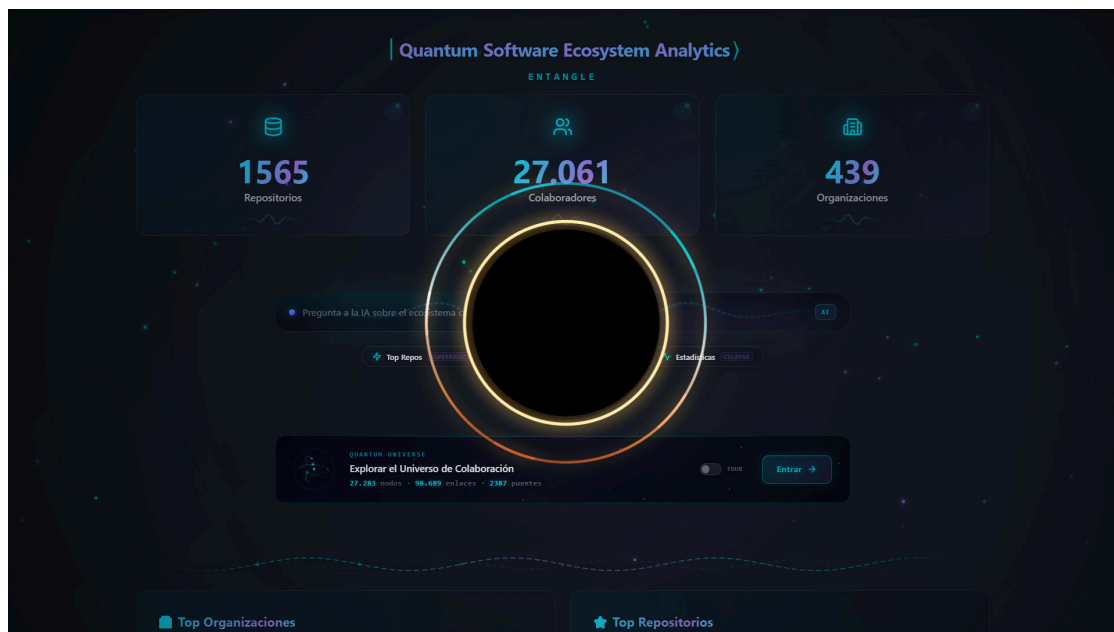


Figura J.13: *Black Hole Collapse*: animación de salida durante la fase de *Event Horizon*.

J.5.3 Tour Cósmico: Guion Narrativo Opcional

El Tour Cósmico es una experiencia narrada **opcional**, activable por el usuario desde el control correspondiente del universo, que aporta una capa cinematográfica complementaria a las animaciones de entrada y salida descritas más arriba. No depende de ellas ni se ejecuta automáticamente: el universo puede explorarse libremente con `OrbitControls` sin necesidad de iniciar nunca el tour.

El tour genera dinámicamente entre 12 y 16 *waypoints* a partir de los datos reales del ecosistema. Cada *waypoint* define una posición y objetivo de cámara, una duración (12–18 s), un título, un texto narrativo con datos interpolados y una lista de entidades prioritarias a resaltar. La secuencia narrativa estándar sigue este orden:

1. **El Vacío** (18 s): oscuridad total. Narra el silencio previo a la era *open-source* cuántica.
2. **Preludio** (12 s): “Entonces la industria decidió abrir sus puertas...”. Introduce las primeras entidades del ecosistema sobre la escena ya materializada por la animación de entrada.
3. **Génesis** (16 s): primer año del ecosistema. Detecta hitos como IBM Quantum Experience (2016), Qiskit y Q# (2017) o el Forest SDK de Rigetti.
4. **Primeros Nodos** (12 s): primeros repositorios y organizaciones, distinguiendo entre industria y proyectos experimentales.
5. **Primer Gigante** (13 s): enfoca la primera organización industrial clave (IBM, Google, Microsoft, Rigetti, D-Wave, Xanadu o Zapata, según los datos).
6. **Aceleración** (~12 s): año de punto de inflexión en la creación de repositorios.
7. **Competencia Estructurada** (~12 s): equilibrio, estándares y formación de comunidades (2020–2021).
8. **Entrelazamiento** (13 s): colaboraciones reales y vínculos inter-organizacionales.
9. **Usuarios Puente** (13 s): desarrolladores que conectan múltiples organizaciones.
10. **Inflación Cósmica** (13 s): año con mayor crecimiento relativo.
11. **Quantum Babel** (12 s): diversidad de lenguajes de programación en el ecosistema.
12. **Consolidación** (13 s): ecosistema maduro, especialización (2022+).
13. **Panorámica Final** (14 s): visión completa con el total de repositorios, organizaciones y desarrolladores.

Los textos de cada *waypoint* se generan con plantillas que interpolan valores reales (ej.: “1.247 repos, 86 organizations and 3.492 developers will materialize in this universe”), de modo que el tour refleja siempre el estado actual de la base de datos. El usuario controla el avance con las teclas → o Espacio (siguiente *waypoint*), ← (anterior) y un botón de salto rápido.

A continuación se ilustran tres *waypoints* concretos del tour. El *Genesis* (Figura J.14) muestra el inicio del recorrido, con las primeras entidades del ecosistema cuántico (años más tempranos del filtro cronológico) visibles sobre el universo ya materializado. El *Entrelazamiento* (Figura J.15) destaca los canales inter-organizacionales con los pulsos de *tunneling* en plena propagación. La *Panorámica Final* (Figura J.16) ofrece la visión completa del universo con la totalidad de repositorios, organizaciones y desarrolladores ya visibles tras desactivar el filtro temporal.

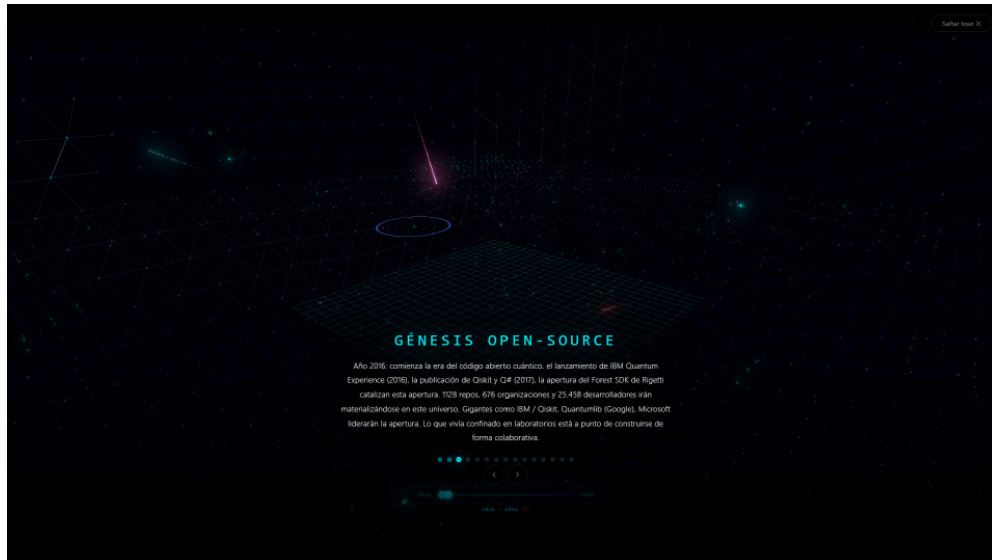


Figura J.14: *Waypoint Genesis*: inicio del recorrido del tour.

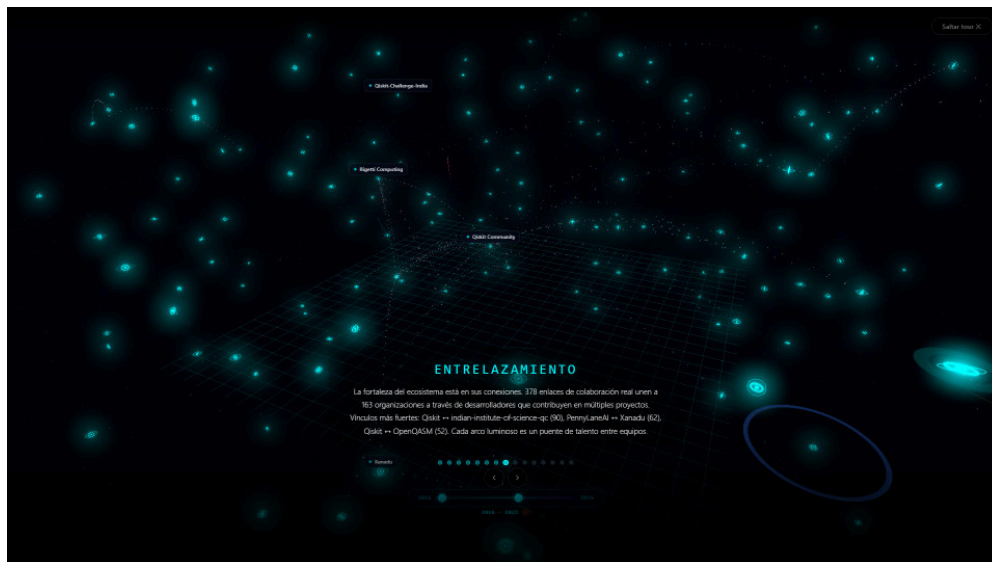


Figura J.15: *Waypoint Entrelazamiento*: canales inter-organizacionales con pulsos de *tunneling*.



Figura J.16: *Waypoint Panorámica Final.*

J.6 Efectos Visuales y Fenómenos Cuánticos: Parámetros Detallados

Esta sección documenta los parámetros numéricos exactos de los efectos visuales que se introducen de forma resumida en la Sección 5.8.6 del cuerpo principal. Cada uno refuerza la metáfora cuántica del universo:

- **Vacío cuántico:** rejilla *lattice* en el plano XZ (líneas cian cada 30 unidades sobre una extensión de 400×400) con 400 partículas de **fluctuación del vacío** que aparecen y desaparecen siguiendo un ciclo de vida sinusoidal, simulando la creación y aniquilación de pares virtuales.
- **Campo de interferencia cuántica:** 600 partículas azules dispuestas en un plano de fondo ($z = -200$) que oscilan verticalmente con ondas estacionarias $y += \sin(t + \phi) \cdot 0,01 \cdot t$, reproduciendo un patrón de interferencia.
- **Quantum Foam:** partículas con un ciclo de vida basado en dos armónicos superpuestos (un destello principal y un segundo armónico al 70 % de frecuencia) que simulan las fluctuaciones del espacio-tiempo a escala de Planck. Cada partícula experimenta su propia vibración de incertidumbre de Heisenberg.
- **Ondas de decoherencia:** anillos expansivos (RingGeometry) que se generan aleatoriamente desde los procesadores cuánticos visibles cada 8 ± 6 segundos, expandiéndose durante 3,5 s con una curva *easeOutCubic* antes de desaparecer. Representan la pérdida de coherencia cuántica que se propaga desde las organizaciones más activas.
- **Rayos cósmicos:** *ribbons* triangulares que cruzan el universo en seis colores ciclados (cian, magenta, dorado, verde esmeralda, naranja plasma, violeta). El 55 % de los rayos se dirigen hacia la Dyson Shell (radio 3.500 unidades), generando destellos de impacto con propagación gaussiana al alcanzarla.

- **Anillos de energía:** pulsos toroidales que se expanden exponencialmente ($e^{t+\phi \cdot 6}$) con desvanecimiento radial, representando transferencias de energía entre nodos.
- **Dyson Shell:** un icosaedro subdividido cuatro veces (≈ 1.280 triángulos) con radio de 3.500 unidades que envuelve todo el universo. Sus aristas conducen 12 pulsos de energía que saltan por adyacencia topológica con un brillo base cian, mientras que sus vértices pulsan suavemente con $0,15 + 0,15 \cdot \sin(0,3t + \phi)$. La esfera rota a velocidad ultra-lenta (0,005 rad/s). Los impactos de rayos cósmicos generan ondas de propagación 2D a 1.500 u/s sobre su superficie.

Adicionalmente se aplican efectos de post-procesado (*bloom* para realzar las áreas luminosas y *depth of field* para enfocar selectivamente la atención del usuario) que completan la composición visual de la escena.

Anexo K

Galería del Agente de IA Conversacional

Este anexo recoge ejemplos detallados de interacción con el agente de IA conversacional de Entangle. El análisis funcional y la evaluación cuantitativa se encuentran en la Sección 5.14 del Capítulo 5; aquí se documentan los detalles técnicos de la canalización de RAG y se muestran capturas adicionales de los seis *workers* especializados (DATA, DASHBOARD, UNIVERSE, KNOWLEDGE, INSIGHTS y DEEP_RESEARCH), de la memoria conversacional y de las acciones automáticas disponibles.

K.1 Catálogo de Workers

La Tabla K.1 resume los seis *workers* del agente, su función y la fuente o herramientas que emplea cada uno. Tres de ellos (DATA, DASHBOARD, UNIVERSE) constituyen el núcleo de consulta de datos y guía de la interfaz; los tres restantes (KNOWLEDGE, INSIGHTS, DEEP_RESEARCH) incorporan las capacidades cualitativas, analíticas y de acceso externo descritas en las secciones siguientes.

Tabla K.1: Workers especializados del agente de IA conversacional.

Worker	Función	Fuente / Herramientas
DATA	Consultas cuantitativas: rankings, conteos, agregaciones	MongoDB (<i>function calling</i>)
DASHBOARD	Explicación de la interfaz 2D, métricas y metodología; acciones de UI	<i>System prompt</i> (sin herramientas)
UNIVERSE	Guía del universo 3D, lentes y navegación; acciones de UI	<i>System prompt</i> (sin herramientas)
KNOWLEDGE	Preguntas cualitativas («¿qué hace X?», comparativas)	RAG sobre README (búsqueda vectorial)
INSIGHTS	Análisis cruzado: comparativas, similitud, colaboración	MongoDB + vectores + grafo
DEEP_RESEARCH	Investigación externa: <i>papers</i> y web	arXiv + Tavily (<i>endpoint</i> separado)

K.2 Consultas al Agente DATA

El agente DATA es el único con acceso de consulta **de propósito general** a MongoDB, formulando directamente las *aggregation pipelines* mediante *function calling*; los *workers* KNOWLEDGE e INSIGHTS también consultan los datos, pero a través de herramientas especializadas (búsqueda vectorial y análisis predefinidos). La Figura K.1 muestra una consulta típica en la que el usuario solicita los repositorios cuánticos con más estrellas; el panel de razonamiento permite observar cómo el agente inspecciona el esquema, formula una *aggregation pipeline* y presenta los resultados con datos reales.

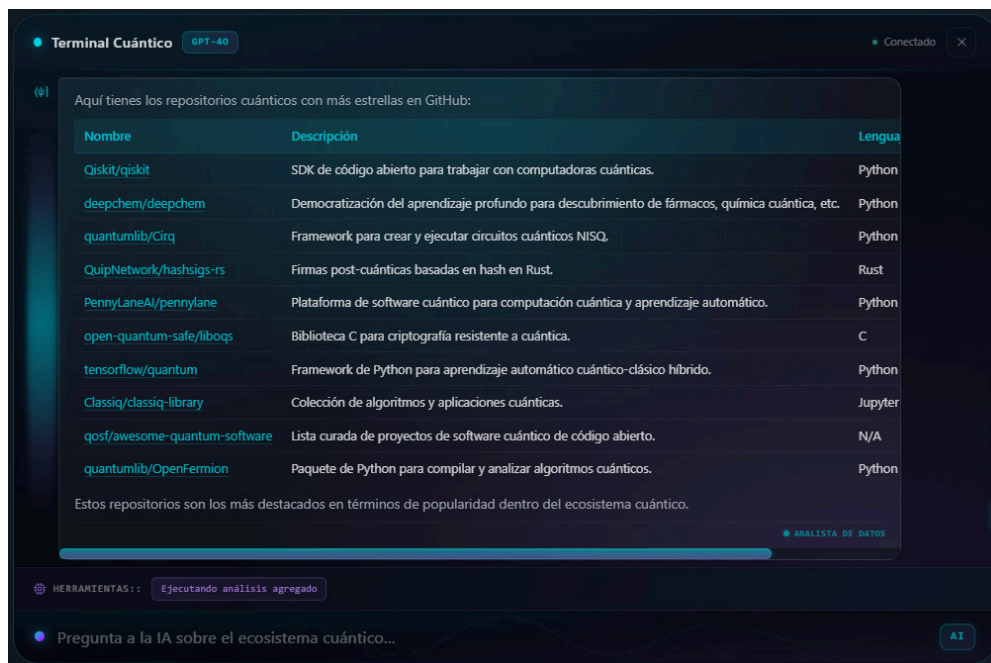


Figura K.1: Consulta al agente DATA con datos reales de la base de datos.

La Figura K.2 ilustra una consulta con agregación: ante «¿Qué porcentaje de repositorios usan Python como lenguaje principal?», el agente ejecuta un análisis agregado y devuelve el porcentaje calculado (42,05 %).

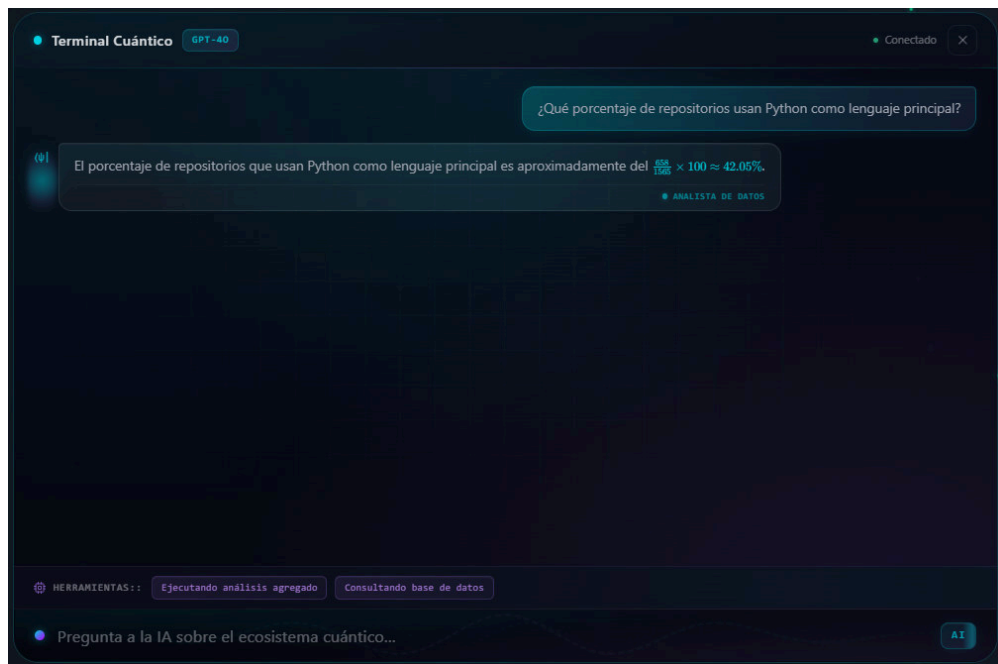


Figura K.2: Consulta con agregación al agente DATA.

K.3 Razonamiento de los Agentes y Acciones Automáticas

Los agentes DASHBOARD y UNIVERSE responden a partir del conocimiento embebido en sus *system prompts*. La Figura K.3 muestra cómo el agente DASHBOARD explica el significado de una métrica.

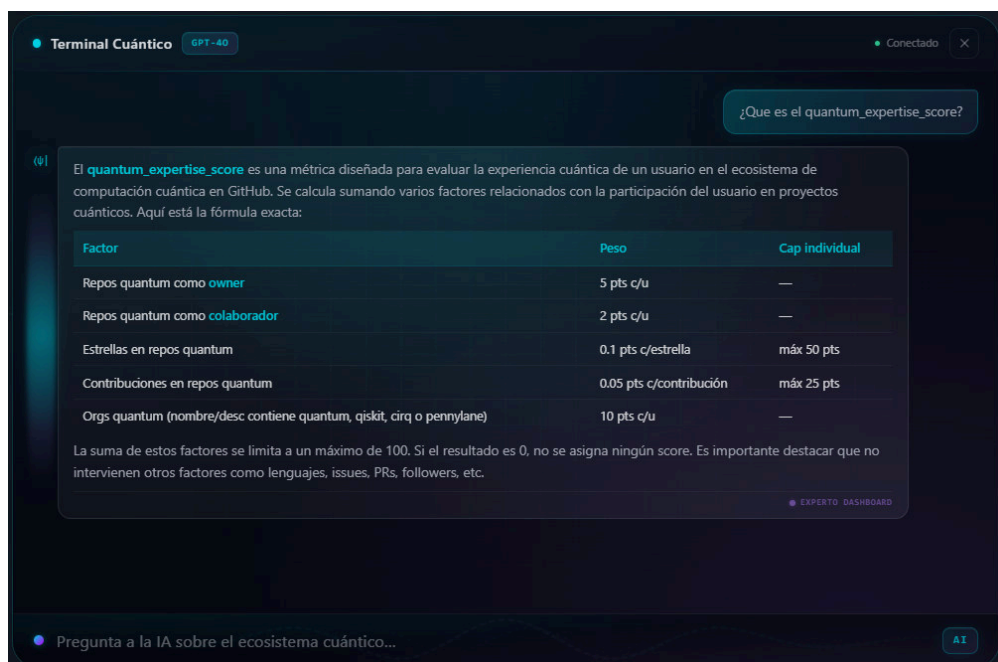


Figura K.3: Respuesta contextual del agente DASHBOARD.

Mientras un agente trabaja, el chat muestra un panel de razonamiento en vivo con las herramientas que está invocando y los resultados intermedios. La Figura K.4 captura este

panel para el agente DATA (etiquetado en la interfaz como *Analista*) durante la resolución de una consulta agregada: se observan las llamadas sucesivas a `run_aggregation` y a la consulta del esquema de `repositories`, con sus correspondientes confirmaciones de *Datos recibidos* y el contador de resultados obtenidos junto al tiempo total transcurrido.

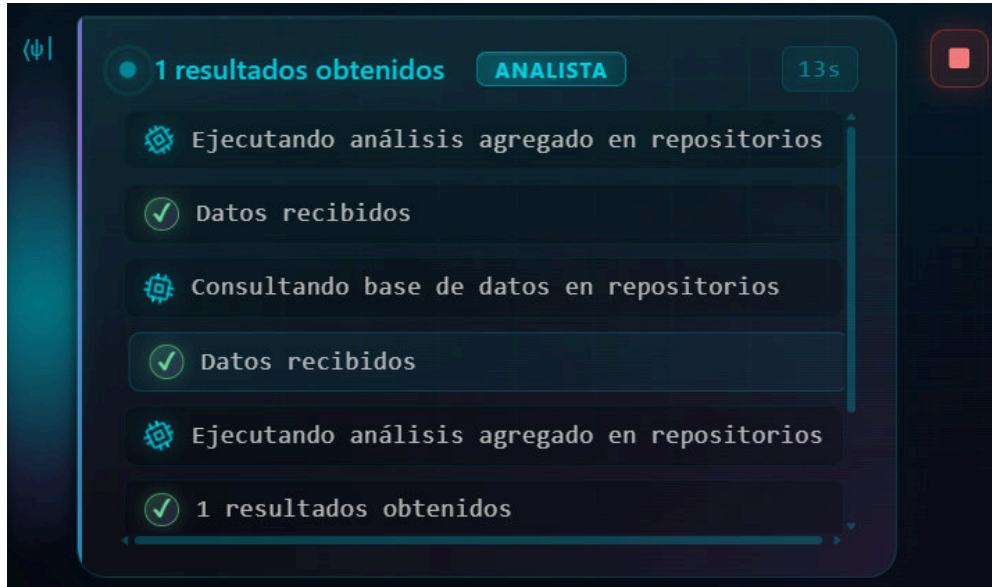


Figura K.4: Panel de razonamiento en vivo del agente DATA (*Analista*).

La Figura K.5 ilustra la ejecución de una acción automática `CREATE_VIEW`: ante la petición imperativa «Crea una vista con Microsoft, Qiskit, Quantumlib y Mathworks», el agente `DASHBOARD` (*Experto Dashboard*) genera una nueva vista personalizada en el sistema de favoritos del cuadro de mando (visible en la cabecera como *Vista activa: Vista Autogenerada #5*) y reduce los KPIs y el resto de visualizaciones al subconjunto seleccionado de organizaciones.

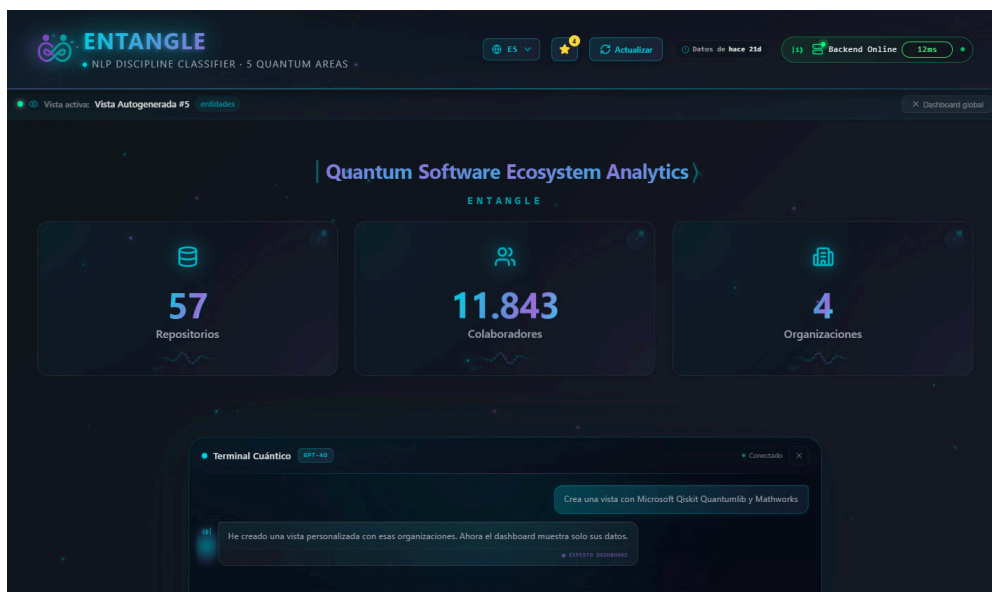


Figura K.5: Acción automática `CREATE_VIEW` ejecutada por el agente `DASHBOARD`.

K.4 Detalle de la Canalización de RAG

El *worker* KNOWLEDGE se apoya en una canalización de RAG [LPP⁺20] sobre los ficheros README de los repositorios indexados, cuya estructura conceptual (fragmentación, vectorización, almacenamiento y búsqueda semántica) se describe en la Sección 5.6.3. Este anexo detalla sus aspectos de implementación.

El indexado se ejecuta de forma idempotente mediante el módulo `indexer.py`, que solo procesa los README nuevos o modificados, de modo que reanudar el proceso no reprocesa el corpus completo. La fragmentación reside en `chunker.py`, que respeta la estructura *Markdown* para que ningún bloque parta un encabezado, una lista o un fragmento de código a la mitad. La vectorización emplea el modelo `text-embedding-3-small` de Azure AI Foundry, que produce vectores de 1536 dimensiones [VSP⁺17], y los vectores se persisten en una colección dedicada de Cosmos DB indexada con el algoritmo IVF y distancia coseno, reutilizando el clúster existente como almacén vectorial para evitar infraestructura y coste adicionales.

El corpus resultante asciende a unos 11.000 fragmentos sobre los 1.565 repositorios del *dataset*. Las tres herramientas del *worker* INSIGHTS (`compare_repos`, `find_similar_repos` y `collaboration_strength`) reutilizan esta misma infraestructura vectorial, combinándola con consultas estructuradas sobre MongoDB.

K.5 Conocimiento Cualitativo: Worker KNOWLEDGE

El *worker* KNOWLEDGE responde preguntas cualitativas mediante la canalización de RAG descrita en la Sección K.4. Ante una consulta como «¿En qué se diferencian Qiskit y Cirq?», ejecuta una búsqueda semántica sobre los aproximadamente 11.000 fragmentos indexados, recupera los pasajes más afines de los README de ambos proyectos y elabora la respuesta **citando las fuentes reales**; cuando el repositorio consultado no figura en el corpus, lo admite explícitamente y ofrece la información parcial disponible, evitando la fabricación de datos característica de las alucinaciones de los LLM [JLF⁺23]. Esta consulta comparativa se muestra en la Figura 5.15 (Sección 5.14.3 del cuerpo).

K.6 Análisis Cruzado: Worker INSIGHTS

El *worker* INSIGHTS ofrece análisis comparativos estructurados mediante tres herramientas que reutilizan la infraestructura vectorial del RAG: `compare_repos` genera una comparativa tabulada de hasta cinco repositorios con métricas reales (estrellas, contribuidores, lenguaje, licencia y actividad); `find_similar_repos` aprovecha la búsqueda vectorial para descubrir proyectos semánticamente afines a uno dado; y `collaboration_strength` cuantifica el solapamiento de contribuidores entre dos organizaciones mediante el índice de Jaccard. A modo de ejemplo, la medición entre las organizaciones `qiskit` y `qiskit-community` arroja 260 contribuidores compartidos, lo que se traduce en un índice de Jaccard de 0,19 indicativo de

una colaboración significativa. Una comparativa tabulada de tres repositorios generada con `compare_repos` se muestra en la Figura 5.16 (Sección 5.14.3 del cuerpo).

K.7 Investigación Externa: Worker DEEP_RESEARCH

El *worker* DEEP_RESEARCH dota al agente de acceso a internet para responder preguntas que exceden el *dataset*, como la búsqueda de publicaciones académicas recientes. Dispone de dos herramientas: `search_arxiv`, que consulta la API pública de arXiv para recuperar *papers* científicos con su título, autores y enlace, y `web_search`, que emplea el servicio Tavily para búsquedas web optimizadas para LLM. Dos decisiones de diseño gobiernan este *worker*: queda **fuera del enrutado automático** y solo se activa mediante una acción explícita del usuario en la interfaz, evitando que el agente se salga del dominio sin intención; y opera bajo una **política estricta de honestidad de fuentes**, de modo que si la fuente primaria (arXiv) falla y se recurre a la búsqueda web como alternativa, la respuesta lo declara explícitamente y no presenta los resultados de respaldo como si procedieran de arXiv. La robustez frente a los límites de tasa de arXiv se aborda mediante reintentos con espera incremental. La Figura K.6 muestra una búsqueda de *papers* recientes sobre VQE con mitigación de ruido.

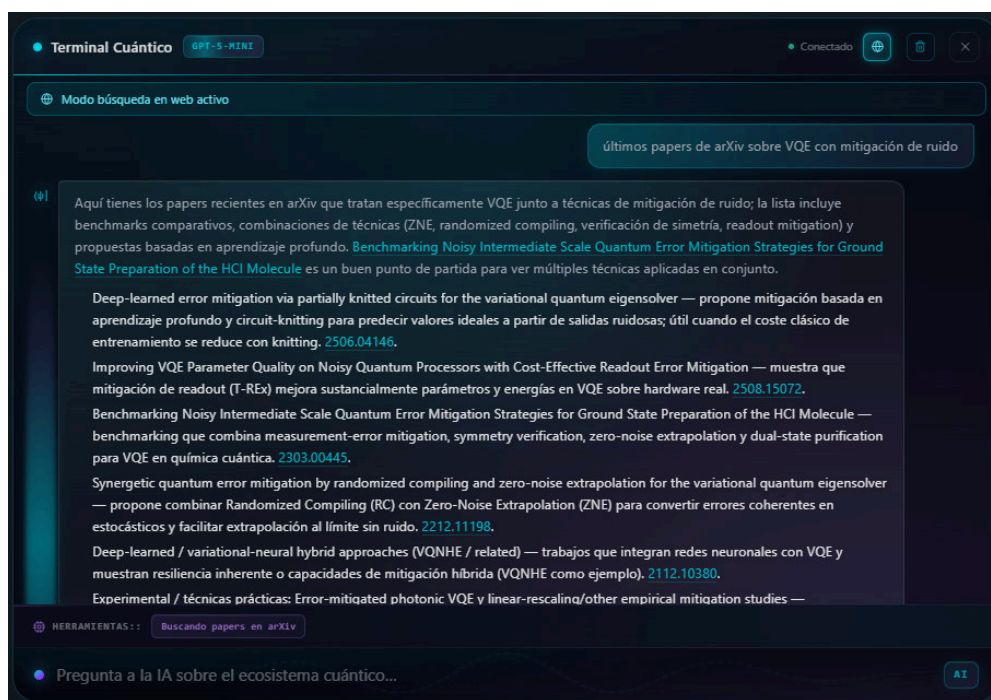


Figura K.6: Consulta al *worker* DEEP_RESEARCH: recuperación de publicaciones reales de arXiv con título, autores y enlace.

K.8 Memoria Conversacional

Para superar la ausencia de contexto entre mensajes, el agente incorpora memoria conversacional persistente en servidor. Cada sesión se identifica con un UUID y se almacena como documento independiente en la colección `chat_sessions` de Cosmos DB, con un índice TTL

que expira automáticamente las sesiones inactivas a los 30 días. El identificador de sesión se conserva en el *localStorage* del navegador, de modo que el contexto sobrevive a recargas de página y permite resolver referencias anafóricas. El número de mensajes por sesión se acota para no exceder la ventana de contexto del modelo, y al recuperar el historial se aplica una sanitización que descarta secuencias incompletas de llamadas a herramientas, garantizando la validez del *payload* enviado al modelo. En la interacción de ejemplo, el usuario formula primero la consulta «¿Cuáles son los cinco repositorios cuánticos con más estrellas?», que el agente resuelve con la lista correspondiente. Acto seguido, sin nombrar ninguna entidad, pregunta «¿Y de esos, cuál tiene más forks?». La Figura K.7 recoge la respuesta a esta segunda consulta: el agente interpreta correctamente la referencia anafórica «de esos» recuperando la lista de repositorios del turno anterior desde la sesión persistida.

Terminal Cuántico GPT-5-MINI

Conectado

Expandir a los top 10.
Mostrar además forks, colaboradores o fecha de última actualización.
Filtrar por lenguaje (p. ej. solo Python) o por organización. ¿Qué prefieres?

ANALISTA DE DATOS

¿Y de esos, cuál tiene más forks?

He consultado la colección **repositories** y ordené esos cinco por **fork_count**. El repositorio con más forks es:

Rank	Repositorio (full_name)	Forks	Estrellas	Lenguaje	URL
1	Qiskit/qiskit	2778	7092	Python	https://github.com/Qiskit/qiskit
2	deepchem/deepchem	2115	6570	Python	https://github.com/deepchem/deepchem
3	quantumlib/Cirq	1182	4881	Python	https://github.com/quantumlib/Cirq
4	PennyLaneAI/pennylane	747	3086	Python	https://github.com/PennyLaneAI/pennylane
5	QuipNetwork/hashsig-rs	21	4145	Rust	https://github.com/QuipNetwork/hashsig-rs

¿Quieres que muestre además la fecha de la última actualización (pushed_at) o el número de colaboradores para estos repositorios?

ANALISTA DE DATOS

HERRAMIENTAS: Consultando base de datos

Pregunta a la IA sobre el ecosistema cuántico...

AI

Figura K.7: Memoria conversacional: respuesta del agente a una pregunta anafórica («¿y de esos...?») que solo es resoluble recuperando el contexto del turno anterior desde la sesión persistida.

Anexo L

Galería del Dashboard 2D Interactivo

Este anexo recoge las capturas detalladas del cuadro de mando interactivo 2D de Entangle. El análisis funcional condensado se encuentra en la Sección 5.12 del Capítulo 5; aquí se reúnen las capturas de las tarjetas KPI en sus dos estados, el detalle del panel de *bridge profiles* y el grafo de colaboración inter-organizacional.

L.1 Vista General de la Página Principal y Tarjetas KPI

La Figura L.1 presenta una vista panorámica de la página principal del cuadro de mando, donde se aprecian de un vistazo los cuatro bloques que componen la *landing* del producto: la cabecera con el logotipo ENTANGLE y los indicadores de estado (selector de idioma, *badge* del *backend* y marca temporal de última actualización), las tres tarjetas KPI con las métricas agregadas del ecosistema (repositorios, colaboradores y organizaciones), la barra de acceso al chat de IA con sus accesos rápidos a *Top Repos*, *Orgs Líderes* y *Estadísticas*, y el banner inferior de acceso al universo 3D con sus contadores de nodos, enlaces y puentes.

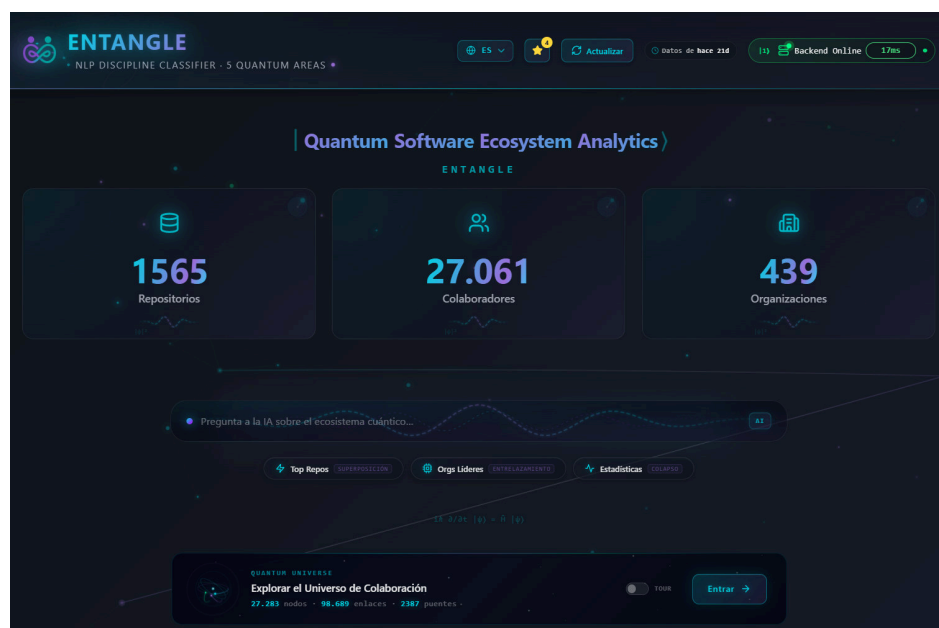


Figura L.1: Vista panorámica de la página principal de Entangle.

La Figura L.2 ilustra el estado de una tarjeta KPI tras la interacción del usuario: la onda de probabilidad que en estado inicial recorre la base de la tarjeta colapsa a un delta de Dirac $\delta(x)$, simbolizando la medición cuántica y materializando visualmente la metáfora del colapso de la función de onda descrita en la Sección 5.8.2.



Figura L.2: Tarjeta KPI tras la interacción del usuario: colapso de la función de onda.

L.2 Panel de *Bridge Profiles*

La Figura L.3 muestra en detalle el donut de distribución por disciplina junto al panel de puentes interdisciplinarios, que permite identificar qué usuarios cruzan fronteras entre disciplinas distintas y funcionan como conectores del ecosistema.

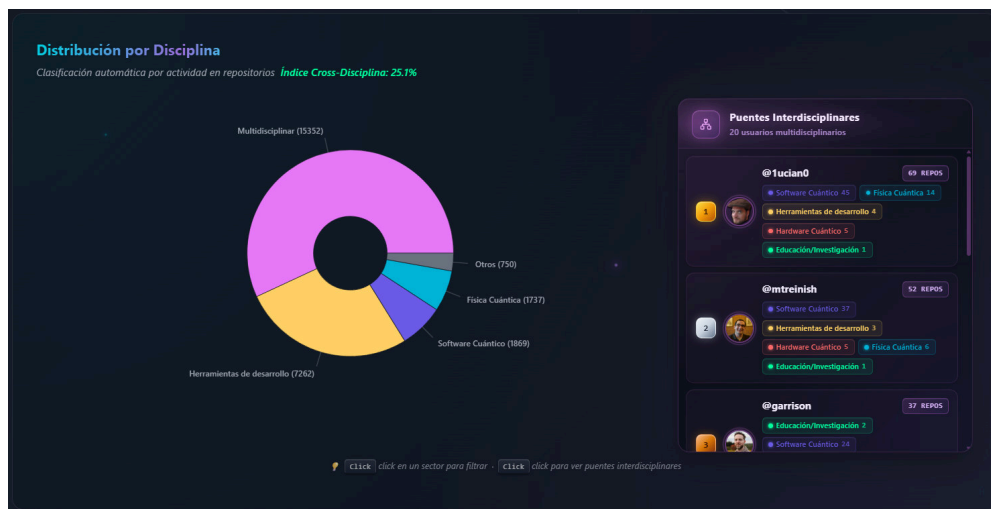


Figura L.3: Detalle del donut de distribución por disciplina y panel de puentes interdisciplinares.

L.3 Grafo de Colaboración

La Figura L.4 muestra el grafo de colaboración con *layout* circular jerárquico obtenido tras seleccionar varias organizaciones. Los *bridge users* (nodos ámbar en el centro) conectan comunidades que de otro modo permanecerían aisladas, y las aristas inter-organizacionales (cian) se distinguen visualmente de las intra-organizacionales.

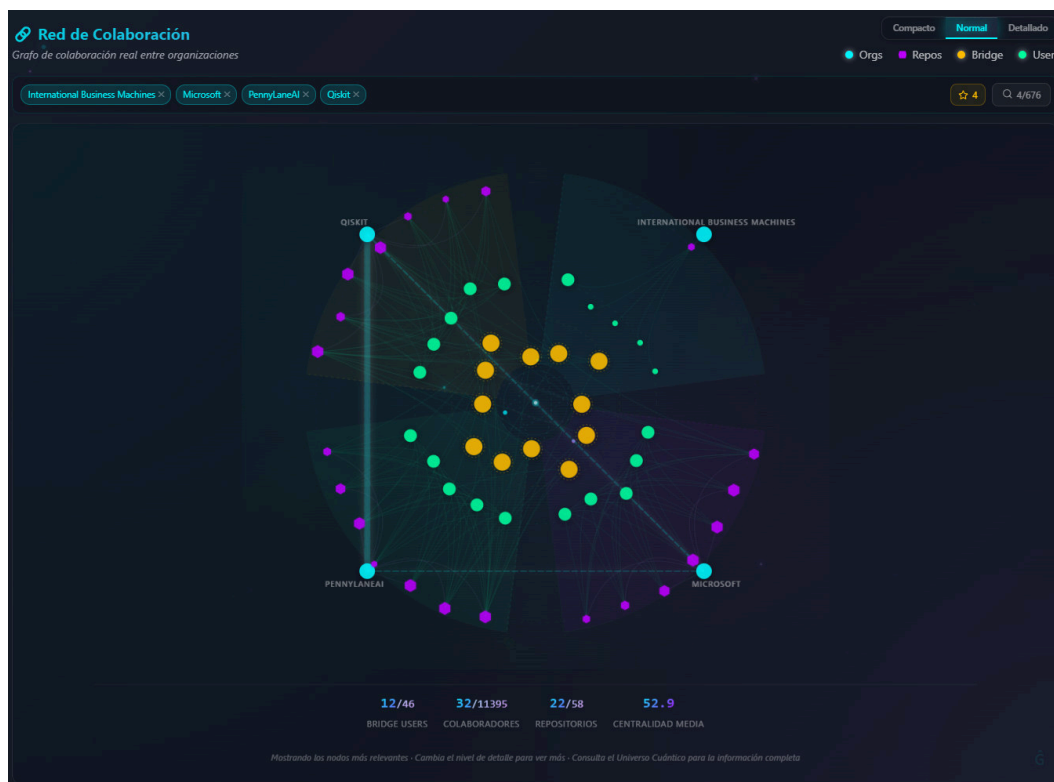


Figura L.4: Grafo de colaboración con *layout* circular jerárquico y aristas diferenciadas por tipo.

L.4 Cabecera y Pie de Página

Tras la primera entrega del cuadro de mando se introdujo un rediseño de los componentes de marco (cabecera y pie de página) y se reforzó la responsividad de la aplicación, sin alterar la composición ni el comportamiento del cuerpo del dashboard documentado en las secciones anteriores.

La cabecera (Figura L.5) integra varios componentes nuevos: LogoQuantumParticles (logotipo con animación de partículas cuánticas), TaglineRotator (lema rotatorio con varias frases descriptivas del producto), LanguageSelector (selector de idioma EN/ES), BackendStatusBadge (indicador en tiempo real del estado del backend, consultado mediante *health check* periódico), LastUpdatedBadge (marca temporal de la última actualización del dataset) y un BellCircuit decorativo a modo de favicon dinámico. Estos componentes proporcionan información operativa del sistema sin desplazar contenido del cuerpo.

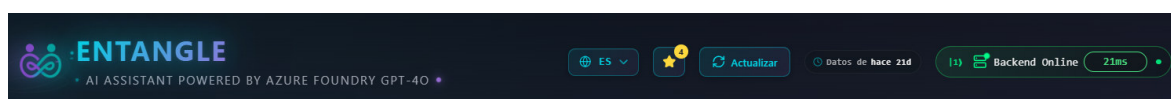


Figura L.5: Cabecera del cuadro de mando.

El pie de página (Figura L.6) lo constituye el componente FooterExtra, que muestra estadísticas en vivo del ecosistema (totales de repositorios, organizaciones y desarrolladores

indexados) y enlaces sociales y de contacto del proyecto. Es el complemento informativo de la cabecera y cierra visualmente la página.



Figura L.6: Pie de página FooterExtra del cuadro de mando.

Adicionalmente, se realizó un **rediseño responsivo** transversal mediante CSS Modules con *breakpoints* adaptativos en `App.module.css` y en los *stylesheets* de los principales componentes (`ChartsSection`, `CollaborationBanner`, `ContributorSankey`, `UniverseView`). Este trabajo permite que el cuadro de mando, originalmente concebido para resoluciones de escritorio amplias, conserve la legibilidad y la usabilidad en pantallas reducidas sin sacrificar la densidad informativa de las visualizaciones.

Anexo M

Detalle de los resultados de rendimiento

Este anexo amplía la Sección 5.15 con el detalle técnico de las optimizaciones de renderizado, base de datos y pipeline que sostienen las cifras presentadas en el cuerpo de la memoria.

M.1 Detalle del Renderizado del Universo 3D

El universo 3D (Sección 5.8) renderiza simultáneamente ~ 28.000 entidades con ~ 98.000 conexiones mediante React Three Fiber [pmn24a]. Mantener una tasa de fotogramas fluida con este volumen de geometría requiere un conjunto de optimizaciones que se cuantifican a continuación.

Consolidación de *draw calls*. Cada tipo de entidad utiliza `InstancedMesh` [Cab24] para consolidar miles de objetos en un número mínimo de llamadas de dibujo a la GPU. La Tabla M.1 resume el impacto.

Tabla M.1: *Draw calls* estimados sin y con `InstancedMesh` (IM).

Componente	Instancias	<i>Draw calls</i> sin IM	<i>Draw calls</i> con IM
Organizaciones (toros)	~ 420	~ 1.680	4
Repositorios (esferas)	~ 1.600	~ 3.200	2
Anillos de energía	~ 420	~ 420	1
Pulsos de entrelazamiento	variable	variable	1

Los ~ 27.000 usuarios se renderizan como puntos en un único `ShaderMaterial` GLSL que calcula la posición orbital, el color de lente y la vibración de incertidumbre íntegramente en la GPU, sin iteraciones en el hilo principal de JavaScript.

Canales de entrelazamiento. Las ~ 98.000 conexiones (Sección 5.8.3) se representan mediante líneas de 35 puntos cada una, lo que genera $\sim 1,33$ millones de vértices. Un *shader* GLSL personalizado gestiona la animación de flujo cuántico y la visibilidad de cada canal. Cuando el progreso de montaje es inferior al 0,1 %, el *shader* desplaza el vértice fuera del

clip space (`gl_Position = vec4(9999)`) en lugar de establecer un tamaño cero, evitando así el *clamping* a 1 px que aplican numerosas GPUs.

Montaje progresivo. La escena no se construye de golpe: un sistema de 9 etapas con retardos configurables (200–900 ms entre etapas) añade los componentes de forma secuencial. Esto distribuye la carga de inicialización a lo largo de ~ 4 segundos y permite que el usuario perciba la interfaz como interactiva desde la etapa 5, antes de que la escena esté completa. Durante la carga, el bucle de renderizado opera en modo demand (solo redibuja cuando hay cambios); al completarse, conmuta a always para la animación continua.

Web Workers. El cálculo del *layout* espacial (Sección 5.8.4) se ejecuta en un Web Worker dedicado para no bloquear el hilo principal. Este *worker* aplica el mapeo logarítmico, la colocación estocástica con 80 intentos por organización y la clasificación de Jenks Natural Breaks [Jen67]. Un segundo *worker* procesa en tres fases los datos de detalle: datos básicos (~ 50 ms), simulaciones (~ 200 – 500 ms) y entidades similares (la más costosa). Los resultados se devuelven en *chunks* de 500 posiciones y 1.000 conexiones, intercalando cesiones al hilo principal cada 8 ms (`yieldToMain`) para mantener la capacidad de respuesta [Moz24d].

Post-procesamiento. La cadena de post-procesamiento (Sección 5.8.6) se limita a un único efecto Bloom (`intensity=1.4`, `threshold=0.2`, 4 niveles de *mipmap*) con multisampling desactivado. Esta combinación proporciona el efecto de luminiscencia cuántica sin el coste de efectos adicionales como Screen Space Ambient Occlusion (SSAO) o *motion blur*. La resolución de píxeles se limita a `min(devicePixelRatio, 2)` para evitar el sobre coste de pantallas 4K.

M.2 Detalle de Base de Datos y Consultas

La conexión a Azure Cosmos DB for MongoDB vCore [Mic24c] utiliza un *pool* de 100 conexiones máximas con un mínimo de 10 mantenidas activas, gestionado por Py-Mongo [Mon24b]. Los *timeouts* se ajustan al perfil de carga: 10 s para conexión, 30 s para operaciones (incrementado para soportar *bulk upserts* del *pipeline*) y 5 s para selección de servidor.

Las consultas de agregación incluyen un `maxTimeMS` explícito (5.000–15.000 ms según la complejidad) que actúa como salvaguarda contra consultas que bloqueen el servidor. La centralidad de intermediación (*betweenness centrality*) [Fre77], calculada durante la fase de análisis de red (Sección 5.3.1), es la métrica más costosa computacionalmente. Utiliza un muestreo de $k = 50$ nodos para grafos con más de 5.000 nodos, lo que reduce el tiempo de cálculo en un factor de $\sim 4\times$ frente al muestreo de $k = 200$ sin degradar significativamente la precisión de los *rankings* resultantes.

M.3 Detalle de la Caché *Chunked* en MongoDB

Dado que Azure Cosmos DB for MongoDB vCore impone un límite de 16 MB por documento Binary JSON (BSON), los *arrays* grandes se fragmentan en *chunks* de 1,5 MB con un margen del 30 % para el *overhead* BSON (Sección 5.3.1). En la práctica, el grafo completo se distribuye en aproximadamente 10 documentos (1 meta + 2 de nodos + 6 de enlaces + 1 de *bridge users*). La lectura utiliza una única consulta `find({$in})` en lugar de N lecturas secuenciales, lo que mantiene la latencia muy por debajo del segundo en el endpoint `network-metrics`.

Anexo N

Detalle del análisis de calidad de código (SonarQube)

Este anexo amplía el análisis agregado presentado en la Sección 5.17 con el detalle por repositorio: configuración de la *Quality Gate*, métricas individuales del backend y del frontend, lista de incidencias corregidas en el primer escaneo y *dashboards* de SonarQube de cada proyecto.

N.1 Configuración de SonarQube

Se desplegó una instancia local de SonarQube Community Edition sobre Docker para analizar los dos repositorios del proyecto. Se definió una *Quality Gate* personalizada denominada «Entangle» con las condiciones recogidas en la Tabla N.1.

Tabla N.1: Condiciones de la *Quality Gate* «Entangle».

Métrica	Condición	Umbral
Fiabilidad (<i>Reliability Rating</i>)	$\leq$	C
Seguridad (<i>Security Rating</i>)	$\leq$	A
Mantenibilidad (<i>Maintainability Rating</i>)	$\leq$	B
Cobertura de tests	$\geq$	60 %
Líneas duplicadas	$\leq$	5 %
Duplicación en código nuevo	$\leq$	3 %
Incidencias en código nuevo	$\leq$	0
<i>Security Hotspots</i> revisados	$\geq$	80 %
Vulnerabilidades	$\leq$	0

La cobertura del backend se genera con `pytest-cov` en formato Cobertura XML, mientras que la del frontend se obtiene mediante el proveedor V8 de Vitest. SonarScanner [Son24c] analiza ambos proyectos e importa los informes de cobertura correspondientes.

N.2 Detalle del Backend

La Tabla N.2 resume las métricas obtenidas para el backend Python.

Tabla N.2: Métricas de SonarQube para el backend (entangle-backend).

Métrica	Valor
Líneas de código (<i>ncloc</i>)	14 687
Líneas totales	21 768
Ficheros	35
Funciones	389
Clases	39
Densidad de comentarios	22,5 %
Cobertura de tests	61,0 %
Líneas duplicadas	2,1 %
Bugs	0
Vulnerabilidades	0
<i>Security Hotspots</i>	0 (100 % revisados)
<i>Code Smells</i>	294
Deuda técnica	72 h 4 min
Complejidad ciclomática	2 395
Complejidad cognitiva	3 803
<i>Reliability Rating</i>	A
<i>Security Rating</i>	A
<i>Maintainability Rating</i>	A
Quality Gate	OK

Corrección de incidencias detectadas en el primer escaneo. El análisis inicial del backend detectó **18 bugs** y **3 *security hotspots*** que impedían superar la *Quality Gate*. La Tabla N.3 detalla las correcciones aplicadas, tras las cuales el backend pasó de *Reliability Rating* E (5 bugs BLOCKER) a A.

Tabla N.3: Incidencias corregidas en el backend tras el primer análisis de SonarQube.

Fichero	Sev.	Corrección aplicada
models/ ___.init__.py	BLOCKER (×17)	La lista <code>__all__</code> exportaba 17 nombres de clase inexistentes (<code>Language</code> , <code>LanguageEdge</code> , <code>License</code> , etc.) procedentes de versiones anteriores del modelo de datos. Se sustituyeron por los nombres realmente importados (<code>LanguageInfo</code> , <code>LicenseInfo</code> , <code>OwnerInfo</code> , etc.).
api/ chat_.routes.py	MAJOR (×1)	La tarea creada con <code>asyncio.ensure_future()</code> no se almacenaba en ninguna variable, exponiéndola a recolección de basura prematura. Se asignó a una variable local <code>_task</code> para retener la referencia.
api/routes.py	LOW (×2)	Se utilizaba <code>hashlib.md5()</code> para generar claves de caché. Aunque no constituye un riesgo real (no se usa con fines criptográficos), SonarQube lo marca como <i>hotspot</i> . Se migró a <code>hashlib.sha256()</code> truncado a 16 caracteres.
ai/agent.py	MEDIUM (×1)	La expresión regular <code>_CODE_FENCE_ACTION</code> contenía cuantificadores anidados (<code>\n*\s*</code>) con <code>re.DOTALL</code> , susceptibles de <i>backtracking</i> polinómico. Se simplificó a <code>\s+</code> sin <code>re.DOTALL</code> .

N.3 Detalle del Frontend

La Tabla N.4 resume las métricas obtenidas para el frontend React.

El frontend acumula 734 *code smells* con 70 horas de deuda técnica y una complejidad ciclomática de 4 217, valores superiores a los del backend debido a la naturaleza del código React/JSX, que genera un elevado número de funciones (1 353 entre componentes, *hooks* y *handlers*). La duplicación del 3,9 % y los 2 bugs de severidad baja sitúan la fiabilidad en B, umbral aceptado por la *Quality Gate* (condición $\leq C$). La cobertura del 62,2 % se centra en los *stores* Zustand, la capa HTTP y la lógica algorítmica, excluyendo deliberadamente los componentes 3D de Three.js cuya validación es visual.

Tabla N.4: Métricas de SonarQube para el frontend (entangle-frontend).

Métrica	Valor
Líneas de código (<i>ncloc</i>)	14 040
Líneas totales	16 923
Ficheros	34
Funciones	1 353
Densidad de comentarios	10,8 %
Cobertura de tests	62,2 %
Líneas duplicadas	3,9 %
Bugs	2
Vulnerabilidades	0
<i>Security Hotspots</i>	0 (100 % revisados)
<i>Code Smells</i>	734
Deuda técnica	70 h
Complejidad ciclomática	4 217
Complejidad cognitiva	2 608
<i>Reliability Rating</i>	B
<i>Security Rating</i>	A
<i>Maintainability Rating</i>	A
Quality Gate	OK

N.4 Dashboards de SonarQube por Repositorio

entangle-backend / Overview

Overview

15k Lines of Code · Version not provided · Last analysis 51 seconds ago

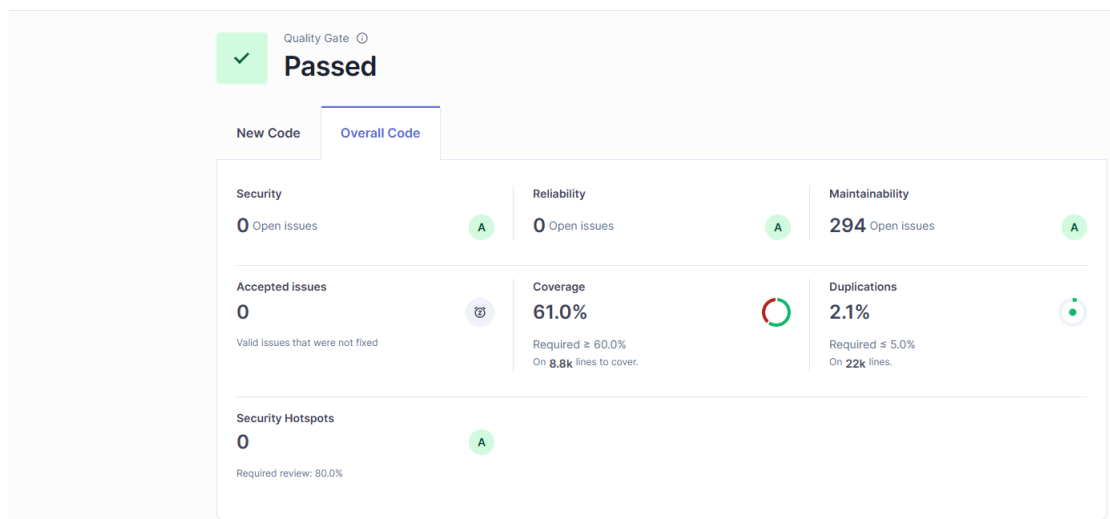


Figura N.1: Dashboard de SonarQube del proyecto entangle-backend.

Overview

14k Lines of Code • Version not provided • Last analysis 3 minutes ago

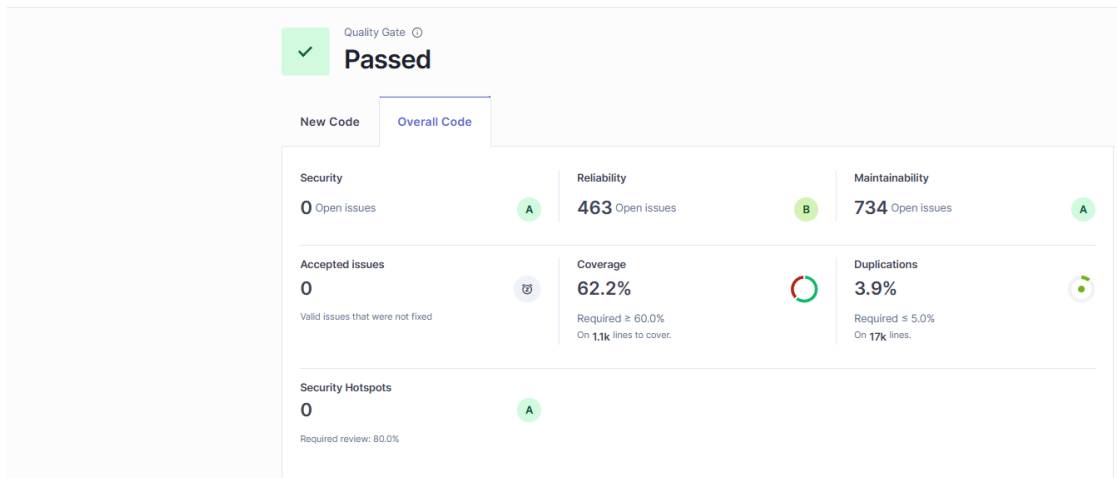


Figura N.2: Dashboard de SonarQube del proyecto entangle-frontend.

N.5 Integración Continua con SonarCloud

El análisis local descrito en las secciones anteriores se complementa con una capa de **integración continua** desplegada sobre *SonarCloud* [Son24a], el servicio gestionado de SonarSource. Esta capa no sustituye a la instancia local de SonarQube, que se conserva para análisis profundos puntuales con la *Quality Gate Entangle* de la Tabla N.1, sino que añade un control automático sobre cada cambio que entra al repositorio.

Cada repositorio del proyecto incorpora un *workflow* de GitHub Actions [Git24a] ubicado en `.github/workflows/sonarcloud.yml` que se dispara en push a las ramas `main` y `hotfix` y en cualquier `pull_request` contra `main`. El *workflow* ejecuta la suite completa de pruebas con cobertura, sube el informe a SonarCloud y delega la evaluación de la *Quality Gate* en el servicio remoto.

N.5.1 Workflow del Backend

El Listado N.1 muestra una versión sintetizada del *workflow* del backend. Levanta un servicio MongoDB 7 efímero, requerido por la suite de tests de la capa de persistencia, instala `pytest-cov`, `pytest-mock` y `pytest-timeout` (con un *timeout* de 30 s por test para evitar bloqueos por servicios externos no disponibles en CI) y publica `coverage.xml` en formato Cobertura, que SonarCloud importa según la propiedad `sonar.python.coverage.reportPaths` declarada en `sonar-project.properties`.

```

1  name: SonarCloud (Backend)
2  on:
3    push: { branches: [main, hotfix] }
4    pull_request: { branches: [main] }
5  jobs:
6    sonar:
7      runs-on: ubuntu-latest
8      services:
9        mongodb:
10         image: mongo:7
11         ports: [27017:27017]
12      steps:
13        - uses: actions/checkout@v4
14          with: { fetch-depth: 0 } # Sonar necesita historial completo
15        - uses: actions/setup-python@v5
16          with: { python-version: '3.11', cache: 'pip' }
17        - run: pip install -r requirements.txt pytest-cov pytest-timeout
18        - name: Tests + cobertura
19          env:
20            MONGO_URI: mongodb://localhost:27017/
21            ENVIRONMENT: test
22          run: |
23            pytest --cov=src --cov-report=xml:coverage.xml \
24              --timeout=30 --timeout-method=thread -q
25        - uses: SonarSource/sonarcloud-github-action@v3
26          env:
27            SONAR_TOKEN: ${ secrets.SONAR_TOKEN }

```

Listado N.1: Extracto de .github/workflows/sonarcloud.yml (Backend).

N.5.2 Workflow del Frontend

El *workflow* del frontend (Listado N.2) es análogo al del backend pero sin servicios externos: Vitest se invoca con `npm run test:coverage`, que genera `coverage/lcov.info`, importado por SonarCloud (propiedad `sonar.javascript.lcov.reportPaths`).

```

1  name: SonarCloud (Frontend)
2  on:
3    push: { branches: [main, hotfix] }
4    pull_request: { branches: [main] }
5  jobs:
6    sonar:
7      runs-on: ubuntu-latest
8      steps:
9        - uses: actions/checkout@v4
10         with: { fetch-depth: 0 }
11        - uses: actions/setup-node@v4
12         with: { node-version: '20', cache: 'npm' }
13        - run: npm ci
14        - run: npm run test:coverage
15          env: { CI: 'true', VITE_API_URL: http://localhost:8000/api/v1 }
16        - uses: SonarSource/sonarcloud-github-action@v3
17          env: { SONAR_TOKEN: '${ secrets.SONAR_TOKEN }' }

```

Listado N.2: Extracto de .github/workflows/sonarcloud.yml (Frontend).

N.5.3 Configuración del Proyecto

El fichero `sonar-project.properties` declara, en cada repositorio, la `projectKey` de SonarCloud, la organización (`angel-tfg-uclm`), las rutas de fuentes y tests, las exclusiones específicas (scripts utilitarios, *infra-as-code*, datos estáticos, módulos *bootstrap*, escena 3D no testeable y mocks) y los *paths* de los informes de cobertura. Esta separación entre `sonar-project.properties` y el *workflow* permite modificar las exclusiones sin tocar el *pipeline* de CI.

N.5.4 Quality Gate Sonar Way y resultados

A diferencia de la *Quality Gate Entangle* usada en el análisis local, que evalúa el proyecto en su totalidad, la *Sonar Way* se centra en el **código nuevo** desde la rama de referencia (`previous_version`). La Tabla N.5 resume las condiciones evaluadas y el valor real observado en el último *run* de cada repositorio.

Tabla N.5: Condiciones de la *Quality Gate Sonar Way* y resultados sobre código nuevo en SonarCloud.

Métrica (código nuevo)	Umbral	Backend	Frontend
<i>Reliability Rating</i> (new)	A	A	A
<i>Security Rating</i> (new)	A	A	A
<i>Maintainability Rating</i> (new)	A	A	A
Cobertura sobre código nuevo	≥ 80 %	N/A *	100,0 %
Duplicación en código nuevo	≤ 3 %	0,0 %	0,0 %
<i>Security Hotspots</i> revisados	100 %	100 %	100 %
Quality Gate	N/A	OK	OK

*La condición de cobertura sobre código nuevo se omite cuando el último cambio no introduce líneas ejecutables (por ejemplo, cuando el commit solo modifica documentación, configuración o tests existentes).

N.5.5 Dashboards y Resultados Visuales en SonarCloud

Las Figuras N.3, N.4 y N.5 muestran las vistas reales del servicio SonarCloud para la organización `angel-tfg-uclm` y para cada uno de sus dos proyectos. La vista consolidada de organización (Figura N.3) presenta de un vistazo el estado de la *Quality Gate Sonar Way* para los dos repositorios y su última ejecución, mientras que las vistas individuales (Figuras N.4 y N.5) detallan los paneles *New Code* y *Overall Code* con todas las métricas de cobertura, duplicación, fiabilidad, seguridad y mantenibilidad medidas por el servicio.

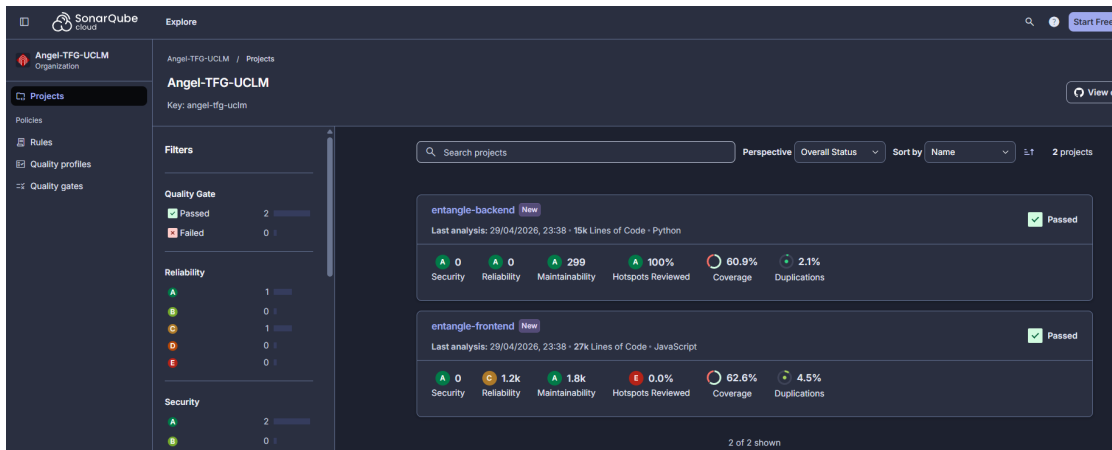


Figura N.3: Vista consolidada de la organización `angel-tfg-uclm` en SonarCloud.

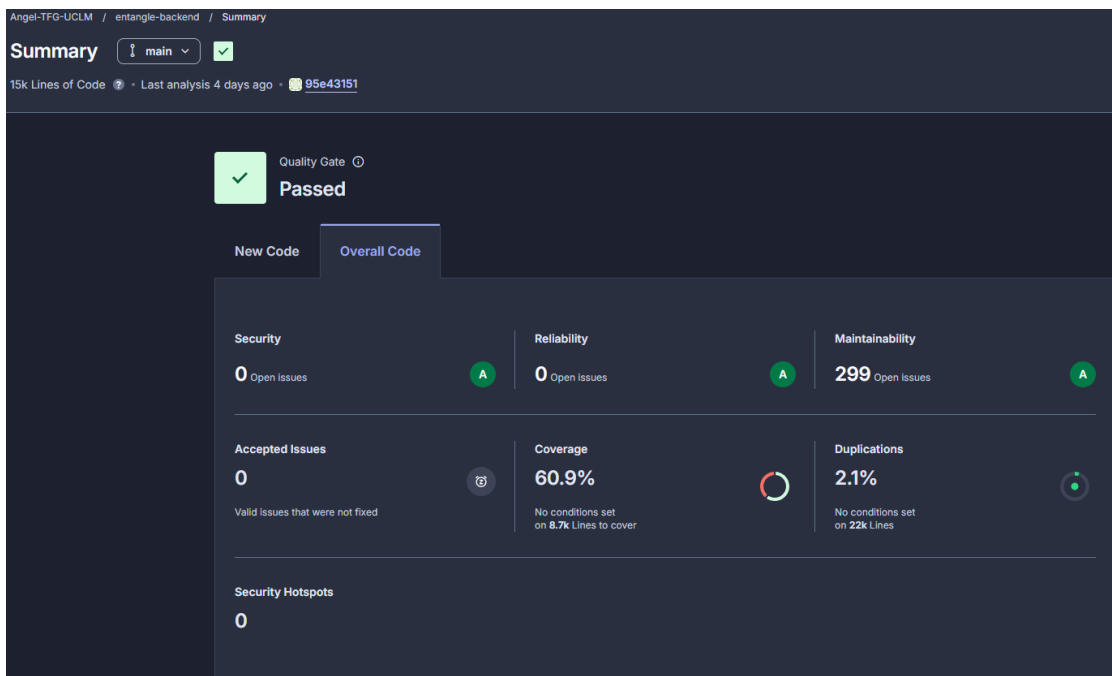


Figura N.4: Dashboard de SonarCloud del proyecto `Angel-TFG-UCLM_Entangle-Core` (backend).

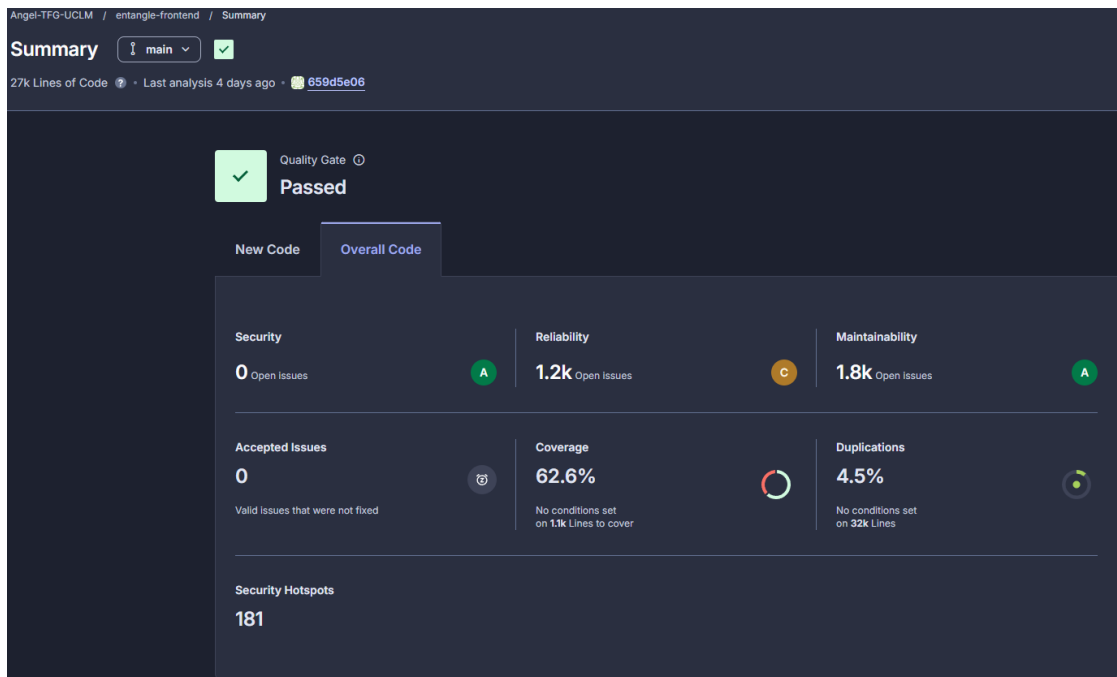


Figura N.5: Dashboard de SonarCloud del proyecto Angel-TFG-UCLM_Entangle-Visualizer (frontend).

Adicionalmente, la integración con GitHub Actions hace que cada *pull request* reciba un comentario automático del bot SonarCloud (Figura N.6) con el resultado de la *Quality Gate* sobre los cambios introducidos. Este *check* actúa como guardarraíl preventivo: cualquier degradación en código nuevo es detectada y notificada antes del *merge*, permitiendo iterar sobre el cambio sin contaminar la rama *main*.

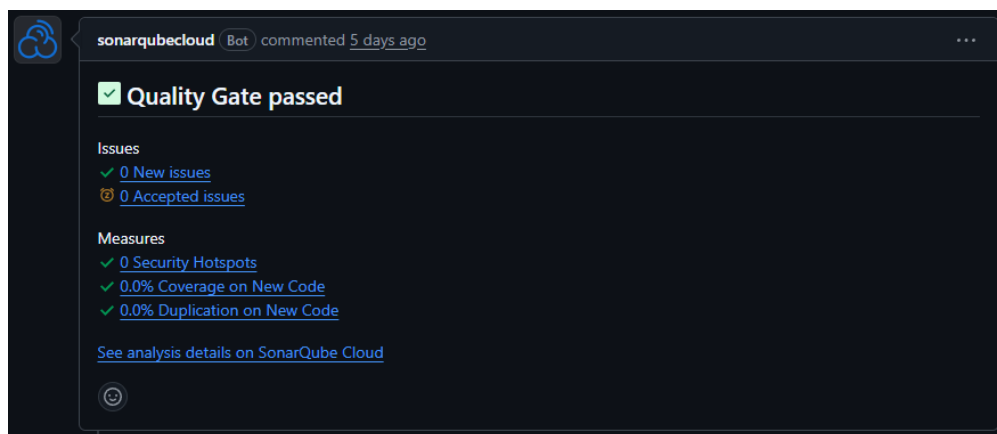


Figura N.6: Comentario automático del bot SonarCloud en un *pull request*.

N.5.6 Análisis local vs. CI continuo: complementariedad

Las dos capas de análisis son complementarias y obedecen a propósitos distintos:

- La **instancia local de SonarQube** con la *Quality Gate Entangle* (Sección N.1) se utiliza para auditorías en profundidad sobre la totalidad del código, con umbrales

adaptados al estado heredado del proyecto (cobertura global ≥ 60 %, fiabilidad $\leq C$). Estos resultados son los que figuran en las Tablas N.2 y N.4.

- **SonarCloud** con la *Sonar Way* actúa como *gatekeeper* sobre cada cambio entrante, garantizando que *ninguna línea nueva* degrada la calidad medida del proyecto, independientemente del estado heredado.

Esta combinación cubre las dos preguntas relevantes: «¿en qué estado está el proyecto en su conjunto?» (responde el análisis local) y «¿cada cambio que entra mantiene o mejora ese estado?» (responde la CI continua de SonarCloud).

Anexo Ñ

Fragmentos de código adicionales

Este anexo recoge los fragmentos de código que sostienen los algoritmos clásicos del proyecto pero cuya inclusión en el cuerpo de la memoria desplazaría otros contenidos más relevantes. Se complementan con los fragmentos centrales del proyecto, integrados en sus secciones de contexto del Capítulo 5: la detección de comunidades mediante Louvain (Listado 5.2) y la clasificación de intención del agente conversacional (Listado 5.1).

Ñ.1 Ejecución Segura de Agregaciones MongoDB

El Listado Ñ.1 muestra la función `run_aggregation`, invocada por el agente DATA mediante *function calling*. Incorpora validaciones de seguridad: solo permite colecciones predefinidas, bloquea *stages* de escritura (`$out`, `$merge`) y fuerza un `$limit` si el *pipeline* no lo incluye.

```
1 def run_aggregation(collection: str, pipeline: List[Dict]) -> str:
2     if collection not in _COLLECTIONS:
3         return json.dumps({"error": f"Coleccion no valida: {collection}"})
4
5     FORBIDDEN_STAGES = {"$out", "$merge"}
6     for stage in pipeline:
7         for key in stage:
8             if key in FORBIDDEN_STAGES:
9                 return json.dumps({"error": f"Stage '{key}' no permitido"})
10
11     if not any("$limit" in s for s in pipeline):
12         pipeline.append({"$limit": _MAX_RESULTS})
13
14     repo = _COLLECTIONS[collection]
15     cursor = repo.collection.aggregate(pipeline)
16     results = list(cursor)
17
18     for doc in results:
19         if "_id" in doc and not isinstance(doc["_id"], (str, int, float)):
20             doc["_id"] = str(doc["_id"])
21
22     return json.dumps({"count": len(results), "results": results}, default=str)
```

Listado Ñ.1: Ejecución segura de pipelines de agregación.

Ñ.2 Clasificación por Jenks Natural Breaks

El Listado Ñ.2 implementa el algoritmo de Jenks Natural Breaks [Jen67] mediante programación dinámica. El algoritmo particiona un vector de valores numéricos en k clases minimizando la varianza intraclass. Se utiliza para clasificar organizaciones en tres zonas de densidad (núcleo, media, periferia) en la visualización 3D.

```
1  def jenks_natural_breaks(data, n_classes):
2      sorted_data = sorted(data)
3      n = len(sorted_data)
4      if n <= n_classes:
5          step = (sorted_data[-1] - sorted_data[0]) / n_classes if n > 1 else 0
6          return [sorted_data[0] + step * (i + 1) for i in range(n_classes - 1)]
7
8      lower = [[0] * (n_classes + 1) for _ in range(n + 1)]
9      vari = [[float('inf')] * (n_classes + 1) for _ in range(n + 1)]
10     for j in range(1, n_classes + 1):
11         lower[1][j] = 1
12         vari[1][j] = 0
13
14     for l in range(2, n + 1):
15         s = s2 = 0; w = 0
16         for m in range(1, l + 1):
17             i3 = l - m + 1
18             val = sorted_data[i3 - 1]
19             w += 1; s += val; s2 += val * val
20             v = s2 - (s * s) / w
21             if i3 > 1:
22                 for j in range(2, n_classes + 1):
23                     cost = v + vari[i3 - 1][j - 1]
24                     if cost < vari[l][j]:
25                         lower[l][j] = i3
26                         vari[l][j] = cost
27         lower[l][1] = 1
28         vari[l][1] = s2 - (s * s) / w
29
30     class_starts = [0] * n_classes
31     k = n
32     for j in range(n_classes, 1, -1):
33         class_starts[j - 1] = lower[k][j] - 1
34         k = lower[k][j] - 1
35
36     boundaries = []
37     for c in range(1, n_classes):
38         boundaries.append(
39             (sorted_data[class_starts[c] - 1] + sorted_data[class_starts[c]]) / 2
40         )
41     return boundaries
```

Listado Ñ.2: Algoritmo de Jenks Natural Breaks.

Ñ.3 Camino Mínimo entre Entidades (Quantum Tunnel)

El Listado Ñ.3 muestra la función que encuentra el camino más corto entre dos entidades del grafo mediante el algoritmo de Dijkstra [Dij59], utilizada en la funcionalidad *Quantum Tunnel* del universo 3D. La implementación filtra nodos bot como intermediarios para obtener rutas más significativas y recurre al grafo completo si no existe ruta libre de bots.

```
1  def find_path(self, source_id, target_id):
2      if not self.G.has_node(source_id):
3          return {"found": False, "error": f"Node origen no encontrado"}
4      if not self.G.has_node(target_id):
5          return {"found": False, "error": f"Node destino no encontrado"}
6      try:
7          non_bot_nodes = [
8              n for n in self.G.nodes
9              if not self.G.nodes[n].get("is_bot", False)
10             or n in (source_id, target_id)
11          ]
12          G_no_bots = self.G.subgraph(non_bot_nodes)
13          try:
14              path_nodes = nx.shortest_path(G_no_bots, source_id, target_id)
15          except nx.NetworkXNoPath:
16              path_nodes = nx.shortest_path(self.G, source_id, target_id)
17
18          path_details = []
19          for node_id in path_nodes:
20              data = self.G.nodes[node_id]
21              path_details.append({
22                  "id": node_id,
23                  "type": data.get("type", "unknown"),
24                  "name": data.get("name") or data.get("login")
25                  or data.get("full_name", node_id),
26              })
27          return {
28              "found": True, "path": path_details,
29              "length": len(path_nodes) - 1,
30          }
31      except nx.NetworkXNoPath:
32          return {"found": False, "error": "No existe canal entre entidades"}
```

Listado Ñ.3: Camino mínimo con filtrado de bots (Dijkstra).

Ñ.4 Cálculo del Bus Factor

El Listado Ñ.4 calcula el *bus factor* de cada repositorio: el número mínimo de contribuidores que acumulan el 50 % de las contribuciones totales [APHV16]. El resultado se clasifica en cuatro niveles de riesgo que alimentan la lente de resiliencia del universo 3D.

```
1  def compute_bus_factor(self):
2      bus_factors = {}
3      for full_name, repo in self._repos_data.items():
4          collabs = repo.get("collaborators", [])
5          if not collabs:
6              bus_factors[f"repo_{full_name}"] = {
7                  "bus_factor": 0, "risk": "critical"
8              }
9              continue
10
11         sorted_c = sorted(
12             collabs, key=lambda c: c.get("contributions", 0), reverse=True
13         )
14         total = sum(c.get("contributions", 0) for c in sorted_c)
15         if total == 0:
16             bus_factors[f"repo_{full_name}"] = {
17                 "bus_factor": len(sorted_c),
18                 "risk": "low" if len(sorted_c) > 3 else "medium",
19             }
20             continue
21
22         threshold = total * 0.5
23         cumulative = bus_factor = 0
24         for c in sorted_c:
25             cumulative += c.get("contributions", 0)
26             bus_factor += 1
27             if cumulative >= threshold:
28                 break
29
30         if bus_factor <= 1: risk = "critical"
31         elif bus_factor <= 2: risk = "high"
32         elif bus_factor <= 4: risk = "medium"
33         else: risk = "low"
34
35         bus_factors[f"repo_{full_name}"] = {
36             "bus_factor": bus_factor, "risk": risk,
37             "total_contributions": total,
38             "total_contributors": len(sorted_c),
39         }
40     return bus_factors
```

Listado Ñ.4: Cálculo del bus factor por repositorio.

Ñ.5 Gestión del Rate Limit de GitHub

El Listado Ñ.5 muestra el decorador que gestiona automáticamente los límites de tasa de la API de GitHub. El sistema verifica preventivamente el número de peticiones restantes y, si se agota, espera hasta el *reset* con un margen de seguridad de 5 segundos.

```
1  def with_rate_limit_handling(max_retries: int = 3):
2      def decorator(func):
3          @wraps(func)
4          def wrapper(*args, **kwargs):
5              retries = 0
6              while retries <= max_retries:
7                  try:
8                      check_rate_limit_before_request()
9                      return func(*args, **kwargs)
10                 except Exception as e:
11                     if "rate limit" in str(e).lower():
12                         rate_limit = get_rate_limit_info()
13                         wait_for_rate_limit_reset(rate_limit["resetAt"])
14                         retries += 1
15                         continue
16                     raise
17                 raise Exception(f"Reintentos agotados ({max_retries})")
18             return wrapper
19     return decorator
```

Listado Ñ.5: Decorador con reintentos ante rate limit.

Anexo O

Detalle de la suite de pruebas

Este anexo amplía la Sección 5.16 con la estrategia de aislamiento de la suite del *backend*, las pruebas específicas de resiliencia frente a *throttling* de Cosmos DB, las pruebas del `TestClient` de FastAPI y las propiedades de idempotencia que aseguran la consistencia del *pipeline* de ingesta.

O.1 Estrategia de Aislamiento del Backend

Todas las pruebas operan sobre *mocks* y *fixtures* definidos en `conftest.py`: un cliente GraphQL simulado, una base de datos en memoria y un repositorio MongoDB con operaciones Create, Read, Update, Delete (CRUD) completas. Esto permite ejecutar la suite sin depender de servicios externos (MongoDB, GitHub API), manteniendo tiempos de ejecución bajos. Los marcadores `@pytest.mark.integration` y `@pytest.mark.slow` están registrados para distinguir las pruebas que requieren conexión real, aunque en la práctica toda la suite se ejecuta con *mocks*.

O.2 Pruebas de Resiliencia frente a *Throttling*

Varios ficheros de test incluyen pruebas específicas para el mecanismo de reintento ante errores HTTP 429 de Azure Cosmos DB [Mic24c]. Cuando el servicio devuelve un código 429 (*Request Rate Too Large*), la cabecera `Retry-After` indica cuántos milisegundos debe esperar el cliente antes de reintentar. El *backend* implementa un bucle de reintento con retroceso exponencial que respeta este valor, hasta un máximo configurable de intentos. Los tests verifican tres escenarios concretos:

- **Sin *throttle***: la operación se completa en el primer intento y devuelve los datos esperados.
- **Reintento exitoso**: la primera llamada devuelve 429, el sistema espera y reintenta, y la segunda llamada tiene éxito. Se verifica que los datos devueltos son correctos y que se realizaron exactamente dos llamadas.
- **Degradación controlada**: todas las llamadas devuelven 429 hasta agotar los reintentos. Se verifica que el sistema lanza una excepción controlada en lugar de bloquearse indefinidamente o propagar un error no gestionado.

La función `time.sleep` se mockea para evitar esperas reales durante la ejecución.

O.3 Pruebas de la API REST

Los tests de `test_api.py` y `test_routes_*.py` utilizan el `TestClient` de FastAPI [Ram24], que permite realizar peticiones HTTP al servidor sin levantarlo en un puerto real. Se cubren los *endpoints* de salud, listado y obtención por ID de las tres entidades (repositorios, usuarios y organizaciones), filtros por lenguaje y estrellas mínimas, paginación, lanzamiento del *pipeline*, administración y búsqueda de entidades. De forma complementaria, durante el desarrollo se utilizaron *scripts* puntuales para tareas de validación manual: auditoría y limpieza del *dataset* (eliminación de falsos positivos como QuantumultX o Firefox Quantum), verificación de la migración a Azure Cosmos DB for MongoDB vCore y comprobación de los cortes de Jenks Natural Breaks [Jen67] sobre las distribuciones reales.

O.4 Idempotencia y Consistencia del Pipeline

Todas las operaciones de escritura del *pipeline* de ingesta utilizan `bulk_upsert` con el campo `id` de GitHub como clave de deduplicación: cada documento se inserta o actualiza mediante `UpdateOne({id: doc[id]}, {"$set": doc}, upsert=True)`. Esta estrategia garantiza que re-ejecutar cualquier fase del *pipeline* (repositorios, usuarios u organizaciones) no duplica datos ni corrompe registros previos: el resultado es idéntico independientemente de cuántas veces se ejecute. En caso de fallo a mitad de una fase, basta con relanzarla; los documentos ya procesados simplemente se sobrescriben con valores idénticos, y los pendientes se insertan por primera vez. Dado que el identificador es el `id` numérico inmutable de GitHub y no el `login` del usuario, los cambios de nombre de cuenta no generan duplicados ni referencias huérfanas. Esta propiedad de idempotencia simplifica tanto la operación en producción como la estrategia de recuperación ante errores.

Anexo P

Declaración de Uso Responsable de la Inteligencia Artificial

Durante la elaboración de este Trabajo Fin de Grado se han empleado herramientas de inteligencia artificial generativa (asistentes de programación y modelos de lenguaje de gran tamaño) como apoyo al proceso de desarrollo. El autor declara que dicho uso se ha ajustado en todo momento a los principios de integridad académica y a las recomendaciones de la Universidad de Castilla-La Mancha sobre el empleo responsable de estas tecnologías en trabajos académicos.

P.1 Ámbito del uso de la inteligencia artificial

Las herramientas de inteligencia artificial se han utilizado, con supervisión humana constante, en los siguientes ámbitos:

- **Asistencia a la programación.** Apoyo en la escritura, refactorización y documentación de código, así como en la depuración de errores y la generación de pruebas. La totalidad del código resultante ha sido revisada, comprendida, adaptada y validada por el autor antes de su incorporación al proyecto.
- **Apoyo a la redacción.** Sugerencias de estilo, corrección ortográfica y gramatical, y mejora de la claridad expositiva de la memoria. La estructura, los argumentos, las conclusiones y la interpretación de los resultados son obra y responsabilidad exclusiva del autor.
- **Consulta técnica.** Resolución de dudas sobre tecnologías, bibliotecas y buenas prácticas, siempre contrastadas con la documentación oficial y la bibliografía citada en este documento.

P.2 Compromisos asumidos

El autor de este Trabajo Fin de Grado declara expresamente que:

1. Todo el contenido elaborado con asistencia de inteligencia artificial ha sido revisado, verificado y validado críticamente, asumiendo la plena responsabilidad sobre su corrección y adecuación.

2. Las decisiones de diseño, la arquitectura del sistema, los análisis realizados y las conclusiones alcanzadas son fruto del trabajo propio y no han sido delegadas a ninguna herramienta automática.
3. No se ha presentado como propio ningún contenido generado por inteligencia artificial sin la debida comprensión, adaptación y verificación.
4. Los datos, métricas y resultados expuestos en esta memoria son reales y reproducibles, y proceden de la ejecución efectiva del sistema desarrollado.

Coherentemente con estos principios, el propio agente conversacional que forma parte del sistema Entangle se ha diseñado bajo criterios de uso responsable de la inteligencia artificial: fundamenta sus respuestas en datos reales mediante recuperación aumentada (Sección 5.6.2), reconoce explícitamente cuándo carece de información en lugar de inventarla, y opera con herramientas de solo lectura que impiden la alteración de los datos (Sección 5.14).

P.3 Firma

Y para que así conste a los efectos oportunos, el autor firma la presente declaración de uso responsable de la inteligencia artificial.

En Talavera de la Reina, a 13 de julio de 2026.

Este documento ha sido firmado digitalmente por el autor.

Fdo.: Ángel Luis Lara Martín

Este documento fue editado y tipografiado con $\text{\LaTeX}$
empleando la clase **gita-tfg** que se puede encontrar en:

<https://github.com/anarubioruiz/gita-tfg>

La clase **gita-tfg** está basada en la clase **esi-tfg**:

<https://github.com/UCLM-ESI/esi-tfg>

